

# 尚硅谷大数据项目之电商实时数仓 (电商数据仓库系统)

(作者：尚硅谷研究院)

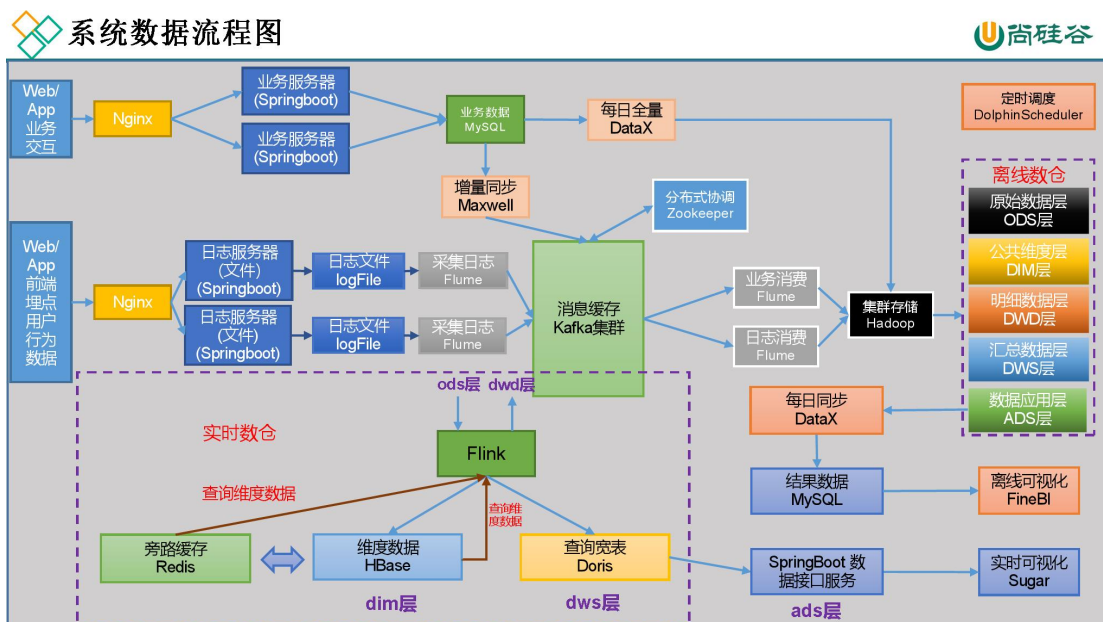
版本：V5.0

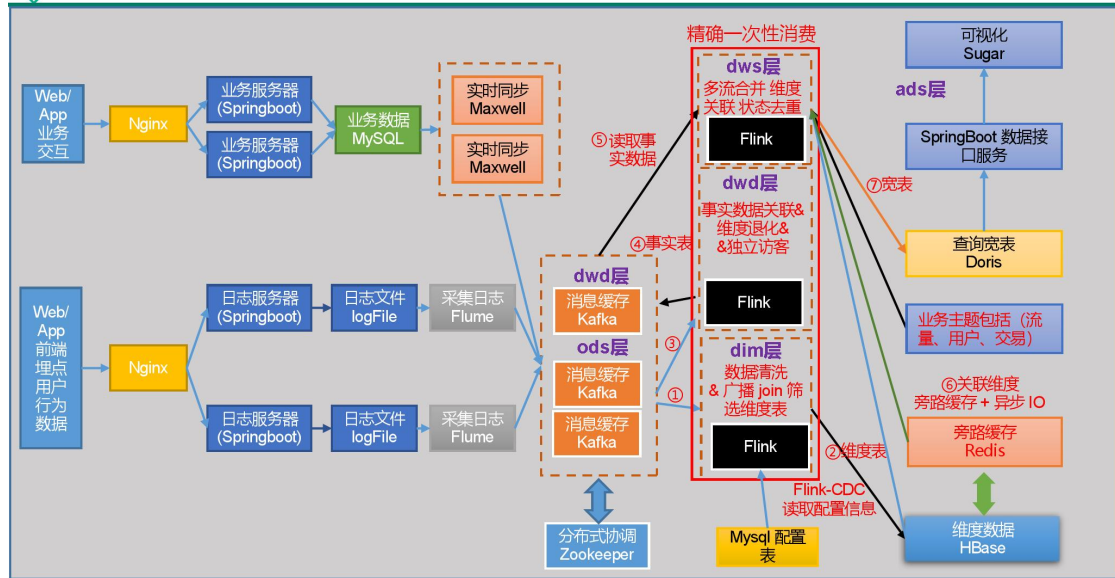
## 第 1 章 数据仓库概述

### 1.1 数据仓库概念

数据仓库是一个为数据分析而设计的企业级数据管理系统。数据仓库可集中、整合多个信息源的大量数据，借助数据仓库的分析能力，企业可从数据中获得宝贵的信息进而改进决策。同时，随着时间的推移，数据仓库中积累的大量历史数据对于数据科学家和业务分析师也是十分宝贵的。

### 1.2 数据仓库核心架构





### 1.3 普通实时计算与实时数仓比较

普通的实时计算优先考虑时效性，所以从数据源采集经过实时计算直接得到结果。如此做时效性更好，但是弊端是由于计算过程中的中间结果没有沉淀下来，所以当面对大量实时需求的时候，计算的复用性较差，开发成本随着需求增加直线上升。



实时数仓基于一定的数据仓库理念，对数据处理流程进行规划、分层，目的是提高数据的复用性。



## 1.4 离线计算与实时计算的比较

**离线计算：**就是在计算开始前已知所有输入数据，输入数据不会产生变化，一般计算量级较大，计算时间也较长。例如今天早上一点，把昨天累积的日志，计算出所需结果。最经典的就是 Hadoop 的 MapReduce 方式；

一般是根据前一日的数据生成报表，虽然统计指标、报表繁多，但是对时效性不敏感。从技术操作的角度，这部分属于批处理的操作。即根据确定范围的数据一次性计算。

**实时计算：**输入数据是可以以序列化的方式一个个输入并进行处理的，也就是说在开始的时候并不需要知道所有的输入数据。与离线计算相比，运行时间短，计算量级相对较小。强调计算过程的时间要短，即所查当下给出结果。

主要侧重于对当日数据的实时监控，通常业务逻辑相对离线需求简单一些，统计指标也少一些，但是更注重数据的时效性，以及用户的交互性。从技术操作的角度，这部分属于流处理的操作。根据数据源源不断地到达进行实时的运算。

## 第 2 章 数据仓库建模概述

### 2.1 数据仓库建模的意义

如果把数据看作图书馆里的书，我们希望看到它们在书架上分门别类地放置；如果把数据看作城市的建筑，我们希望城市规划布局合理；如果把数据看作电脑文件和文件夹，我们希望按照自己的习惯有很好的文件夹组织方式，而不是糟糕混乱的桌面，经常为找一个文件而不知所措。

数据模型就是数据组织和存储方法，它强调从业务、数据存取和使用角度合理存储数据。只有将数据有序的组织 and 存储起来之后，数据才能得到高性能、低成本、高效率、高质量的使用。

高性能：良好的数据模型能够帮助我们快速查询所需要的数据。

低成本：良好的数据模型能减少重复计算，实现计算结果的复用，降低计算成本。

高效率：良好的数据模型能极大的改善用户使用数据的体验，提高使用数据的效率。

高质量：良好的数据模型能改善数据统计口径的混乱，减少计算错误的可能性。

## 2.2 数据仓库建模方法论

### 2.2.1 ER 模型

数据仓库之父 Bill Inmon 提出的建模方法是从全企业的高度，用实体关系（Entity Relationship, ER）模型来描述企业业务，并用规范化的方式表示出来，在范式理论上符合 3NF。

#### 1) 实体关系模型

实体关系模型将复杂的数据抽象为两个概念——实体和关系。实体表示一个对象，例如学生、班级，关系是指两个实体之间的关系，例如学生和班级之间的从属关系。

#### 2) 数据库规范化

数据库规范化是使用一系列范式设计数据库（通常是关系型数据库）的过程，其目的是减少数据冗余，增强数据的一致性。

这一系列范式就是指在设计关系型数据库时，需要遵从的不同的规范。关系型数据库的范式一共有六种，分别是第一范式（1NF）、第二范式（2NF）、第三范式（3NF）、巴斯-科德范式（BCNF）、第四范式（4NF）和第五范式（5NF）。遵循的范式级别越高，数据冗余性就越低。

#### 3) 三范式

##### （1）第一范式



## 第一范式

1、第一范式1NF核心原则就是：**属性不可切割**

表 不符合一范式的表格设计

| ID  | 商品    | 商家ID   | 用户ID  |
|-----|-------|--------|-------|
| 001 | 5 台电脑 | XXX旗舰店 | 00001 |

很明显上图所示的表格设计是不符合第一范式的，商品列中的数据不是原子数据项，是可以进行分割的，因此对表格进行修改，让表格符合第一范式的要求，修改结果如下图所示：

表 符合一范式的表格设计

| ID  | 商品 | 数量 | 商家ID   | 用户ID  |
|-----|----|----|--------|-------|
| 001 | 电脑 | 5  | XXX旗舰店 | 00001 |

实际上，**1NF是所有关系型数据库的最基本要求**，你在关系型数据库管理系统（RDBMS），例如SQL Server，Oracle，MySQL中创建数据表的时候，**如果数据表的设计不符合这个最基本的要求，那么操作一定是不能成功的**。也就是说，只要在RDBMS中已经存在的数据表，一定是符合1NF的。

让天下没有难学的技术

## (2) 第二范式

## 第二范式

2、第二范式2NF核心原则：**不能存在“部分函数依赖”**

| 学号         | 姓名  | 系名  | 系主任 | 课名    | 分数 |
|------------|-----|-----|-----|-------|----|
| 1022211101 | 李小明 | 经济系 | 王强  | 高等数学  | 95 |
| 1022211101 | 李小明 | 经济系 | 王强  | 大学英语  | 87 |
| 1022211101 | 李小明 | 经济系 | 王强  | 普通化学  | 76 |
| 1022211102 | 张莉莉 | 经济系 | 王强  | 高等数学  | 72 |
| 1022211102 | 张莉莉 | 经济系 | 王强  | 大学英语  | 98 |
| 1022211102 | 张莉莉 | 经济系 | 王强  | 计算机基础 | 88 |
| 1022511101 | 高芳芳 | 法律系 | 刘玲  | 高等数学  | 82 |
| 1022511101 | 高芳芳 | 法律系 | 刘玲  | 法学基础  | 82 |

以上表格明显存在，部分依赖。比如，这张表的主键是（学号，课名），分数确实完全依赖于（学号，课名），但是姓名并不完全依赖于（学号，课名）

| 学号         | 课名    | 分数 |
|------------|-------|----|
| 1022211101 | 高等数学  | 95 |
| 1022211101 | 大学英语  | 87 |
| 1022211101 | 普通化学  | 76 |
| 1022211102 | 高等数学  | 72 |
| 1022211102 | 大学英语  | 98 |
| 1022211102 | 计算机基础 | 88 |
| 1022511101 | 高等数学  | 82 |
| 1022511101 | 法学基础  | 82 |

| 学号         | 姓名  | 系名  | 系主任 |
|------------|-----|-----|-----|
| 1022211101 | 李小明 | 经济系 | 王强  |
| 1022211102 | 张莉莉 | 经济系 | 王强  |
| 1022511101 | 高芳芳 | 法律系 | 刘玲  |

以上符合第二范式，去掉部分函数依赖

让天下没有难学的技术

## (3) 第三范式

### 3、第三范式 3NF 核心原则：不能存在传递函数依赖

在下面这张表中，存在传递函数依赖：学号->系名->系主任，但是系主任推不出学号。

| 学号         | 姓名  | 系名  | 系主任 |
|------------|-----|-----|-----|
| 1022211101 | 李小明 | 经济系 | 王强  |
| 1022211102 | 张莉莉 | 经济系 | 王强  |
| 1022511101 | 高芳芳 | 法律系 | 刘玲  |

上面表需要再次拆解：

| 学号         | 姓名  | 系名  |
|------------|-----|-----|
| 1022211101 | 李小明 | 经济系 |
| 1022211102 | 张莉莉 | 经济系 |
| 1022511101 | 高芳芳 | 法律系 |

| 系名  | 系主任 |
|-----|-----|
| 经济系 | 王强  |
| 法律系 | 刘玲  |

让天下没有难学的技术

### (4) 函数依赖

| 学号         | 姓名  | 系名  | 系主任 | 课程    | 分数 |
|------------|-----|-----|-----|-------|----|
| 1022211101 | 李小明 | 经济系 | 王强  | 高等数学  | 95 |
| 1022211101 | 李小明 | 经济系 | 王强  | 大学英语  | 87 |
| 1022211101 | 李小明 | 经济系 | 王强  | 普通化学  | 76 |
| 1022211102 | 张莉莉 | 经济系 | 王强  | 高等数学  | 72 |
| 1022211102 | 张莉莉 | 经济系 | 王强  | 大学英语  | 98 |
| 1022211102 | 张莉莉 | 经济系 | 王强  | 计算机基础 | 88 |
| 1022511101 | 高芳芳 | 法律系 | 刘玲  | 高等数学  | 82 |
| 1022511101 | 高芳芳 | 法律系 | 刘玲  | 法学基础  | 82 |

#### 1、完全函数依赖：

设X, Y是关系R的两个属性集合，X'是X的真子集，存在  $X \rightarrow Y$ ，但对每一个X'都有  $X' \not\rightarrow Y$ ，则称Y完全函数依赖于X。记做：  $X \twoheadrightarrow Y$

人类语言：

比如通过，(学号，课程) 推出分数，但是单独用学号推断不出来分数，那么可以说：分数 完全依赖于(学号，课程)。

即：通过AB能得出C，但是AB单独得不出C，那么说C完全依赖于AB。

#### 2、部分函数依赖

假如Y函数依赖于X，但同时Y并不完全函数依赖于X，那么我们就称Y部分函数依赖于X，记做：  $X \twoheadrightarrow Y$

人类语言：

比如通过，(学号，课程) 推出姓名，因为其实直接可以通过，学号推出姓名，所以：姓名 部分依赖于(学号，课程) 即：通过AB能得出C，通过A也能得出C，或者通过B也能得出C，那么说C部分依赖于AB。

#### 3、传递函数依赖

传递函数依赖：设X, Y, Z是关系R中互不相同的属性集合，存在  $X \rightarrow Y (Y \not\rightarrow X), Y \rightarrow Z$ ，则称Z传递函数依赖于X。记做：  $X \twoheadrightarrow Z$

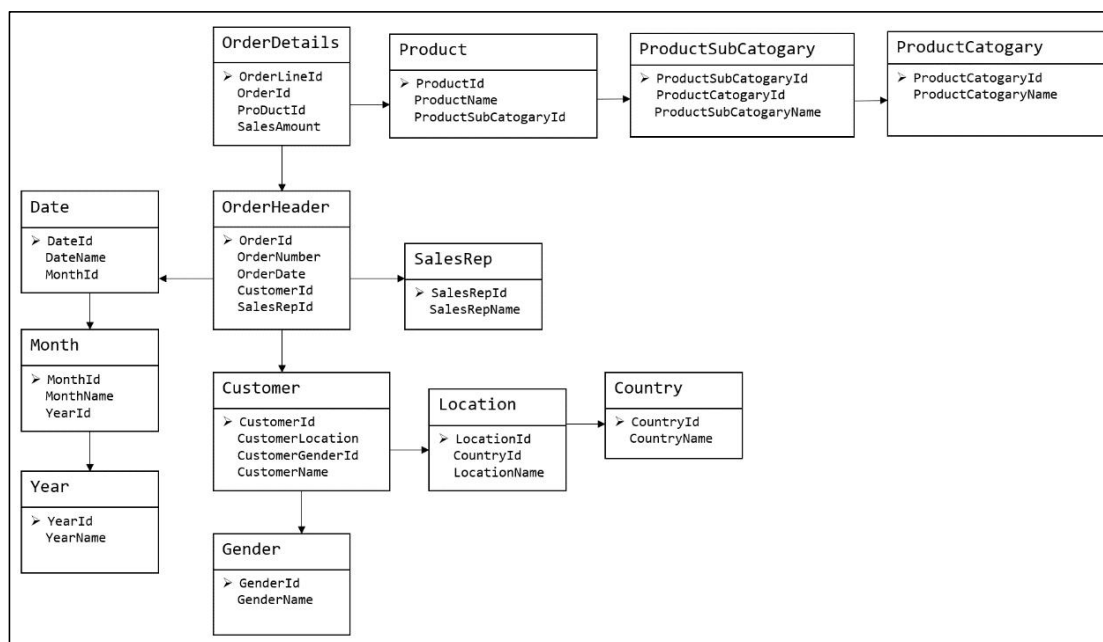
人类语言：

比如：学号 推出 系名，系名 推出 系主任，但是，系主任推不出学号，系主任主要依赖于系名。这种情况可以说：系主任 传递依赖于 学号

通过A得到B，通过B得到C，但是C得不到A，那么说C传递依赖于A。

让天下没有难学的技术

下图为一个采用 Bill Inmon 倡导的建模方法构建的模型，从图中可以看出，较为松散、零碎，物理表数量多。



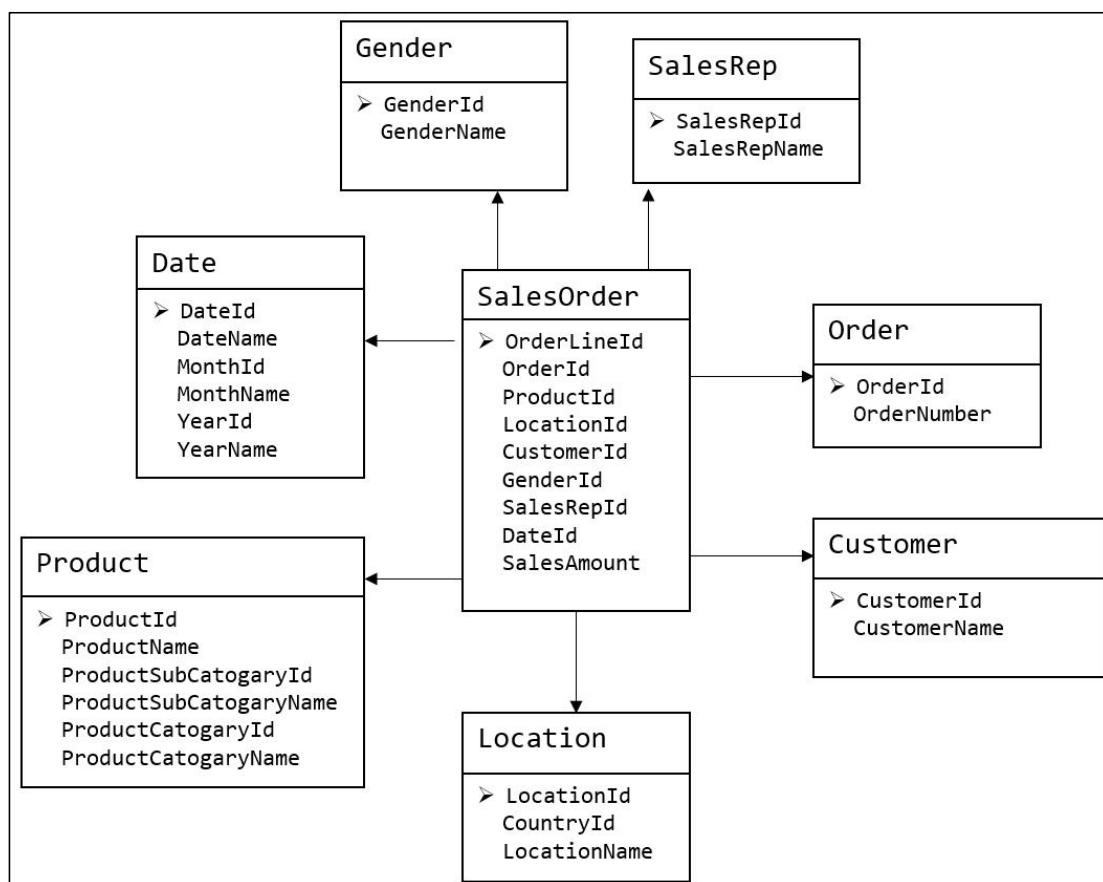
这种建模方法的出发点是整合数据，其目的是将整个企业的数据进行组合和合并，并进行规范处理，减少数据冗余性，保证数据的一致性。这种模型并不适合直接用于分析统计。

## 2.2.2 维度模型

数据仓库领域的另一位大师——Ralph Kimball 倡导的建模方法为维度建模。维度模型将复杂的业务通过**事实**和**维度**两个概念进行呈现。**事实通常对应业务过程，而维度通常对应业务过程发生时所处的环境。**

注：业务过程可以概括为一个个不可拆分的行为事件，例如电商交易中的下单，取消订单，付款，退单等，都是业务过程。

下图为一个典型的维度模型，其中位于中心的 SalesOrder 为事实表，其中保存的是下单这个业务过程的所有记录。位于周围每张表都是维度表，包括 Date（日期），Customer（顾客），Product（产品），Location（地区）等，这些维度表就组成了每个订单发生时所处的环境，即何人、何时、在何地下单了何种产品。从图中可以看出，模型相对清晰、简洁。



维度建模以数据分析作为出发点，为数据分析服务，因此它关注的重点的用户如何更快的完成需求分析以及如何实现较好的大规模复杂查询的响应性能。

## 第 3 章 维度建模理论之事实表

事实表作为数据仓库维度建模的核心，紧紧围绕着业务过程来设计。其包含与该业务过程有关的维度引用（维度表外键）以及该业务过程的度量（通常是可累加的数字类型字段）。

事实表通常比较“细长”，即列较少，但行较多，且行的增速快。

事实表有三种类型：分别是事务事实表、周期快照事实表和累积快照事实表，每种事实表都具有不同的特点和适用场景，下面逐个介绍。

### 3.1 事务型事实表

#### 3.1.1 概述

事务事实表用来记录各业务过程，它保存的是各业务过程的原子操作事件，即最细粒度的操作事件。粒度是指事实表中一行数据所表达的业务细节程度。

事务型事实表可用于分析与各业务过程相关的各项统计指标,由于其保存了最细粒度的记录,可以提供最大限度的灵活性,可以支持无法预期的各种细节层次的统计需求。

### 3.1.2 设计流程

设计事务事实表时一般可遵循以下四个步骤:

选择业务过程→声明粒度→确认维度→确认事实

#### 1) 选择业务过程

在业务系统中,挑选我们感兴趣的业务过程,业务过程可以概括为一个个不可拆分的行为事件,例如电商交易中的下单,取消订单,付款,退单等,都是业务过程。通常情况下,一个业务过程对应一张事务型事实表。

#### 2) 声明粒度

业务过程确定后,需要为每个业务过程声明粒度。即精确定义每张事务型事实表的每行数据表示什么,应该尽可能选择最细粒度,以此来应各种细节程度的需求。

典型的粒度声明如下:

订单事实表中一行数据表示的是一个订单中的一个商品项。

#### 3) 确定维度

确定维度具体是指,确定与每张事务型事实表相关的维度有哪些。

确定维度时应尽量多的选择与业务过程相关的环境信息。因为维度的丰富程度就决定了维度模型能够支持的指标丰富程度。

#### 4) 确定事实

此处的“事实”一词,指的是每个业务过程的度量值(通常是可累加的数字类型的值,例如:次数、个数、件数、金额等)。

经过上述四个步骤,事务型事实表就基本设计完成了。第一步选择业务过程可以确定有哪些事务型事实表,第二步可以确定每张事务型事实表的每行数据是什么,第三步可以确定每张事务型事实表的维度外键,第四步可以确定每张事务型事实表的度量值字段。

### 3.1.3 不足

事务型事实表可以保存所有业务过程的最细粒度的操作事件,故理论上其可以支撑与各业务过程相关的各种统计粒度的需求。但对于某些特定类型的需求,其逻辑可能会比较复杂,或者效率会比较低下。例如:



### 1) 存量型指标

例如商品库存，账户余额等。此处以电商中的虚拟货币为例，虚拟货币业务包含的业务过程主要包括获取货币和使用货币，两个业务过程各自对应一张事务型事实表，一张存储所有的获取货币的原子操作事件，另一张存储所有使用货币的原子操作事件。

假定现有一个需求，要求统计截至当日的各用户虚拟货币余额。由于获取货币和使用货币均会影响到余额，故需要对两张事务型事实表进行聚合，且需要区分两者对余额的影响（加或减），另外需要对两张表的全表数据聚合才能得到统计结果。

可以看到，不论是从逻辑上还是效率上考虑，这都不是一个好的方案。

### 2) 多事务关联统计

例如，现需要统计最近 30 天，用户下单到支付的时间间隔的平均值。统计思路应该是找到下单事务事实表和支付事务事实表，过滤出最近 30 天的记录，然后按照订单 id 对两张事实表进行关联，之后用支付时间减去下单时间，然后再求平均值。

逻辑上虽然并不复杂，但是其效率较低，因为下单事务事实表和支付事务事实表均为大表，大表 join 大表的操作应尽量避免。

可以看到，在上述两种场景下事务型事实表的表现并不理想。下面要介绍的另外两种类型的事实表就是为了弥补事务型事实表的不足的。

## 3.2 周期型快照事实表

### 3.2.1 概述

周期快照事实表以具有规律性的、可预见的时间间隔来记录事实，主要用于分析一些存量型（例如商品库存，账户余额）或者状态型（空气温度，行驶速度）指标。

对于商品库存、账户余额这些存量型指标，业务系统中通常会计算并保存最新结果，所以定期同步一份全量数据到数据仓库，构建周期型快照事实表，就能轻松应对此类统计需求，而无需再对事务型事实表中大量的历史记录进行聚合了。

对于空气温度、行驶速度这些状态型指标，由于它们的值往往是连续的，我们无法捕获其变动的原子事务操作，所以无法使用事务型事实表统计此类需求。而只能定期对其进行采样，构建周期型快照事实表。

### 3.2.2 设计流程

#### 1) 确定粒度

周期型快照事实表的粒度可由采样周期和维度描述,故确定采样周期和维度后即可确定粒度。

采样周期通常选择每日。

维度可根据统计指标决定,例如指标为统计每个仓库中每种商品的库存,则可确定维度为仓库和商品。

确定完采样周期和维度后,即可确定该表粒度为每日-仓库-商品。

## 2) 确认事实

事实也可根据统计指标决定,例如指标为统计每个仓库中每种商品的库存,则事实为商品库存。

### 3.2.3 事实类型

此处的事实类型是指度量值的类型,而非事实表的类型。事实(度量值)共分为三类,分别是可加事实,半可加事实和不可加事实。

#### 1) 可加事实

可加事实是指可以按照与事实表相关的所有维度进行累加,例如事务型事实表中的事实。

#### 2) 半可加事实

半可加事实是指只能按照与事实表相关的一部分维度进行累加,例如周期型快照事实表中的事实。以上述各仓库中各商品的库存每天快照事实表为例,这张表中的库存事实可以按照仓库或者商品维度进行累加,但是不能按照时间维度进行累加,因为将每天的库存累加起来是没有任何意义的。

#### 3) 不可加事实

不可加事实是指完全不具备可加性,例如比率型事实。不可加事实通常需要转化为可加事实,例如比率可转化为分子和分母。

## 3.3 累积型快照事实表

### 3.3.1 概述

累积快照事实表是基于一个业务流程中的多个关键业务过程联合处理而构建的事实表,如交易流程中的下单、支付、发货、确认收货业务过程。

累积型快照事实表通常具有多个日期字段,每个日期对应业务流程中的一个关键业务过程(里程碑)。

| 订单 id | 用户 id | 下单日期       | 支付日期       | 发货日期       | 确认收货日期     | 订单金额 | 支付金额 |
|-------|-------|------------|------------|------------|------------|------|------|
| 1001  | 1234  | 2020-06-14 | 2020-06-15 | 2020-06-16 | 2020-06-17 | 1000 | 1000 |

累积型快照事实表主要用于分析业务过程（里程碑）之间的时间间隔等需求。例如前文提到的用户下单到支付的平均时间间隔，使用累积型快照事实表进行统计，就能避免两个事务事实表的关联操作，从而变得十分简单高效。

### 3.3.2 设计流程

累积型快照事实表的设计流程同事务型事实表类似，也可采用以下四个步骤，下面重点描述与事务型事实表的不同之处。

选择业务过程→声明粒度→确认维度→确认事实。

#### 1) 选择业务过程

选择一个业务流程中需要关联分析的多个关键业务过程，多个业务过程对应一张累积型快照事实表。

#### 2) 声明粒度

精确定义每行数据表示的是什么，尽量选择最小粒度。

#### 3) 确认维度

选择与各业务过程相关的维度，需要注意的是，每各业务过程均需要一个日期维度。

#### 4) 确认事实

选择各业务过程的度量值。

## 第 4 章 维度建模理论之维度表

### 4.1 维度表概述

维度表是维度建模的基础和灵魂。前文提到，事实表紧紧围绕业务过程进行设计，而维度表则围绕业务过程所处的环境进行设计。维度表主要包含一个主键和各种维度字段，维度字段称为维度属性。

### 4.2 表设计步骤

#### 1) 确定维度（表）

在设计事实表时，已经确定了与每个事实表相关的维度，理论上每个相关维度均需对应一张维度表。需要注意到，可能存在多个事实表与同一个维度都相关的情况，这种情况需保证维度的唯一性，即只创建一张维度表。另外，如果某些维度表的维度属性很少，例如只有

一个名称，则可不创建该维度表，而把该表的维度属性直接增加到与之相关的事实表中，这个操作称为维度退化。

## 2) 确定主维表和相关维表

此处的主维表和相关维表均指业务系统中与某维度相关的表。例如业务系统中与商品相关的表有 sku\_info, spu\_info, base\_trademark, base\_category3, base\_category2, base\_category1 等，其中 sku\_info 就称为商品维度的主维表，其余表称为商品维度的相关维表。维度表的粒度通常与主维表相同。

## 3) 确定维度属性

确定维度属性即确定维度表字段。维度属性主要来自于业务系统中与该维度对应的主维表和相关维表。维度属性可直接从主维表或相关维表中选择，也可通过进一步加工得到。

确定维度属性时，需要遵循以下要求：

### (1) 尽可能生成丰富的维度属性

维度属性是后续做分析统计时的查询约束条件、分组字段的基本来源，是数据易用性的关键。维度属性的丰富程度直接影响到数据模型能够支持的指标的丰富程度。

### (2) 尽量不使用编码，而使用明确的文字说明，一般可以编码和文字共存。

### (3) 尽量沉淀出通用的维度属性

有些维度属性的获取需要进行比较复杂的逻辑处理，例如需要通过多个字段拼接得到。为避免后续每次使用时的重复处理，可将这些维度属性沉淀到维度表中。

## 4.3 维度设计要点

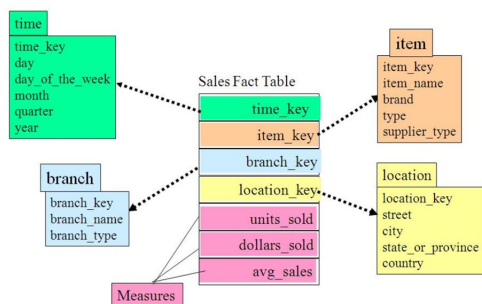
### 1) 规范化与反规范化

规范化是指使用一系列范式设计数据库的过程，其目的是减少数据冗余，增强数据的一致性。通常情况下，规范化之后，一张表的字段会拆分到多张表。

反规范化是指将多张表的数据冗余到一张表，其目的是减少 join 操作，提高查询性能。

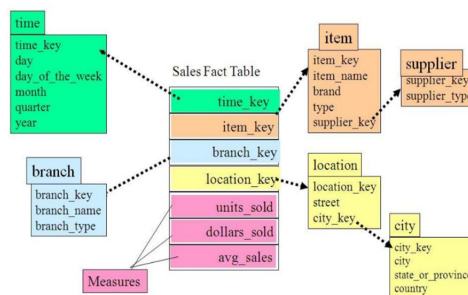
在设计维度表时，如果对其进行规范化，得到的维度模型称为雪花模型，如果对其进行反规范化，得到的模型称为星型模型。

### 1、星型模型



雪花模型与星型模型的区别主要在于维度表是否进行规范化。

### 2、雪花模型



雪花模型，比较靠近3NF，但是无法完全遵守，因为遵循3NF的性能成本太高。

让天下没有难学的技术

数据仓库系统的主要目的是用于数据分析和统计，所以是否方便用户进行统计分析决定了模型的优劣。采用雪花模型，用户在统计分析的过程中需要大量的关联操作，使用复杂度高，同时查询性能很差，而采用星型模型，则方便、易用且性能好。所以出于易用性和性能的考虑，离线维度表一般是很不规范化的-星型模型。

## 4.4 实时数仓维度设计

在电商离线数仓中，普通维度表是通过主维表和相关维表做关联查询生成的。与之对应的业务数表数据是通过每日一次全量同步导入到 HDFS 的，只须每日做一次全量数据的关联查询即可。而实时数仓中，系统上线后我们采集的是所有表的变化数据，这样就会导致一旦主维表或相关维表中的某张表数据发生了变化，就需要和其它表的历史数据做关联。

此时我们会面临一个问题：如何获取历史数据？

对于这个问题，一种方案是在某张与维度表相关的业务表数据发生变化时，执行一次 maxwell-bootstrap 命令，将相关业务数据库维度表的数据导入 Kafka。但是这样做又会面临三个问题：

- (1) Kafka 中存储冗余数据；
- (2) maxwell-bootstrap 命令交给谁去执行？必然要引入调度组件或功能；
- (3) 实时数仓中的数据是以流的形式存在的，如果不同流中数据进入程序的机器时间差异过大就会出现 join 不上的情况。如何保证导入的历史数据和变化数据可以关联上？势必要尽可能及时地执行历史数据导入命令且在 Flink 程序中设置足够的延迟。而前者难以保证，后者又会影响整个实时数仓的时效性。综上，这种方案并不合理。

另一种方案是维度表发生变化时去 HBase 中读取关联后的维表，筛选受影响的数据，与变化或新增的维度信息（通常生产环境的业务数据库是不会删除的）做关联，再把关联后的数据写入 HBase。但是考虑这样一种情况，以商品表为例，主维表为 sku\_info，相关维表有 spu\_info，base\_trademark，base\_category1，base\_category2，base\_category3 等，假设 base\_category1 表的某条数据发生了变化，HBase 表受影响的数据非常多（base\_category1 表的粒度较粗），我们需要把这些数据取出来，修改，然后再写回 HBase。显然，这种方案也不合理。

第三种方案是将分表导入 HBase，关联操作在 HBase 中完成。首先 HBase 的 join 性能很差，其次，关联操作不在流处理的 DAG 图中，需要单独调度，增加了系统复杂度。最后，当粒度较粗的维表数据发生变化时，受影响的数据很多。综上，这种方案也不合理。

基于上述分析，对业务表做 join 形成维度表的方式并不适用于实时数仓。

因此，在实时数仓中，我们不再对业务数据库中的维度表进行合并，仅对一些不需要的字段进行过滤，然后将维度数据写入 HBase 的维度表中，业务数据库的维度表和 HBase 的维度表是一一对应的。

写入维度数据使用 HBase 的 put 方法，实现幂等写入。当维度数据发生变化时，程序会用变化后的新数据覆盖旧数据。从而保证 HBase 中保存的是一份全量最新的维度数据。

这样做会产生一个问题：实时数仓没有保存历史维度数据，与数仓特征（保存历史数据）相悖。那么，维度表可以按照上述思路设计吗？

首先，我们要明确：数仓之所以要保存历史数据，是为了运用历史数据做一些相关指标的计算，而实时数仓本就是对最新的业务数据做分析计算，不涉及历史数据，因此无须保存。

此外，生产环境中实时数仓的上线通常不会早于离线数仓，如果有涉及到历史数据的指标，在离线数仓中计算即可。因此，实时数仓中只需要保留一份最新的维度数据，上述方案是切实可行的。

特别地，对于字典表，数据一般不会变化，而且我们至多只会用到 dic\_code，dic\_name 和 parent\_code 三个字段，建立单独的维度表意义不大，选择将维度字段退化到事实表中。

## 第 5 章 数据仓库设计

### 5.1 数据仓库分层规划



优秀可靠的数仓体系，需要良好的数据分层结构。合理的分层，能够使数据体系更加清晰，使复杂问题得以简化。以下是该项目的分层规划。



## 数据仓库分层规划

### 数据应用层 (ADS)

存放各项统计指标结果。

### 汇总数据层 (DWS)

基于上层的指标需求，以分析的主题对象作为建模驱动，构建公共统计粒度的汇总表。

### 明细数据层 (DWD)

基于维度建模理论进行构建，存放维度模型中的事实表，保存各业务过程最小粒度的操作记录。

### 原始数据层 (ODS)

存放未经处理的原始数据，结构上与源系统保持一致，是数据仓库的数据准备区。

### 公共维度层 (DIM)

基于维度建模理论进行构建，存放维度模型中的维度表，保存一致性维度信息。

| 分层简称 | 全称                       |
|------|--------------------------|
| ODS  | Operation Data Store     |
| DWD  | Data Warehouse Detail    |
| DIM  | Dimension                |
| DWS  | Data Warehouse Summary   |
| ADS  | Application Data Service |

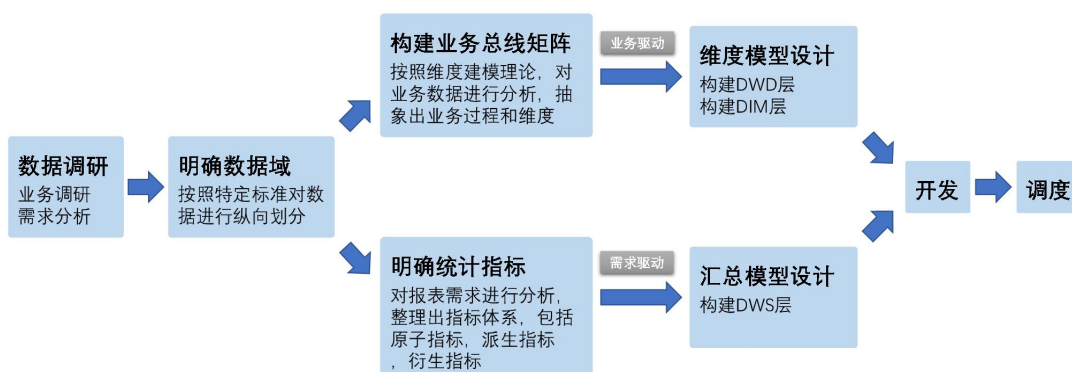
让天下没有难学的技术

## 5.2 数据仓库构建流程

以下是构建数据仓库的完整流程。



## 数据仓库构建流程



让天下没有难学的技术

### 5.2.1 数据调研

数据调研重点要做两项工作，分别是业务调研和需求分析。这两项工作做的是否充分，直接影响着数据仓库的质量。

#### 1) 业务调研

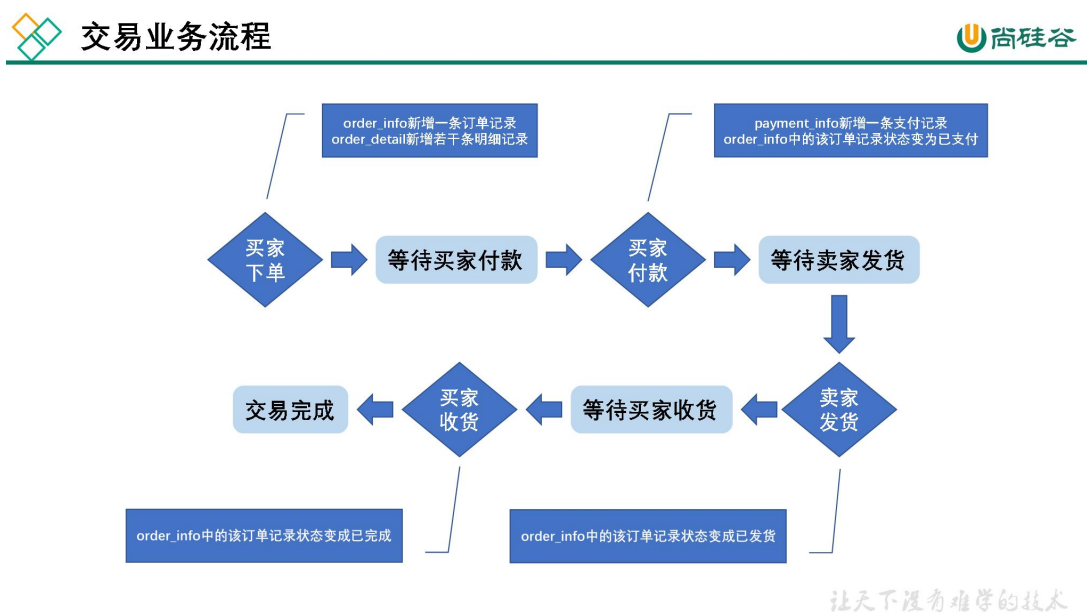
更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

业务调研的主要目标是**熟悉业务流程、熟悉业务数据**。

**熟悉业务流程**要求做到，明确每个业务的具体流程，需要将该业务所包含的每个**业务过程**一一列举出来。

**熟悉业务数据**要求做到，将数据（包括埋点日志和业务数据表）与业务过程对应起来，明确每个业务过程会对哪些表的数据产生影响，以及产生什么影响。产生的影响，需要具体到，是新增一条数据，还是修改一条数据，并且需要明确新增的内容或者是修改的逻辑。

下面业务电商中的交易为例进行演示，交易业务涉及到的业务过程有买家下单、买家支付、卖家发货，买家收货，具体流程如下图。



## 2) 需求分析

典型的需求指标如，最近一天各省份手机品类订单总额。

分析需求时，需要明确需求所需的**业务过程及维度**，例如该需求所需的业务过程就是买家下单，所需的维度有日期，省份，商品品类。

## 3) 总结

做完业务分析和需求分析之后，要保证每个需求都能找到与之对应的业务过程及维度。若现有数据无法满足需求，则需要和业务方进行沟通，例如某个页面需要新增某个行为的埋点。

### 5.2.2 明确数据域

数据仓库模型设计除横向的分层外，通常也需要根据业务情况进行纵向划分数据域。

划分数据域的意义是**便于数据的管理和应用**。

通常可以根据业务过程或者部门进行划分，本项目根据业务过程进行划分，需要注意的是一个业务过程只能属于一个数据域。

下面是本数仓项目所需的所有业务过程及数据域划分详情。

| 数据域 | 业务过程                      |
|-----|---------------------------|
| 交易域 | 加购、下单、取消订单、支付成功、退单、退款成功   |
| 流量域 | 页面浏览、启动应用、动作、曝光、错误        |
| 用户域 | 注册、登录                     |
| 互动域 | 收藏、评价                     |
| 工具域 | 优惠券领取、优惠券使用（下单）、优惠券使用（支付） |

## 5.2.3 构建业务总线矩阵

业务总线矩阵中包含维度模型所需的所有事实（业务过程）以及维度，以及各业务过程与各维度的关系。矩阵的行是一个个业务过程，矩阵的列是一个个的维度，行列的交点表示业务过程与维度的关系。


业务总线矩阵


| 数据域 | 业务过程      | 维度 |    |    |    |      |      |        |   |
|-----|-----------|----|----|----|----|------|------|--------|---|
|     |           | 时间 | 用户 | 商品 | 地区 | 支付方式 | 退单类型 | 退单原因类型 |   |
| 交易域 | 加购物车      | √  | √  | √  |    |      |      |        |   |
|     | 下单        | √  | √  | √  | √  |      |      |        |   |
|     | 取消订单      | √  | √  | √  | √  |      |      |        |   |
|     | 支付成功      | √  | √  | √  | √  | √    |      |        |   |
|     | 退单        | √  | √  | √  | √  |      | √    |        | √ |
|     | 退款成功      | √  | √  | √  | √  | √    |      |        |   |
| 用户域 | 注册        | √  | √  |    |    |      |      |        |   |
| 工具域 | 领取优惠券     | √  | √  |    |    |      |      |        |   |
|     | 使用优惠券(下单) | √  | √  |    |    |      |      |        |   |
|     | 使用优惠券(支付) | √  | √  |    |    |      |      |        |   |
| 互动域 | 收藏商品      | √  | √  | √  |    |      |      |        |   |
|     | 评价        | √  | √  | √  |    |      |      |        |   |

业务过程与维度的关系

让天下没有难学的技术

一个业务过程对应维度模型中一张事务型事实表，一个维度则对应维度模型中的一张维度表。**所以构建业务总线矩阵的过程就是设计维度模型的过程。**但是需要注意的是，总线矩阵中通常只包含事务型事实表，另外两种类型的事实表需单独设计。

按照事务型事实表的设计流程，**选择业务过程→声明粒度→确认维度→确认事实**，得到的最终的业务总线矩阵见以下表格。



后续的 DWD 层以及 DIM 层的搭建需参考业务总线矩阵。

## 5.2.4 明确统计指标

明确统计指标具体的工作是，深入分析需求，构建指标体系。构建指标体系的主要意义就是指标定义标准化。所有指标的定义，都必须遵循同一套标准，这样能有效的避免指标定义存在歧义，指标定义重复等问题。

### 1) 指标体系相关概念

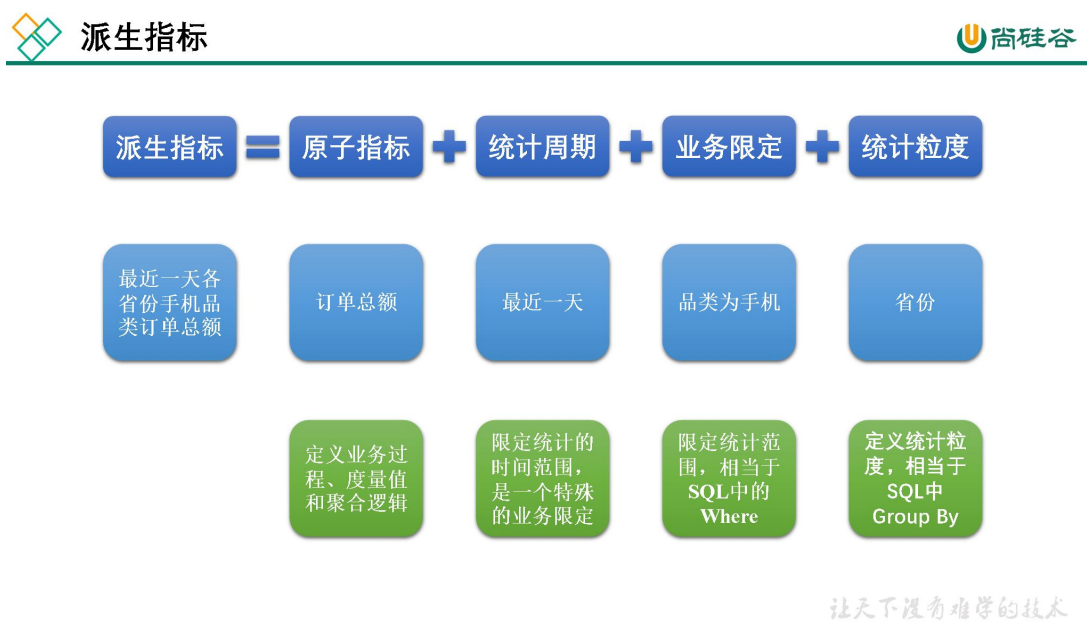
#### (1) 原子指标

原子指标基于某一**业务过程**的**度量值**，是业务定义中不可再拆解的指标，原子指标的核心功能就是对指标的**聚合逻辑**进行了定义。我们可以得出结论，原子指标包含三要素，分别是业务过程、度量值和聚合逻辑。

例如**订单总额**就是一个典型的原子指标，其中的业务过程为用户下单、度量值为订单金额，聚合逻辑为 sum() 求和。需要注意的是原子指标只是用来辅助定义指标一个概念，通常不会对应有实际统计需求与之对应。

#### (2) 派生指标

派生指标基于原子指标，其与原子指标的关系如下图所示。



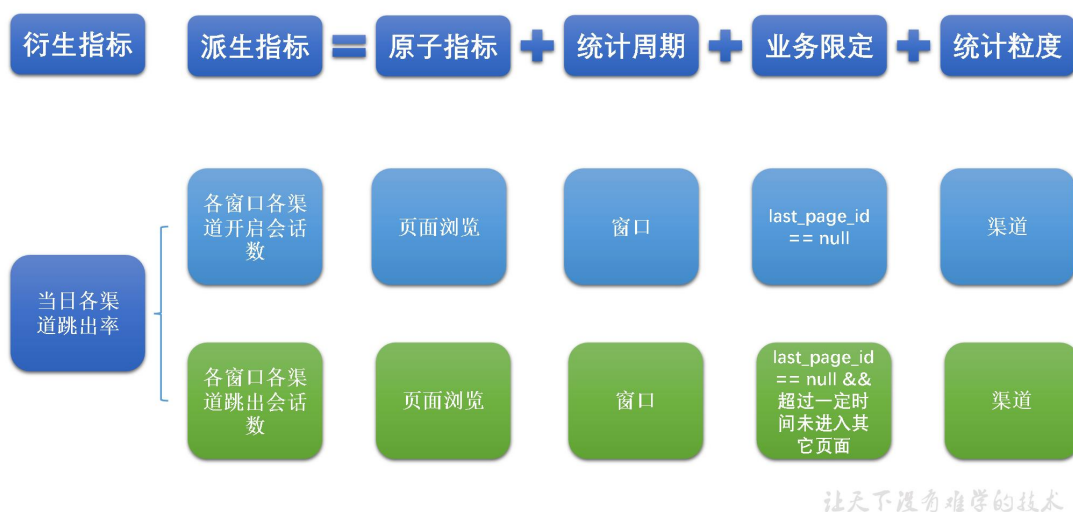
与原子指标不同，派生指标通常会对应实际的统计需求。请从图中的例子中，体会指标定义标准化的含义。

### (3) 衍生指标

衍生指标是在一个或多个派生指标的基础上，通过各种逻辑运算复合而成的。例如比率、比例等类型的指标。衍生指标也会对应实际的统计需求。



#### 衍生指标



## 2) 指标体系对于数仓建模的意义

通过上述两个具体的案例可以看出，绝大多数的统计需求，都可以使用原子指标、派生指标以及衍生指标这套标准去定义。同时能够发现这些统计需求都直接的或间接的对应一个或者是多个派生指标。

当统计需求足够多时，必然会出现部分统计需求对应的派生指标相同的情况。这种情况下，我们就可以考虑将这些公共的派生指标保存下来，这样做的主要目的就是减少重复计算，提高数据的复用性。

这些公共的派生指标统一保存在数据仓库的 DWS 层。因此 DWS 层设计，就可以参考我们根据现有的统计需求整理出的派生指标。

从上述指标体系中抽取出来的所有派生指标见如下表格。



gmall-实时重构指标体系.xlsx

## 5.2.5 维度模型设计

维度模型的设计参照上述得到的业务总线矩阵即可。事实表存储在 DWD 层，维度表存储在 DIM 层。

### 5.2.6 汇总模型设计

汇总模型的设计参考上述整理出的指标体系（主要是派生指标）即可。汇总表与派生指标的对应关系是，**一张汇总表通常包含**业务过程相同、统计周期相同、统计粒度相同的**多个派生指标**。

## 第 6 章 IDEA 开发环境准备

### 6.1 项目架构与代码管理



尚硅谷大数据技术  
之代码管理.docx

### 6.2 项目架构设计

本项目准备一个父工程，父工程下创建四个 module，如下：

- (1) realtime-common：用于引入公共的第三方依赖，编写工具类、实体类等。
- (2) realtime-dim：用于编写 DIM 层业务代码。
- (3) realtime-dwd：用于编写 DWD 层业务代码。
- (4) realtime-dws：用于编写 DWS 层业务代码。

其中，后三个 module 统称为业务模块，业务模块都要将 realtime-common 模块作为依赖引入。

#### 6.2.1 父工程

##### 1) 知识储备

dependencyManagement 是 Maven 中的一个元素，它用于集中管理项目中所有依赖项的版本，而不是真正地声明这些依赖。当项目中有多个模块时，可以在父 POM 中集中管理所有模块的依赖版本。这确保了所有模块都使用相同的依赖版本，从而避免了版本冲突。当子模块添加已被管理的依赖时不需要声明版本，因为它已经在父工程定义了。

##### 2) POM 文件

(1) 创建父工程，将打包时不必包含在内的依赖添加到父工程的 pom 文件中，这部分依赖的 scope 均为 provided，主要有三类：



- Flink 相关：集群 Flink 的 lib 目录下存在的依赖，包括 Flink 运行环境及日志框架 Log4j 有关的依赖。
- Hadoop 相关：hadoop-client 和 hadoop-auth。Flink 的检查点保存在 HDFS，hadoop-client 用于和 HDFS 交互。本项目还要读写 HBase，操作 HBase 需要引入 hadoop-auth。
- Lombok 依赖：Lombok 是一个 Java 库，它提供了一系列注解，旨在通过自动化某些常见的样板代码生成来简化 Java 代码。通过使用 Lombok，开发者可以减少大量重复和冗长的代码，从而使得代码更加简洁、易读和易维护。

前两类都是集群已有的依赖，因而打包时不必包含，而 Lombok 只在编译时生效，编译完成后代码的运行和它再无关系，打包时也不需要包含在内。

(2) 在父工程的 POM 文件中通过 dependencyManagement 标签声明所有依赖的版本。

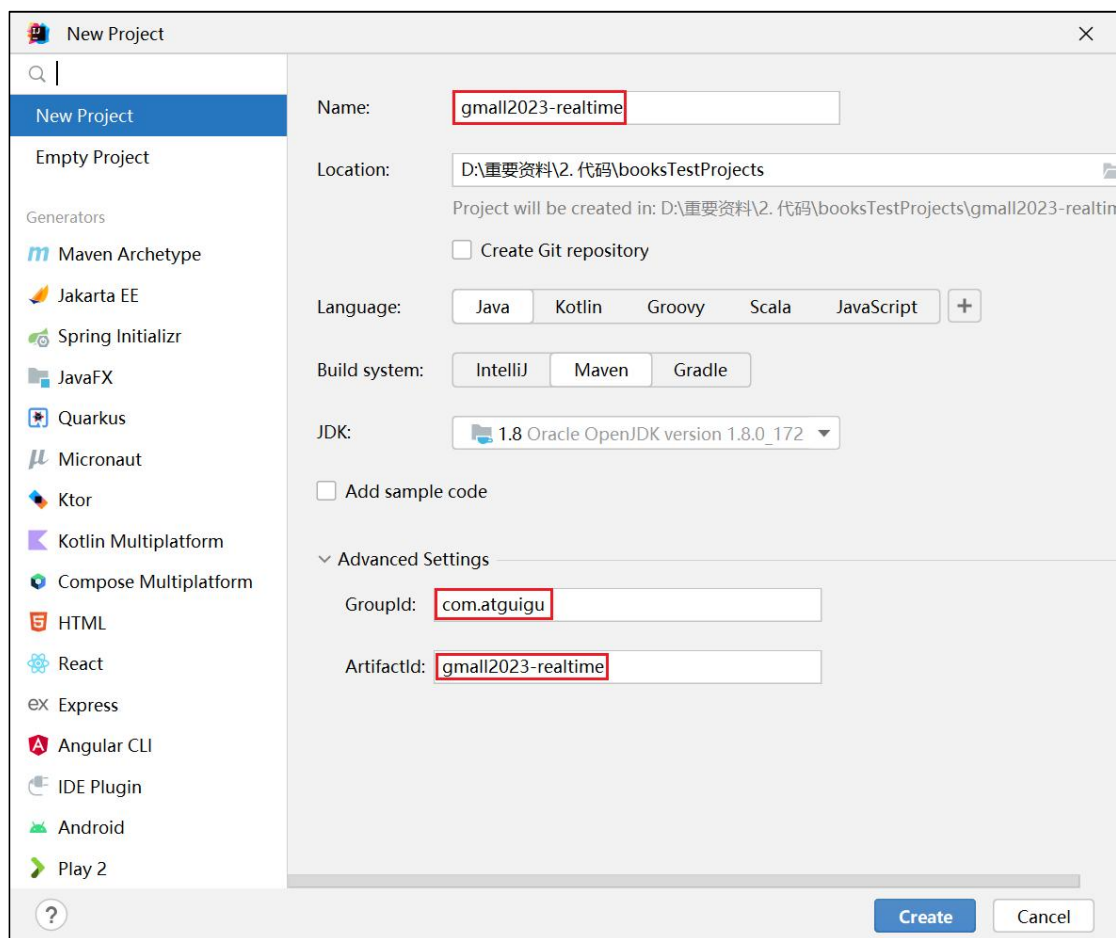
### 6.2.2 realtime-common

本项目需要引入大量依赖，除去父类中引入的部分（集群已有的依赖），还有很多集群没有的依赖（下文称为第三方依赖），打包时都要包含在 jar 包里，这样就会导致业务模块打包时 jar 包很大，打包时间很长，部署效率不高。为了改善这一情况，我们构建公共模块 realtime-common，将第三方依赖都在公共模块的 POM 文件中引入，业务模块只需要引入 realtime-common 依赖即可。但要注意，如果 realtime-common 的 scope 为默认的 compile，业务模块打包时依然会包含所有的第三方依赖，不能解决问题，所以我们将 scope 更改为 provided，并将打好的 realtime-common 包放到集群 Flink 的 lib 目录下。

为了精简子模块的代码，我们将基类、工具类、实体类、常量类、通用的自定义函数全部放到 realtime-common 中。

## 6.3 创建 gmall2023-realtime

### 6.3.1 创建项目



### 6.3.2 修改 pom 文件

在 pom.xml 中添加以下内容

```
<properties>
  <java.version>1.8</java.version>
  <maven.compiler.source>${java.version}</maven.compiler.source>
  <maven.compiler.target>${java.version}</maven.compiler.target>

  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
  <dependency>
```

```
<groupId>org.apache.flink</groupId>
<artifactId>flink-streaming-java</artifactId>
<version>${flink.version}</version>
<scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-clients</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<dependency><!--在 idea 运行的时候,可以打开 web 页面-->
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-runtime-web</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-json</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-csv</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<!--如果保存检查点到 hdfs 上, 需要引入此依赖-->
<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-client</artifactId>
  <version>${hadoop.version}</version>
  <scope>provided</scope>
</dependency>

<dependency>
```

```
<groupId>org.apache.hadoop</groupId>
<artifactId>hadoop-auth</artifactId>
<scope>provided</scope>
<exclusions>
  <exclusion>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-reload4j</artifactId>
  </exclusion>
</exclusions>
</dependency>
```

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.20</version>
  <scope>provided</scope>
</dependency>
```

<!--Flink 默认使用的是 slf4j 记录日志，相当于一个日志的接口，我们这里使用 log4j 作为具体的日志实现-->

```
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.25</version>
  <scope>provided</scope>
</dependency>
```

```
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-to-slf4j</artifactId>
  <version>2.14.0</version>
  <scope>provided</scope>
</dependency>
```

```
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-api-java-bridge</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>
```

```
<dependency>
  <groupId>org.apache.flink</groupId>
```

```
<artifactId>flink-table-planner-loader</artifactId>
<version>${flink.version}</version>
<scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-table-runtime</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-files</artifactId>
  <version>${flink.version}</version>
  <scope>provided</scope>
</dependency>

</dependencies>

<dependencyManagement>

  <dependencies>

    <dependency>
      <groupId>com.alibaba</groupId>
      <artifactId>fastjson</artifactId>
      <version>${fastjson.version}</version>
    </dependency>

    <dependency>
      <groupId>org.apache.flink</groupId>
      <artifactId>flink-connector-kafka</artifactId>
      <version>${flink.version}</version>
    </dependency>

    <dependency>
      <groupId>com.ververica</groupId>
      <artifactId>flink-connector-mysql-cdc</artifactId>
      <version>${flink-cdc.version}</version>
    </dependency>

  </dependencies>

</dependencyManagement>
```

```
<!-- hbase 依赖-->
<dependency>
  <groupId>org.apache.hbase</groupId>
  <artifactId>hbase-client</artifactId>
  <version>${hbase.version}</version>
</dependency>

<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-auth</artifactId>
  <version>${hadoop.version}</version>
  <scope>provided</scope>
  <exclusions>
    <exclusion>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-reload4j</artifactId>
    </exclusion>
  </exclusions>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-hbase-2.2</artifactId>
  <version>${flink.version}</version>
</dependency>

<!--
https://mvnrepository.com/artifact/org.apache.doris/flink-doris-connector-1.17 -->
<dependency>
  <groupId>org.apache.doris</groupId>
  <artifactId>flink-doris-connector-1.17</artifactId>
  <version>1.5.2</version>
</dependency>

<dependency>
  <groupId>commons-beanutils</groupId>
  <artifactId>commons-beanutils</artifactId>
  <version>1.9.4</version>
</dependency>

<dependency>
  <groupId>com.janeluo</groupId>
  <artifactId>ikanalyzer</artifactId>
```



```
<version>2012_u6</version>
</dependency>

<dependency>
  <groupId>redis.clients</groupId>
  <artifactId>jedis</artifactId>
  <version>3.3.0</version>
</dependency>

<dependency>
  <groupId>io.lettuce</groupId>
  <artifactId>lettuce-core</artifactId>
  <version>6.2.4.RELEASE</version>
</dependency>

</dependencies>

</dependencyManagement>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-shade-plugin</artifactId>
      <version>3.1.1</version>
      <executions>
        <execution>
          <phase>package</phase>
          <goals>
            <goal>shade</goal>
          </goals>
          <configuration>
            <artifactSet>
              <excludes>
                <exclude>com.google.code.findbugs:jsr305</exclude>
                <exclude>org.slf4j:*</exclude>
                <exclude>log4j:*</exclude>
                <exclude>org.apache.hadoop:*</exclude>
              </excludes>
            </artifactSet>
            <filters>
              <filter>
                <!-- Do not copy the signatures in the
```

```

META-INF folder. Otherwise, this might cause SecurityExceptions when
using the JAR. -->
        <!-- 打包时不复制 META-INF 下的签名文件，避免
        报非法签名文件的 SecurityExceptions 异常-->
        <artifact>*:*</artifact>
        <excludes>
            <exclude>META-INF/*.SF</exclude>
            <exclude>META-INF/*.DSA</exclude>
            <exclude>META-INF/*.RSA</exclude>
        </excludes>
    </filter>
</filters>

    <transformers combine.children="append">
        <!-- The service transformer is needed to
        merge META-INF/services files -->
        <!-- connector 和 format 依赖的工厂类打包时会相互
        覆盖，需要使用 ServicesResourceTransformer 解决-->
        <transformer

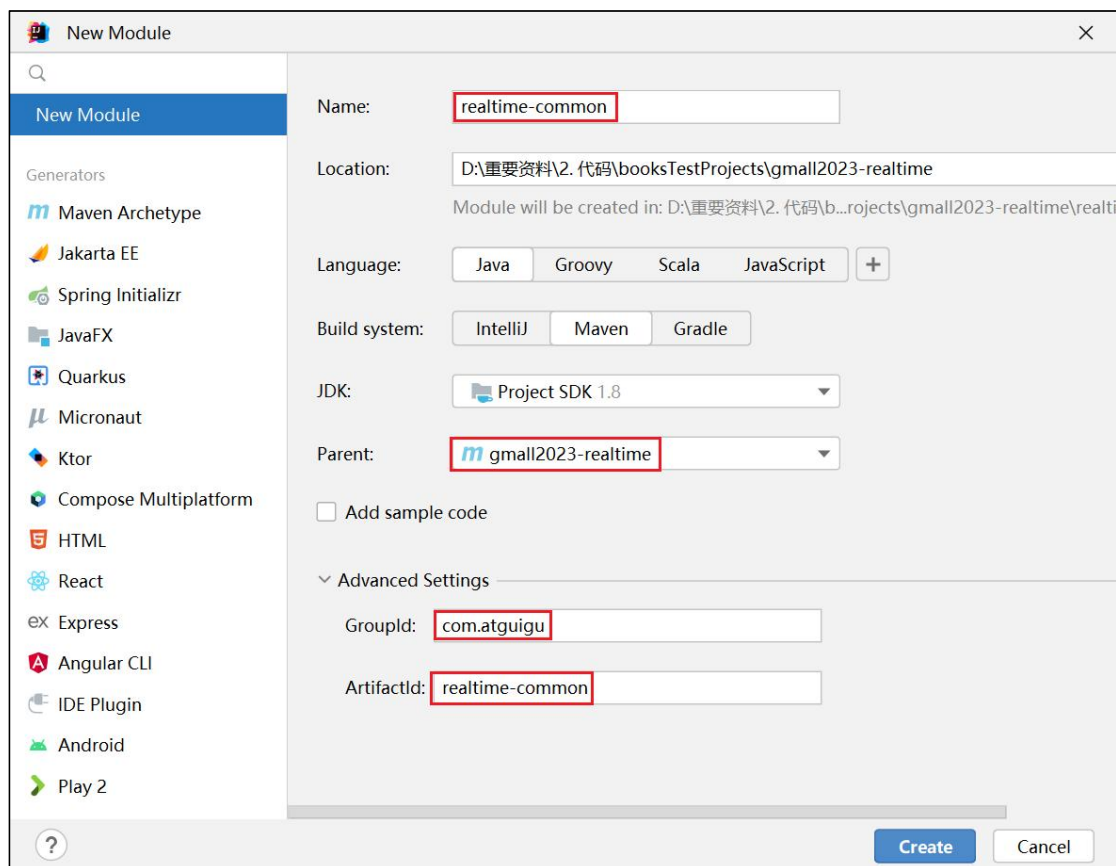
implementation="org.apache.maven.plugins.shade.resource.ServicesR
esourceTransformer"/>
    </transformers>
</configuration>
</execution>
</executions>
</plugin>
</plugins>
</build>

```

## 6.4 在 gmall2023-realtime 下创建子模块 realtime-common

### 6.4.1 创建 module





The image shows the 'New Module' dialog in IntelliJ IDEA. The 'Name' field is 'realtime-common'. The 'Location' is 'D:\重要资料\2. 代码\booksTestProjects\gmall2023-realtime'. The 'Language' is 'Java'. The 'Build system' is 'Maven'. The 'JDK' is 'Project SDK 1.8'. The 'Parent' is 'gmall2023-realtime'. The 'Advanced Settings' section shows 'GroupId' as 'com.atguigu' and 'ArtifactId' as 'realtime-common'. The 'Create' button is highlighted.

## 6.4.2 引入依赖

```
<dependencies>

<dependency>
  <groupId>com.alibaba</groupId>
  <artifactId>fastjson</artifactId>
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-kafka</artifactId>
</dependency>

<dependency>
  <groupId>com.ververica</groupId>
  <artifactId>flink-connector-mysql-cdc</artifactId>
</dependency>

<!-- hbase 依赖-->
<dependency>
  <groupId>org.apache.hbase</groupId>
  <artifactId>hbase-client</artifactId>
```

```
</dependency>

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-hbase-2.2</artifactId>
</dependency>

<!--
https://mvnrepository.com/artifact/org.apache.doris/flink-doris-connector-1.17 -->
<dependency>
  <groupId>org.apache.doris</groupId>
  <artifactId>flink-doris-connector-1.17</artifactId>
</dependency>

<dependency>
  <groupId>commons-beanutils</groupId>
  <artifactId>commons-beanutils</artifactId>
</dependency>

<dependency>
  <groupId>com.janeluo</groupId>
  <artifactId>ikanalyzer</artifactId>
</dependency>

<dependency>
  <groupId>redis.clients</groupId>
  <artifactId>jedis</artifactId>
</dependency>

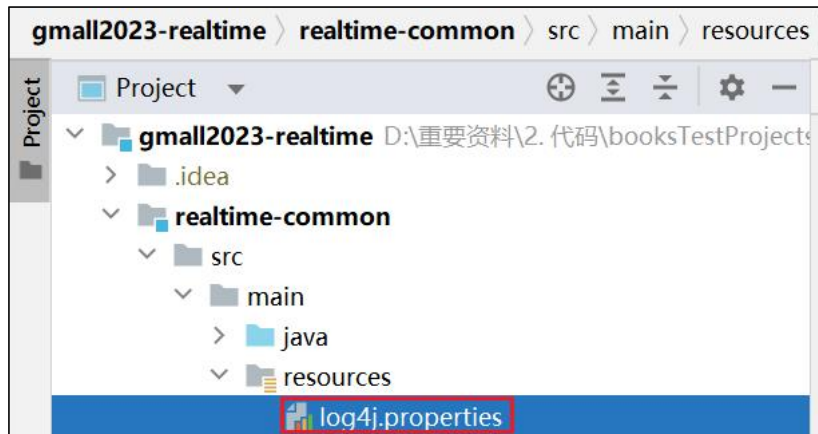
<dependency>
  <groupId>io.lettuce</groupId>
  <artifactId>lettuce-core</artifactId>
</dependency>
</dependencies>
```

### 6.4.3 在 resources 目录下创建 log4j.properties 文件

Flink 使用 log4j 作为默认的日志框架，启动应用程序，控制台会打印日志，但有时默认的日志级别并不能满足我们的需求，此时就要在 module 的 resources 目录下创建 log4j.properties 文件，在其中对日志级别等做出调整。

本项目有数十个 module，如果每个 module 下都准备一份 log4j.properties 文件，将会非常繁琐。实际上，本项目的 module 只有两类：common 模块和业务模块，而所有的业务模块都依赖的 common 模块，只需要在 common 模块的 resources 目录下准备一份 log4j 配置文件即可对所有模块生效。

#### 1) 在 realtime-common 模块的 resources 目录下创建 log4j.properties



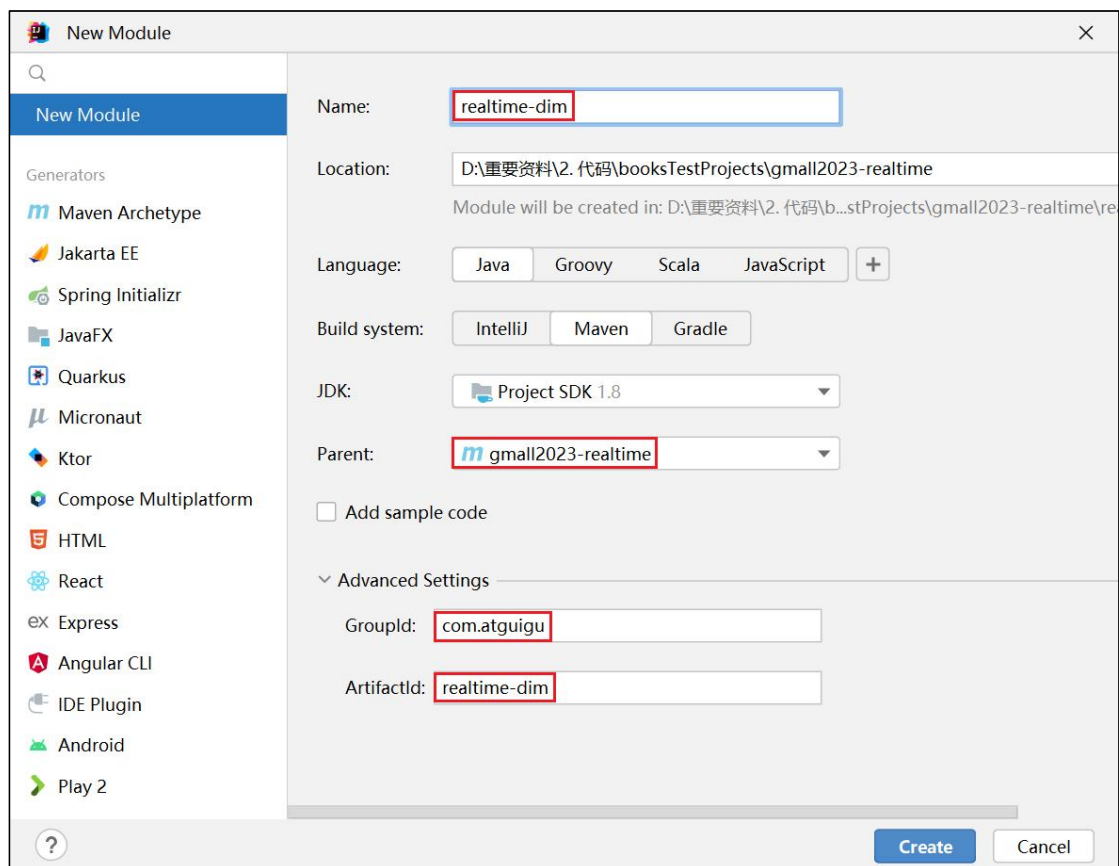
#### 2) 写入以下内容

```
log4j.appender.stdout=org.apache.log4j.ConsoleAppender
log4j.appender.stdout.target=System.out
log4j.appender.stdout.layout=org.apache.log4j.PatternLayout
log4j.appender.stdout.layout.ConversionPattern=%d{yyyy-MM-dd
HH:mm:ss} %10p (%c:%M) - %m%n
log4j.rootLogger=error,stdout
```

## 6.5 创建子模块 realtime-dim

如前所述，业务模块只需要依赖 realtime-common 即可，下文不再赘述。

### 6.5.1 创建 module



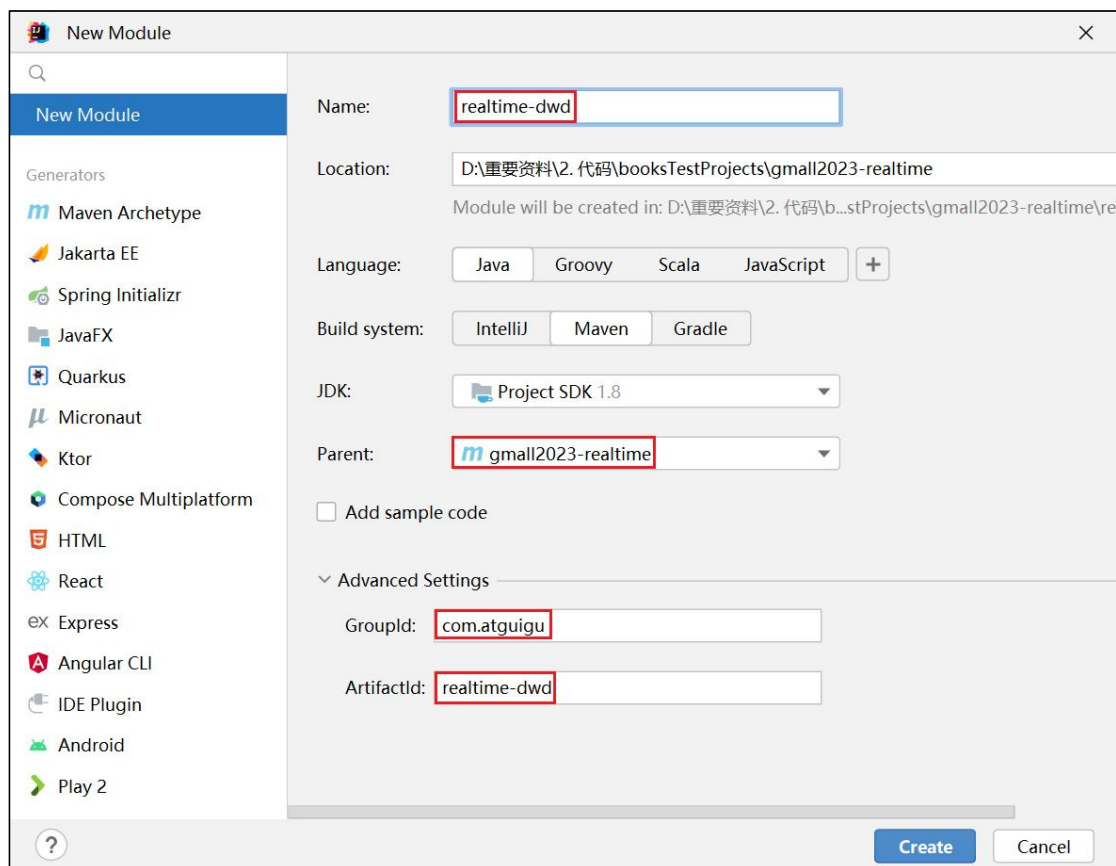
## 6.5.2 导入依赖

```
<dependencies>
  <dependency>
    <groupId>com.atguigu</groupId>
    <artifactId>realtime-common</artifactId>
    <version>1.0-SNAPSHOT</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

## 6.6 创建子模块 realtime-dwd

### 6.6.1 创建 module



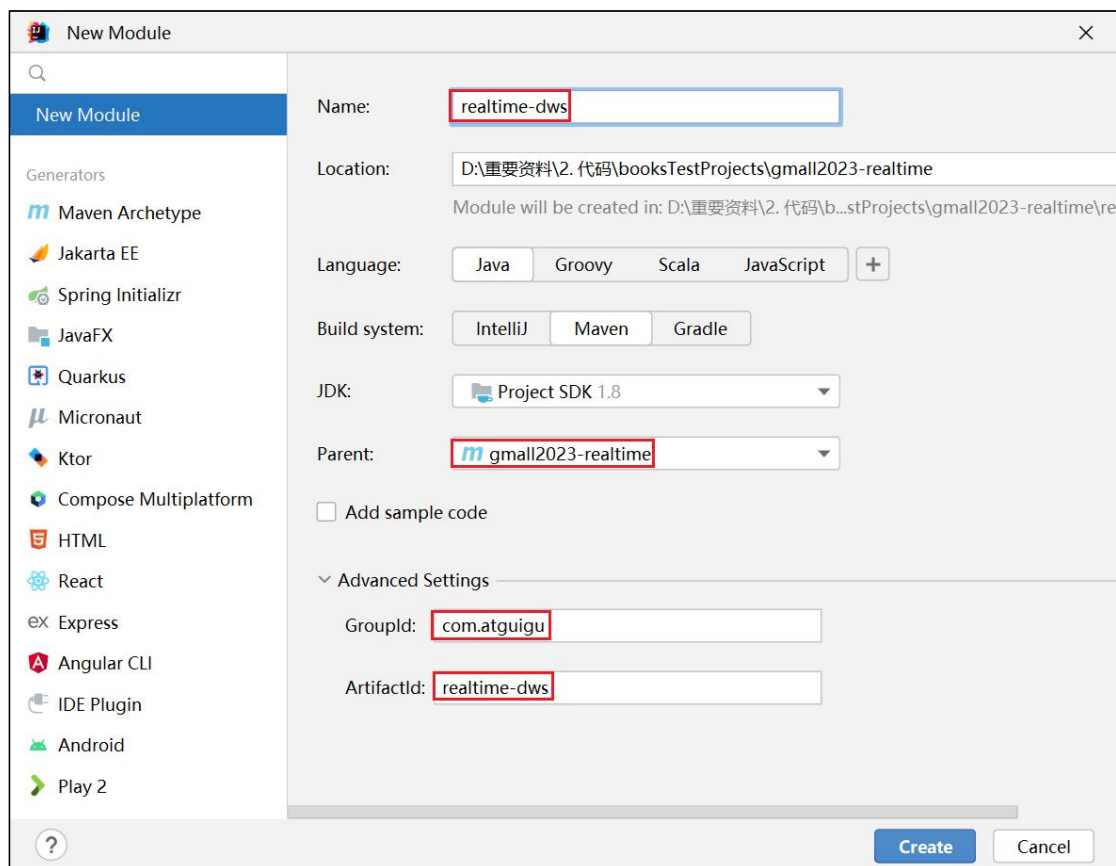


## 6.6.2 导入依赖

```
<dependencies>
  <dependency>
    <groupId>com.atguigu</groupId>
    <artifactId>realtime-common</artifactId>
    <version>1.0-SNAPSHOT</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

## 6.7 创建子模块 realtime-dws

### 6.7.1 创建 module

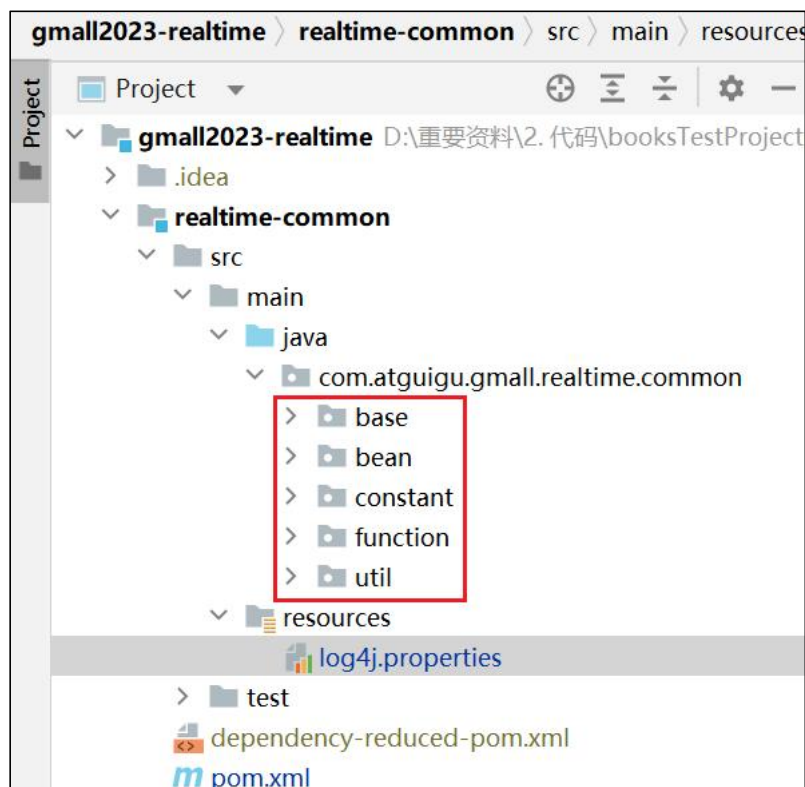


## 6.7.2 导入依赖

```
<dependencies>
  <dependency>
    <groupId>com.atguigu</groupId>
    <artifactId>realtime-common</artifactId>
    <version>1.0-SNAPSHOT</version>
    <scope>provided</scope>
  </dependency>
</dependencies>
```

## 6.8 package 设计

在 realtime-common 包下创建如下包结构



#### 1) base 包

包含所有业务代码的基类。

#### 2) bean 包

包含所有实体类。

#### 3) constant 包

包含所有常量类。

#### 4) function 包

包含所有函数类。

#### 5) util 包

包含所有工具类。

## 第 7 章 GitLab 环境准备

### 7.1 GitLab 集成

#### 7.1.1 GitLab 部署和 IDEA 集成

##### 1) 部署和集成

参见以下文档。



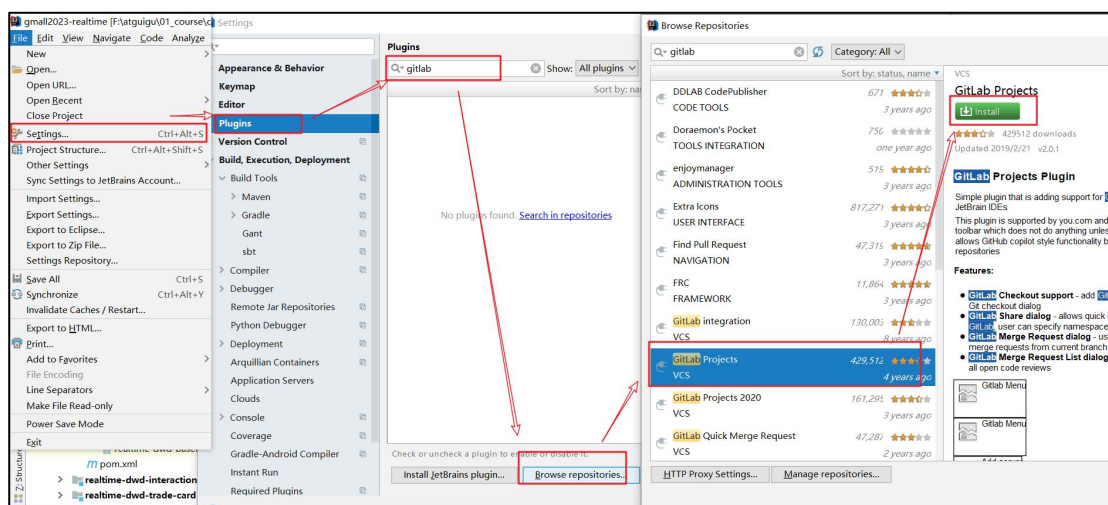
尚硅谷大数据技术  
之GitLab.docx

## 7.1.2 初始化项目

确保已完成 GitLab 部署及 IDEA 和 GitLab 的集成。

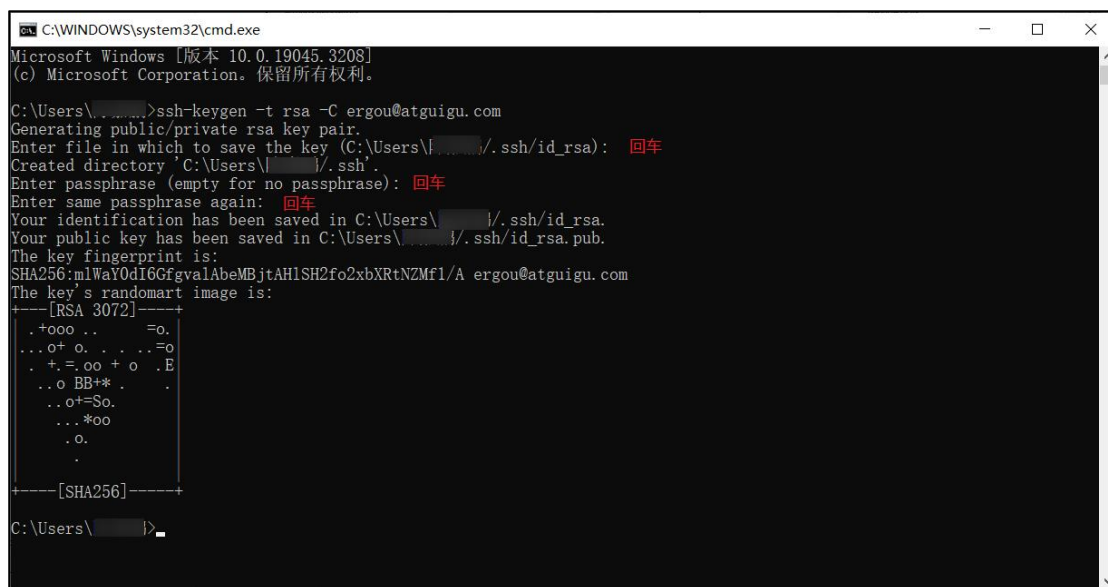
我们将实时数仓项目分享到 GitLab 上：

### 1) 安装 GitLab 插件

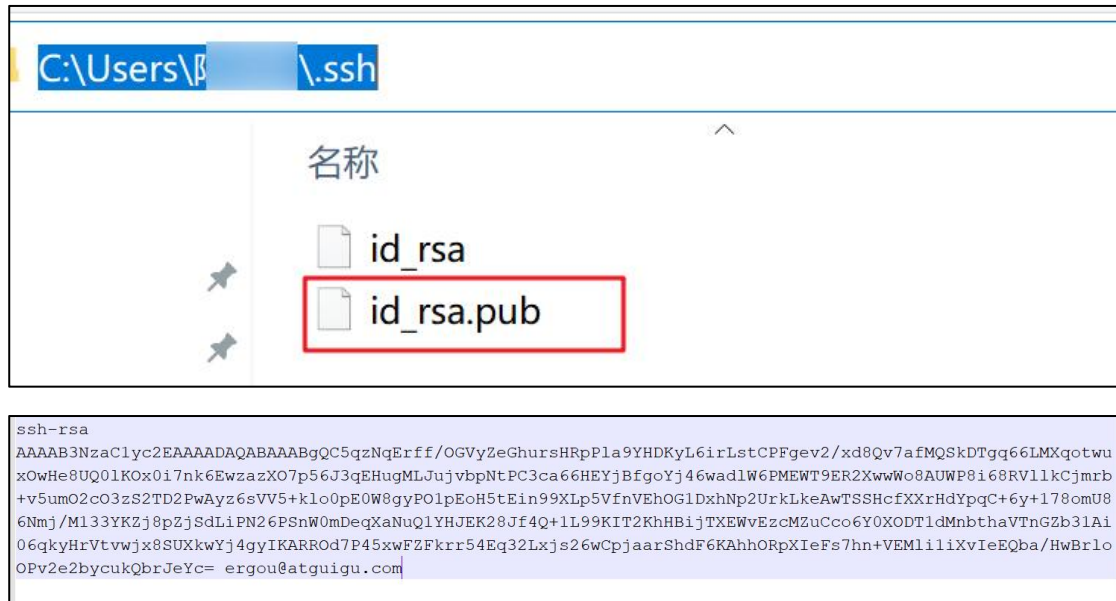


### 2) 配置 SSH 免密登录

```
ssh-keygen -t rsa -C ergou@atguigu.com
```



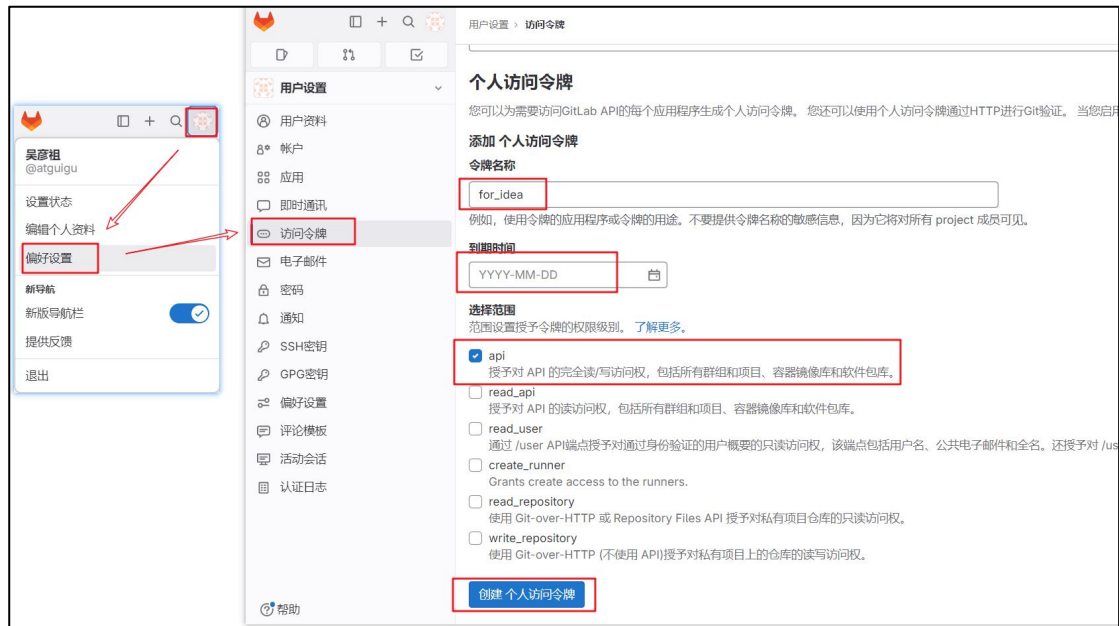
到用户目录下.ssh 查看 id\_rsa.pub 文件



添加到 GitLab 中：



### 3) 获取 GitLab 个人令牌

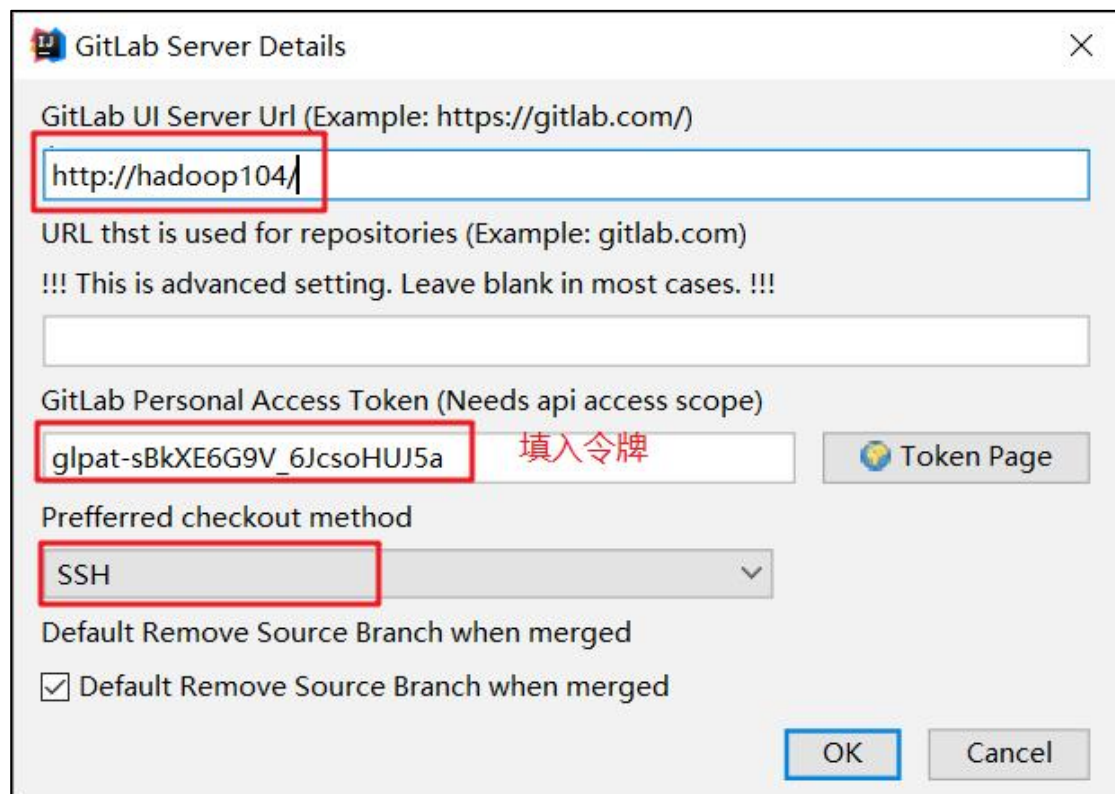
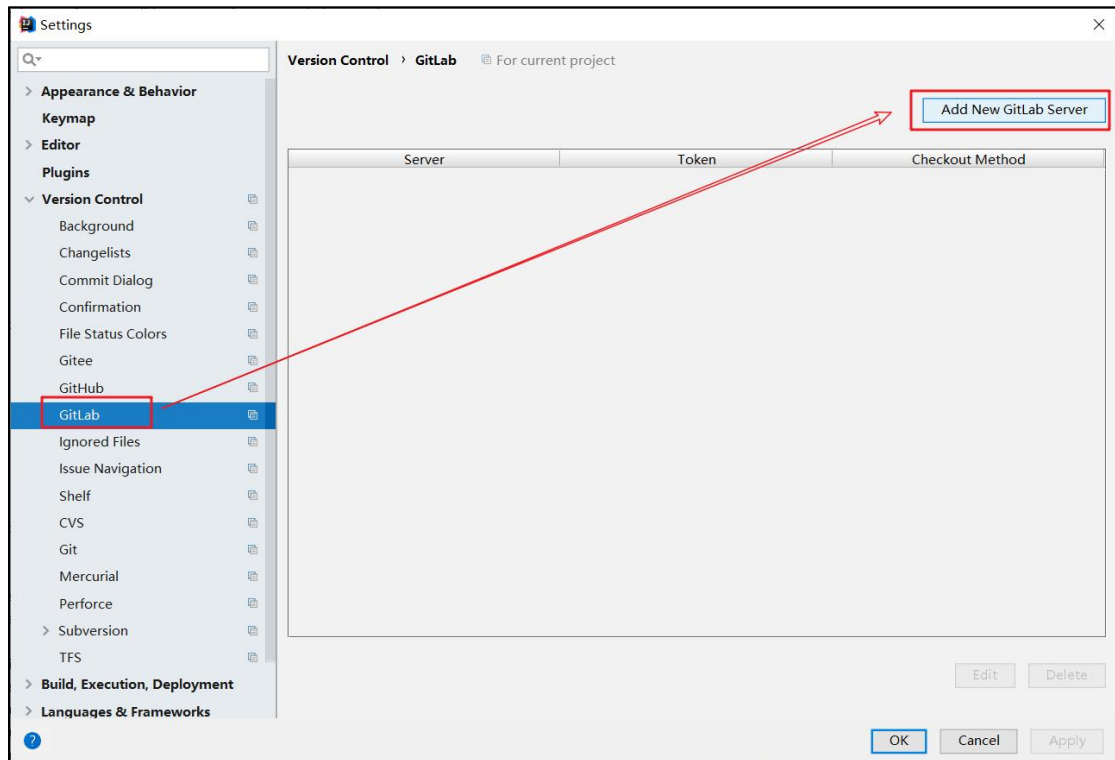


创建后，可以查看和复制生成的 token：



#### 4) 添加 GitLab 服务





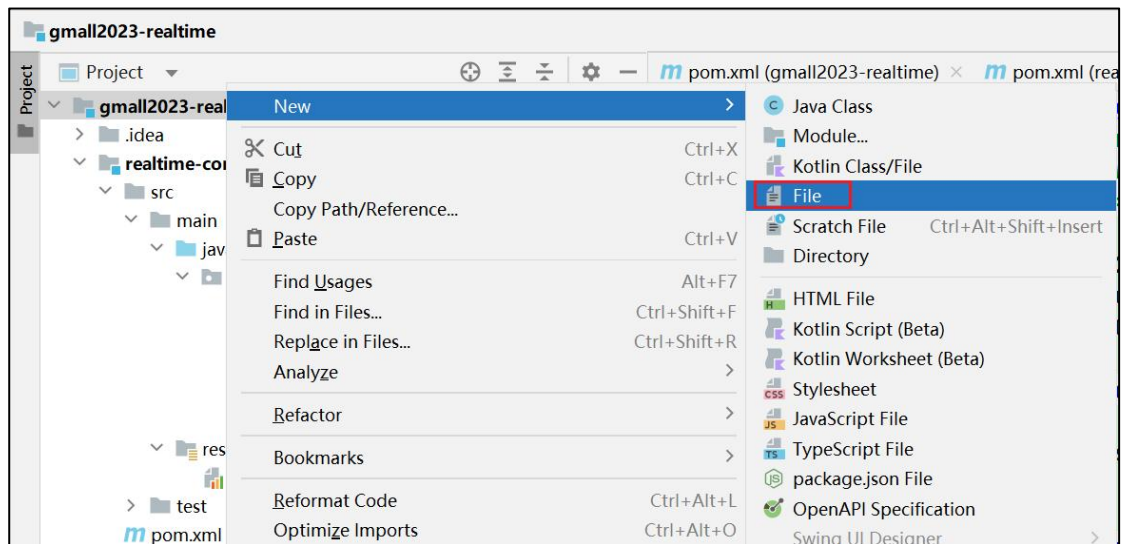
##### 5) 修改默认分支的保护策略

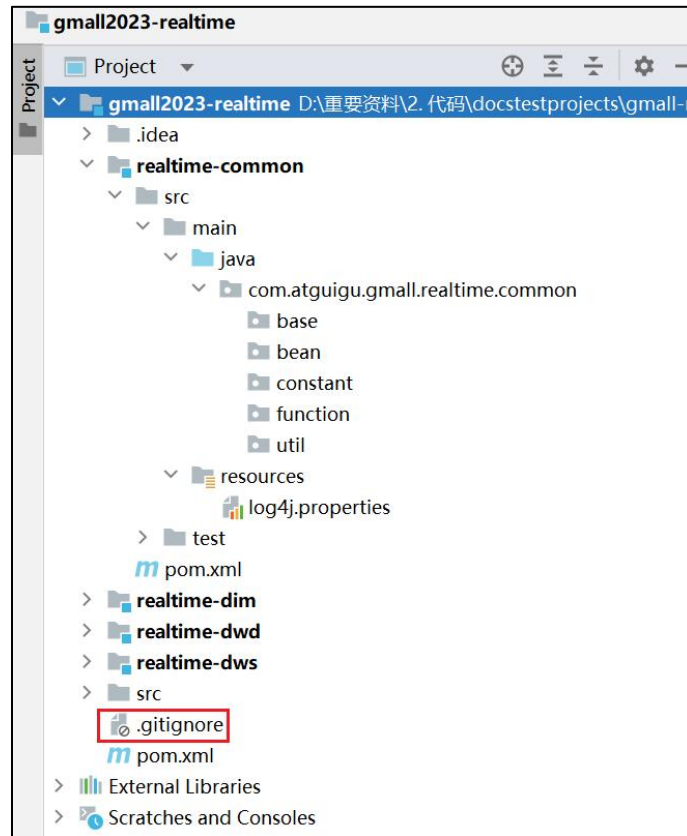




## 6) 创建.gitignore

(1) 在父工程根目录下创建.gitignore 文件



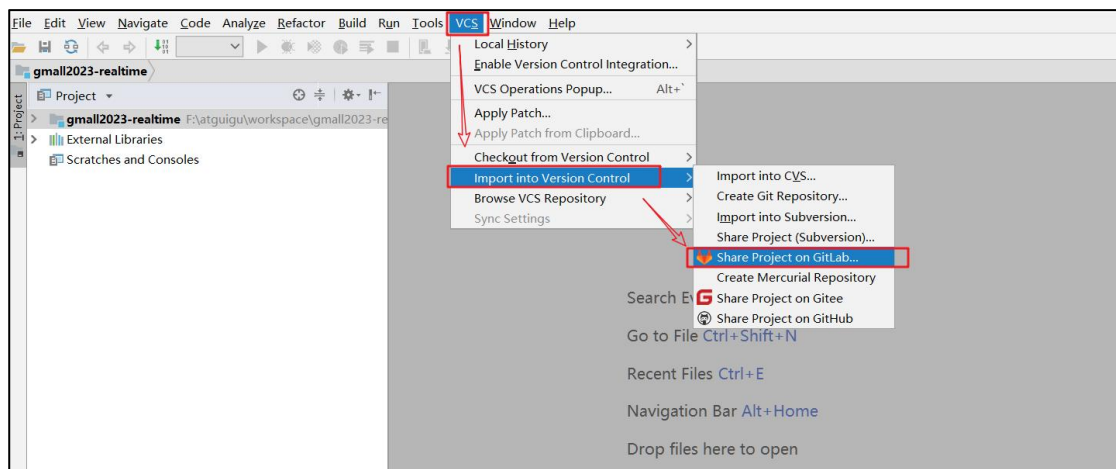


(2) 写入以下内容

```
/**/target
/**/.idea
*.iml
*.class

**/**/dependency-reduced-pom.xml
```

## 7) 分享到 GitLab



有了 token，才能获取 group 列表，才能选择 group：

Share on GitLab

Gitlab Server: http://hadoop104/

Name: gmall2023-realtime

Group: bigdata

Auth Type: ☒ SSH ☐ HTTP

Message: Initial commit

☒ Private  
☐ Internal  
☐ Public

Share

Cancel

因为之前填了token, 才可选

GitLab 的 bigdata 群组下，可以看到分享的项目：

rdc > bigdata > gmall2023-realtime

g

gmall2023-realtime

项目ID: 8

1次提交

1个分支

0个标签

82 KB 项目存储

init

ergou 编辑于 16分钟前

f8a8e772

master

gmall2023-realtime

+

查找文件

编辑

下载

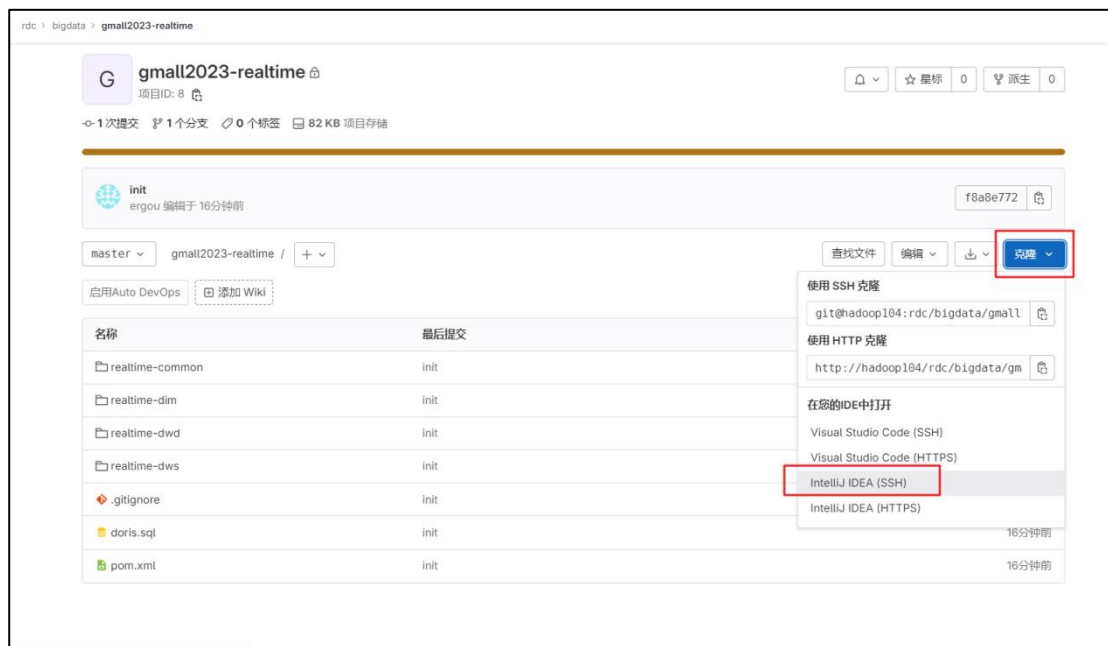
克隆

启用Auto DevOps

添加 Wiki

| 名称              | 最后提交 | 最后更新  |
|-----------------|------|-------|
| realtime-common | init | 16分钟前 |
| realtime-dim    | init | 16分钟前 |
| realtime-dwd    | init | 16分钟前 |
| realtime-dws    | init | 16分钟前 |
| .gitignore      | init | 16分钟前 |
| doris.sql       | init | 16分钟前 |
| pom.xml         | init | 16分钟前 |

团队其他人员，克隆项目可以通过 GitLab 页面快速操作：



## 第 8 章 数据仓库涉及的组件环境准备

### 8.1 Flink 环境搭建

Flink 集群部署请参考如下文档。



尚硅谷大数据技术  
之Flink.docx

### 8.2 Hbase 环境搭建

Hbase 集群部署请参考如下文档。



尚硅谷大数据技术  
之HBase.docx

### 8.3 Redis 环境搭建

#### 8.3.1 Redis 安装

Redis 安装请参考如下文档



尚硅谷大数据技术  
之 Redis.docx

### 8.3.2 Redis 配置文件修改

1) 切换到 atguigu 用户, 拷贝 redis.conf 配置文件, 置于 atguigu 家目录下

若已按照 1 中文档操作, 则这一步可以省略

```
[root@hadoop102 atguigu]# su atguigu
[atguigu@hadoop102 redis-6.0.8]$ cp
/opt/module/redis-6.0.8/redis.conf ~/my_redis.conf
```

2) 注销 bind 配置项

```
# bind 127.0.0.1
```

3) 修改 protected-mode

```
protected-mode no
```

## 8.4 Doris 环境搭建

Doris 安装请参考如下文档



尚硅谷大数据技术  
之DorisV2.x.docx

## 第 9 章 数仓开发之 ODS 层

### 9.1 概述

采集到 Kafka 的 topic\_log 和 topic\_db 主题的数据即为实时数仓的 ODS 层, 这一层的作用是对数据做原样展示和备份。参见采集文档。

需要注意的是, 离线数仓模拟数据的业务时间是打散在一天不同时刻的, 而实时要求数据的业务时间为系统时间, 修改 application.yml, 将 mock.if-realtime 置为 1 即可, 如下。

```
mock.if-realtime: 1
```

### 9.2 Kafka 数据有序

本项目要求 Flink 单个并行度的数据严格有序 (主要用于数据去重), 为了保证这一点, 做了如下配置。

1) 修改 Kafka 配置文件 server.properties, 将 topic 分区数设置为 4

```
[atguigu@hadoop102 ~]$ cd /opt/module/kafka/
[atguigu@hadoop102 kafka]$ vim config/server.properties
# 在配置文件中修改 Kafka 主题的默认分区数
num.partitions=4
```

三台服务器的配置文件都要修改，修改完毕重启 Kafka，不可直接分发配置文件，因为每个节点的 `broker.id` 必须唯一。分发后必须修改 `broker.id` 才能重启 Kafka，否则重启失败。

## 2) 在 Flink 程序中设置并行度为 4

Flink 程序从 Kafka 消费数据时会启动同属于一个消费者组的四个消费者，Kafka 消费者的默认分区分配策略是 `Range + CooperativeSticky`，消费者数和分区数相同时，每个消费者消费一个分区的数据。只要单分区数据有序，即可保证 Flink 单个并行度数据有序。

本项目 Kafka 版本为 3.0.0，在 Kafka 1.x 及之后的版本中，保证数据单分区有序，条件如下：

### (1) 未开启幂等性

`max.in.flight.requests.per.connection` 需要设置为 1。

该参数指定了生产者在收到服务器响应之前可以发送多少条消息

### (2) 开启幂等性

`max.in.flight.requests.per.connection` 需要设置小于等于 5。

原因说明：因为在 kafka 1.x 以后，启用幂等后，kafka 服务端会缓存 producer 发来的最近 5 个 request 的元数据，故无论如何，都可以保证最近 5 个 request 的数据都是有序的。

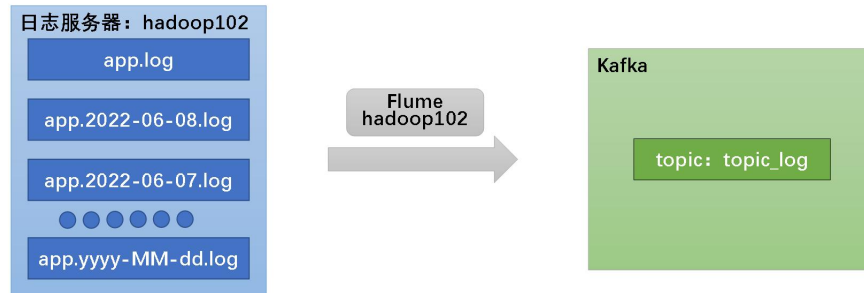
默认情况下幂等性是开启的，`max.in.flight.requests.per.connection` 默认值为 5，所以单分区数据默认是有序的，不需要做任何配置。

综上，我们可以保证 Flink 程序单个并行度的数据有序。

## 9.3 日志数据采集

## 9.3.1 数据通道

### 用户行为日志数据通道



让天下没有难学的技术

## 9.3.2 上传模拟生成日志数据的 jar 包

将\2.资料\mock\下 application.yml、gmall-remake-mock-2023-05-15-3.jar、path.json、logback.xml 上传到 hadoop102 的/opt/module/applog 目录下

## 9.3.3 根据需要修改配置文件

### (1) application.yml 文件

可以根据需求生成对应日期的用户行为日志。

```
[atguigu@hadoop102 applog]$ vim application.yml
```

修改如下内容

```
# 外部配置打开
logging.config: ./logback.xml

#http 模式下, 发送的地址
mock:
  log:
    type: "file"      #"file" "http" "kafka" "none"
    http:
      url: "http://localhost:8090/applog"
    kafka:
      server: "hadoop102:9092,hadoop102:9092,hadoop102:9092"
      topic: "topic_log"

spring:
  datasource:
    type: com.alibaba.druid.pool.DruidDataSource
    druid:
      url:
jdbc:mysql://hadoop102:3306/gmall?characterEncoding=utf-8&allowPublicKeyRetrieval=true
```



```
&useSSL=false&serverTimezone=GMT%2B8
    username: root
    password: "000000"
    driver-class-name: com.mysql.cj.jdbc.Driver
    max-active: 20
    test-on-borrow: true

mybatis-plus.global-config.db-config.field-strategy: not_null
mybatis-plus:
    mapper-locations: classpath:mapper/*.xml

mybatis:
    mapper-locations: classpath:mapper/*.xml
```

#业务日期, 并非Linux系统时间的日期, 而是生成模拟数据的日期

```
mock.date: "2022-06-08"
```

# 日志是否写入数据库一份 写入 z\_log 表中

```
mock.log.db.enable: 1
```

# 清空

```
mock.clear.busi: 1
```

# 清空用户

```
mock.clear.user: 0
```

# 批量生成新用户

```
mock.new.user: 0
```

#session 次数

```
mock.user-session.count: 200
```

#设备最大值

```
mock.max.mid: 1000000
```

# 是否针对实时生成数据, 若启用(置为1)则数据的 yyyy-MM-dd 与 mock.date 一致而 HH:mm:ss 与系统时间一致; 若禁用则数据的 yyyy-MM-dd 与 mock.date 一致而 HH:mm:ss 随机分布, 此处开启

```
mock.if-realtime: 1
```

#访问时间分布权重

```
mock.start-time-weight:
```

```
"10:5:0:0:0:0:5:5:5:10:10:15:20:10:10:10:10:20:25:30:35:30:20"
```

#支付类型占比 支付宝 : 微信 : 银联

```
mock.payment_type_weight: "40:50:10"
```

#页面平均访问时间

```
mock.page.during-time-ms: 20000
```

#错误概率 百分比

```
mock.error.rate: 3
```

#每条日志发送延迟 ms

```
mock.log.sleep: 100
```

#课程详情来源 用户查询, 商品推广, 智能推荐, 促销活动

```
mock.detail.source-type-rate: "40:25:15:20"
```

```
mock.if-cart-rate: 100
```

```
mock.if-favor-rate: 70
```

```
mock.if-order-rate: 100
```

```
mock.if-refund-rate: 50
```

```
#搜索关键词
mock.search.keyword: "java,python,多线程,前端,数据库,大数据,hadoop,flink"

#用户数据变化概率
mock.user.update-rate: 20

# 男女浏览品牌比重 (11 品牌)
mock.tm-weight.male: "3:2:5:5:5:1:1:1:1:1:1"
mock.tm-weight.female: "1:5:1:1:2:2:2:5:5:5:5"

# 外连类型比重 (5 种)
mock.refer-weight: "10:2:3:4:5"

# 线程池相关配置
mock.pool.core: 20
mock.pool.max-core: 100
```

## (2) path.json, 该文件用来配置访问路径

根据需求, 可以灵活配置用户点击路径。

```
[
  {
    "path":["start_app","home", "search",
    "good_list","good_detail","good_detail", "good_detail","cart","order","payment","mine",
    "order_list","end"],"rate":100 },
    {
      "path":["start_app","home",
      "good_list","good_detail","good_detail", "good_detail","cart","end"],"rate":30 },
      {
        "path":["start_app","home",
        "activity1111","good_detail", "cart","good_detail","cart","order","payment","end"],"r
        ate":30 },

        {
          "path":["activity1111","good_detail", "activity1111", "good_detail","order","payment",
          "end"],"rate":200 },

          {
            "path":["start_app","home", "activity1111", "good_detail","order","payment","end"],"ra
            te":200 },
            {
              "path":["start_app","home", "good_detail","order","payment","end"],"rate":30 },
              {
                "path":["good_detail","order","payment","end"],"rate":650 },
                {
                  "path":["good_detail"],"rate":30 },
                  {
                    "path":["start_app","home","mine","good_detail"],"rate":30 },
                    {
                      "path":["start_app","home",
                      "good_detail","good_detail","good_detail","cart","order","payment","end"],"rate":200
                      },
                      {
                        "path":["start_app","home",
                        "search","good_list","good_detail","cart","order","payment","end"],"rate":200 }
                        ]

```

## (3) logback 配置文件

可配置日志生成路径, 修改内容如下。

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <property name="LOG_HOME" value="/opt/module/applog/log" />
  <appender name="console" class="ch.qos.logback.core.ConsoleAppender">
    <target>System.out</target>
    <encoder>
```

```
<pattern>%msg%n</pattern>
</encoder>
</appender>

<appender name="console_em" class="ch.qos.logback.core.ConsoleAppender">
  <target>System.err</target>
  <encoder>
    <pattern>%msg%n</pattern>
  </encoder>
</appender>

<appender name="rollingFile"
class="ch.qos.logback.core.rolling.RollingFileAppender">
  <file>${LOG_HOME}/app.log</file>
  <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
    <fileNamePattern>${LOG_HOME}/app.%d{yyyy-MM-dd}.log</fileNamePattern>
  </rollingPolicy>
  <encoder>
    <pattern>%msg%n</pattern>
  </encoder>
</appender>

<!-- 将某一个包下日志单独打印日志 -->
<logger name="com.atguigu.mock.util.LogUtil"
  level="INFO" additivity="false">
  <appender-ref ref="rollingFile" />
<!--
  <appender-ref ref="console" />-->
</logger>
<logger name="com.atguigu.gmallre.mock.task.UserMockTask" level="INFO"
additivity="false" >
  <appender-ref ref="console_em" />
</logger>

<!-- <logger name="com.alibaba.druid.pool" level="error" additivity="false" -->
<!--   <appender-ref ref="console" />-->
<!-- </logger>-->

<!-- <logger name="com.atguigu.edu2021.mock.mapper" level="debug">-->
<!--   <appender-ref ref="console" />-->
<!-- </logger>-->

<!-- <logger name="com.atguigu.edu2021.mock.service.impl.UserInfoServiceImpl"
level="debug">
  <appender-ref ref="console" />
</logger>-->

<root level="error" >
  <appender-ref ref="console_em" />
  <!-- <appender-ref ref="async-rollingFile" /> -->
</root>
</configuration>
```

### 9.3.4 在 hadoop102 上生成日志并落盘

(1) 进入到/opt/module/applog 路径, 执行以下命令

```
[atguigu@hadoop102 applog]$ java -jar gmall-remake-mock-2023-05-15-3.jar test 100
2022-06-08
```

➤ 增加 test 参数为测试模式, 只生成用户行为数据不生成业务数据。

- 100 为产生的用户 session 数一个 session 默认产生 1 条启动日志和 5 条页面方法日志。

- 第三个参数为日志数据的日期，测试模式下不会加载配置文件，要指定数据日期只能通过命令行传参实现。

- 三个参数的顺序必须与示例保持一致
- 第二个参数和第三个参数可以省略，如果 test 后面不填写参数，默认为 1000

(2) 在/opt/module/applog/log 目录下查看生成日志

```
[atguigu@hadoop102 log]$ ll
```

### 9.3.5 集群日志生成脚本

(1) 在/home/atguigu/bin 目录下创建脚本 lg.sh

```
[atguigu@hadoop102 bin]$ vim lg.sh
```

(2) 在脚本中编写如下内容

```
#!/bin/bash
echo "===== hadoop102 ====="
ssh hadoop102 "cd /opt/module/applog/; java -jar gmall-remake-mock-2023-05-15-3.jar $1 $2
$3 >/dev/null 2>&1 &"
```

注：

- /opt/module/applog/为 jar 包及配置文件所在路径
- /dev/null 代表 Linux 的空设备文件，所有往这个文件里面写入的内容都会丢失，俗称“黑洞”。

标准输入 0：从键盘获得输入 /proc/self/fd/0

标准输出 1：输出到屏幕（即控制台） /proc/self/fd/1

错误输出 2：输出到屏幕（即控制台） /proc/self/fd/2

(3) 修改脚本执行权限

```
[atguigu@hadoop102 bin]$ chmod u+x lg.sh
```

(4) 启动脚本

```
[atguigu@hadoop102 module]$ lg.sh test 100
```

(5) 在 hadoop102 的/opt/module/applog/log 目录查看生成的数据

```
[atguigu@hadoop102 logs]$ ls
app.log
```

### 9.3.6 通过 flume 将落盘的日志数据采集到 kafka 主题

实时项目中，用户行为日志是没有历史数据的，故日志只需要准备某一天的数据

#### (1) 准备 flume 配置文件

在 hadoop102 节点的 Flume 的 job 目录下创建 file\_to\_kafka.conf

```
[atguigu@hadoop102 flume]$ vim job/file_to_kafka.conf

#定义组件
a1.sources = r1
a1.channels = c1

#配置 source
a1.sources.r1.type = TAILDIR
a1.sources.r1.filegroups = f1
a1.sources.r1.filegroups.f1 = /opt/module/applog/log/app.*
a1.sources.r1.positionFile = /opt/module/flume/taildir_position.json

#配置 channel
a1.channels.c1.type = org.apache.flume.channel.kafka.KafkaChannel
a1.channels.c1.kafka.bootstrap.servers = hadoop102:9092,hadoop103:9092
a1.channels.c1.kafka.topic = topic_log
a1.channels.c1.parseAsFlumeEvent = false

#组装
a1.sources.r1.channels = c1
```

#### (2) 编写 flume 启停脚本

在 hadoop102 节点的/home/atguigu/bin 目录下创建脚本 fl.sh 并授权

```
#!/bin/bash

case $1 in
"start"){
    echo " -----启动 hadoop102 采集 flume-----"
    ssh hadoop102 "nohup /opt/module/flume/bin/flume-ng agent -n a1 -c
/opt/module/flume/conf/ -f /opt/module/flume/job/file_to_kafka.conf >/dev/null 2>&1 &"
};;
"stop"){
    echo " -----停止 hadoop102 采集 flume-----"
    ssh hadoop102 "ps -ef | grep file_to_kafka | grep -v grep | awk '{print \$2}' | xargs
-n1 kill -9 "
};;
esac
```

#### (3) 启动 Zookeeper

#### (4) 启动 Kafka

注意：为了后续 Flink 读取和 kafka 并行度一直，我们统一设置并行度为 4

#### (5) 启动一个命令行 Kafka 消费者，消费 topic\_log 主题的数据

```
/opt/module/kafka/bin/kafka-console-consumer.sh --bootstrap-server hadoop102:9092
--topic topic_log
```

#### (6) 执行 flume 采集脚本

```
f1.sh start
```

(7) 执行日志生成脚本 lg.sh

(8) 观察命令行 Kafka 消费者是否消费到数据。

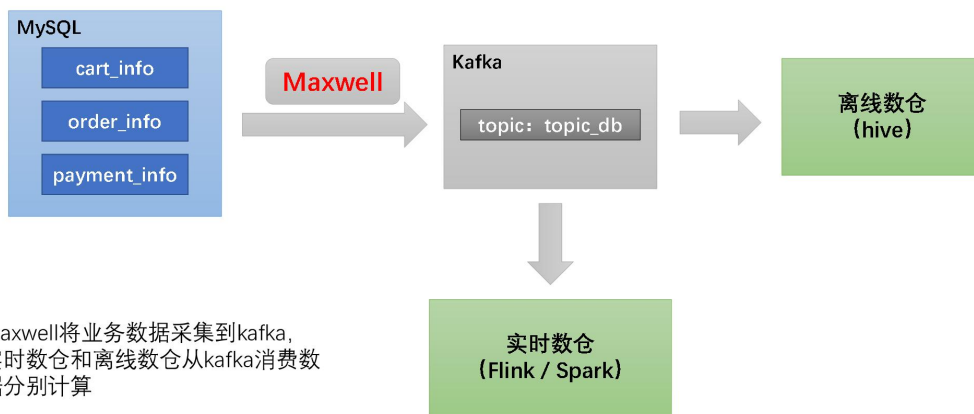
```
{
  "actions": [
    {
      "action_id": "cart_add",
      "extend1": "",
      "extend2": "",
      "item": "23",
      "item_type": "sku_id",
      "ts": 1680402512483
    }
  ],
  "common": {
    "ar": "11",
    "ba": "OPPO",
    "ch": "xiaomi",
    "is_new": "0",
    "md": "OPPO Oneplus 10",
    "mid": "mid_255",
    "os": "Android 11.0",
    "vc": "v2.1.134"
  },
  "displays": [
    {
      "item": "20",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 0
    },
    {
      "item": "24",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 1
    },
    {
      "item": "27",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 2
    },
    {
      "item": "2",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 3
    },
    {
      "item": "34",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 4
    },
    {
      "item": "26",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 5
    }
  ],
  "page": {
    "during_time": 13063,
    "page_id": "good_detail",
    "ts": 1680402512483
  }
},
{
  "actions": [
    {
      "action_id": "cart_add",
      "extend1": "",
      "extend2": "",
      "item": "4",
      "item_type": "sku_id",
      "ts": 1680402512483
    }
  ],
  "common": {
    "ar": "11",
    "ba": "OPPO",
    "ch": "xiaomi",
    "is_new": "0",
    "md": "OPPO Oneplus 10",
    "mid": "mid_255",
    "os": "Android 11.0",
    "vc": "v2.1.134"
  },
  "displays": [
    {
      "item": "14",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 0
    },
    {
      "item": "19",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 1
    },
    {
      "item": "25",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 2
    },
    {
      "item": "24",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 3
    },
    {
      "item": "15",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 4
    },
    {
      "item": "23",
      "item_type": "sku_id",
      "pos_id": 4,
      "pos_seq": 5
    }
  ],
  "page": {
    "during_time": 13050,
    "page_id": "good_detail",
    "ts": 1680402512483
  }
}
```

## 9.4 业务数据采集

### 9.4.1 数据通道



#### 业务数据通道

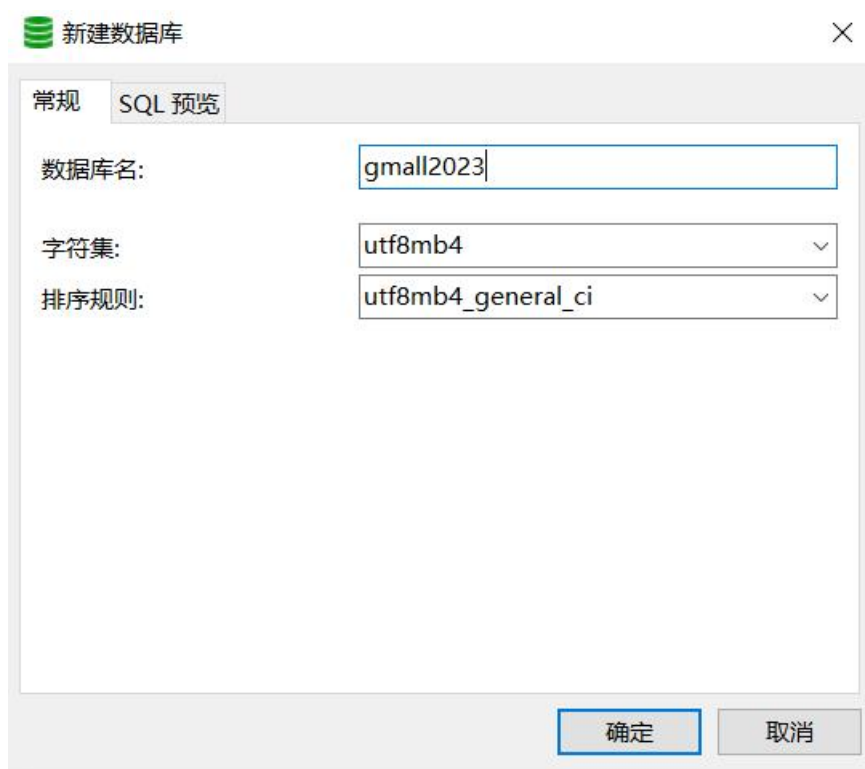


Maxwell将业务数据采集到kafka，  
实时数仓和离线数仓从kafka消费数据  
分别计算

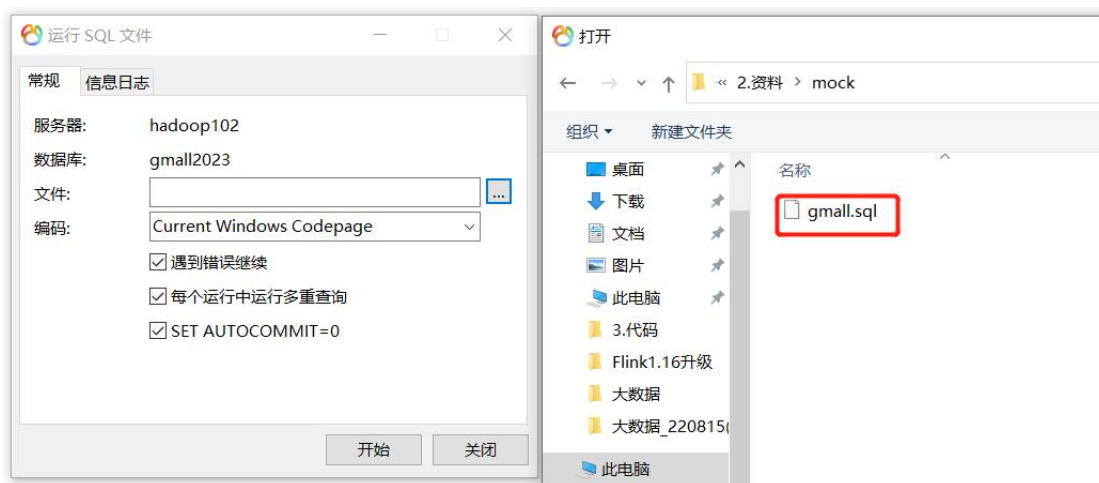
让天下没有难学的技术

### 9.4.2 MySQL 的准备

1) 创建实时业务数据库



## 2) 导入建表数据



## 3) 修改/etc/my.cnf 文件

```
[atguigu@hadoop102 module]$ sudo vim /etc/my.cnf
server-id= 1
log-bin=mysql-bin
binlog_format=row
binlog-do-db=gmall2023
注意: binlog-do-db 根据自己的情况进行修改, 指定具体要同步的数据库
```

## 4) 重启 MySQL 使配置生效

```
sudo systemctl restart mysqld
```

到/var/lib/mysql 目录下查看初始文件大小 157，注：mysql8 自定开启 binlog



```
-rw-r----- 1 mysql mysql      157 4月  2 10:58 mysql-bin.000001
-rw-r----- 1 mysql mysql       19 4月  2 10:58 mysql-bin.index
```

## 5) 模拟生成数据

### (1) 修改/opt/module/applog/application.yml 中数据库连接信息

```
spring:
  datasource:
    type: com.alibaba.druid.pool.DruidDataSource
    druid:
      url: jdbc:mysql://hadoop102:3306/gmall12023?characterEncoding=utf-8&allowPublicKeyRetrieval=true&useSSL=false&serverTimezone=GMT%2B8
      username: root
      password: "123456"
      driver-class-name: com.mysql.cj.jdbc.Driver
      max-active: 20
      test-on-borrow: true
```

### ➤ 实时数仓中需要修改/opt/module/applog/application.yml 中生成数据的时间

```
mock.if-realtime: 1
```

### ➤ 运行 lg.sh

注意：这个时候，直接让 jar 包会直接读取 application.yml，所以不需要指定 test 模式

```
[atguigu@hadoop102 dblog]$ lg.sh
```

注意：如果生成较慢，可根据配置情况适当调整配置项

### ➤ 再次到到/var/lib/mysql 目录下，查看 index 文件的大小

```
-rw-r----- 1 mysql mysql      180 4月  2 11:00 mysql-bin.000001
-rw-r----- 1 mysql mysql    2464619 4月  2 11:09 mysql-bin.000002
-rw-r----- 1 mysql mysql       38 4月  2 11:00 mysql-bin.index
```

## 9.4.3 安装 Maxwell

### ➤ 解压 maxwell-1.29.2.tar.gz 到/opt/module 目录

```
[atguigu@hadoop102 module]$ tar -zxvf /opt/software/maxwell-1.29.2.tar.gz -C /opt/module/
```

### ➤ 修改名称

```
mv maxwell-1.29.2/ maxwell
```

## 9.4.4 初始化 Maxwell 元数据库

### ➤ 在 MySQL 中建立一个 maxwell 库用于存储 Maxwell 的元数据

```
[atguigu@hadoop102 module]$ mysql -uroot -p123456
mysql> CREATE DATABASE maxwell ;
```

### ➤ 设置安全级别

```
mysql> set global validate_password.length=4;
mysql> set global validate_password.policy=0;
```

➤ 创建 maxwell 用户并赋予必要权限

```
mysql> CREATE USER 'maxwell'@'%' IDENTIFIED BY '123456';
mysql> GRANT ALL ON maxwell.* TO 'maxwell'@'%';
mysql> GRANT SELECT, REPLICATION CLIENT, REPLICATION SLAVE ON *.* TO 'maxwell'@'%';
```

## 9.4.5 使用 Maxwell 监控抓取 MySQL 数据

➤ 拷贝配置文件

```
[atguigu@hadoop102 maxwell]$ cp config.properties.example config.properties
```

➤ 修改配置文件

```
producer=kafka
kafka.bootstrap.servers=hadoop102:9092,hadoop103:9092,hadoop104:9092
#需要添加
kafka_topic=topic_db
# mysql login info
host=hadoop102
user=maxwell
password=123456
jdbc_options=useSSL=false&serverTimezone=Asia/Shanghai&allowPublicKeyRetrieval=true
#需要添加 后续初始化会用
client_id=maxwell_1
```

# 过滤 gmall2023 中的 z\_log 表数据, 该表是日志数据的备份, 无须采集

```
filter=exclude:gmall2023.z_log
```

注意: 默认还是输出到指定 Kafka 主题的一个 kafka 分区, 因为多个分区并行可能会打乱 binlog 的顺序

如果要提高并行度, 首先设置 kafka 的分区数>1, 然后设置 producer\_partition\_by 属性  
可选值 producer\_partition\_by=database|table|primary\_key|random|column

➤ 在 /home/atguigu/bin 目录下编写 maxwell.sh 启动脚本

```
[atguigu@hadoop102 maxwell]$ vim /home/atguigu/bin/maxwell.sh
/opt/module/maxwell/bin/maxwell --config
/opt/module/maxwell/config.properties >/dev/null 2>&1 &
```

➤ 授予执行权限

```
[atguigu@hadoop102 maxwell]$ sudo chmod +x /home/atguigu/bin/maxwell.sh
```

➤ 运行启动程序

```
[atguigu@hadoop102 maxwell]$ maxwell.sh
[atguigu@hadoop202 maxwell-1.25.0]$ xcall.sh jps
----- hadoop202 -----
6320 Application
5252 Kafka
7896 Jps
4811 QuorumPeerMain
7835 Maxwell
```

➤ 启动 Kafka 消费客户端, 观察结果

```
[atguigu@hadoop102 kafka]$ bin/kafka-console-consumer.sh --bootstrap-server
hadoop102:9092 --topic topic_db
```

### ➤ 执行 lg.sh 生成模拟数据

```
[atguigu@hadoop102 dblog]$ lg.sh
```

```
{ "database": "gmall2021", "table": "user_info", "type": "update", "ts": "1610504502", "xid": "118213", "commit": true, "data": { "id": 1, "login_name": "66axzspu60", "nick_name": "茗茗1", "passwd": null, "name": "茗茗", "phone_num": "13717127677", "email": "66axzspu60@0355.net", "head_img": null, "user_level": "2", "birthday": "1974-01-13", "gender": "F", "create_time": "2021-01-13 09:52:56", "operate_time": null, "status": null }, "old": { "nick_name": "茗茗" } }
```

注意：如果需要监控 DDL 变化，在启动 Maxwell 的时候添加参数 `-output_ddl`

## 9.4.6 同步维度历史数据

实时计算不考虑历史的事实数据，但要考虑历史维度数据。因此要对维度相关的业务表做一次全量同步

### (1) 与维度相关的业务表如下

```
activity_info
activity_rule
activity_sku
base_category1
base_category2
base_category3
base_dic
base_province
base_region
base_trademark
coupon_info
coupon_range
financial_sku_cost
sku_info
spu_info
user_info
```

### (2) 切换到 /home/atguigu/bin 目录，创建 mysql\_to\_kafka\_init.sh 文件

```
[atguigu@hadoop102 maxwell]$ cd ~/bin
[atguigu@hadoop102 bin]$ vim mysql_to_kafka_init.sh
```

### (3) 在脚本中添加如下内容

```
#!/bin/bash

# 该脚本的作用是初始化所有的业务数据，只需执行一次

MAXWELL_HOME=/opt/module/maxwell

import_data() {
    $MAXWELL_HOME/bin/maxwell-bootstrap --database gmall12023 --table $1 --config
    $MAXWELL_HOME/config.properties
}

case $1 in
    "activity_info")
        import_data activity_info
        ;;
    "activity_rule")
        import_data activity_rule
        ;;
    "activity_sku")
        import_data activity_sku
        ;;
    *)
        echo "Invalid table name"
        exit 1
    esac
```

```
;;
"base_category1")
import_data base_category1
;;
"base_category2")
import_data base_category2
;;
"base_category3")
import_data base_category3
;;
"base_dic")
import_data base_dic
;;
"base_province")
import_data base_province
;;
"base_region")
import_data base_region
;;
"base_trademark")
import_data base_trademark
;;
"coupon_info")
import_data coupon_info
;;
"coupon_range")
import_data coupon_range
;;
"financial_sku_cost")
import_data financial_sku_cost
;;
"sku_info")
import_data sku_info
;;
"spu_info")
import_data spu_info
;;
"user_info")
import_data user_info
;;
"all")
import_data activity_info
import_data activity_rule
import_data activity_sku
import_data base_category1
import_data base_category2
import_data base_category3
import_data base_dic
import_data base_province
import_data base_region
import_data base_trademark
import_data coupon_info
import_data coupon_range
import_data financial_sku_cost
import_data sku_info
import_data spu_info
import_data user_info
;;
esac
```

#### (4) 增加执行权限

```
[atguigu@hadoop102 bin]$ chmod +x mysql_to_kafka_init.sh
```

(5) 执行脚本(注意: 需要启动 maxwell 本身进程)

```
[atguigu@hadoop102 bin]$ mysql_to_kafka_init.sh all
```

(6) 启动 Kafka 消费者, 观察数据是否写入 Kafka

```
[atguigu@hadoop102 bin]$ kafka-console-consumer.sh --bootstrap-server hadoop102:9092  
--topic topic_db
```

## 第 10 章 数仓开发之 DIM 层

### 10.1 DIM 层设计要点

1) DIM 层的设计依据是维度建模理论, 该层存储维度模型的维度表。

2) DIM 层的数据存储在 HBase 表中

DIM 层表是用于维度关联的, 要通过主键去获取相关维度信息, 这种场景下 K-V 类型数据库的效率较高。常见的 K-V 类型数据库有 Redis、HBase, 而 Redis 的数据常驻内存, 会给内存造成较大压力, 因而选用 HBase 存储维度数据。

3) DIM 层表名的命名规范为 dim\_表名

### 10.2 配置表

#### 10.2.1 Flink CDC

Flink CDC 的介绍及配置请参考如下文档。



尚硅谷大数据技术  
之Flink-CDCV2.4.2

#### 10.2.2 配置表设计

1) 字段解析

我们将为维度分流准备配置表 table\_process\_dim, 为该表设计五个字段, 如下:

- (1) source\_table: 作为数据源的业务数据表名
- (2) sink\_table: 作为数据目的地的 HBase 表名
- (3) sink\_family: 作为数据目的地的 HBase 列族
- (4) sink\_columns: 写入 HBase 的字段
- (5) sink\_row\_key: 写入 HBase 需要指定的 row\_key

将 sink\_table 作为配置表的主键，可以通过它获取唯一的配置信息，明确维度分流的数据源及目的地等信息。

## 2) Mysql 中创建数据库建表并开启 Binlog

(1) 创建数据库 gmall2023\_config，注意:和 gmall 业务库区分开

```
[atguigu@hadoop102 dblog]$ mysql -uroot -p123456 -e"create database gmall2023_config charset utf8mb4 default collate utf8mb4_general_ci"
```

(2) 执行建表语句以及数据导入 SQL 文件如下



table\_process\_dim  
m维度初始配置.sql

(3) 建表语句如下

```
DROP TABLE IF EXISTS `table_process_dim`;
CREATE TABLE `table_process_dim` (
  `source_table` varchar(200) NOT NULL COMMENT '来源表',
  `sink_table` varchar(200) NOT NULL COMMENT '输出表',
  `sink_family` varchar(200) COMMENT '当输出类型是 dim 的时候,输出到 hbase 的列族',
  `sink_columns` varchar(2000) COMMENT '输出字段',
  `sink_row_key` varchar(200) COMMENT '当输出类型是 dim 时的 rowkey',
  PRIMARY KEY (`sink_table`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

(4) 在 MySQL 配置文件中增加 gmall2023\_config 开启 Binlog,

```
[atguigu@hadoop102 ~]$ sudo vim /etc/my.cnf
```

增加对于 gmall2023\_config 的监控

```
#数据库 id
server-id = 1
#启动 binlog, 该参数的值会作为 binlog 的文件名
log-bin=mysql-bin
#binlog 类型, maxwell 要求为 row 类型
binlog_format=row
#启用 binlog 的数据库, 需根据实际情况作出修改
binlog-do-db=gmall
binlog-do-db=gmall2023_config
```

保存退出

(5) 重启 MySQL 服务

```
[atguigu@hadoop102 ~]$ sudo systemctl restart mysqld
```

## 10.3 主要任务

更多 Java - 大数据 - 前端 - python 人工智能资料下载，可百度访问：尚硅谷官网

### 10.3.1 接收 Kafka 数据，过滤空值数据

对 Maxwell 抓取的数据进行 ETL，有用的部分保留，没用的过滤掉。

### 10.3.2 动态拆分维度表功能

实时数仓的维度表用于补充事实数据的维度信息（维度关联），事实数据是以流的形式存在的，如果维度信息也是这样，维度关联就要通过双流 join 来实现，而通常维度数据的生成时间可能远远早于事实数据，要保证二者可以正确关联，就要把维度流的数据缓存下来，设置一个极大的 ttl，使其常驻内存。对于数据量比较小的维度而言，这样做没有什么问题，但如果是用户这样数据量在百 G 以上的维度，必然会带来极大的内存开销，不是一个很好的解决方案。

通常，我们会将维度数据写入外部存储介质，通常是 HBase，Redis 或 MySQL 这样的 K-V 数据库，以便通过主键高效地获取维度信息。本项目选用 HBase，主要是考虑到大数据场景下某些维度表数据量极大，Redis 和 MySQL 可能扛不住这样的压力。HBase 的设计初衷就是支持海量数据的读写操作，可以托管数十亿行、数百万列的大表，并且支持随机查询，可以通过 row\_key 快速定位目标数据。

所有维度都存在 Kafka 的 topic\_db 主题，不同的维度表上下游、字段信息不同，可以把这些信息记录在配置文件中，在程序启动时加载，根据配置信息完成维度数据的处理。这种方案有一个非常严重的问题，程序启动后并不能动态感知配置变化，当业务端需求变化时必须重启计算程序读取更新后的配置文件，效率极低。此处需要一种动态配置方案，当配置变化时，流处理程序可以自动感知。此处提出三种动态配置方案：

（1）用 Zookeeper 存储配置信息，通过 Watch 感知数据变化

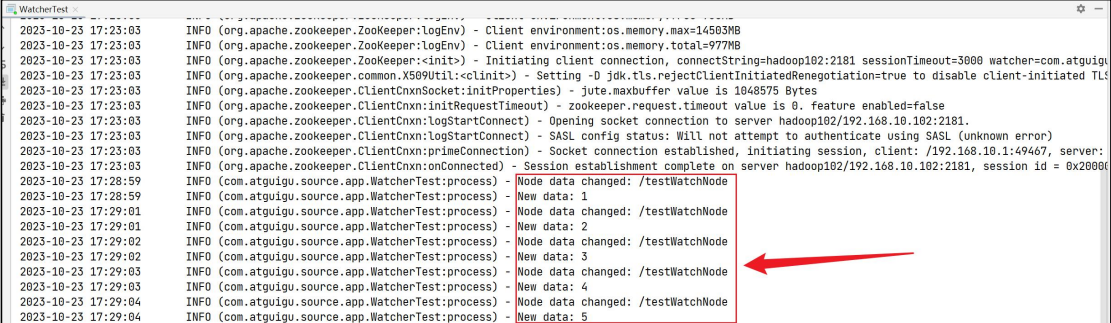
在命令行客户端修改结点内容

WATCHER::

```
WatchedEvent state:SyncConnected type:None path:null
[zk: localhost:2181 (CONNECTED) 0] set /testWatchNode '1'
[zk: localhost:2181 (CONNECTED) 1] set /testWatchNode '2'
[zk: localhost:2181 (CONNECTED) 2] set /testWatchNode '3'
[zk: localhost:2181 (CONNECTED) 3] set /testWatchNode '4'
[zk: localhost:2181 (CONNECTED) 4] set /testWatchNode '5'
```

观察 IDEA 控制台日志





```
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ZooKeeper:logEnv) - Client environment:os.memory.max=14583MB
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ZooKeeper:logEnv) - Client environment:os.memory.total=977MB
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ZooKeeper:Init) - Initiating client connection, connectString=hadoop102:2181 sessionTimeout=3000 watcher=com.atguigu
2023-10-23 17:23:03 INFO (org.apache.zookeeper.common.XS99Util:<clinit>) - Setting -Djdk.tls.rejectClientInitiatedRenegotiation=true to disable client-initiated TLS
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ClientCnxnSocket:initProperties) - jute.maxbuffer value is 1048576 Bytes
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ClientCnxn:initRequestTimeout) - zookeeper.request.timeout value is 8, feature enabled=false
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ClientCnxn:logStartConnect) - Opening socket connection to server hadoop102/192.168.10.102:2181.
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ClientCnxn:logStartConnect) - SASL config status: Will not attempt to authenticate using SASL (unknown error)
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ClientCnxn:primeConnection) - Socket connection established, initiating session, client: /192.168.10.1:49467, server:
2023-10-23 17:23:03 INFO (org.apache.zookeeper.ClientCnxn:onConnected) - Session establishment complete on server hadoop102/192.168.10.102:2181, session id = 0x20000
2023-10-23 17:28:59 INFO (com.atguigu.source.app.WatcherTest:process) - Node data changed: /testWatchNode
2023-10-23 17:28:59 INFO (com.atguigu.source.app.WatcherTest:process) - New data: 1
2023-10-23 17:29:01 INFO (com.atguigu.source.app.WatcherTest:process) - Node data changed: /testWatchNode
2023-10-23 17:29:01 INFO (com.atguigu.source.app.WatcherTest:process) - New data: 2
2023-10-23 17:29:02 INFO (com.atguigu.source.app.WatcherTest:process) - Node data changed: /testWatchNode
2023-10-23 17:29:02 INFO (com.atguigu.source.app.WatcherTest:process) - New data: 3
2023-10-23 17:29:03 INFO (com.atguigu.source.app.WatcherTest:process) - Node data changed: /testWatchNode
2023-10-23 17:29:03 INFO (com.atguigu.source.app.WatcherTest:process) - New data: 4
2023-10-23 17:29:04 INFO (com.atguigu.source.app.WatcherTest:process) - Node data changed: /testWatchNode
2023-10-23 17:29:04 INFO (com.atguigu.source.app.WatcherTest:process) - New data: 5
```

结点数据变更，IDEA 程序捕捉到了变更操作

将变更告知 Flink 应用

上述程序将 Zookeeper 结点的变更打印到了控制台，我们可以在程序的相应位置监控变更，将上述 demo 中的 data 同步给 Flink 的状态即可满足需求。

(2) 用 mysql 数据库存储，周期性同步

只需要在分程序的指定位置写一个死循环，周期性拉取 MySQL 配置表的数据即可。

(3) 用 mysql 数据库存储，通过 FlinkCDC 监控配置表数据变更

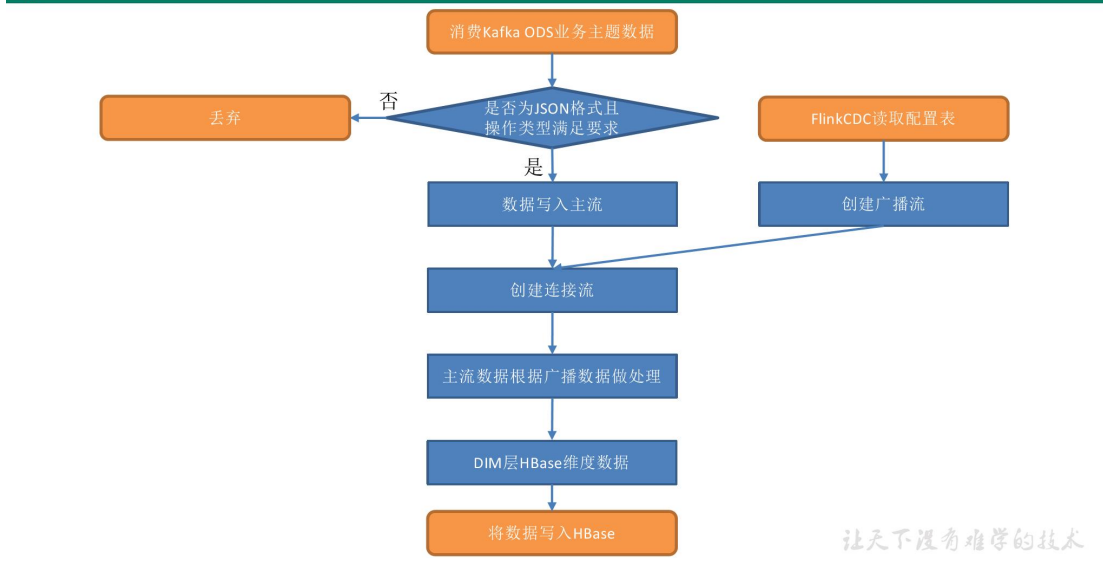
第二种方案无法实时获知数据变更，配置信息的更新至多需要等待一个周期，为了提升时效性，周期应尽可能短，但周期越短，执行查询的频率越高，开销越大，并且周期再短都无法实现实时更新，必然存在延迟。

而 Zookeeper 是大数据生态中的协调者，一般不会用于存储数据，且 Zookeeper 的结点存储的是字符串，我们可以通过 JSON 数组存储配置信息，但是与第三种方案相比，对于用户而言，操作繁琐，且配置信息不够清晰。

第三种方案可以实现变更数据的实时监控，且配置信息存储在 MySQL，符合开发习惯，易于维护，配置变更操作简便。

综上，此处选择第三种方案。通过 FlinkCDC 读取配置表生成配置流，将其作为广播流与主流关联，将配置变更实时传达给主流。

流程如下



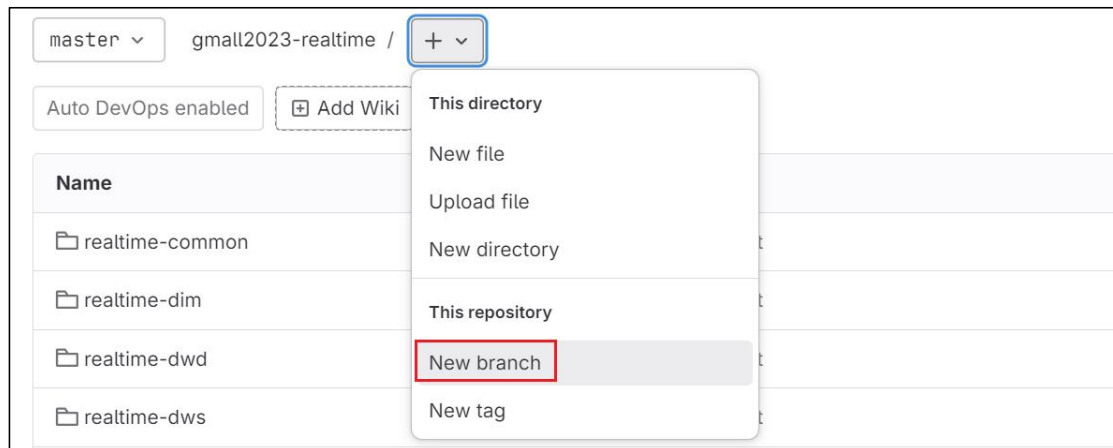
### 10.3.3 把流中的数据保存到对应的维度表

维度数据保存到 HBase 的表中。

## 10.4 新建本地和远端分支

### 10.4.1 新建远端 dev 分支

在 GitLab 的 gmall2023-realtime 仓库下新建分支 dev



## New Branch

Branch name

Create from

master

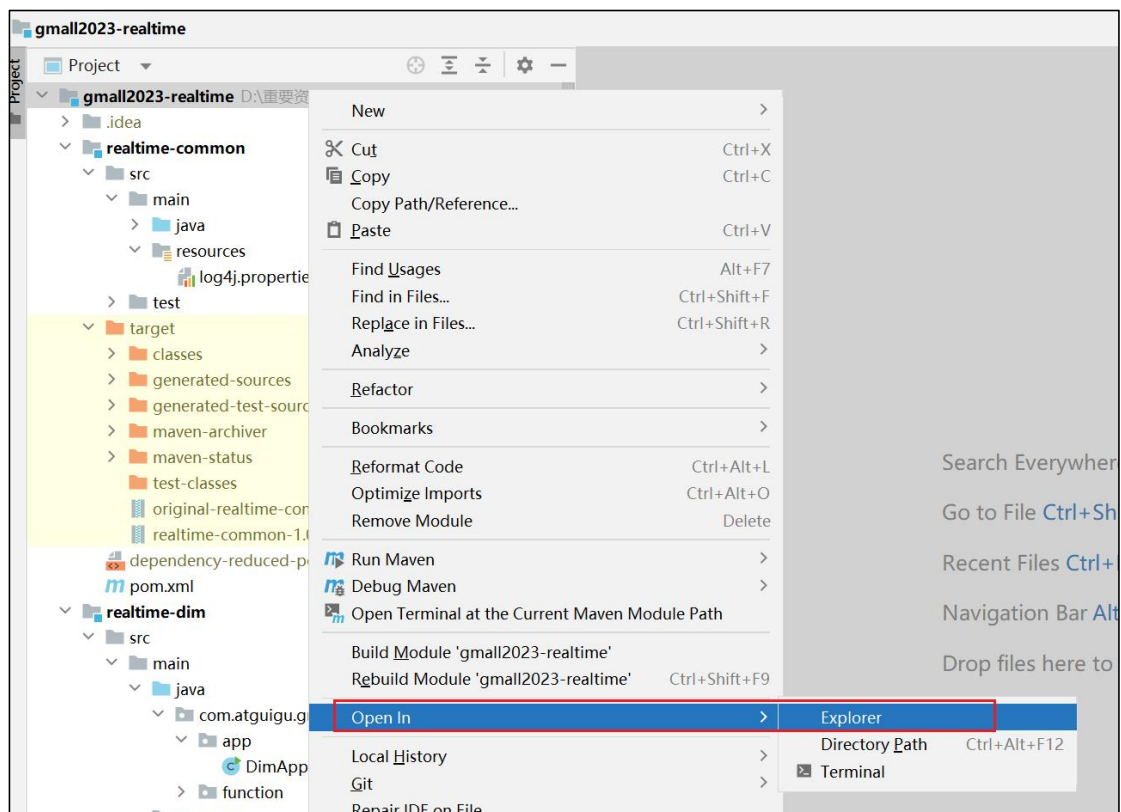
Existing branch name, tag, or commit SHA

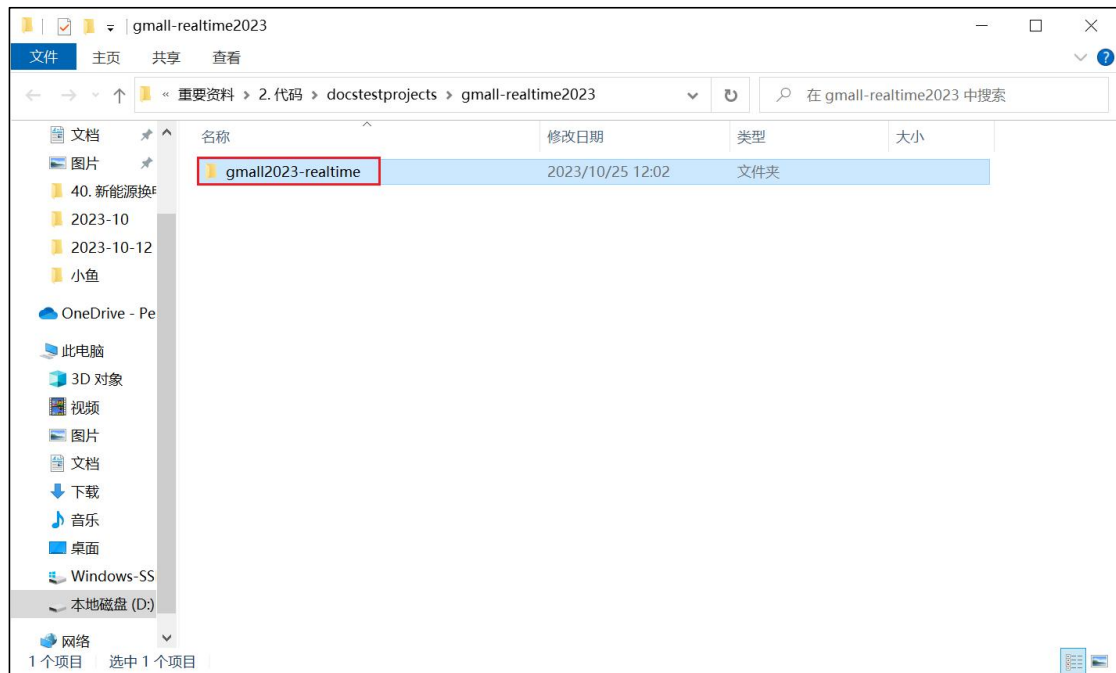
Create branch

Cancel

## 10.4.2 新建本地 dev 分支

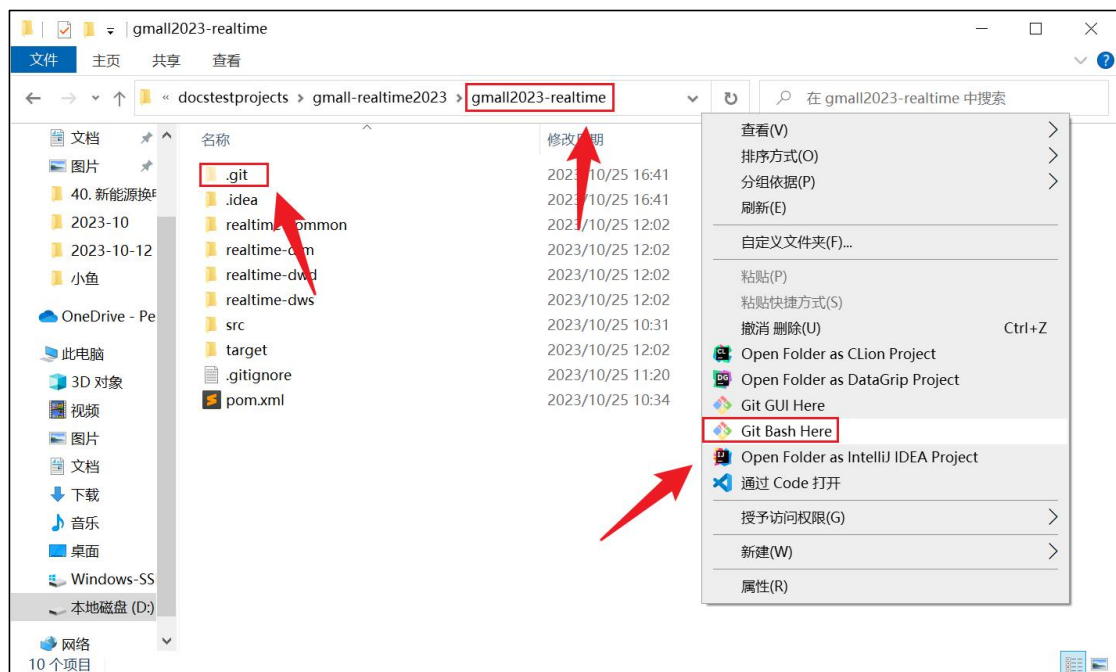
1) 在资源管理器中打开父工程所在目录





## 2) 打开 git 命令行终端

进入项目根目录，可以看到名为.git的隐藏目录，右键空白处，选择“Git Bash Here”  
打开 git 命令行终端，如下。



## 3) 创建本地 dev 分支，并追踪远端 dev 分支

(1) 查看远程仓库别名

```
$ git remote
```

返回如下

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```

master bcf4d2d [gmall2023-realtime/master: ahead 1] DI
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docst
time2023/gmall2023-realtime (dev)
$ git remote
gmall2023-realtime
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docst
time2023/gmall2023-realtime (dev)
$ |

```

(2) 获取远程仓库 gmall2023-realtime 的分支和标签

```
$ git fetch gmall2023-realtime
```

本项目只有一个远端仓库，此时仓库别名可以省略，即

```
$ git fetch
```

可以看到上一节新建的 dev 分支

```

Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-real
time2023/gmall2023-realtime (dev)
$ git fetch
From hadoop104:rdc/bigdata/gmall2023-realtime
* [new branch]      dev          -> gmall2023-realtime/dev

```

(3) 创建本地分支 dev，并追踪远端仓库 dev 分支

```
$ git checkout dev
```

上述命令的执行结果如下

```

Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-real
time2023/gmall2023-realtime (master)
$ git checkout dev
Switched to a new branch 'dev'
branch 'dev' set up to track 'gmall2023-realtime/dev'.
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-real
time2023/gmall2023-realtime (dev)
$ |

```

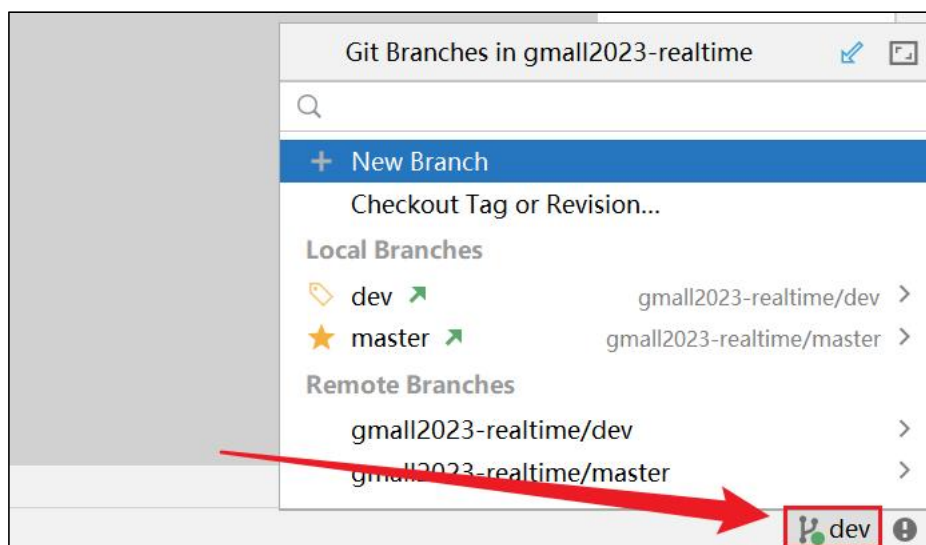
上述命令的作用是：

- ① 如果本地仓库已存在 dev 分支，则切换至该分支；
- ② 如果本地仓库不存在，且远端仓库存在 dev 分支，则在本地创建同名分支，并设置本地的 dev 分支追踪远程的 dev 分支；
- ③ 如果本地和远端都不存在 dev 分支，则会抛出异常。此时本地和远端都不存在分支 dev111，如果执行 git checkout dev111，则抛出异常，如下：



```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realtime2023/gmall2023-realtime (dev)
$ git checkout dev111
error: pathspec 'dev111' did not match any file(s) known to git
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realtime2023/gmall2023-realtime (dev)
$
```

### 10.4.3 在 IDEA 查看分支是否切换

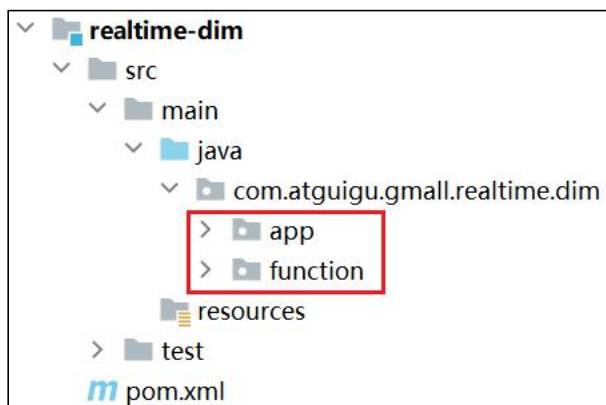


如上图所示，在命令行切换至 dev 后，IDEA 已同步更新。

我们的开发都在本地的 dev 分支完成，每完成一个模块的开发就提交至远端 dev 分支，管理员会将 dev 分支合并至某个 release 分支和 master 分支，本地的 master 分支只需要和远端的 master 分支保持一致即可，我们不会将本地 master 的内容提交至远端 master 分支（通常没有权限）。

## 10.5 package 设计

在 realtime-dim 包下创建以下包结构。



### 1) app 包

包含 DIM 层维度分流应用主程序代码。

### 2) function 包

包含维度分流用到的函数类代码。

## 10.6 App 基类设计

### 10.6.1 模板方法设计模式

#### 1) 定义

在父类中定义完成某一个功能的核心算法骨架，具体的实现可以延迟到子类中完成。模板方法类一定是抽象类，里面有一套具体的实现流程（可以是抽象方法也可以是普通方法）。这些方法可能由上层模板继承而来。

#### 2) 优点

在不改变父类核心算法骨架的前提下，每一个子类都可以有不同的实现。我们只需要关注具体方法的实现逻辑而不必在实现流程上分心。

### 10.6.2 基类设计思路

Flink Job 的处理流程大致可以分为以下几步：

- (1) 初始化流处理环境，配置检查点，从 Kafka 中读取目标主题数据
- (2) 执行核心处理逻辑
- (3) 执行

其中，所有 Job 的第一步和第三步基本相同，我们可以定义基类，将这两步交给基类完成。定义抽象方法，用于实现核心处理逻辑，子类只需要重写该方法即可实现功能。省去了大量的重复代码，且不必关心整体的处理流程。

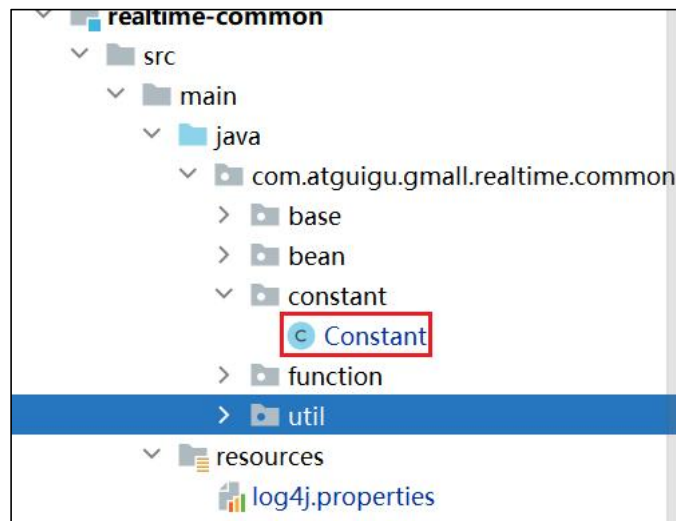
### 10.6.3 代码实现

#### 1) 创建 Constant 常量类

本项目会用到大量的配置信息，如主题名、Kafka 集群地址等，为了减少冗余代码，保证一致性，将这些配置信息统一抽取到常量类中。

在 `com.atguigu.gmall.realtime.common` 模块的包下创建常量类 `Constant`





写入以下内容

```
package com.atguigu.gmall.realtime.common.constant;

public class Constant {
    public static final String KAFKA_BROKERS =
        "hadoop102:9092,hadoop103:9092,hadoop104:9092";

    public static final String TOPIC_DB = "topic_db";
    public static final String TOPIC_LOG = "topic_log";

    public static final String MYSQL_HOST = "hadoop102";
    public static final int MYSQL_PORT = 3306;
    public static final String MYSQL_USER_NAME = "root";
    public static final String MYSQL_PASSWORD = "000000";
    public static final String HBASE_NAMESPACE = "gmall";

    public static final String MYSQL_DRIVER =
        "com.mysql.cj.jdbc.Driver";
    public static final String MYSQL_URL =
        "jdbc:mysql://hadoop102:3306?useSSL=false";

    public static final String TOPIC_DWD_TRAFFIC_START =
        "dwd_traffic_start";
    public static final String TOPIC_DWD_TRAFFIC_ERR =
        "dwd_traffic_err";
    public static final String TOPIC_DWD_TRAFFIC_PAGE =
        "dwd_traffic_page";
    public static final String TOPIC_DWD_TRAFFIC_ACTION =
        "dwd_traffic_action";
    public static final String TOPIC_DWD_TRAFFIC_DISPLAY =
```

```
"dwd_traffic_display";

    public static final String TOPIC_DWD_INTERACTION_COMMENT_INFO =
"dwd_interaction_comment_info";
    public static final String TOPIC_DWD_TRADE_CART_ADD =
"dwd_trade_cart_add";

    public static final String TOPIC_DWD_TRADE_ORDER_DETAIL =
"dwd_trade_order_detail";

    public static final String TOPIC_DWD_TRADE_ORDER_CANCEL =
"dwd_trade_order_cancel";

    public static final String TOPIC_DWD_TRADE_ORDER_PAYMENT_SUCCESS
= "dwd_trade_order_payment_success";
    public static final String TOPIC_DWD_TRADE_ORDER_REFUND =
"dwd_trade_order_refund";

    public static final String
TOPIC_DWD_TRADE_REFUND_PAYMENT_SUCCESS =
"dwd_trade_refund_payment_success";

    public static final String TOPIC_DWD_USER_REGISTER =
"dwd_user_register";
}
```

## 2) 创建 FlinkSourceUtil 工具类

### (1) 所需依赖

Flink 程序和 Kafka 交互需要引入以下依赖，已在 common 模块引入。

```
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-kafka</artifactId>
</dependency>
```

### (2) 代码实现

和 Kafka 交互要用到 Flink 提供的 KafkaSource 及 KafkaSink，Flink-1.16 开始，原先用于交互的 FlinkKafkaConsumer、FlinkKafkaProducer 被标记为过时，不推荐使用。为了提高模板代码的复用性，将 KafkaSource 的构建封装到 FlinkSourceUtil 工具类中。

在 common 模块的 com.atguigu.gmall.realtime.common.util 包下创建 FlinkSourceUtil，写入以下内容。

```
package com.atguigu.gmall.realtime.common.util;

import com.atguigu.gmall.realtime.common.constant.Constant;
import
org.apache.flink.api.common.serialization.DeserializationSchema;
import org.apache.flink.api.common.typeinfo.TypeInformation;
import org.apache.flink.api.common.typeinfo.Types;
import org.apache.flink.connector.kafka.source.KafkaSource;
import
org.apache.flink.connector.kafka.source.enumerator.initializer.OffsetsInitializer;

import java.io.IOException;
import java.nio.charset.StandardCharsets;

public class FlinkSourceUtil {
    public static KafkaSource<String> getKafkaSource(String groupId,
                                                    String topic) {
        return KafkaSource.<String>builder()
            .setBootstrapServers(Constant.KAFKA_BROKERS)
            .setGroupId(groupId)
            .setTopics(topic)
            .setStartingOffsets(OffsetsInitializer.latest())
            .setValueOnlyDeserializer(new
DeserializationSchema<String>() {
                @Override
                public String deserialize(byte[] message) throws
IOException {
                    if (message != null) {
                        return new String(message,
StandardCharsets.UTF_8);
                    }
                    return null;
                }

                @Override
                public boolean isEndOfStream(String nextElement) {
                    return false;
                }

                @Override
                public TypeInformation<String> getProducedType() {
                    return Types.STRING;
                }
            })
    }
}
```

```
        })
        .build();
    }
}
```

### 3) 在 base 包下创建基类 BaseApp, 编写以下内容

```
package com.atguigu.gmall.realtime.common.base;

import com.atguigu.gmall.realtime.common.util.FlinkSourceUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.connector.kafka.source.KafkaSource;
import org.apache.flink.runtime.state.hashmap.HashMapStateBackend;
import org.apache.flink.streaming.api.CheckpointingMode;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

import static org.apache.flink.streaming.api.environment.CheckpointConfig.ExternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION;

public abstract class BaseApp {
    public abstract void handle(StreamExecutionEnvironment env,
                                DataStreamSource<String> stream);

    public void start(int port, int parallelism, String ckAndGroupId,
String topic) {
        // 1. 环境准备
        // 1.1 设置操作 Hadoop 的用户名为 Hadoop 超级用户 atguigu
        System.setProperty("HADOOP_USER_NAME", "atguigu");

        // 1.2 获取流处理环境, 并指定本地测试时启动 WebUI 所绑定的端口
        Configuration conf = new Configuration();
        conf.setInteger("rest.port", port);
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment(conf);

        // 1.3 设置并行度
        env.setParallelism(parallelism);

        // 1.4 状态后端及检查点相关配置
        // 1.4.1 设置状态后端
```

```
env.setStateBackend(new HashMapStateBackend());

// 1.4.2 开启 checkpoint
env.enableCheckpointing(5000);
// 1.4.3 设置 checkpoint 模式：精准一次

env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.
EXACTLY_ONCE);
// 1.4.4 checkpoint 存储

env.getCheckpointConfig().setCheckpointStorage("hdfs://hadoop102:
8020/gmall2023/stream/" + ckAndGroupId);
// 1.4.5 checkpoint 并发数
env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);
// 1.4.6 checkpoint 之间的最小间隔

env.getCheckpointConfig().setMinPauseBetweenCheckpoints(5000);
// 1.4.7 checkpoint 的超时时间
env.getCheckpointConfig().setCheckpointTimeout(10000);
// 1.4.8 job 取消时 checkpoint 保留策略

env.getCheckpointConfig().setExternalizedCheckpointCleanup(RETAIN
_ON_CANCELLATION);

// 1.5 从 Kafka 目标主题读取数据，封装为流
KafkaSource<String> source =
FlinkSourceUtil.getKafkaSource(ckAndGroupId, topic);

DataStreamSource<String> stream = env.fromSource(source,
WatermarkStrategy.noWatermarks(), "kafka_source");

// 2. 执行具体的处理逻辑
handle(env, stream);

// 3. 执行 Job
try {
    env.execute();
} catch (Exception e) {
    e.printStackTrace();
}
}
```

## 10.6.4 基类 start()方法参数说明

基类提供了 start()方法，子类只需要调用该方法即可启动数据处理，该方法有四个形参，如下：

### 1) port

测试环境下启动本地 WebUI 的端口，为了避免本地端口冲突，做出以下规定：

- (1) DIM 层维度分流应用使用 10001 端口
- (2) DWD 层应用程序按照在本文档中出现的先后顺序，端口从 10011 开始，自增 1
- (3) DWS 层应用程序按照在本文档中出现的先后顺序，端口从 10021 开始，自增 1

### 2) parallelism

并行度，本项目统一设置为 4。

### 3) ckAndGroupId

消费 Kafka 主题时的消费者组 ID 和检查点路径的最后一级目录名称，二者取值相同，为 Job 主程序类名的下划线命名形式。如 DimApp 的该参数取值为 dim\_app。

### 4) topic

消费的 Kafka 主题名称。

## 10.7 代码实现

### 10.7.1 接收 Kafka 数据，过滤空值数据

#### 1) 知识储备

@Slf4j 注解：Lombok 提供的注解，用于在类中自动生成 Slf4j 的日志对象，添加在类名上方。当我们在一个类上使用该注解，Lombok 会自动为该类注入一个名为 log 的静态日志实例，用于日志记录。

具体来说，在 DimApp 上方添加 @Slf4j 注解后，相当于在类中加入了如下的代码：

```
private static final org.slf4j.Logger log =
org.slf4j.LoggerFactory.getLogger(DimApp.class);
```

#### 2) 所需依赖

此处将数据解析为 JSON 对象，需要用到 fastjson，依赖如下。

```
<dependency>
  <groupId>com.alibaba</groupId>
  <artifactId>fastjson</artifactId>
</dependency>
```

已在 common 模块引入。

需要用到 Lombok，依赖如下。

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.20</version>
  <scope>provided</scope>
</dependency>
```

已在父工程的 POM 文件中引入。

### 3) 代码实现

在 app 包下创建维度分流主程序 DimApp，使其继承 BaseApp，重写 handle 方法。在 DimApp 的 main 方法中创建当前类的实例，调用 start() 方法，即可启动数据处理流程。本节数据来源于 Kafka 的 topic\_db 主题，只须将主题名称传递给 start() 方法的第四个形参，基类就会帮助我们该主题读取数据，封装为流。

代码如下。

```
package com.atguigu.gmall.realtime.dim.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import lombok.extern.slf4j.Slf4j;
import org.apache.flink.api.common.functions.FilterFunction;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import org.apache.flink.streaming.api.datastream.SingleOutputStreamOperator;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

@Slf4j
public class DimApp extends BaseApp {
    public static void main(String[] args) {
        new DimApp().start(10001,
            4,
            "dim_app",
            Constant.TOPIC_DB
        );
    }
}
```



```
}

@Override
public void handle(StreamExecutionEnvironment env,
                  DataStreamSource<String> stream) {
    // 1. 对消费的数据, 做数据清洗
    SingleOutputStreamOperator<JSONObject> etlStream =
etl(stream);
}

private SingleOutputStreamOperator<JSONObject>
etl(DataStreamSource<String> stream) {
    return stream
        .filter(new FilterFunction<String>() {
            @Override
            public boolean filter(String value) {
                try {
                    JSONObject jsonObj =
JSON.parseObject(value);
                    String db = jsonObj.getString("database");
                    String type = jsonObj.getString("type");
                    String data = jsonObj.getString("data");

                    return "gmall".equals(db)
                        && ("insert".equals(type)
                            || "update".equals(type)
                            || "delete".equals(type)
                            || "bootstrap-insert".equals(type))
                        && data != null
                        && data.length() > 2;

                } catch (Exception e) {
                    log.warn("不是正确的 json 格式的数据: " +
value);

                    return false;
                }
            }
        })
        .map(JSON::parseObject);
}
}
```

## 10.7.2 读取配置表数据

## 1) 所需依赖

编写 Flink 业务代码必然要用到 Flink 运行环境相关的依赖，集群 Flink lib 目录下提供了这部分依赖，此处不列出，下文同理，不再赘述。

此处要引入 FlinkCDC 相关依赖，如下。

```
<dependency>
  <groupId>com.ververica</groupId>
  <artifactId>flink-connector-mysql-cdc</artifactId>
</dependency>
```

已在 realtime-common 的 POM 文件中引入，此处不必执行任何修改。

## 2) 创建配置表实体类

在 common 模块的 com.atguigu.gmall.realtime.common.bean 包下创建 TableProcessDim 实体类，如下

```
package com.atguigu.gmall.realtime.common.bean;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class TableProcessDim {
    // 来源表名
    String sourceTable;

    // 目标表名
    String sinkTable;

    // 输出字段
    String sinkColumns;

    // 数据到 hbase 的列族
    String sinkFamily;

    // sink 到 hbase 的时候的主键字段
    String sinkRowKey;

    // 配置表操作类型
    String op;
```

```
}
```

### 3) 读取配置表数据

(1) 在 DimApp 中创建 readTableProcess 方法

```
private SingleOutputStreamOperator<TableProcessDim>
readTableProcess(StreamExecutionEnvironment env) {
    // useSSL=false
    Properties props = new Properties();
    props.setProperty("useSSL", "false");
    props.setProperty("allowPublicKeyRetrieval", "true");
    MySqlSource<String> mySqlSource = MySqlSource.<String>builder()
        .hostname(Constant.MYSQL_HOST)
        .port(Constant.MYSQL_PORT)
        .databaseList("gmall2023_config") // set captured database,
        // If you need to synchronize the whole database, Please set tableList
        // to ".*".
        .tableList("gmall2023_config.table_process_dim") // set
        // captured table
        .username(Constant.MYSQL_USER_NAME)
        .password(Constant.MYSQL_PASSWORD)
        .jdbcProperties(props)
        .deserializer(new JsonDebeziumDeserializationSchema()) //
        // converts SourceRecord to JSON String
        .startupOptions(StartupOptions.initial()) // 默认值:
        // initial 第一次启动读取所有数据(快照), 然后通过 binlog 实时监控变化数据
        .build();

    return env
        .fromSource(mySqlSource,
            WatermarkStrategy.noWatermarks(), "cdc-source")
        .setParallelism(1) // 并行度设置为 1
        .map(new MapFunction<String, TableProcessDim>() {
            @Override
            public TableProcessDim map(String value) throws
            Exception {
                JSONObject obj = JSON.parseObject(value);
                String op = obj.getString("op");
                TableProcessDim tableProcessDim;
                if ("d".equals(op)) {
                    tableProcessDim = obj.getJSONObject("before",
                        TableProcessDim.class);
                } else {
                    tableProcessDim = obj.getJSONObject("after",
                        TableProcessDim.class);
                }
            }
        })
```

```
        }  
        tableProcessDim.setOp(op);  
  
        return tableProcessDim;  
    }  
})  
.setParallelism(1);  
}
```

(2) 在 DimApp 的 main 方法中调用上述方法

```
// 2. 通过 flink cdc 读取配置表的数据  
SingleOutputStreamOperator<TableProcessDim> configStream =  
readTableProcess(env);
```

#### 4) 注意

配置流的并行度必须为 1，否则在配置信息变更时会出现乱序，进而导致一致性问题。比如某条配置信息由 A 变为 B，再由 B 变为 C，当配置流并行度不为 1，且第一次变更和第二次变更日志进入不同分区时，第二次变更可能先到达，下游获取的配置信息会先由 A 变为 C，再由 C 变为 B，与上游不一致。

### 10.7.3 在 HBase 中建表

#### 1) 定义 HBaseUtil

(1) 所需依赖

与 HBase 交互需要引入如下依赖。

```
<dependency>  
  <groupId>org.apache.hbase</groupId>  
  <artifactId>hbase-client</artifactId>  
</dependency>  
  
<dependency>  
  <groupId>org.apache.hadoop</groupId>  
  <artifactId>hadoop-auth</artifactId>  
  <scope>provided</scope>  
  <exclusions>  
    <exclusion>  
      <groupId>org.slf4j</groupId>  
      <artifactId>slf4j-reload4j</artifactId>  
    </exclusion>  
  </exclusions>  
</dependency>
```

hbase-client 及 flink-connector-hbase-2.2 已在 common 模块引入, hadoop-auth 已在父工程引入。

## (2) 实现

在 common 模块的 util 包下创建工具类 HBaseUtil, 在其中创建 getHBaseConnection、closeHBaseConn、createHBaseTable、dropHBaseTable 方法, 如下

```
package com.atguigu.gmall.realtime.common.util;

import lombok.extern.slf4j.Slf4j;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.hbase.TableName;
import org.apache.hadoop.hbase.client.*;

import java.io.IOException;

@Slf4j
public class HBaseUtil {

    public static Connection getHBaseConnection() throws IOException {
        Configuration conf = new Configuration();
        conf.set("hbase.zookeeper.quorum", "hadoop102");
        conf.set("hbase.zookeeper.property.clientPort", "2181");

        return ConnectionFactory.createConnection(conf);
    }

    public static void closeHBaseConn(Connection hbaseConn) throws IOException {
        if (hbaseConn != null && !hbaseConn.isClosed()) {
            hbaseConn.close();
        }
    }

    public static void createHBaseTable(Connection hbaseConn,
                                         String nameSpace,
                                         String table,
                                         String family) throws IOException {
        Admin admin = hbaseConn.getAdmin();
        TableName tableName = TableName.valueOf(nameSpace, table);
        // 判断要建的表是否存在
    }
}
```

```
        if (admin.tableExists(tableName)) {
            return;
        }
        // 列族描述器
        ColumnFamilyDescriptor cfDesc =
ColumnFamilyDescriptorBuilder.of(family);
        // 表的描述器
        TableDescriptor desc =
TableDescriptorBuilder.newBuilder(tableName)
            .setColumnFamily(cfDesc) // 给表设置列族
            .build();
        admin.createTable(desc);
        admin.close();
        log.info(nameSpace + " " + table + " 建表成功");
    }

    public static void dropHBaseTable(Connection hbaseConn,
                                       String nameSpace,
                                       String table) throws IOException {

        Admin admin = hbaseConn.getAdmin();
        TableName tableName = TableName.valueOf(nameSpace, table);
        if (admin.tableExists(tableName)) {
            admin.disableTable(tableName);
            admin.deleteTable(tableName);
        }
        admin.close();
        log.info(nameSpace + " " + table + " 删除成功");
    }
}
```

## 2) 在 DimApp 中定义 createHBaseTable 方法

```
private SingleOutputStreamOperator<TableProcessDim>
createHBaseTable(
    SingleOutputStreamOperator<TableProcessDim> tpStream) {
    return tpStream.map(new RichMapFunction<TableProcessDim,
TableProcessDim>() {

        private Connection hbaseConn;

        @Override
        public void open(Configuration parameters) throws
Exception {
```

```
// 1. 获取到 HBase 的连接
hbaseConn = HBaseUtil.getHBaseConnection();
}

@Override
public void close() throws Exception {
    // 2. 关闭连接
    HBaseUtil.closeHBaseConn(hbaseConn);
}

@Override
public TableProcessDim map(TableProcessDim
tableProcessDim) throws Exception {
    String op = tableProcessDim.getOp();
    if ("d".equals(op)) {
        dropTable(tableProcessDim);
    } else if ("r".equals(op) || "c".equals(op)) {
        createTable(tableProcessDim);
    } else { // u 应该先删除表,再建表. 表的历史数据需要重新同步
        dropTable(tableProcessDim);
        createTable(tableProcessDim);
    }
    return tableProcessDim;
}

private void createTable(TableProcessDim
tableProcessDim) throws IOException {
    // namespace
    HBaseUtil.createHBaseTable(hbaseConn,
        Constant.HBASE_NAMESPACE,
        tableProcessDim.getSinkTable(),
        tableProcessDim.getSinkFamily());
}

private void dropTable(TableProcessDim tableProcessDim)
throws IOException {
    HBaseUtil.dropHBaseTable(hbaseConn,
        Constant.HBASE_NAMESPACE, tableProcessDim.getSinkTable());
}

})
.setParallelism(1);
}
```



### 3) 在 main 方法中调用上述方法

```
// 3. 根据配置表的数据, 在 HBase 中建表
configStream = createHBaseTable(configStream);
```

### 4) 注意

与上文同理, 配置流并行度必须为 1。

## 10.7.4 将主流和配置流进行关联, 并对关联后的流中数据进行处理

### 1) 定义 JdbcUtil

#### (1) 所需依赖

此处要通过反射为对象的属性赋值, org.apache.commons.beanutils.BeanUtils 提供了相关工具类, 依赖如下。

```
<dependency>
  <groupId>commons-beanutils</groupId>
  <artifactId>commons-beanutils</artifactId>
</dependency>
```

已在 common 模块引入。

#### (2) 实现

在 common 模块的 util 包下创建工具类 JdbcUtil, 创建 getMysqlConnection、queryList 和 closeConnection 方法, 如下。

```
package com.atguigu.gmall.realtime.common.util;

import org.apache.commons.beanutils.BeanUtils;
import org.apache.flink.shaded.guava30.com.google.common.base.CaseFormat;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;

import static com.atguigu.gmall.realtime.common.constant.Constant.*;

public class JdbcUtil {

    public static Connection getMysqlConnection() throws
        ClassNotFoundException, SQLException {
        // 获取 jdbc 连接
```

```
// 1. 加载驱动
Class.forName(MYSQL_DRIVER);
return DriverManager.getConnection(MYSQL_URL,
MYSQL_USER_NAME, MYSQL_PASSWORD);
}

/**
 * 执行一个查询语句，把查询到的结果封装 T 类型的对象中。
 *
 * @param conn      mysql 连接
 * @param querySql  查询的 sql 语句：必须是查询语句
 * @param tClass    T 类
 * @param <T>       每行封装的类型
 * @return          查询到的多行结果
 */
public static <T> List<T> queryList(Connection conn,
                                     String querySql,
                                     Class<T> tClass,
                                     boolean... isUnderlineToCamel)
throws Exception {
    boolean defaultIsUToC = false; // 默认不执行下划线转驼峰

    if (isUnderlineToCamel.length > 0) {
        defaultIsUToC = isUnderlineToCamel[0];
    }

    List<T> result = new ArrayList<>();
    // 1. 预编译
    PreparedStatement preparedStatement =
conn.prepareStatement(querySql);
    // 2. 执行查询，获得结果集
    ResultSet resultSet = preparedStatement.executeQuery();
    ResultSetMetaData metaData = resultSet.getMetaData();
    // 3. 解析结果集，把数据封装到一个 List 集合中
    while (resultSet.next()) {
        // 变量到一行数据，把这个行数据封装到一个 T 类型的对象中
        T t = tClass.newInstance(); // 使用反射创建一个 T 类型的对象
        // 遍历这一行的每一列数据
        for (int i = 1; i <= metaData.getColumnCount(); i++) {
            // 获取列名
            // 获取列值
            String name = metaData.getColumnLabel(i);
            Object value = resultSet.getObject(name);
```

```
        if (defaultIsUToC) { // 需要下划线转驼峰: a_a => aA
a_aaaa_aa => aAaaaAa
            name =
CaseFormat.LOWER_UNDERSCORE.to(CaseFormat.LOWER_CAMEL, name);
        }

        // t.name=value
        BeanUtils.setProperty(t, name, value);
    }
    result.add(t);
}
return result;
}

public static void closeConnection(Connection conn) throws
SQLException {
    if (conn != null && !conn.isClosed()) {
        conn.close();
    }
}
}
```

## 2) 在 DimApp 中定义 connect 方法

### (1) 广播配置流

定义广播状态描述器，将配置流处理为广播流

### (2) 用主流关联广播流

### (3) 对关联后的流中数据做处理

定义 BroadcastProcessFunction 的匿名内部类完成处理，我们需要重写三个方法：open、processElement 和 processBroadcastElement，每个方法的任务如下：

#### ① processBroadcastElement

处理广播流数据，将配置信息记录在广播状态中。

#### ② processElement

根据广播状态中的配置信息对流中数据做筛选和处理。此处要将主流数据的操作类型保留下来发送给下游，下游写入 HBase 之前会按照数据的操作类型做相应处理。

#### ③ open 方法

用于完成配置信息的初始化操作。

我们是按照配置信息对主流数据做筛选的,如果可以在广播状态中找到相应的配置则主流数据可以保留,否则舍弃。若主流先于配置流到达,此时广播状态尚未完成配置加载,某些本应被保留的主流数据找不到相应的配置信息,被舍弃,从而导致数据丢失。

首次上线,维度分流应用启动时我们会通过 **Maxwell-Bootstrap** 对维度信息做全表扫描,测试环境下,只要在分流应用启动后再执行维度数据首日全量同步脚本即可避免数据丢失。但是生产环境中,通常是把很多应用的提交命令放在脚本里一键执行的,无法保证维度信息的全表扫描在动态分流应用启动后执行。此外,故障重启时,主流和广播流数据的加载顺序同样无法控制。

针对上述问题,考虑对配置表信息进行预加载。只要在 **Flink** 应用初始化时将配置信息预加载到内存,主流获取不到对应状态时读取内存中的配置信息,就可以避免数据丢失。初始化逻辑写在 **open** 方法中。

```
private SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDim>> connect(
    SingleOutputStreamOperator<JSONObject> dataStream,
    SingleOutputStreamOperator<TableProcessDim> configStream) {

    // 1. 把配置流做成广播流
    // key: 表名   user_info
    // value: TableProcess
    MapStateDescriptor<String, TableProcessDim> mapStateDescriptor =
new MapStateDescriptor<String, TableProcessDim>("table_process_dim",
String.class, TableProcessDim.class);
    BroadcastStream<TableProcessDim> broadcastStream =
configStream.broadcast(mapStateDescriptor);
    // 2. 数据流去 connect 广播流
    return dataStream
        .connect(broadcastStream)
        .process(new BroadcastProcessFunction<JSONObject,
TableProcessDim, Tuple2<JSONObject, TableProcessDim>>() {

            private HashMap<String, TableProcessDim> map;

            @Override
            public void open(Configuration parameters) throws
Exception {
                // open 中没有办法访问状态!!!
                map = new HashMap<>();
                // 1. 去 mysql 中查询 table_process 表所有数据
```

```
        java.sql.Connection mysqlConn =
JdbcUtil.getMysqlConnection();

        List<TableProcessDim> tableProcessDimList =
JdbcUtil.queryList(mysqlConn,
                    "select * from
gmall2023_config.table_process_dim",
                    TableProcessDim.class,
                    true
                );

        for (TableProcessDim tableProcessDim :
tableProcessDimList) {
            String key = tableProcessDim.getSourceTable();
            map.put(key, tableProcessDim);
        }
        JdbcUtil.closeConnection(mysqlConn);
    }

    // 2. 处理广播流中的数据：把配置信息存入到广播状态中
    @Override
    public void processBroadcastElement(TableProcessDim
tableProcessDim,

                                   Context context,

Collector<Tuple2<JSONObject, TableProcessDim>> out) throws Exception
    {
        BroadcastState<String, TableProcessDim> state =
context.getBroadcastState(mapStateDescriptor);
        String key = tableProcessDim.getSourceTable();

        if ("d".equals(tableProcessDim.getOp())) {
            // 删除状态
            state.remove(key);
            // map 中的配置也要删除
            map.remove(key);
        } else {
            // 更新或者添加状态
            state.put(key, tableProcessDim);
        }
    }

    // 3. 处理数据流中的数据：从广播状态中读取配置信息
    @Override
    public void processElement(JSONObject jsonObj,
```

```
        ReadOnlyContext context,
        Collector<Tuple2<JSONObject,
TableProcessDim>> out) throws Exception {
    ReadOnlyBroadcastState<String, TableProcessDim>
state = context.getBroadcastState(mapStateDescriptor);
    String key = jsonObj.getString("table");
    TableProcessDim tableProcessDim = state.get(key);

    if (tableProcessDim == null) { // 如果状态中没有查到,
则去 map 中查找
        tableProcessDim = map.get(key);
        if (tableProcessDim != null) {
            log.info("在 map 中查找到 " + key);
        }
    } else {
        log.info("在 状态 中查找到 " + key);
    }
    if (tableProcessDim != null) { // 这条数据找到了对应
的配置信息
        JSONObject data =
jsonObj.getJSONObject("data");
        data.put("op_type", jsonObj.getString("type"));
// 后期需要
        out.collect(Tuple2.of(data, tableProcessDim));
    }
}
});
}
```

### 3) 在 main 方法中调用上述方法

```
// 4. 主流 connect 配置流, 并对关联后的流做处理
SingleOutputStreamOperator<Tuple2<JSONObject, TableProcessDim>>
dimDataToTpStream = connect(etlStream, configStream);
```

## 10.7.5 删除不需要的字段

### 1) 在 DimApp 中定义 deleteNotNeedColumns 方法

```
private SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDim>> deleteNotNeedColumns (
    SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDim>> dimDataToTpStream) {
    return dimDataToTpStream
        .map(new MapFunction<Tuple2<JSONObject, TableProcessDim>,
Tuple2<JSONObject, TableProcessDim>>() {
            @Override
```

```

        public Tuple2<JSONObject, TableProcessDim>
map(Tuple2<JSONObject, TableProcessDim> dataWithConfig) {
            JSONObject data = dataWithConfig.f0;
            List<String> columns = new
ArrayList<>(Arrays.asList(dataWithConfig.f1.getSinkColumns().spli
t(", ")));

            columns.add("op_type");

            data.keySet().removeIf(key
-> !columns.contains(key));
            return dataWithConfig;
        }
    });
}

```

## 2) 在 main 方法中调用上述方法

```

// 5. 删除不需要的字段
SingleOutputStreamOperator<Tuple2<JSONObject, TableProcessDim>>
resultStream = deleteNotNeedColumns(dimDataToTpStream);

```

## 10.7.6 写出到 HBase 目标表

### 1) 自定义函数 HBaseSinkFunction

#### (1) 思路分析

自定义 HBaseSinkFunction 继承 RichSinkFunction，定义 putDim 和 delDim 方法，用于向 HBase 中写入数据和删除数据。重写 open、close 和 invoke 方法。open 和 close 方法分别负责 HBase 连接的获取和释放。invoke 方法用于对主流数据逐条处理，当数据到来时，获取其操作类型，若为 delete，则调用 delDim 方法删除 HBase 中的维度数据，否则调用 putDim 方法写入即可。

#### (2) 代码实现

##### ① 在 HBaseUtil 中补充 putRow 和 delRow 方法

```

/**
 * 用于向 HBase 目标表写入数据
 * @param conn HBase 连接对象
 * @param nameSpace HBase 命名空间
 * @param table HBase 表名
 * @param rowKey 数据的 row_key
 * @param family 数据所属的列族
 * @param data 待写出的数据
 * @throws IOException 可能抛出的异常
 */

```



```
public static void putRow(Connection conn,
                          String nameSpace,
                          String table,
                          String rowKey,
                          String family,
                          JSONObject data) throws IOException {
    // 1. 获取 table 对象
    TableName tableName = TableName.valueOf(nameSpace, table);
    Table t = conn.getTable(tableName);

    // 2. 创建 put 对象
    Put put = new Put(Bytes.toBytes(rowKey));
    // 3. 把每列放入 put 对象中
    for (String key : data.keySet()) {
        String value = data.getString(key);
        if (value != null) {
            put.addColumn(Bytes.toBytes(family), Bytes.toBytes(key),
Bytes.toBytes(value));
        }
    }
    // 4. 向 table 对象中 put 数据
    t.put(put);

    t.close();
}

/**
 * 根据 row_key 删除 HBase 目标表指定数据
 * @param conn HBase 连接对象
 * @param nameSpace HBase 命名空间
 * @param table HBase 表名
 * @param rowKey HBase row_key
 * @throws IOException 可能抛出的异常
 */
public static void delRow(Connection conn,
                          String nameSpace,
                          String table,
                          String rowKey) throws IOException {
    TableName tableName = TableName.valueOf(nameSpace, table);
    Table t = conn.getTable(tableName);
    Delete delete = new Delete(Bytes.toBytes(rowKey));
    // 删除整行
    t.delete(delete);
    t.close();
}
```

```
}
```

② 在 `realtime-dim` 模块的 `function` 包下创建 `HBaseSinkFunction`，写入以下内容

```
package com.atguigu.gmall.realtime.dim.function;

import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.bean.TableProcessDim;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.HBaseUtil;
import lombok.extern.slf4j.Slf4j;
import org.apache.flink.api.java.tuple.Tuple2;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.functions.sink.RichSinkFunction;
import org.apache.hadoop.hbase.client.Connection;

import java.io.IOException;

@Slf4j
public class HBaseSinkFunction extends
    RichSinkFunction<Tuple2<JSONObject, TableProcessDim>> {

    private Connection conn;

    @Override
    public void open(Configuration parameters) throws Exception {
        conn = HBaseUtil.getHBaseConnection();
    }

    @Override
    public void close() throws Exception {
        HBaseUtil.closeHBaseConn(conn);
    }

    @Override
    public void invoke(Tuple2<JSONObject, TableProcessDim>
        dataWithConfig,
                        Context context) throws Exception {
        // insert update delete bootstrap-insert
        JSONObject data = dataWithConfig.f0;
        String opType = data.getString("op_type");

        if ("delete".equals(opType)) {
            // 删除维度信息
        }
    }
}
```

```
        delDim(dataWithConfig);
    }else{
        // insert update 和 bootstrap-insert 的时候, 写入维度数据
        putDim(dataWithConfig);
    }
}

private void putDim(Tuple2<JSONObject, TableProcessDim>
dataWithConfig) throws IOException {
    JSONObject data = dataWithConfig.f0;
    TableProcessDim tableProcessDim = dataWithConfig.f1;

    String rowKey =
data.getString(tableProcessDim.getSinkRowKey());
    log.info("向 HBase 写入数据 dataWithConfig: " +
dataWithConfig);
    // data 中有多少 kv 就写多少列 - 1
    data.remove("op_type");
    HBaseUtil.putRow(conn,
        Constant.HBASE_NAMESPACE,
        tableProcessDim.getSinkTable(),
        rowKey,
        tableProcessDim.getSinkFamily(),
        data);
}

private void delDim(Tuple2<JSONObject, TableProcessDim>
dataWithConfig) throws IOException {
    JSONObject data = dataWithConfig.f0;
    TableProcessDim tableProcessDim = dataWithConfig.f1;

    String rowKey =
data.getString(tableProcessDim.getSinkRowKey());

    HBaseUtil.delRow(conn,
        Constant.HBASE_NAMESPACE,
        tableProcessDim.getSinkTable(),
        rowKey);
}
}
```

## 2) 在 DimApp 中定义 writeToHBase 方法

```
private void
```

```
writeToHBase(SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDim>> resultStream) {
    /*
    1. 有没有专门的 HBase 连接器
        没有
    2. sql 有专门的 HBase 连接器, 由于一次只能写到一个表中, 所以也不能把流转
    成表再写

    3. 自定义 sink
    */
    resultStream.addSink(new HBaseSinkFunction());
}
```

### 3) 在 main 方法中调用上述方法

```
// 6. 写出到 HBase 目标表
writeToHBase(resultStream);
```

## 10.8 测试

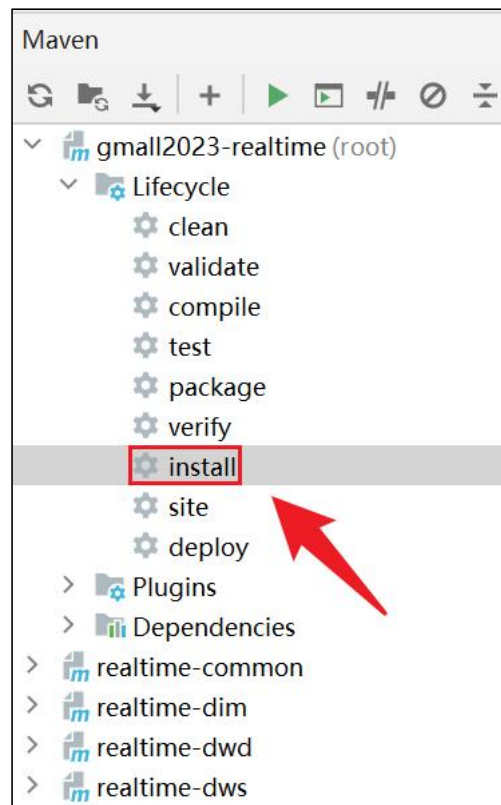
### 1) 启动 HDFS、ZK、Kafka、Maxwell、HBase

### 2) 在 HBase 中创建 namespace

```
hbase:002:0> create_namespace "gmall"
```

### 3) 运行 IDEA 中的 DimApp

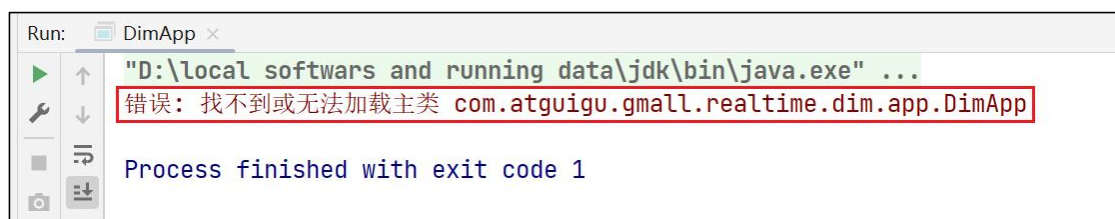
(1) install 父工程



后期要对某个子模块单独打包就必须先对父工程执行 `install` 操作，此外，`realtime-common` 模块作为业务模块的依赖，必须被 `install` 到本地仓库，业务模块才可以通  
过编译。

### (2) 首次启动 DimApp

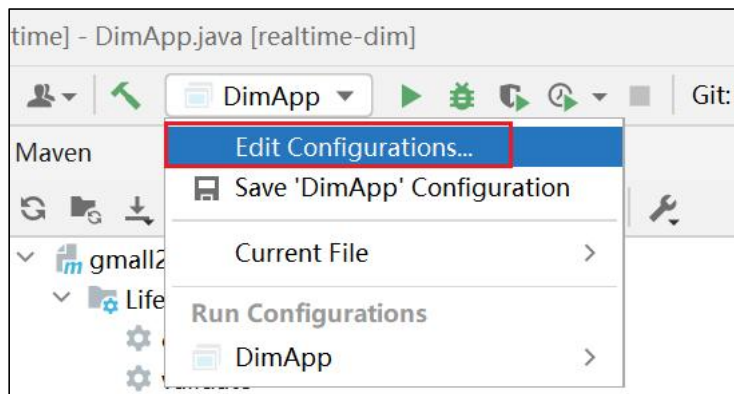
报错如下



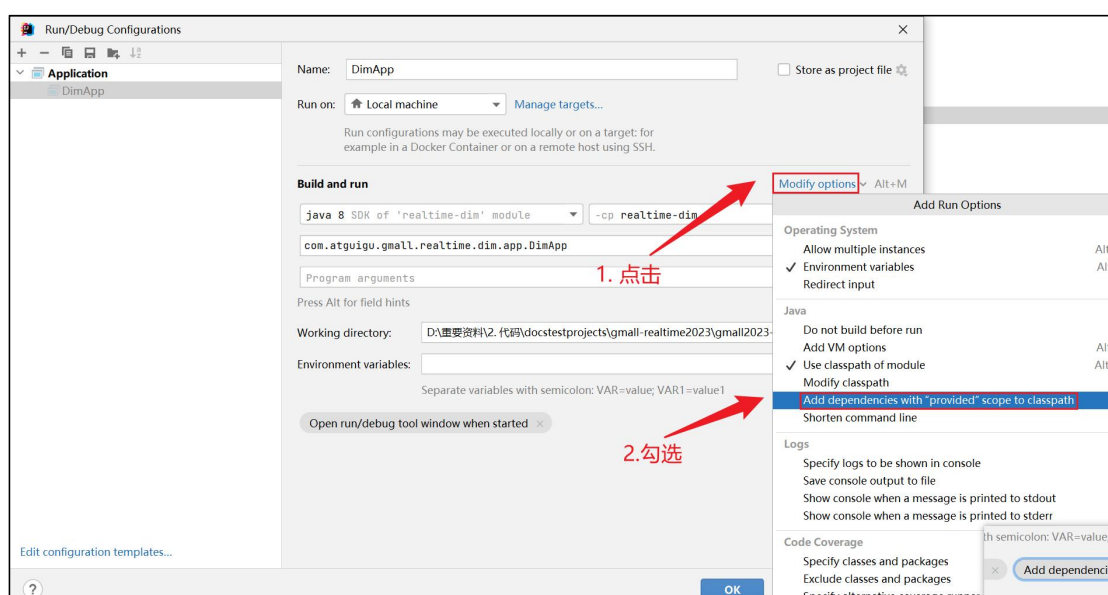
这是因为 `DimApp` 继承了 `BaseApp`，后者位于 `realtime-common` 模块下，`realtime-dim` 引  
入了 `common` 模块，但 `scope` 为 `provided`，在 IDEA 运行时，默认情况下 `ClassPath` 中不会包  
含 `scope` 为 `provided` 的依赖，因此找不到 `BaseApp` 的字节码文件，`DimApp` 自然也无法加载。

### (3) 修改 DimApp 的 Configuration

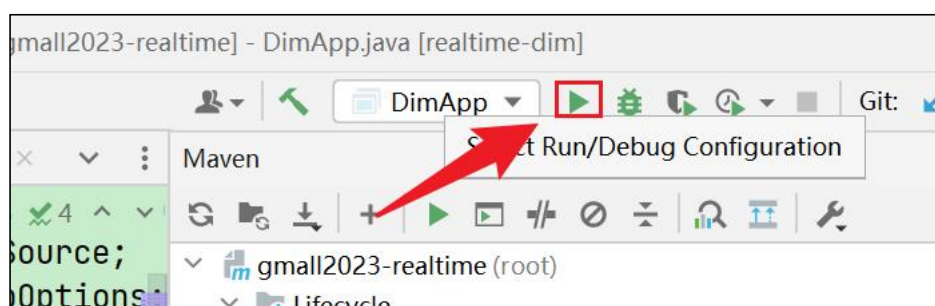
只需要修改 `DimApp` 的 `Application` 配置即可解决上述问题。



勾选: Add dependencies with "provided" scope to classpath



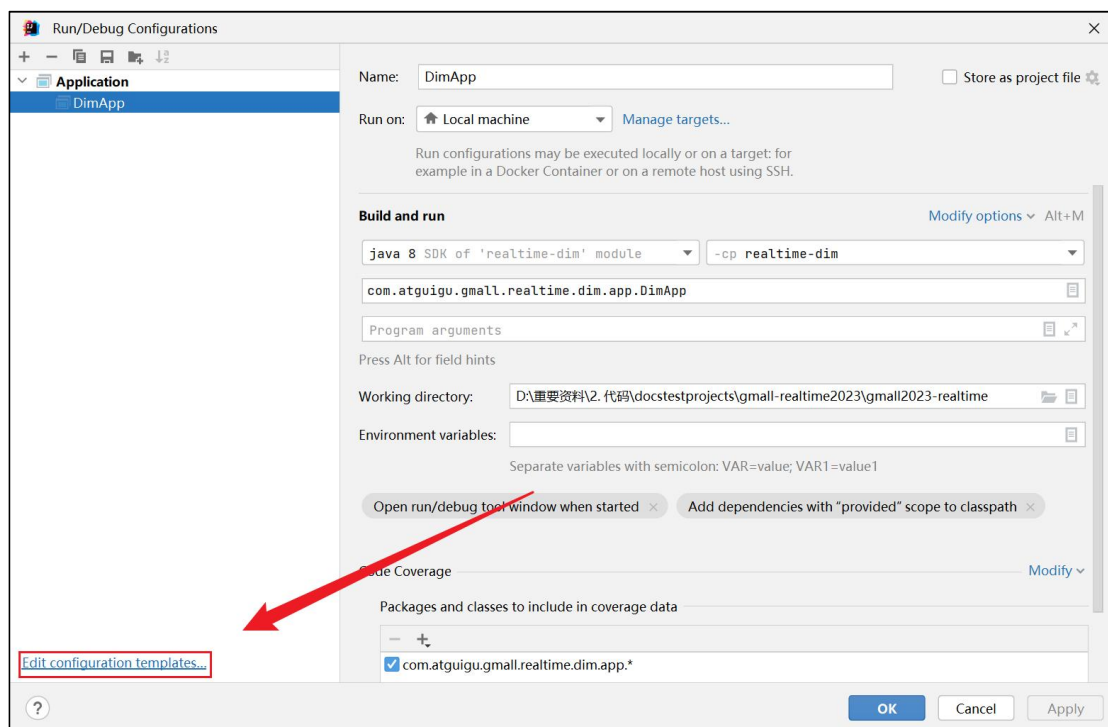
(4) 再次运行



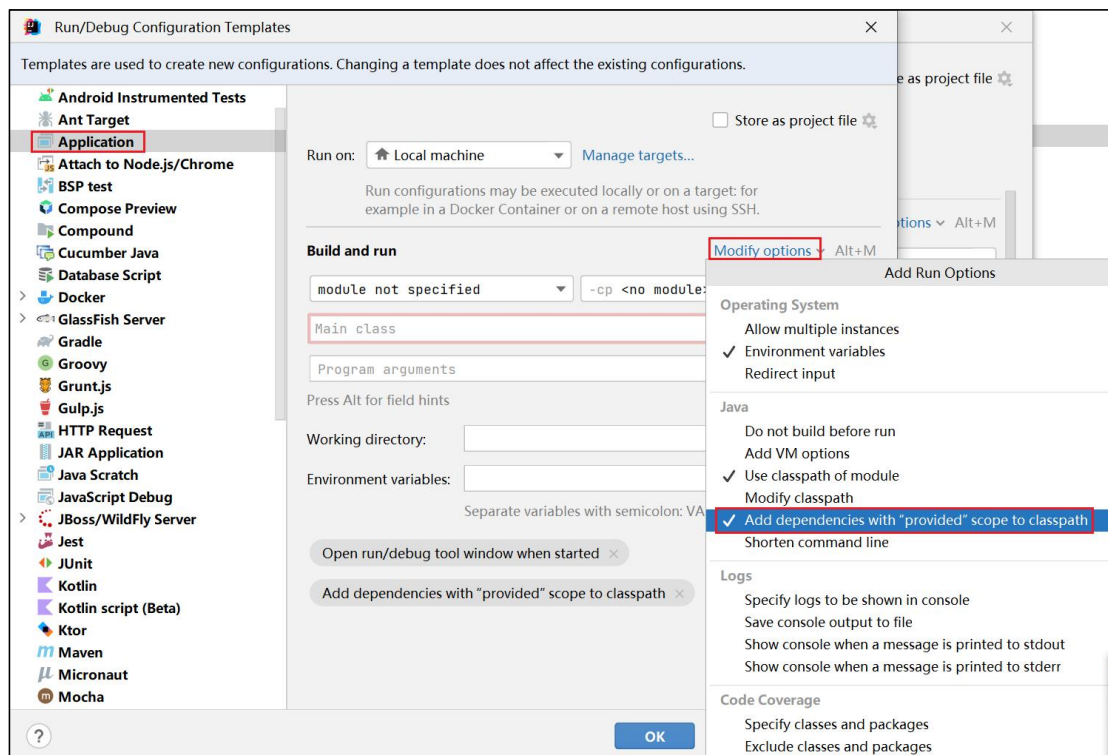
(5) 编辑 Application 的默认模板

本项目包含数十个应用程序，每个程序运行前都修改一次 Application 的配置，非常繁琐。可以编辑 Application 的默认模板，只要在模板中勾选 Add dependencies with "provided" scope to classpath，所有应用程序的运行配置都会自动勾选该配置。

① 编辑模板



② 选择 Application，勾选配置



4) 执行 mysql\_to\_kafka\_dim\_init.sh 脚本

```
[atguigu@hadoop102 ~]$ mysql_to_kafka_dim_init.sh all
```

5) 通过 HBase Shell 查看建表和写入情况

(1) 查看建表情况



```
hbase:038:0> list_namespace_tables "gmall"
```

```
hbase:038:0> list_namespace_tables "gmall"
TABLE
dim_activity_info
dim_activity_rule
dim_activity_sku
dim_base_category1
dim_base_category2
dim_base_category3
dim_base_dic
dim_base_province
dim_base_region
dim_base_trademark
dim_coupon_info
dim_coupon_range
dim_financial_sku_cost
dim_sku_info
dim_spu_info
dim_user_info
16 row(s)
Took 0.0291 seconds
=> ["dim_activity_info", "dim_activity_rule", "dim_activity_sku", "dim_b
"dim_base_province", "dim_base_region", "dim_base_trademark", "dim_coupo
spu_info", "dim_user_info"]
hbase:039:0>
```

## (2) 部分表数据

```
["FINANCIAL_LEASE_REALTIME:dim_business_partner", "FINANCIAL_LEASE_REALTIME:dim_department", "FINANCIAL_LEASE_REALTIME:dim_employee", "
LEASE_REALTIME:dim_industry", "MEDICAL_REALTIME:dim_dict", "MEDICAL_REALTIME:dim_doctor", "MEDICAL_REALTIME:dim_hospital", "MEDICAL_REALTI
icine", "MEDICAL_REALTIME:dim_patient", "MEDICAL_REALTIME:dim_user", "gmall:dim_activity_info", "gmall:dim_activity_rule", "gmall:dim_acti
"gmall:dim_base_category1", "gmall:dim_base_category2", "gmall:dim_base_category3", "gmall:dim_base_dic", "gmall:dim_base_province", "gma
e_region", "gmall:dim_base_trademark", "gmall:dim_coupon_info", "gmall:dim_coupon_range", "gmall:dim_financial_sku_cost", "gmall:dim_sku_i
ll:dim_spu_info", "gmall:dim_user_info"]
hbase:002:0> |scan "gmall:dim_user_info"
ROW COLUMN+CELL
1 column=info:birthday, timestamp=2023-10-19T11:36:05.255, value=1982-04-07
1 column=info:create_time, timestamp=2023-10-19T11:36:05.255, value=2023-06-07 00:00:00
1 column=info:id, timestamp=2023-10-19T11:36:05.255, value=1
1 column=info:login_name, timestamp=2023-10-19T11:36:05.255, value=diqomkg3f9
1 column=info:name, timestamp=2023-10-19T11:36:05.255, value=\xE7\xA9\x86\xE9\xB9\x8F
1 column=info:user_level, timestamp=2023-10-19T11:36:05.255, value=1
10 column=info:birthday, timestamp=2023-10-19T11:36:05.265, value=1973-03-07
10 column=info:create_time, timestamp=2023-10-19T11:36:05.265, value=2023-06-07 00:00:00
10 column=info:id, timestamp=2023-10-19T11:36:05.265, value=10
10 column=info:login_name, timestamp=2023-10-19T11:36:05.265, value=zp3agb5c
10 column=info:name, timestamp=2023-10-19T11:36:05.265, value=\xE6\x97\xB6\xE8\xBE\xBE\xE5\xAE\x89
10 column=info:operate_time, timestamp=2023-10-19T11:36:05.265, value=2023-06-16 00:00:00
10 column=info:user_level, timestamp=2023-10-19T11:36:05.265, value=3
```

## 10.9 提交代码至 gitlab

(1) 打开 git 命令行终端

(2) 将所有更改添加到暂存区

本项目准备了.gitignore 文件, 将所有文件添加到暂存区即可。

```
$ git add *
```

(3) 提交至本地仓库

```
$ git commit -m "DIM 层完成"
```

(4) 将更改同步至远端仓库的 dev 分支

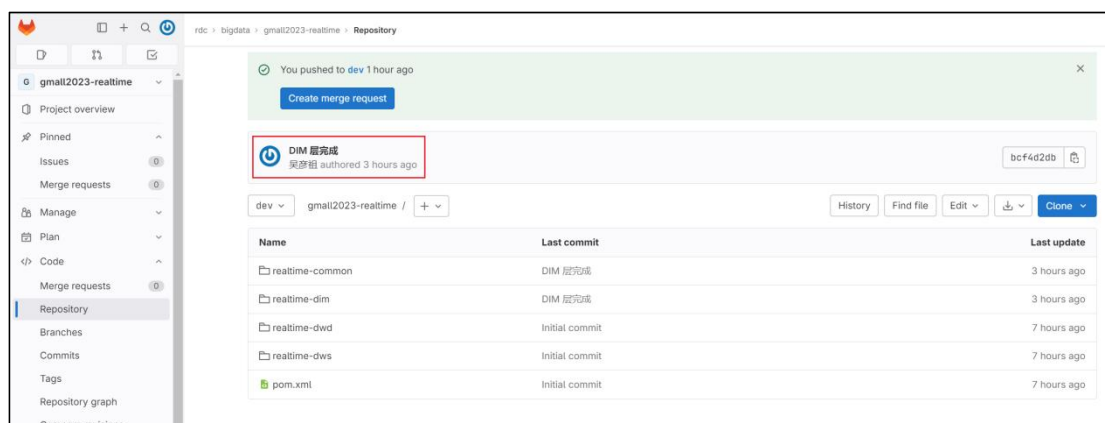
```
$ git push
```

结果如下

```
Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realtime2023/gmall2023-realtime (dev)
$ git push
Enumerating objects: 43, done.
Counting objects: 100% (43/43), done.
Delta compression using up to 20 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (36/36), 31.82 KiB | 1.27 MiB/s, done.
Total 36 (delta 2), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for dev, visit:
remote: http://hadoop104/rdc/bigdata/gmall2023-realtime/-/merge_requests/new?merge_request%5Bsource_branch%5D=dev
remote:
To hadoop104:rdc/bigdata/gmall2023-realtime.git
e7f666b..bcf4d2d dev -> dev

Lenovo@DESKTOP-67V3UFB MINGW64 /d/重要资料/2. 代码/docstestprojects/gmall-realtime2023/gmall2023-realtime (dev)
$
```

(5) 查看 gitlab



提交成功。

## 第 11 章 数仓开发之 DWD 层

DWD 层设计要点:

DWD 层的设计依据是维度建模理论, 该层存储维度模型的事实表

DWD 层表名的命名规范为 `dwd_数据域_表名`

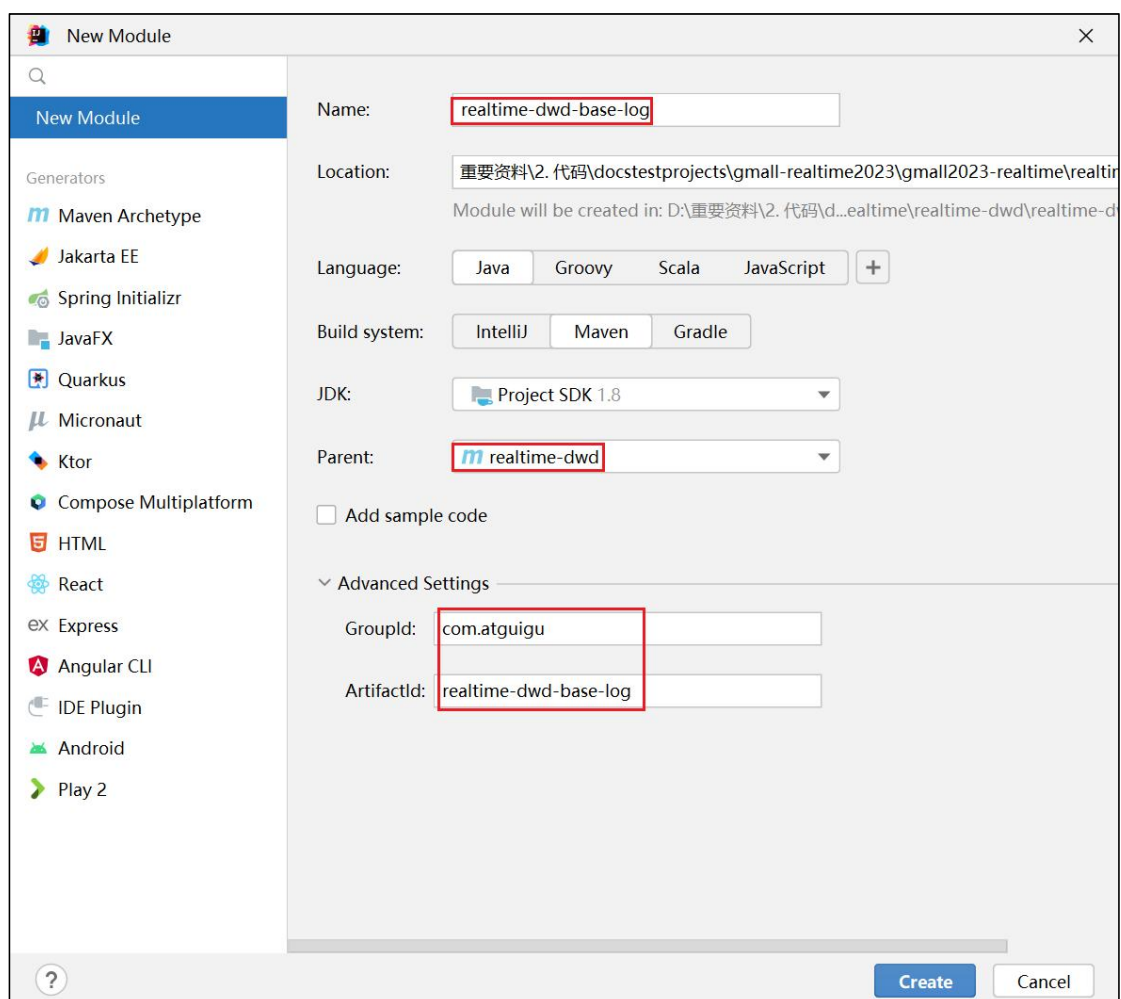
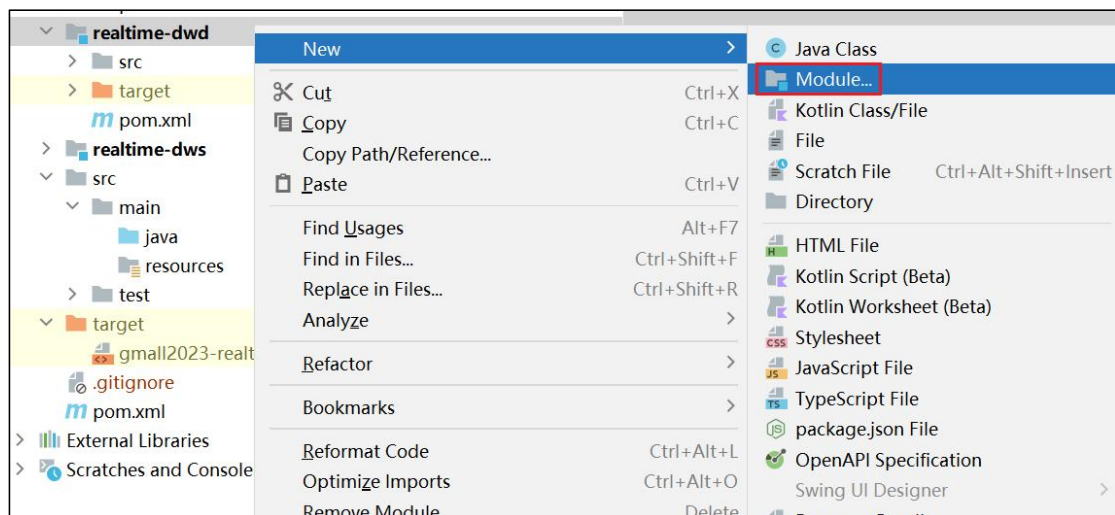
### 11.1 模块设计

DIM 层只有一个应用程序, 放在了 `realtime-dim` 的 `src` 目录下。但 DWD 层有很多应用程序, 为了解耦, 我们为每个应用创建一个单独的模块, 这些模块都是 `realtime-dwd` 的子模块。

### 11.2 流量域未经加工的事务事实表 (日志分流)

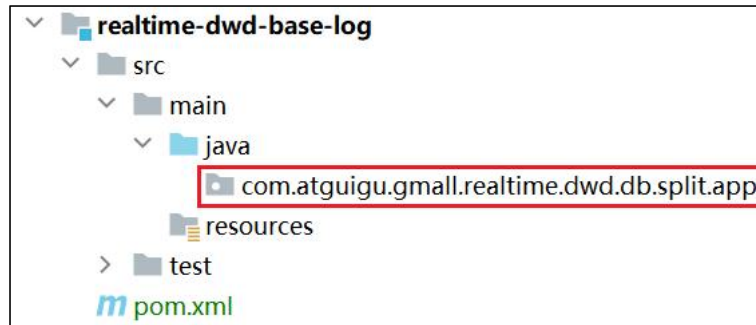
## 11.2.1 创建 realtime-dwd-base-log 模块

### 1) 新建模块



### 2) 创建包

在 realtime-dwd-base-log 模块下创建以下包结构。



## 11.2.2 主要任务

### 1) 数据清洗 (ETL)

数据传输过程中可能会出现部分数据丢失的情况，导致 JSON 数据结构不再完整，因此需要对脏数据进行过滤。

### 2) 新老访客状态标记修复

日志数据 common 字段下的 is\_new 字段是用来标记新老访客状态的，1 表示新访客，0 表示老访客。前端埋点采集到的数据可靠性无法保证，可能会出现老访客被标记为新访客的问题，因此需要对该标记进行修复。

### 3) 分流

本节将通过分流对日志数据进行拆分，生成五张事务事实表写入 Kafka

- (1) 流量域页面浏览事务事实表
- (2) 流量域启动事务事实表
- (3) 流量域动作事务事实表
- (4) 流量域曝光事务事实表
- (5) 流量域错误事务事实表

## 11.2.3 思路

### 1) 数据清洗 (ETL)

对流中数据进行解析，将字符串转换为 JSONObject，如果解析报错则必然为脏数据。定义侧输出流，将脏数据发送到侧输出流，写入 Kafka 脏数据主题

### 2) 新老访客状态标记修复

- (1) 前端埋点新老访客状态标记设置规则

以神策提供的第三方埋点服务中新老访客状态标记设置规则为例

① Web 端：用户第一次访问埋入神策 SDK 页面的当天（即第一天），JS SDK 会在网页的 cookie 中设置一个首日访问的标记，并设置第一天 24 点之前，该标记为 true，即第一天触发的网页端所有事件中，is\_new = 1。第一天之后，该标记则为 false，即第一天之后触发的网页端所有事件中，is\_new = 0；

② 小程序端：用户第一天访问埋入神策 SDK 的页面时，小程序 SDK 会在 storage 缓存中创建一个首日为 true 的标记，并且设置第一天 24 点之前，该标记均为 true。即第一天触发的小程序端所有事件中，is\_new = 1。第一天之后，该标记则为 false，即第一天之后触发的小程序端所有事件中，is\_new = 0；

③ APP 端：用户安装 App 后，第一次打开埋入神策 SDK 的 App 的当天，Android/iOS SDK 会在手机本地缓存内，创建一个首日为 true 的标记，并且设置第一天 24 点之前，该标记均为 true。即第一天触发的 APP 端所有事件中，is\_new = 1。第一天之后，该标记则为 false，即第一天之后触发的 APP 端所有事件中，is\_new = 0。

本项目模拟生成的是 APP 端日志数据。对于此类日志，如果首日之后用户清除了手机本地缓存中的标记，再次启动 APP 会重新设置一个首日为 true 的标记，导致本应为 0 的 is\_new 字段被置为 1，可能会给相关指标带来误差。因此，有必要对新老访客状态标记进行修复。

## （2）新老访客状态标记修复思路

运用 Flink 状态编程，为每个 mid 维护一个键控状态，记录首次访问日期。

### ① 如果 is\_new 的值为 1

- 如果键控状态为 null，认为本次是该访客首次访问 APP，将日志中 ts 对应的日期更新到状态中，不对 is\_new 字段做修改；
- 如果键控状态不为 null，且首次访问日期不是当日，说明访问的是老访客，将 is\_new 字段置为 0；
- 如果键控状态不为 null，且首次访问日期是当日，说明访问的是新访客，不做操作；

### ② 如果 is\_new 的值为 0

- 如果键控状态为 null，说明访问 APP 的是老访客但本次是该访客的页面日志首次进入程序。当前端新老访客状态标记丢失时，日志进入程序被判定为新访客，Flink 程

序就可以纠正被误判的访客状态标记，只要将状态中的日期设置为今天之前即可。本程序选择将状态更新为昨日；

- 如果键控状态不为 `null`，说明程序已经维护了首次访问日期，不做操作。

### 3) 分流：利用侧输出流实现数据拆分

#### (1) 埋点日志结构分析

前端埋点获取的 `JSON` 字符串(日志)可能存在 `common`、`start`、`page`、`displays`、`actions`、`err`、`ts` 七种字段。其中

- `common` 对应的是公共信息，是所有日志都有的字段
- `err` 对应的是错误信息，所有日志都可能有的字段
- `start` 对应的是启动信息，启动日志才有的字段
- `page` 对应的是页面信息，页面日志才有的字段
- `displays` 对应的是曝光信息，曝光日志才有的字段，曝光日志可以归为页面日志，因此必然有 `page` 字段
- `actions` 对应的是动作信息，动作日志才有的字段，同样属于页面日志，必然有 `page` 字段。动作信息和曝光信息可以同时存在。
- `ts` 对应的是时间戳，单位：毫秒，所有日志都有的字段

综上，我们可以将前端埋点获取的日志分为两大类：启动日志和页面日志。二者都有 `common` 字段和 `ts` 字段，都可能有 `err` 字段。页面日志一定有 `page` 字段，一定没有 `start` 字段，可能有 `displays` 和 `actions` 字段；启动日志一定有 `start` 字段，一定没有 `page`、`displays` 和 `actions` 字段。

#### (2) 分流日志分类

本节将按照内容，将日志分为以下五类

- 启动日志
- 页面日志
- 曝光日志
- 动作日志
- 错误日志

#### (3) 分流思路



① 所有日志数据都可能拥有 `err` 字段，所有首先获取 `err` 字段，如果返回值不为 `null` 则将整条日志数据发送到错误侧输出流。然后删掉 `JSONObject` 中的 `err` 字段及对应值；

② 判断是否有 `start` 字段，如果有则说明数据为启动日志，将其发送到启动侧输出流；如果没有则说明为页面日志，进行下一步；

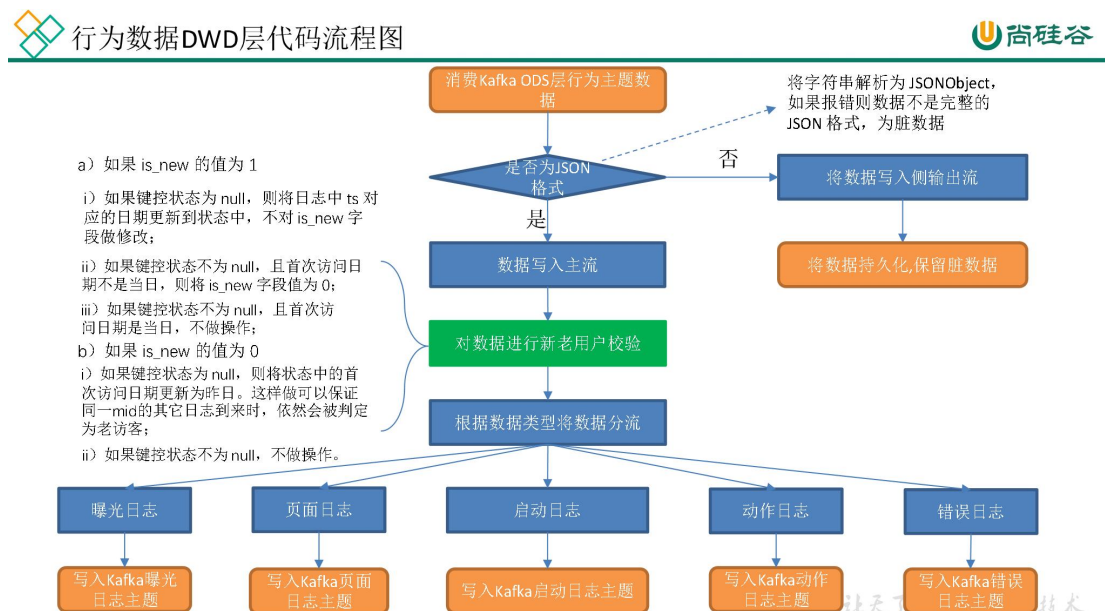
③ 页面日志必然有 `page` 字段、`common` 字段和 `ts` 字段，获取它们的值，`ts` 封装为包装类 `Long`，其余两个字段的值封装为 `JSONObject`；

④ 判断是否有 `displays` 字段，如果有，将其值封装为 `JSONArray`，遍历该数组，依次获取每个元素（记为 `display`），封装为 `JSONObject`。创建一个空的 `JSONObject`，将 `display`、`common`、`page` 和 `ts` 添加到该对象中，获得处理好的曝光数据，发送到曝光侧输出流。动作日志的处理与曝光日志相同（注意：一条页面日志可能既有曝光数据又有动作数据，二者没有任何关系，因此曝光数据不为 `null` 时仍要对动作数据进行处理）；

⑤ 动作日志和曝光日志处理结束后删除 `displays` 和 `actions` 字段，此时主流的 `JSONObject` 中只有 `common` 字段、`page` 字段和 `ts` 字段，即为最终的页面日志。

处理结束后，页面日志数据位于主流，其余四种日志分别位于对应的侧输出流，将五条流的数据写入 `Kafka` 对应主题即可。

## 11.2.4 图解



## 11.2.5 代码实现

### 1) 创建 `FlinkSinkUtil`

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网



在 common 模块的 util 包下创建 FlinkSinkUtil, 创建 getKafkaSink(String topic)方法。

```
package com.atguigu.gmall.realtime.common.util;

import com.atguigu.gmall.realtime.common.constant.Constant;
import org.apache.flink.api.common.serialization.SimpleStringSchema;
import org.apache.flink.api.connector.sink2.Sink;
import org.apache.flink.connector.base.DeliveryGuarantee;
import org.apache.flink.connector.kafka.sink.KafkaRecordSerializationSchema;
import org.apache.flink.connector.kafka.sink.KafkaSink;

import java.util.Random;

public class FlinkSinkUtil {

    public static Sink<String> getKafkaSink(String topic) {

        return KafkaSink.<String>builder()
            .setBootstrapServers(Constant.KAFKA_BROKERS)
            .setRecordSerializer(KafkaRecordSerializationSchema.
<String>builder()
                .setTopic(topic)
                .setValueSerializationSchema(new
SimpleStringSchema())
                .build())
            .setDeliveryGuarantee(DeliveryGuarantee.EXACTLY_ONCE
)
            .setTransactionalIdPrefix("atguigu-" + topic + new
Random().nextLong())
            .setProperty("transaction.timeout.ms", 15 * 60 * 1000
+ "")
            .build();
    }
}
```

## 2) 创建 DateFormatUtil

在 common 模块的 util 包下创建日期处理工具类 DateFormatUtil

```
package com.atguigu.gmall.realtime.common.util;

import java.time.LocalDateTime;
```

```
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.time.format.DateTimeFormatter;
import java.util.Date;

public class DateFormatUtil {
    private static final DateTimeFormatter dtf =
DateTimeFormatter.ofPattern("yyyy-MM-dd");
    private static final DateTimeFormatter dtfForPartition =
DateTimeFormatter.ofPattern("yyyyMMdd");
    private static final DateTimeFormatter dtfFull =
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    /**
     * 2023-07-05 01:01:01 转成 ms 值
     * @param dateTime
     * @return
     */
    public static Long dateTimeToTs(String dateTime) {
        LocalDateTime localDateTime = LocalDateTime.parse(dateTime,
dtfFull);
        return
localDateTime.toInstant(ZoneOffset.of("+8")).toEpochMilli();
    }

    /**
     * 把毫秒值转成 年月日: 2023-07-05
     * @param ts
     * @return
     */
    public static String tsToDate(Long ts) {
        Date dt = new Date(ts);
        LocalDateTime localDateTime =
LocalDateTime.ofInstant(dt.toInstant(), ZoneId.systemDefault());
        return dtf.format(localDateTime);
    }

    /**
     * 把毫秒值转成 年月日时分秒: 2023-07-05 01:01:01
     * @param ts
     * @return
     */
    public static String tsToDateTime(Long ts) {
        Date dt = new Date(ts);
```

```
        LocalDateTime localDateTime =
LocalDateTime.ofInstant(dt.toInstant(), ZoneId.systemDefault());
        return dtfFull.format(localDateTime);
    }

    public static String tsToDateForPartition(long ts) {
        Date dt = new Date(ts);
        LocalDateTime localDateTime =
LocalDateTime.ofInstant(dt.toInstant(), ZoneId.systemDefault());
        return dtfForPartition.format(localDateTime);
    }

    /**
     * 把 年月日转成 ts
     * @param date
     * @return
     */
    public static long dateToTs(String date) {
        return dateTimeToTs(date + " 00:00:00");
    }
}
```

### 3) 主程序

在 realtime-dwd-base-log 模块的 com.atguigu.gmall.realtime.dwd.log.split.app 包下创建应用程序 DwdBaseLog，继承 BaseApp，写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.log.split.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONArray;
import com.alibaba.fastjson.JSONAware;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import lombok.extern.slf4j.Slf4j;
import org.apache.flink.api.common.functions.FilterFunction;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
```

```
org.apache.flink.streaming.api.datastream.SingleOutputStreamOperator;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import org.apache.flink.streaming.api.functions.ProcessFunction;
import org.apache.flink.util.Collector;
import org.apache.flink.util.OutputTag;

import java.util.HashMap;
import java.util.Map;

@Slf4j
public class DwdBaseLog extends BaseApp {

    private final String START = "start";
    private final String ERR = "err";
    private final String DISPLAY = "display";
    private final String ACTION = "action";
    private final String PAGE = "page";

    public static void main(String[] args) {
        new DwdBaseLog().start(
            10011,
            4,
            "dwd_base_log",
            Constant.TOPIC_LOG
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
                      DataStreamSource<String> stream) {
        // 1. etl
        SingleOutputStreamOperator<JSONObject> etledStream =
etl(stream);

        // 2. 纠正新老客户
        SingleOutputStreamOperator<JSONObject> validatedStream =
validateNewOrOld(etledStream);

        // 3. 分流
        Map<String, DataStream<JSONObject>> streams =
```

```
splitStream(validatedStream);

    // 4. 不同的流写出到不同的 topic
    writeToKafka(streams);

}

private void writeToKafka(Map<String, DataStream<JSONObject>>
streams) {
    streams
        .get(START)
        .map(JSONAware::toJSONString)
        .sinkTo(FlinkSinkUtil.getKafkaSink(Constant.TOPIC_DWD_T
RAFFIC_START));

    streams
        .get(ERR)
        .map(JSONAware::toJSONString)
        .sinkTo(FlinkSinkUtil.getKafkaSink(Constant.TOPIC_DWD_T
RAFFIC_ERR));

    streams
        .get(DISPLAY)
        .map(JSONAware::toJSONString)
        .sinkTo(FlinkSinkUtil.getKafkaSink(Constant.TOPIC_DWD_T
RAFFIC_DISPLAY));

    streams
        .get(PAGE)
        .map(JSONAware::toJSONString)
        .sinkTo(FlinkSinkUtil.getKafkaSink(Constant.TOPIC_DWD_T
RAFFIC_PAGE));

    streams
        .get(ACTION)
        .map(JSONAware::toJSONString)
        .sinkTo(FlinkSinkUtil.getKafkaSink(Constant.TOPIC_DWD_T
RAFFIC_ACTION));
}

private Map<String, DataStream<JSONObject>>
splitStream(SingleOutputStreamOperator<JSONObject> stream) {
    OutputTag<JSONObject> displayTag = new
OutputTag<JSONObject>("display"){};
}
```

```
OutputTag<JSONObject> actionTag = new
OutputTag<JSONObject> ("action") {};
OutputTag<JSONObject> errTag = new
OutputTag<JSONObject> ("err") {};
OutputTag<JSONObject> pageTag = new
OutputTag<JSONObject> ("page") {};
/*
    主流：启动日志
    侧输出流：页面 错误 曝光 活动
*/
SingleOutputStreamOperator<JSONObject> startStream = stream
    .process(new ProcessFunction<JSONObject, JSONObject>() {
        @Override
        public void processElement(JSONObject obj,
                                    Context ctx,
                                    Collector<JSONObject> out)
throws Exception {
            JSONObject common = obj.getJSONObject("common");
            Long ts = obj.getLong("ts");
            // 1. 启动
            JSONObject start = obj.getJSONObject("start");
            if (start != null) {
                out.collect(obj);
            }

            // 2. 曝光
            JSONArray displays = obj.getJSONArray("displays");
            if (displays != null) {
                for (int i = 0; i < displays.size(); i++) {
                    JSONObject display =
displays.getJSONObject(i);
                    display.putAll(common);
                    display.put("ts", ts);
                    ctx.output(displayTag, display);
                }

                // 删除 displays
                obj.remove("displays");
            }

            // 3. 活动
            JSONArray actions = obj.getJSONArray("actions");
            if (actions != null) {
                for (int i = 0; i < actions.size(); i++) {
                    JSONObject action =
```

```
actions.getJSONObject(i);
            action.putAll(common);
            ctx.output(actionTag, action);
        }

        // 删除 displays
        obj.remove("actions");
    }

    // 4. err
    JSONObject err = obj.getJSONObject("err");
    if (err != null) {
        ctx.output(errTag, obj);
        obj.remove("err");
    }

    // 5. 页面
    JSONObject page = obj.getJSONObject("page");
    if (page != null) {
        ctx.output(pageTag, obj);
    }
    }
});

Map<String, DataStream<JSONObject>> streams = new HashMap<>();

streams.put(START, startStream);
streams.put(DISPLAY,
startStream.getSideOutput(displayTag));
streams.put(ERR, startStream.getSideOutput(errTag));
streams.put(PAGE, startStream.getSideOutput(pageTag));
streams.put(ACTION, startStream.getSideOutput(actionTag));

return streams;

}

private SingleOutputStreamOperator<JSONObject>
validateNewOrOld(SingleOutputStreamOperator<JSONObject> stream) {
    return stream
        .keyBy(obj ->
obj.getJSONObject("common").getString("mid"))
        .process(new KeyedProcessFunction<String, JSONObject,
```



```
JSONObject>() {

    private ValueState<String> firstVisitDateState;

    @Override
    public void open(Configuration parameters) throws
Exception {
        firstVisitDateState =
getRuntimeContext().getState(new
ValueStateDescriptor<String>("firstVisitDate", String.class));
    }

    @Override
    public void processElement(JSONObject obj,
                                Context ctx,
                                Collector<JSONObject> out)
throws Exception {

        JSONObject common = obj.getJSONObject("common");
        String isNew = common.getString("is_new");

        Long ts = obj.getLong("ts");
        String today = DateFormatUtil.tsToDate(ts);

        // 从状态中获取首次访问日志
        String firstVisitDate =
firstVisitDateState.value();

        if ("1".equals(isNew)) {
            if (firstVisitDate == null) {
                // 这个 mid 的首次访问
                firstVisitDateState.update(today);
            } else if (!today.equals(firstVisitDate)) {
                // 今天和首次访问不一致
                common.put("is_new", "0"); // 把新用户修改为
老用户
            }
        } else {
            if (firstVisitDate == null) {
                // 一个老用户，他的首次访问日志还是 null
                // 把他的首次访问日期设置为昨天
                firstVisitDateState.update(DateFormatUtil.tsToDate(ts - 24 * 60 * 60
```

```
* 1000));  
  
        }  
    }  
    out.collect(obj);  
}  
});  
  
}  
  
private SingleOutputStreamOperator<JSONObject>  
etl(DataStreamSource<String> stream) {  
    return stream  
        .filter(new FilterFunction<String>() {  
            @Override  
            public boolean filter(String value) throws Exception {  
                try {  
                    JSON.parseObject(value );  
                    return true;  
                } catch (Exception e) {  
                    log.error("日志格式不是正确的 JSON 格式: " +  
value);  
                    return false;  
                }  
            }  
        })  
        .map(JSON::parseObject);  
}
```

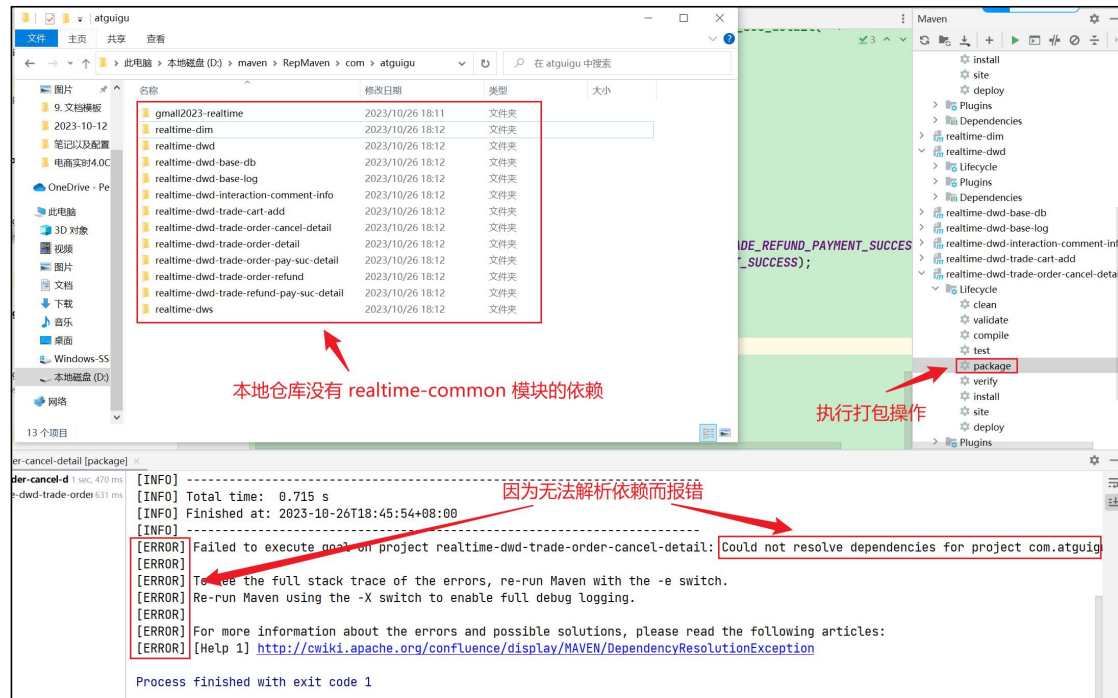
## 11.2.6 测试

### 1) 启动 zk、kafka、flume(fl.sh start)

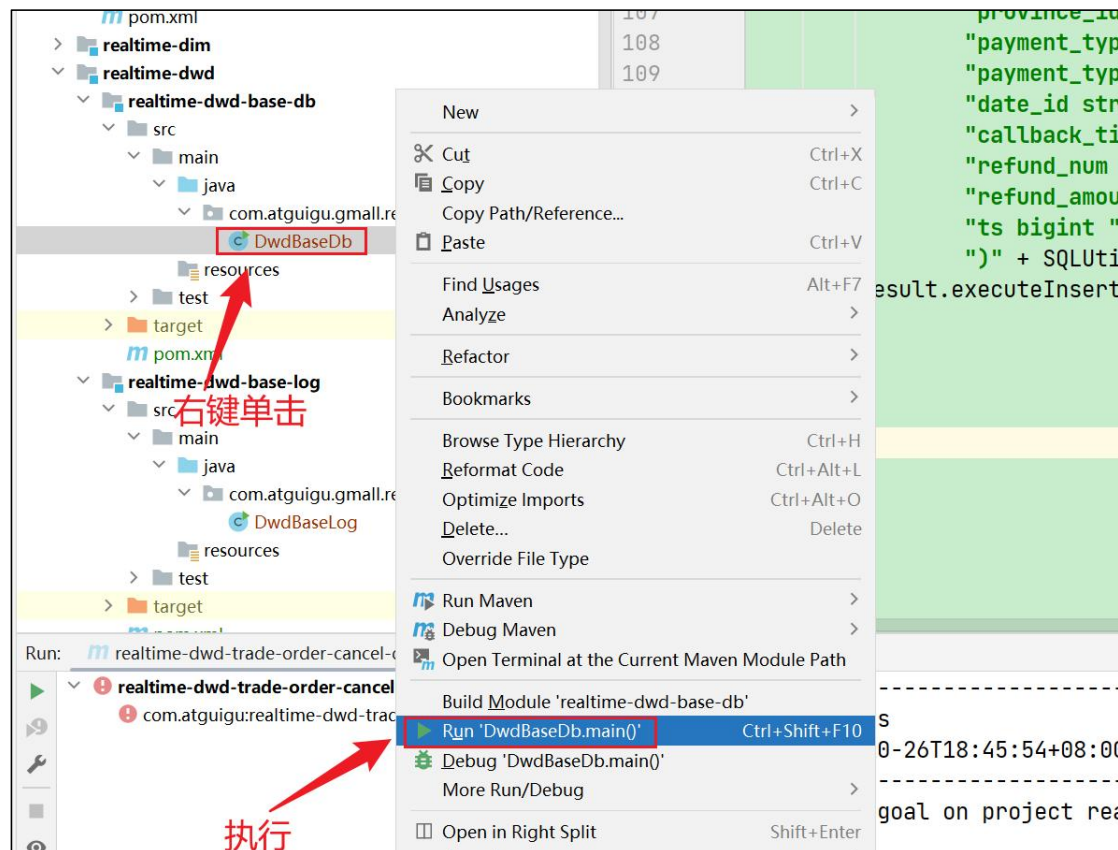
### 2) 运行 DwdBaseLog

(1) 要注意，realtime-dwd-base-log 模块依赖于 common 模块，按照我们对于 Maven 的理解，DwdBaseLog 编译时需要从本地仓库获取 common 模块的字节码文件，本节修改了 common 模块下的代码，要重新将该模块 install 至本地仓库方可完成编译。但是在 IDEA 中直接运行 DwdBaseLog 却不会因为依赖缺失而报错。这是因为 IDEA 做了优化，当父工程下有多个子模块时，IDEA 会尝试优化这些模块间的依赖关系。运行 DwdBaseLog 时，IDEA 会直接使用 common 模块的源码，而不是使用安装到本地仓库的 JAR 包。因此，即便不对 common 模块执行 install 操作，仍然可以运行应用程序。

(2) 但是, 使用 Maven 工具构建仓库, 如执行 compile 或 package 都会因为依赖缺失而失败。如下图所示。



(3) IDEA 的以上优化可以提升开发效率, 测试阶段不需要打包, 因此我们不需要在修改 common 模块代码之后执行 install 操作, 直接右键应用程序运行即可, 如下。



(4) 但要注意，开发完成后，打包之前要先对 `common` 模块执行 `install` 操作。

(5) IDEA 为了提升效率，做了很多缓存，有时更改代码，在执行时并不起效，此时可以清除缓存重启 IDEA，通常可以解决问题。

(6) 上述优化可能因版本差异而有所不同，笔者测试所用的 IDEA 版本为 2022.2.3，此外，某些情况下优化未必起效。此时，对 `common` 模块执行 `install` 操作，清除缓存，通常可以解决问题。

3) 启动命令行消费者监控 `kafka` 各个 `dwd` 日志主题

4) 运行数据生成器

5) 查看控制台 `Kafka` 命令行消费者输出

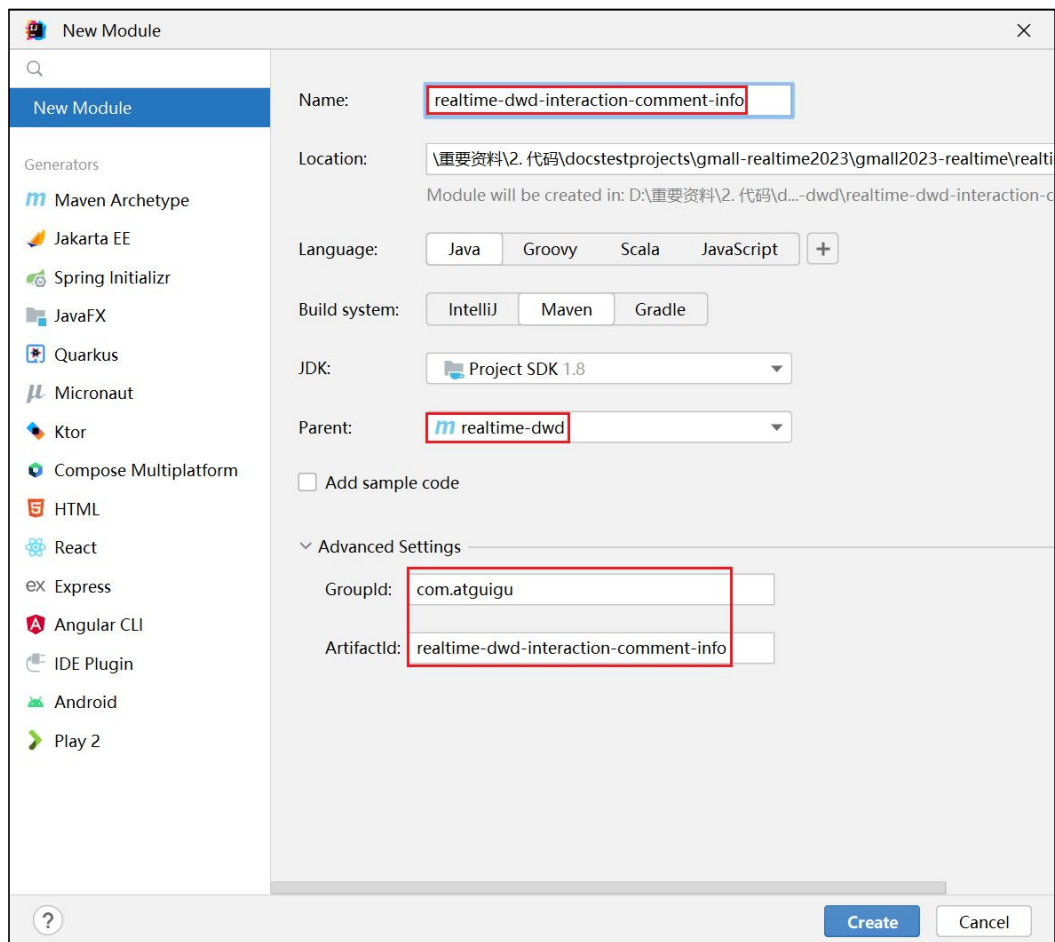
### 11.2.7 提交至 GitLab

操作同上，略。

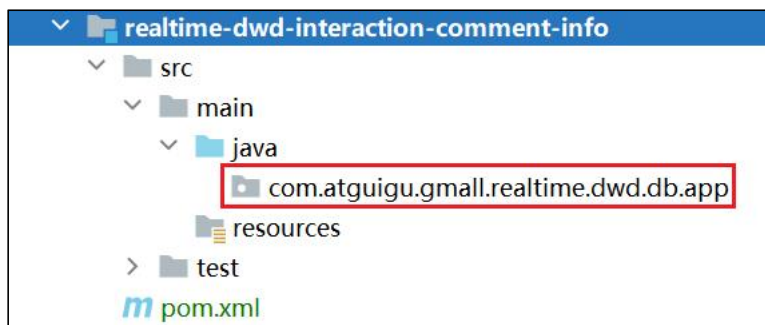
## 11.3 互动域评论事务事实表

### 11.3.1 创建 `realtime-dwd-interaction-comment-info` 模块

1) 新建模块



## 2) 创建包结构



## 11.3.2 主要任务

提取评论操作生成评论表，并将字典表中的相关维度退化到评论表中，写出到 Kafka 对应主题。

## 11.3.3 思路分析

## 11.3.3.1 维度关联（维度退化）实现策略分析

本章业务事实表的构建全部使用 FlinkSQL 实现，字典表数据存储在 MySQL 的业务数据库中，要做维度退化，就要将这些数据从 MySQL 中提取出来封装成 FlinkSQL 表，Flink 的 JDBC SQL Connector 可以实现我们的需求。

## 11.3.3.2 执行步骤

### 1) 设置表状态的 ttl

look up 表的 cache 是由建表时指定的相关参数决定的，与普通连接的 ttl 无关。

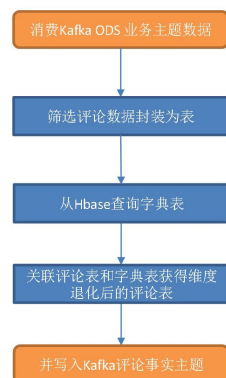
### 2) 读取评论表数据。

### 3) 建立 LookUp 字典表。

### 4) 关联评论表和字典表，维度退化。

## 11.3.4 图解

业务数据DWD层评论业务代码流程图



让天下没有难学的技术

## 11.3.5 代码实现

### 1) 所需依赖

使用 HBase-Connector 需要引入以下依赖。

```

<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-connector-hbase-2.2</artifactId>
</dependency>
    
```

已在 common 模块引入。

### 2) 创建 SQLUtil

在 common 模块的 util 包下创建工具类 SQLUtil，在该类中创建 getKafkaDDLSource 和 getKafkaDDLSink 方法，如下。

```
package com.atguigu.gmall.realtime.common.util;

import com.atguigu.gmall.realtime.common.constant.Constant;

public class SQLUtil {

    public static String getKafkaDDLSource(String groupId, String topic) {

        return "with(" +
            " 'connector' = 'kafka'," +
            " 'properties.group.id' = '" + groupId + "'," +
            " 'topic' = '" + topic + "'," +
            " 'properties.bootstrap.servers' = '" +
Constant.KAFKA_BROKERS + "'," +
            " 'scan.startup.mode' = 'latest-offset'," +
            " 'json.ignore-parse-errors' = 'true'," + // 当 json 解析
失败的时候,忽略这条数据
            " 'format' = 'json' " +
            " )";

    }

    public static String getKafkaDDLSink(String topic) {

        return "with(" +
            " 'connector' = 'kafka'," +
            " 'topic' = '" + topic + "'," +
            " 'properties.bootstrap.servers' = '" +
Constant.KAFKA_BROKERS + "'," +
            " 'format' = 'json' " +
            " )";

    }

}
```

### 3) 创建基类 BaseSQLApp

在 common 模块的 base 包下创建基类 BaseSQLApp，写入以下内容

```
package com.atguigu.gmall.realtime.common.base;

import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.SQLUtil;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.runtime.state.hashmap.HashMapStateBackend;
import org.apache.flink.streaming.api.CheckpointingMode;
import
```



```
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

import
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

import static
org.apache.flink.streaming.api.environment.CheckpointConfig.ExternalizedCheckpointCleanup.RETAIN_ON_CANCELLATION;

public abstract class BaseSQLApp {
    public abstract void handle(StreamExecutionEnvironment env,
                                StreamTableEnvironment tEnv);

    public void start(int port,
                      int parallelism,
                      String ck) {
        System.setProperty("HADOOP_USER_NAME", "atguigu");

        Configuration conf = new Configuration();
        conf.setInteger("rest.port", port);
        StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment(conf);
        env.setParallelism(parallelism);
        // 1. 设置状态后端
        env.setStateBackend(new HashMapStateBackend());

        // 2. 开启 checkpoint
        env.enableCheckpointing(5000);
        // 3. 设置 checkpoint 模式：精准一次

        env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.
EXACTLY_ONCE);
        // 4. checkpoint 存储

        env.getCheckpointConfig().setCheckpointStorage("hdfs://hadoop102:
8020/gmall2023/sql/" + ck);
        // 5. checkpoint 并发数
        env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);
        // 6. checkpoint 之间的最小间隔

        env.getCheckpointConfig().setMinPauseBetweenCheckpoints(500);
        // 7. checkpoint 的超时时间
        env.getCheckpointConfig().setCheckpointTimeout(10000);
```

```
// 8. job 取消的时候的, checkpoint 保留策略

env.getCheckpointConfig().setExternalizedCheckpointCleanup(RETAIN_ON_CANCELLATION);

StreamTableEnvironment tEnv =
StreamTableEnvironment.create(env);

handle(env, tEnv);

}

// 读取 ods_db
public void readOdsDb(StreamTableEnvironment tEnv, String
groupId) {
    tEnv.executeSql("create table topic_db (" +
        " `database` string, " +
        " `table` string, " +
        " `type` string, " +
        " `data` map<string, string>, " +
        " `old` map<string, string>, " +
        " `ts` bigint, " +
        " `pt` as proctime(), " +
        " et as to_timestamp_ltz(ts, 0), " +
        " watermark for et as et - interval '3' second
" +
        ") " + SQLUtil.getKafkaDDLSource(groupId,
Constant.TOPIC_DB));

}

public void readBaseDic(StreamTableEnvironment tEnv){
    tEnv.executeSql(
        "create table base_dic (" +
            " dic_code string," + // 如果字段是原子类型,则表示这个是
rowKey, 字段随意, 字段类型随意
            " info row<dic_name string>, " + // 字段名和 hbase 中的
列族名保持一致. 类型必须是 row. 嵌套进去的就是列
            " primary key (dic_code) not enforced " + // 只能用 rowKey
做主键
        ") WITH (" +
            " 'connector' = 'hbase-2.2'," +
            " 'table-name' = 'gmall:dim_base_dic'," +
            " 'zookeeper.quorum' =
```

```
'hadoop102:2181,hadoop103:2181,hadoop104:2181', " +  
    " 'lookup.cache' = 'PARTIAL', " +  
    " 'lookup.async' = 'true', " +  
    " 'lookup.partial-cache.max-rows' = '20', " +  
    " 'lookup.partial-cache.expire-after-access' = '2  
hour' " +  
    " ) ) );  
}  
}
```

#### 4) 创建主程序

在 `realtime-dwd-interaction-comment-info` 模块的 `com.atguigu.gmall.realtime.dwd.db.app` 包下创建应用程序 `DwdInteractionCommentInfo`，继承 `BaseSQLApp`，写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.db.app;  
  
import com.atguigu.gmall.realtime.common.base.BaseSQLApp;  
import com.atguigu.gmall.realtime.common.constant.Constant;  
import com.atguigu.gmall.realtime.common.util.SQLUtil;  
import  
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;  
import org.apache.flink.table.api.Table;  
import  
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;  
  
public class DwdInteractionCommentInfo extends BaseSQLApp {  
    public static void main(String[] args) {  
        new DwdInteractionCommentInfo().start(  
            10012,  
            4,  
            Constant.TOPIC_DWD_INTERACTION_COMMENT_INFO  
        );  
    }  
  
    @Override  
    public void handle(StreamExecutionEnvironment env,  
        StreamTableEnvironment tEnv) {  
  
        // 1. 通过 ddl 的方式建立动态表：从 topic_db 读取数据 (source)  
        readOdsDb(tEnv,  
Constant.TOPIC_DWD_INTERACTION_COMMENT_INFO);  
        // 2. 过滤出评论表数据
```

```
Table commentInfo = tEnv.sqlQuery(
    "select " +
        " `data`['id'] id, " +
        " `data`['user_id'] user_id, " +
        " `data`['sku_id'] sku_id, " +
        " `data`['appraise'] appraise, " +
        " `data`['comment_txt'] comment_txt, " +
        " `data`['create_time'] comment_time," +
        " ts, " +
        " pt " +
        " from topic_db " +
        " where `database`='gmall' " +
        " and `table`='comment_info' " +
        " and `type`='insert' ");
tEnv.createTemporaryView("comment_info", commentInfo);

// 3. 通过 ddl 方式建表: base_dic hbase 中的维度表 (source)
readBaseDic(tEnv);
// 4. 事实表与维度表的 join: lookup join
Table result = tEnv.sqlQuery("select " +
    "ci.id, " +
    "ci.user_id," +
    "ci.sku_id," +
    "ci.appraise," +
    "dic.info.dic_name
appraise_name," +
    "ci.comment_txt," +
    "ci.ts " +
    "from comment_info ci " +
    "join base_dic for system_time as
of ci.pt as dic " +
    "on ci.appraise=dic.dic_code");

// 5. 通过 ddl 方式建表: 与 kafka 的 topic 关联 (sink)
tEnv.executeSql("create table dwd_interaction_comment_info("
+
    "id string, " +
    "user_id string," +
    "sku_id string," +
    "appraise string," +
    "appraise_name string," +
    "comment_txt string," +
    "ts bigint " +
    ") " +
```

```
SQLUtil.getKafkaDDLSink(Constant.TOPIC_DWD_INTERACTION_COMMENT_IN  
FO));  
  
    // 6. 把 join 的结果写入到 sink 表  
  
result.executeInsert(Constant.TOPIC_DWD_INTERACTION_COMMENT_INFO)  
;  
    }  
}
```

### 11.3.6 测试

- 1) 启动 zk、kafka、maxwell
- 2) 运行 DwdInteractionCommentInfo
- 3) 启动命令行消费者监控 kafka 主题 dwd\_interaction\_comment\_info
- 4) 运行数据生成器
- 5) 查看命令行 Kafka 消费者的输出

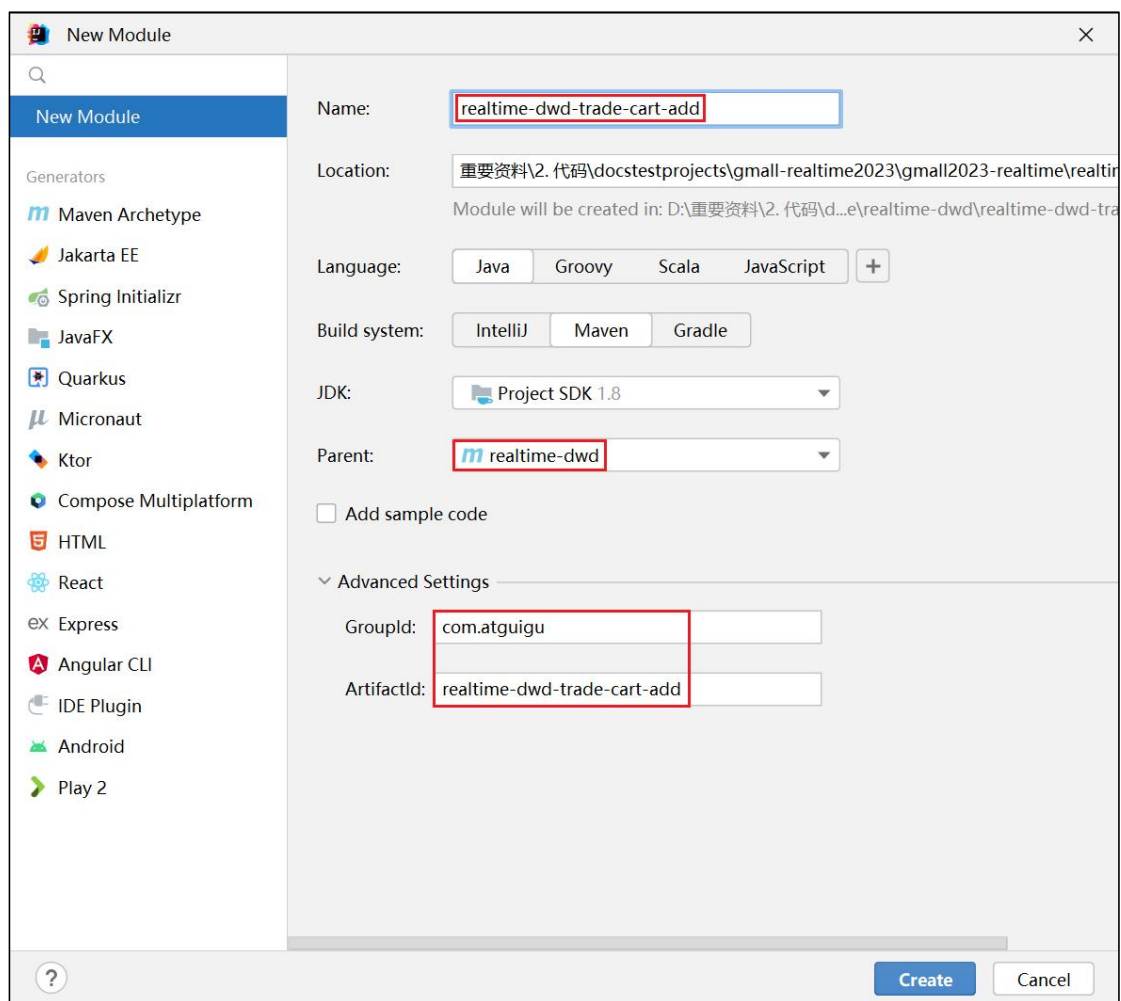
### 11.3.7 提交至 GitLab

略。

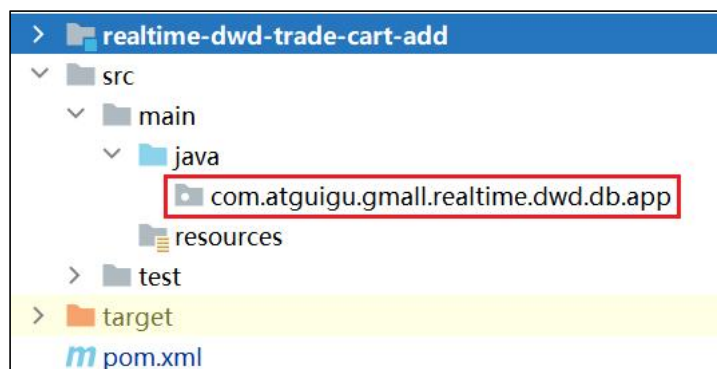
## 11.4 交易域加购事务事实表

### 11.4.1 创建 realtime-dwd-trade-cart-add 模块

- 1) 新建模块



## 2) 创建包结构



## 11.4.2 主要任务

提取加购操作生成加购表，写出到 Kafka 对应主题。

## 11.4.3 思路分析

### 1) 读取业务数据

### 2) 过滤出加购行为

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：尚硅谷官网

3) 将加购表数据写到 kafka

## 11.4.4 图解

 业务数据DWD层加购业务代码流程图



让天下没有难学的技术

## 11.4.5 代码实现

在 realtime-dwd-trade-cart-add 模块的 com.atguigu.gmall.realtime.dwd.db.app 包下创建应用程序 DwdTradeCartAdd，继承基类 BaseSQLApp，写入以下内容。

```

package com.atguigu.gmall.realtime.dwd.db.app;

import com.atguigu.gmall.realtime.common.base.BaseSQLApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.SQLUtil;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import org.apache.flink.table.api.Table;
import
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

public class DwdTradeCartAdd extends BaseSQLApp {
    public static void main(String[] args) {
        new DwdTradeCartAdd().start(
            10013,
            4,
            Constant.TOPIC_DWD_TRADE_CART_ADD
        );
    }
}
    
```



```
@Override
public void handle(StreamExecutionEnvironment env,
                  StreamTableEnvironment tEnv) {
    // 1. 读取 topic_db 数据
    readOdsDb(tEnv, Constant.TOPIC_DWD_TRADE_CART_ADD);
    // 2. 过滤出加购数据
    Table cartAdd = tEnv.sqlQuery(
        "select " +
            " `data`['id'] id," +
            " `data`['user_id'] user_id," +
            " `data`['sku_id'] sku_id," +
            " if(`type`='insert'," +
            "    cast(`data`['sku_num'] as int), " +
            "    cast(`data`['sku_num'] as int) -
cast(`old`['sku_num'] as int)" +
            ") sku_num ," +
            " ts " +
            "from topic_db " +
            "where `database`='gmall' " +
            "and `table`='cart_info' " +
            "and (" +
            "    `type`='insert' " +
            "    or(" +
            "        `type`='update' " +
            "        and `old`['sku_num'] is not null " +
            "        and cast(`data`['sku_num'] as int) >
cast(`old`['sku_num'] as int) " +
            "    )" +
            ")");

    // 3. 写出到 kafka
    tEnv.executeSql("create table dwd_trade_cart_add(" +
        "    id string, " +
        "    user_id string," +
        "    sku_id string," +
        "    sku_num int, " +
        "    ts bigint " +
        ")");
    SQLUtil.getKafkaDDLSink(Constant.TOPIC_DWD_TRADE_CART_ADD));

    cartAdd.executeInsert(Constant.TOPIC_DWD_TRADE_CART_ADD);
}
}
```

### 11.4.6 测试

- 1) 启动 zk、kafka、maxwell
- 2) 运行 DwdTradeCartAdd
- 3) 启动命令行消费者监控 kafka 主题 dwd\_trade\_cart\_add
- 4) 运行数据生成器
- 5) 查看命令行 Kafka 消费者的输出

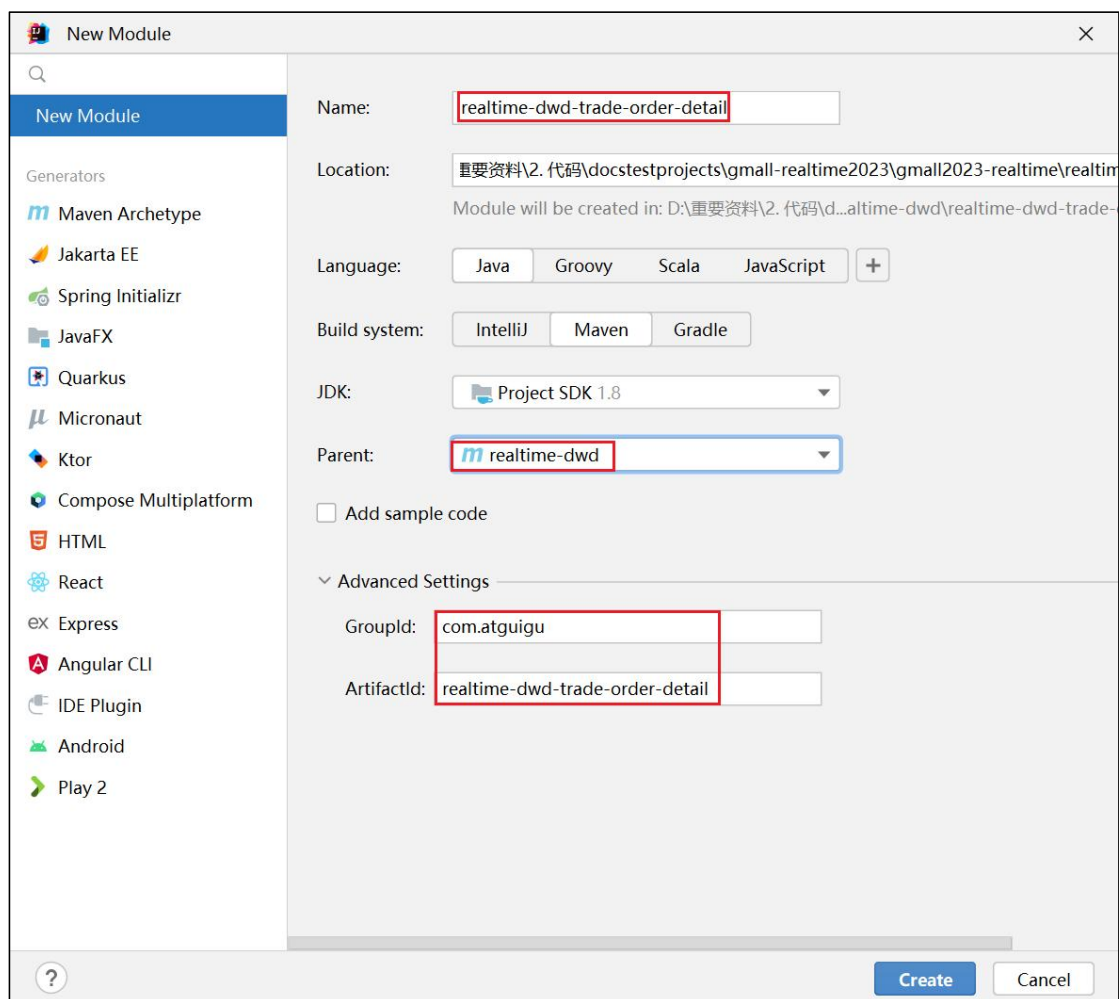
### 11.4.7 提交至 GitLab

略。

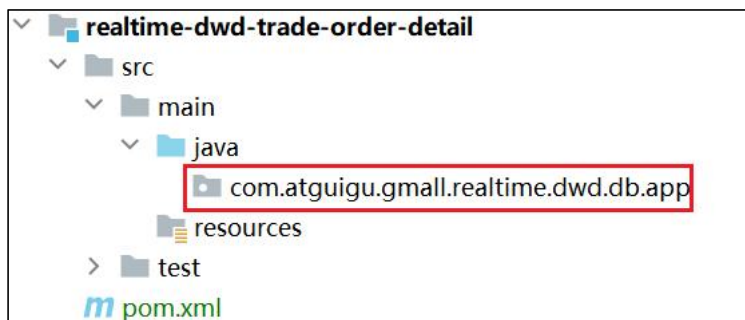
## 11.5 交易域下单事务事实表

### 11.5.1 创建 realtime-dwd-trade-order-detail 模块

- 1) 新建模块



## 2) 创建包结构



## 11.5.2 主要任务

关联订单明细表、订单表、订单明细活动关联表、订单明细优惠券关联表四张事实业务表的 insert 操作，形成下单明细表，写入 Kafka 对应主题。

## 11.5.3 思路分析

### 11.5.3.1 执行步骤

预处理表与订单明细事务事实表的区别只在于前者不会按照订单数据的 type 和 old 字段做过滤，且在表中增加了这两个字段。二者的粒度、聚合逻辑都相同，因此按照订单明细表的思路对预处理表进行分析即可。

#### 1) 设置 ttl;

ttl (time-to-live) 即存活时间。表之间做普通关联时，底层会将两张表的数据维护到状态中，默认情况下状态永远不会清空，这样会对内存造成极大的压力。表状态的 ttl 是 Idle (空闲，即状态未被更新) 状态被保留的最短时间，假设 ttl 为 10s，若状态中的数据在 10s 内未被更新，则未来的某个时间会被清除 (故而 ttl 是最短存活时间)。ttl 默认值为 0，表示永远不会清空状态。

下单操作发生时，订单明细表、订单表、订单明细优惠券关联表和订单明细活动关联表的数据操作类型均为 insert，不存在业务上的滞后问题，只考虑可能的数据乱序即可；因此将 ttl 设置为 5s。

要注意：前文提到，本项目保证了同一分区、同一并行度的数据有序。此处的乱序与之并不冲突，以下单业务过程为例，用户完成下单操作时，订单表中会插入一条数据，订单明细表中会插入与之对应的多条数据，本项目业务数据是按照主键分区进入 Kafka 的，虽然同分区数据有序，但是同一张业务表的数据可能进入多个分区，会乱序。这样一来，订单表

数据与对应的订单明细数据可能被属于其它订单的数据“插队”，因而导致主表或从表数据迟到，可能 join 不上，为了应对这种情况，设置乱序程度，让状态中的数据等待一段时间。

### 2) 从 Kafka topic\_db 主题读取业务数据

### 3) 筛选订单明细表数据

应尽可能保证事实表的粒度为最细粒度，在下单业务过程中，最细粒度的事件为一个订单的一个 SKU 的下单操作，订单明细表的粒度与最细粒度相同，将其作为主表

### 4) 筛选订单表数据

通过该表获取 user\_id 和 province\_id。保留 type 字段和 old 字段用于过滤订单明细数据和取消订单明细数据

### 5) 筛选订单明细活动关联表数据

通过该表获取活动 id 和活动规则 id

### 6) 筛选订单明细优惠券关联表数据；

通过该表获取优惠券 id。

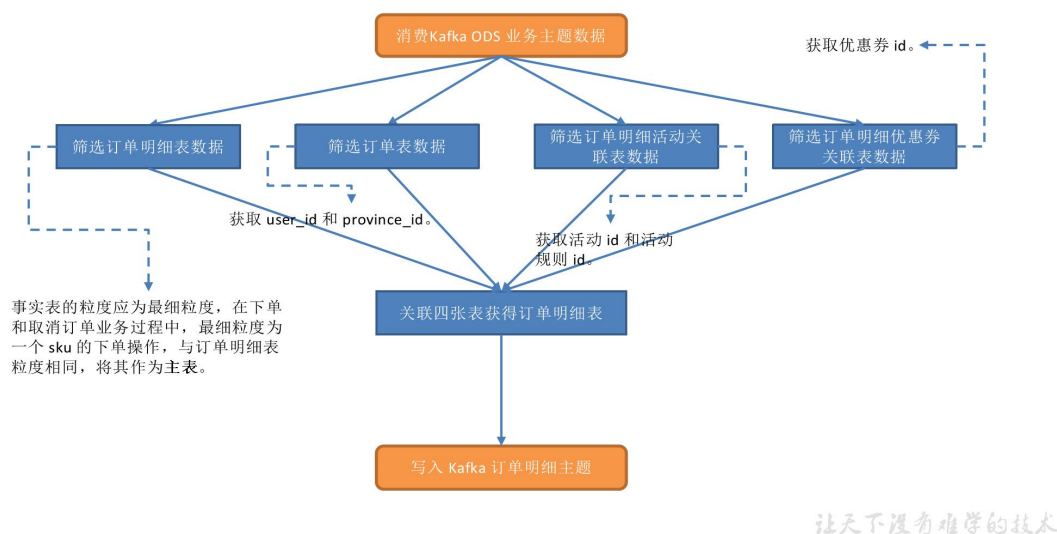
### 7) 关联上述四张表获得订单宽表，写入 Kafka 主题

事实表的粒度应为最细粒度，在下单和取消订单业务过程中，最细粒度为一个 sku 的下单或取消订单操作，与订单明细表粒度相同，将其作为主表。

(1) 订单明细表和订单表的所有记录在另一张表中都有对应数据，内连接即可。

(2) 订单明细数据未必参加了活动也未必使用了优惠券，因此要保留订单明细独有数据，所以与订单明细活动关联表和订单明细优惠券关联表的关联使用 left join。

## 11.5.4 图解



## 11.5.5 代码实现

### 1) 修改 SQLUtil

在 SQLUtil 中新增 getUpsertKafkaDDL 方法，用于生成 Upsert-Kafka SQL 建表语句

```
public static String getUpsertKafkaDDL(String topic) {  
    return "with(" +  
        " 'connector' = 'upsert-kafka'," +  
        " 'topic' = '" + topic + "'," +  
        " 'properties.bootstrap.servers' = '" +  
Constant.KAFKA_BROKERS + "'," +  
        " 'key.json.ignore-parse-errors' = 'true'," +  
        " 'value.json.ignore-parse-errors' = 'true'," +  
        " 'key.format' = 'json'," +  
        " 'value.format' = 'json' " +  
        " )";  
}
```

### 2) 主程序

在 realtime-dwd-trade-order-detail 模块的 com.atguigu.gmall.realtime.dwd.db.app 包下创建应用程序 DwdTradeOrderDetail，继承 BaseSQLApp，写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.db.app;  
  
import com.atguigu.gmall.realtime.common.base.BaseSQLApp;  
import com.atguigu.gmall.realtime.common.constant.Constant;  
import com.atguigu.gmall.realtime.common.util.SQLUtil;  
import
```

```
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.table.api.Table;
import
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

import java.time.Duration;

public class DwdTradeOrderDetail extends BaseSQLApp {
    public static void main(String[] args) {
        new DwdTradeOrderDetail().start(
            10014,
            4,
            Constant.TOPIC_DWD_TRADE_ORDER_DETAIL
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        StreamTableEnvironment tEnv) {
        // 因为有 join: 默认所有表的数据都会一致存储到内存中. 所以要加 ttl

tEnv.getConfig().setIdleStateRetention(Duration.ofSeconds(5));

        // 1. 读取 topic_db
        readOdsDb(tEnv, Constant.TOPIC_DWD_TRADE_ORDER_DETAIL);

        // 2. 过滤出 order_detail 数据: insert
        Table orderDetail = tEnv.sqlQuery(
            "select " +
            "data['id'] id," +
            "data['order_id'] order_id," +
            "data['sku_id'] sku_id," +
            "data['sku_name'] sku_name," +
            "data['create_time'] create_time," +
            "data['source_id'] source_id," +
            "data['source_type'] source_type," +
            "data['sku_num'] sku_num," +
            "cast(cast(data['sku_num'] as decimal(16,2)) * " +
            "    cast(data['order_price'] as decimal(16,2)) as
String) split_original_amount," + // 分摊原始总金额
            "data['split_total_amount'] split_total_amount," + //
```

分摊总金额

```
        "data['split_activity_amount']
split_activity_amount," + // 分摊活动金额
        "data['split_coupon_amount'] split_coupon_amount," +
// 分摊的优惠券金额
        "ts " +
        "from topic_db " +
        "where `database`='gmall' " +
        "and `table`='order_detail' " +
        "and `type`='insert' ");
tEnv.createTemporaryView("order_detail", orderDetail);

// 3. 过滤出 oder_info 数据: insert
Table orderInfo = tEnv.sqlQuery(
    "select " +
        "data['id'] id," +
        "data['user_id'] user_id," +
        "data['province_id'] province_id " +
        "from topic_db " +
        "where `database`='gmall' " +
        "and `table`='order_info' " +
        "and `type`='insert' ");
tEnv.createTemporaryView("order_info", orderInfo);

// 4. 过滤 order_detail_activity 表: insert
Table orderDetailActivity = tEnv.sqlQuery(
    "select " +
        "data['order_detail_id'] order_detail_id, " +
        "data['activity_id'] activity_id, " +
        "data['activity_rule_id'] activity_rule_id " +
        "from topic_db " +
        "where `database`='gmall' " +
        "and `table`='order_detail_activity' " +
        "and `type`='insert' ");
tEnv.createTemporaryView("order_detail_activity",
orderDetailActivity);

// 5. 过滤 order_detail_coupon 表: insert
Table orderDetailCoupon = tEnv.sqlQuery(
    "select " +
        "data['order_detail_id'] order_detail_id, " +
        "data['coupon_id'] coupon_id " +
        "from topic_db " +
        "where `database`='gmall' " +
```

```
        "and `table`='order_detail_coupon' " +
        "and `type`='insert' ");
    tEnv.createTemporaryView("order_detail_coupon",
orderDetailCoupon);
    //orderDetailCoupon.execute().print();

// 6. 四张表 join:
Table result = tEnv.sqlQuery(
    "select " +
        "od.id," +
        "od.order_id," +
        "oi.user_id," +
        "od.sku_id," +
        "od.sku_name," +
        "oi.province_id," +
        "act.activity_id," +
        "act.activity_rule_id," +
        "cou.coupon_id," +
        "date_format(od.create_time, 'yyyy-MM-dd') date_id," +
// 年月日
        "od.create_time," +
        "od.sku_num," +
        "od.split_original_amount," +
        "od.split_activity_amount," +
        "od.split_coupon_amount," +
        "od.split_total_amount," +
        "od.ts " +
        "from order_detail od " +
        "join order_info oi on od.order_id=oi.id " +
        "left join order_detail_activity act " +
        "on od.id=act.order_detail_id " +
        "left join order_detail_coupon cou " +
        "on od.id=cou.order_detail_id ");

// 7. 写出到 kafka 中
tEnv.executeSql(
    "create table dwd_trade_order_detail(" +
        "id string," +
        "order_id string," +
        "user_id string," +
        "sku_id string," +
        "sku_name string," +
        "province_id string," +
        "activity_id string," +
```



```
        "activity_rule_id string," +
        "coupon_id string," +
        "date_id string," +
        "create_time string," +
        "sku_num string," +
        "split_original_amount string," +
        "split_activity_amount string," +
        "split_coupon_amount string," +
        "split_total_amount string," +
        "ts bigint," +
        "primary key(id) not enforced " +
        ") " +

SQLUtil.getUpsertKafkaDDL(Constant.TOPIC_DWD_TRADE_ORDER_DETAIL))
;

result.executeInsert(Constant.TOPIC_DWD_TRADE_ORDER_DETAIL);
    }
}
```

### 11.5.6 测试

- 1) 启动 zk、kafka、maxwell
- 2) 运行 DwdTradeOrderDetail
- 3) 启动命令行消费者监控 kafka 主题 dwd\_trade\_order\_detail
- 4) 运行数据生成器
- 5) 查看命令行 Kafka 消费者的输出

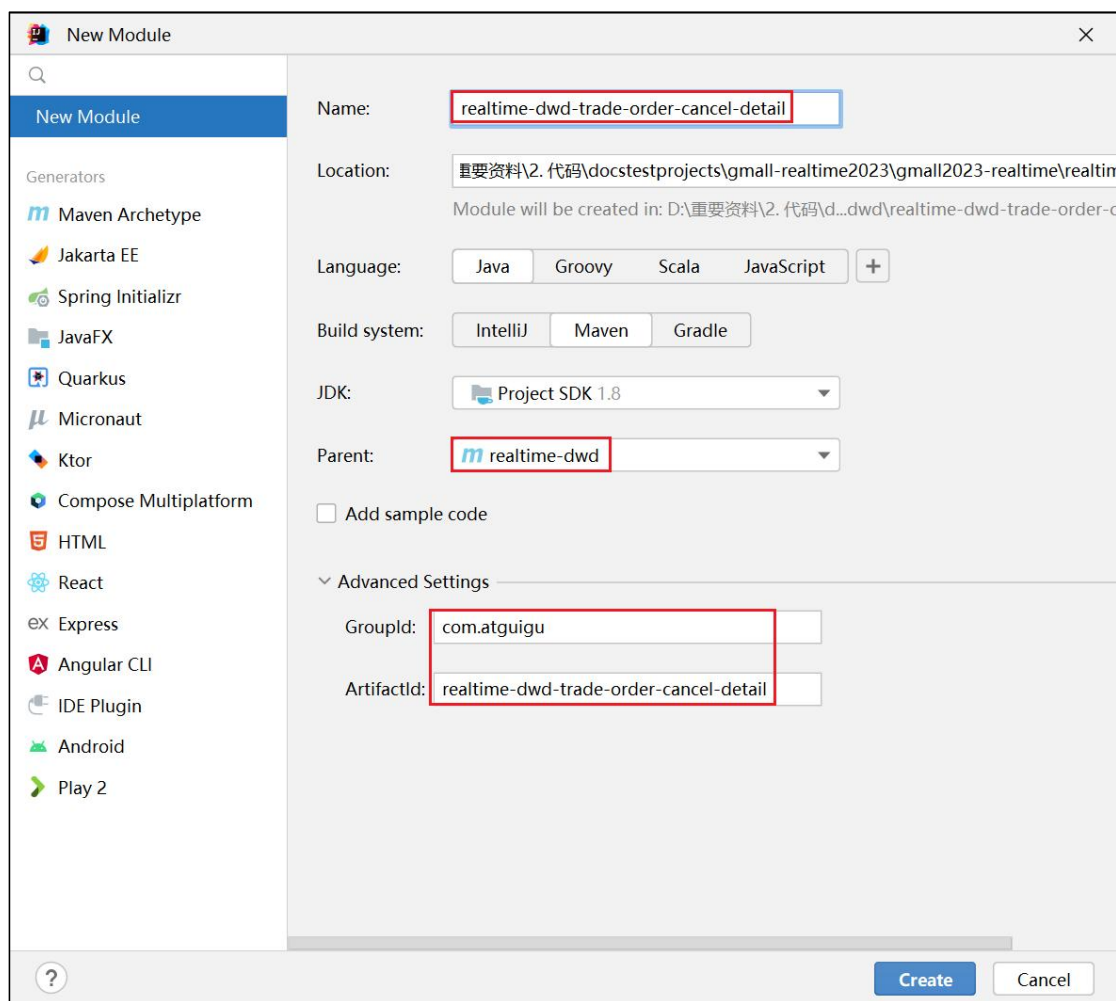
### 11.5.7 提交至 GitLab

略。

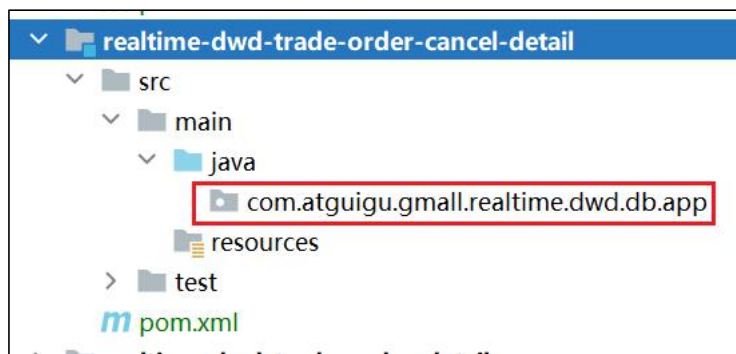
## 11.6 交易域取消订单事务事实表（练习）

### 11.6.1 创建 realtime-dwd-trade-order-cancel-detail 模块

- 1) 新建模块



## 2) 创建包结构



### 11.6.2 主要任务

从 Kafka 读取 topic\_db 主题数据，关联筛选订单明细表、取消订单数据、订单明细活动关联表、订单明细优惠券关联表四张事实业务表形成取消订单明细表，写入 Kafka 对应主题。

### 11.6.3 思路分析

### 1) 设置 ttl

取消订单时只有订单表的状态发生变化,要和其它三张表关联,就需要考虑取消订单的延迟,通常在支付前均可取消订单,因此将 ttl 设置为 15min + 5s。

### 2) 从 Kafka topic\_db 主题读取业务数据

### 3) 筛选订单明细表数据

应尽可能保证事实表的粒度为最细粒度,在取消订单业务过程中,最细粒度的事件为一个订单的一个 SKU 的取消订单操作,订单明细表的粒度与最细粒度相同,将其作为主表。筛选 insert 操作类型的数据即可。

### 4) 筛选取消订单数据

- (1) 通过该表获取 user\_id 和 province\_id。筛选取消订单数据,要满足如下三个条件:
- (2) 操作类型为 update;
- (3) 修改了 order\_status 字段;
- (4) 修改后的 order\_status 字段值为 1003。

### 5) 筛选订单明细活动关联表数据

通过该表获取活动 id 和活动规则 id。筛选 insert 操作类型的数据。

### 6) 筛选订单明细优惠券关联表数据

通过该表获取优惠券 id。筛选 insert 操作类型的数据。

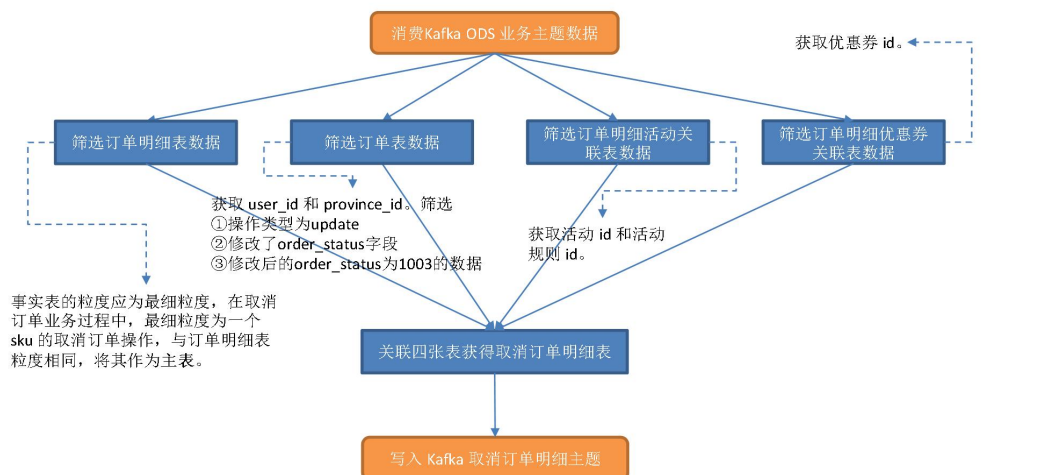
### 7) 关联上述四张表获得取消订单宽表,写入 Kafka 主题

事实表的粒度应为最细粒度,在取消订单业务过程中,最细粒度为一个 sku 的取消订单操作,与订单明细表粒度相同,将其作为主表。

(1) 只需要保留取消订单相关的明细数据,订单明细表与取消订单明细数据内连接即可。

(2) 订单明细数据未必参加了活动也未必使用了优惠券,因此要保留订单明细独有数据,所以与订单明细活动关联表和订单明细优惠券关联表的关联使用 left join。

## 11.6.4 图解



让天下没有难学的技术

## 11.6.5 代码实现

在 realtime-dwd-trade-order-cancel-detail 模块的 com.atguigu.gmall.realtime.dwd.db.app 包下创建应用程序 DwdTradeOrderCancelDetail，继承 BaseSQLApp，写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.db.app;

import com.atguigu.gmall.realtime.common.base.BaseSQLApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.SQLUtil;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.table.api.Table;
import org.apache.flink.table.api.bridge.java.StreamTableEnvironment;
import java.time.Duration;

public class DwdTradeOrderCancelDetail extends BaseSQLApp {
    public static void main(String[] args) {
        new DwdTradeOrderCancelDetail().start(
            10015,
            4,
            Constant.TOPIC_DWD_TRADE_ORDER_CANCEL
        );
    }
}
```

```
@Override
public void handle(StreamExecutionEnvironment env,
                  StreamTableEnvironment tEnv) {

    tEnv.getConfig().setIdleStateRetention(Duration.ofSeconds(30 * 60 +
    5));

    // 1. 读取 topic_db 数据
    readOdsDb(tEnv, Constant.TOPIC_DWD_TRADE_ORDER_CANCEL);
    // 2. 读取 dwd 层下单事务事实表数据
    tEnv.executeSql(
        "create table dwd_trade_order_detail(" +
            "id string," +
            "order_id string," +
            "user_id string," +
            "sku_id string," +
            "sku_name string," +
            "province_id string," +
            "activity_id string," +
            "activity_rule_id string," +
            "coupon_id string," +
            "date_id string," +
            "create_time string," +
            "sku_num string," +
            "split_original_amount string," +
            "split_activity_amount string," +
            "split_coupon_amount string," +
            "split_total_amount string," +
            "ts bigint " +
            ")" +

        SQLUtil.getKafkaDDLSource(Constant.TOPIC_DWD_TRADE_ORDER_CANCEL,
        Constant.TOPIC_DWD_TRADE_ORDER_DETAIL));

    // 3. 从 topic_db 过滤出订单取消数据
    Table orderCancel = tEnv.sqlQuery("select " +
        " `data`['id'] id, " +
        " `data`['operate_time']
operate_time, " +
        " `ts` " +
        "from topic_db " +
        "where `database`='gmall' " +
        "and `table`='order_info' " +
        "and `type`='update' " +
        "and
```

```
`old`['order_status']='1001' " +
                                "and
`data`['order_status']='1003' ");
    tEnv.createTemporaryView("order_cancel", orderCancel);

// 4. 订单取消表和下单表进行 join
Table result = tEnv.sqlQuery(
    "select " +
        "od.id," +
        "od.order_id," +
        "od.user_id," +
        "od.sku_id," +
        "od.sku_name," +
        "od.province_id," +
        "od.activity_id," +
        "od.activity_rule_id," +
        "od.coupon_id," +
        "date_format(oc.operate_time, 'yyyy-MM-dd')
order_cancel_date_id," +
        "oc.operate_time," +
        "od.sku_num," +
        "od.split_original_amount," +
        "od.split_activity_amount," +
        "od.split_coupon_amount," +
        "od.split_total_amount," +
        "oc.ts " +
        "from dwd_trade_order_detail od " +
        "join order_cancel oc " +
        "on od.order_id=oc.id ");

// 5. 写出
tEnv.executeSql(
    "create table dwd_trade_order_cancel (" +
        "id string," +
        "order_id string," +
        "user_id string," +
        "sku_id string," +
        "sku_name string," +
        "province_id string," +
        "activity_id string," +
        "activity_rule_id string," +
        "coupon_id string," +
        "date_id string," +
        "cancel_time string," +
```

```
        "sku_num string," +
        "split_original_amount string," +
        "split_activity_amount string," +
        "split_coupon_amount string," +
        "split_total_amount string," +
        "ts bigint " +
        ") " +
SQLUtil.getKafkaDDLSink(Constant.TOPIC_DWD_TRADE_ORDER_CANCEL));

result.executeInsert(Constant.TOPIC_DWD_TRADE_ORDER_CANCEL);
    }
}
```

### 11.6.6 测试

- 1) 启动 zk、kafka、maxwell
- 2) 运行 DwdTradeOrderDetail 及 DwdTradeOrderCancelDetail
- 3) 启动命令行消费者监控 kafka 主题 dwd\_trade\_order\_cancel\_detail
- 4) 运行数据生成器
- 5) 查看命令行 Kafka 消费者的输出

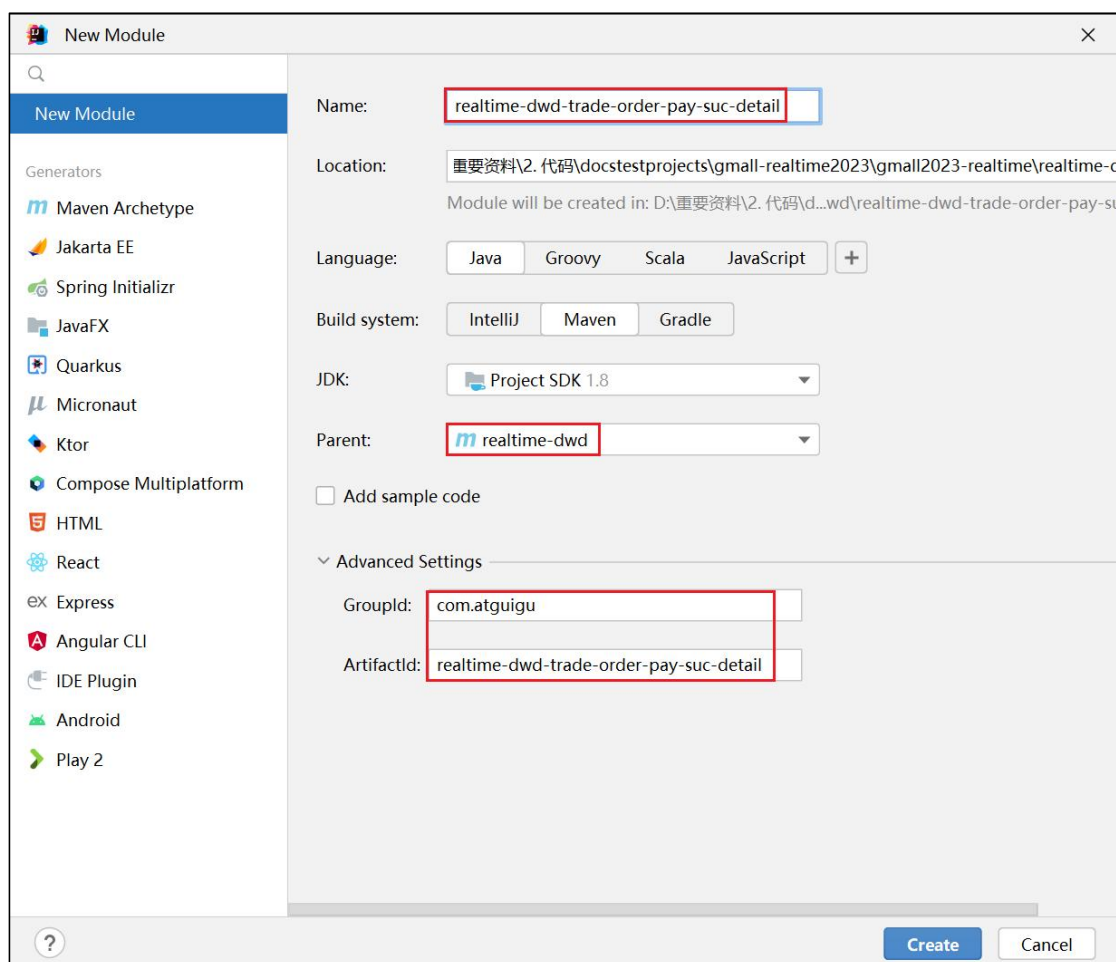
### 11.6.7 提交至 GitLab

略。

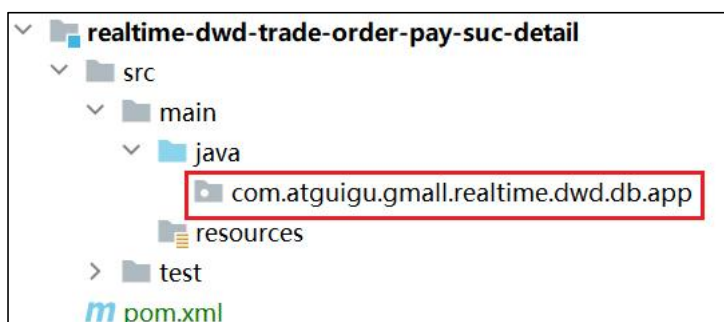
## 11.7 交易域支付成功事务事实表

### 11.7.1 创建 realtime-dwd-trade-order-pay-suc-detail 模块

- 1) 新建模块



## 2) 创建包结构



## 11.7.2 主要任务

从 Kafka topic\_db 主题筛选支付成功数据、从 dwd\_trade\_order\_detail 主题中读取订单事实数据、LookUp 字典表，关联三张表形成支付成功宽表，写入 Kafka 支付成功主题。

## 11.7.3 思路分析

### 11.7.3.1 知识储备

#### 1) 内置函数



本节涉及时间的转换，用到的内置函数如下。

(1) `TO_TIMESTAMP(string1[, string2])`: 将指定格式的日期字符串转化为 Flink SQL 的 `timestamp` 数据类型。第一个参数为格式化日期字符串，第二个参数为日期格式，数据类型也是字符串，可以省略，默认的日期格式为 `'yyyy-MM-dd HH:mm:ss'`。

(2) `FROM_UNIXTIME(numeric[, string])`: 将时间戳转换为指定的格式化字符串。第一个参数是数值，以秒为单位的时间戳，第二个参数是字符串，为日期格式，可以省略，默认为 `'yyyy-MM-dd HH:mm:ss'`。

(3) `CAST(value AS type)`: 将 `value` 的数据类型强制更改为 `type`。

### 11.7.3.2 执行步骤

#### 1) 设置 ttl

本节设置了事件时间，通过 `Interval Join` 实现了左右流的状态管理，无须设置 `ttl`。

#### 2) 获取下单明细数据

用户必然要先下单才有可能支付成功，因此支付成功明细数据集必然是订单明细数据集的子集。要注意：`Interval Join` 要求表中均为 `Append` 数据，即“只能新增，不能修改”，订单明细表数据生成过程中用到了 `left join`，生成了回撤流，看似不满足 `Interval Join` 的条件。但是，回撤数据进入 `Kafka` 会以 `null` 值形式存在，如果用 `Kafka Connector` 将订单明细封装为动态表，`null` 值会被过滤，最终得到的是相同主键存在重复数据的 `Append` 流（动态表本质上就是流），满足 `Interval Join` 的条件。

#### 3) 筛选支付表数据

获取支付类型、回调时间（支付成功时间）、支付成功时间戳。

生产环境下，用户支付后，业务数据库的支付表会插入一条数据，此时的回调时间和回调内容为空。通常底层会调用第三方支付接口，接口会返回回调信息，如果支付成功则回调信息不为空，此时会更新支付表，补全回调时间和回调内容字段的值，并将 `payment_status` 字段的值修改为支付成功对应的状态码（本项目为 1602）。支付成功之后，支付表数据不会发生变化。因此，只要操作类型为 `update` 且状态码为 1602 即为支付成功数据。

由上述分析可知，支付成功对应的业务数据库变化日志应满足两个条件：

(1) `payment_status` 字段的值为 1602；

(2) 操作类型为 `update`。

#### 4) 构建 LookUp 字典表

### 5) 关联上述三张表形成支付成功宽表，写入 Kafka 支付成功主题

支付成功业务过程的最细粒度为一个订单下一个 sku 的支付成功记录。从 topic\_db 主题筛选的支付成功数据与字典表关联后粒度不变，为一个订单的支付成功记录，再关联订单明细表，粒度与支付成功业务过程的最细粒度相同。

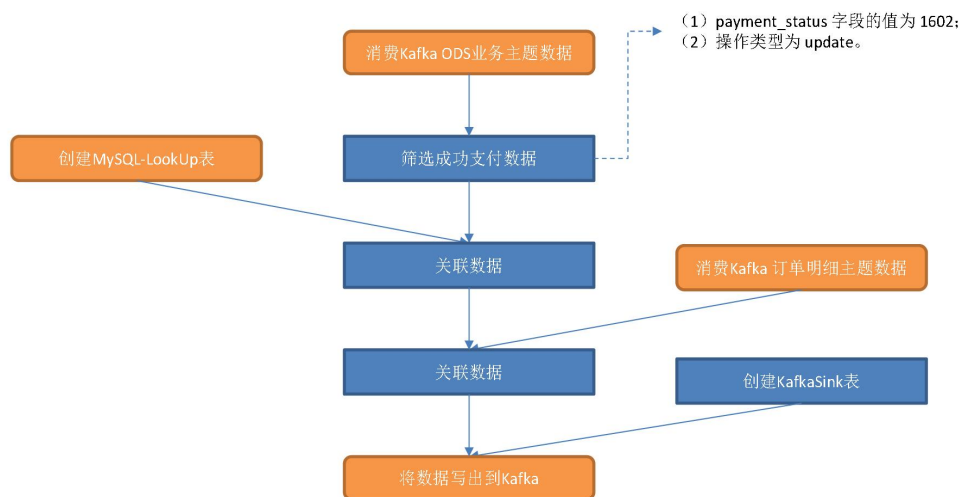
(1) 支付成功数据同字典表关联，获取 payment\_type 对应的支付类型名称，此时字典表相当于维表，不是以流的形式存在，当主流数据到来后去 MySQL 中获取对应的维度数据即可，因此使用内连接即可。下文与字典表的关联同理，不再赘述。

(2) 通过 Interval Join 将下单明细表与上面的结果表关联。订单明细数据的产生必然早于支付数据，通常下单后最晚 15min 内完成支付，超时订单取消。此外，支付数据和订单明细数据位于两条流中，没有严格的先后关系，当支付时间和下单时间非常接近时，可能由于网络问题导致支付数据先到，因此支付数据也要在状态中保留一段时间。本节要求订单明细数据的生成时间处于支付数据生成时间之前 15min 和之后 5s 范围内。

## 11.7.4 图解



业务数据DWD层支付成功明细代码流程图



让天下没有难学的技术

## 11.7.5 代码实现

在 realtime-dwd-trade-order-pay-suc-detail 模块的 com.atguigu.gmall.realtime.dwd.db.app 包下创建应用程序 DwdTradeOrderPaySucDetail，继承 BaseSQLApp，写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.db.app;
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
import com.atguigu.gmall.realtime.common.base.BaseSQLApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.SQLUtil;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

import org.apache.flink.table.api.Table;
import
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

public class DwdTradeOrderPaySucDetail extends BaseSQLApp {
    public static void main(String[] args) {
        new DwdTradeOrderPaySucDetail().start(
            10016,
            4,
            Constant.TOPIC_DWD_TRADE_ORDER_PAYMENT_SUCCESS
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        StreamTableEnvironment tEnv) {
        // 1. 读取下单事务事实表
        tEnv.executeSql(
            "create table dwd_trade_order_detail(" +
                "id string," +
                "order_id string," +
                "user_id string," +
                "sku_id string," +
                "sku_name string," +
                "province_id string," +
                "activity_id string," +
                "activity_rule_id string," +
                "coupon_id string," +
                "date_id string," +
                "create_time string," +
                "sku_num string," +
                "split_original_amount string," +
                "split_activity_amount string," +
                "split_coupon_amount string," +
                "split_total_amount string," +
                "ts bigint," +
                "et as to_timestamp_ltz(ts, 0), " +
```

```
        "watermark for et as et - interval '3' second " +
        ") " +
SQLUtil.getKafkaDDLSource(Constant.TOPIC_DWD_TRADE_ORDER_PAYMENT_
SUCCESS, Constant.TOPIC_DWD_TRADE_ORDER_DETAIL));

// 2. 读取 topic_db
readOdsDb(tEnv,
Constant.TOPIC_DWD_TRADE_ORDER_PAYMENT_SUCCESS);

// 3. 读取 字典表
readBaseDic(tEnv);

// 4. 从 topic_db 中过滤 payment_info
Table paymentInfo = tEnv.sqlQuery("select " +
                                "data['user_id'] user_id," +
                                "data['order_id'] order_id,"
+
                                "data['payment_type']
payment_type," +
                                "data['callback_time']
callback_time," +
                                "`pt`," +
                                "ts, " +
                                "et " +
                                "from topic_db " +
                                "where `database`='gmall' " +
                                "and `table`='payment_info'
" +
                                "and `type`='update' " +
                                "and `old`['payment_status']
is not null " +
                                "and
`data`['payment_status']='1602' ");
tEnv.createTemporaryView("payment_info", paymentInfo);

// 5. 3张 join: interval join 无需设置 ttl
Table result = tEnv.sqlQuery(
    "select " +
        "od.id order_detail_id," +
        "od.order_id," +
        "od.user_id," +
        "od.sku_id," +
        "od.sku_name," +
        "od.province_id," +
```

```
"od.activity_id," +
"od.activity_rule_id," +
"od.coupon_id," +
"pi.payment_type payment_type_code ," +
"dic.dic_name payment_type_name," +
"pi.callback_time," +
"od.sku_num," +
"od.split_original_amount," +
"od.split_activity_amount," +
"od.split_coupon_amount," +
"od.split_total_amount split_payment_amount," +
"pi.ts " +
"from payment_info pi " +
"join dwd_trade_order_detail od " +
"on pi.order_id=od.order_id " +
"and od.et >= pi.et - interval '30' minute " +
"and od.et <= pi.et + interval '5' second " +
"join base_dic for system_time as of pi.pt as dic " +
"on pi.payment_type=dic.dic_code ");

// 6. 写出到 kafka 中
tEnv.executeSql("create table
dwd_trade_order_payment_success (" +
    "order_detail_id string," +
    "order_id string," +
    "user_id string," +
    "sku_id string," +
    "sku_name string," +
    "province_id string," +
    "activity_id string," +
    "activity_rule_id string," +
    "coupon_id string," +
    "payment_type_code string," +
    "payment_type_name string," +
    "callback_time string," +
    "sku_num string," +
    "split_original_amount string," +
    "split_activity_amount string," +
    "split_coupon_amount string," +
    "split_payment_amount string," +
    "ts bigint " +
    ")" +
SQLUtil.getKafkaDDLSink(Constant.TOPIC_DWD_TRADE_ORDER_PAYMENT_SUCCESS));
```

```
result.executeInsert(Constant.TOPIC_DWD_TRADE_ORDER_PAYMENT_SUCCESS);  
    }  
}
```

### 11.7.6 测试

- 1) 启动 zk、kafka、maxwell
- 2) 运行 DwdTradeOrderDetail 及 DwdTradeOrderPaySucDetail
- 3) 启动命令行消费者监控 kafka 主题 dwd\_trade\_order\_pay\_suc\_detail
- 4) 运行数据生成器
- 5) 查看命令行 Kafka 消费者的输出

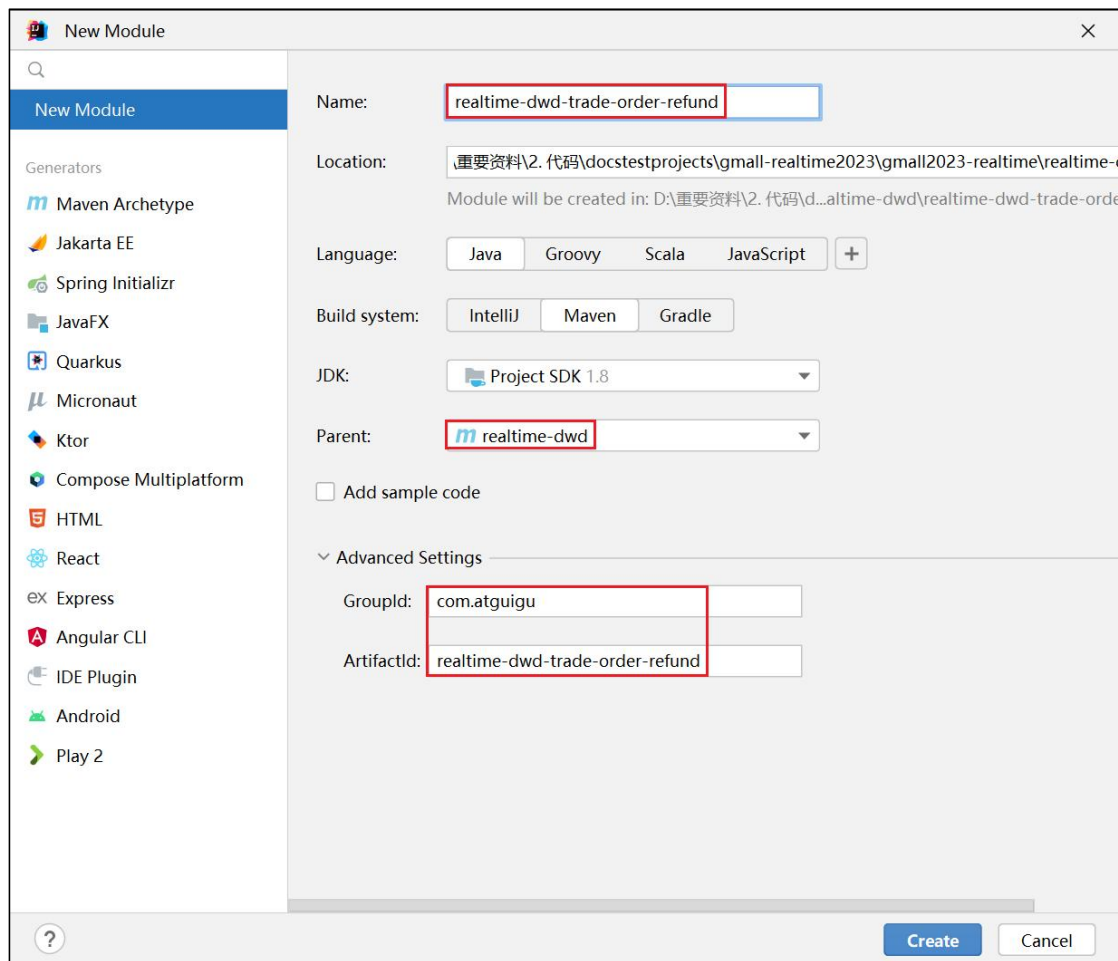
### 11.7.7 提交至 GitLab

略。

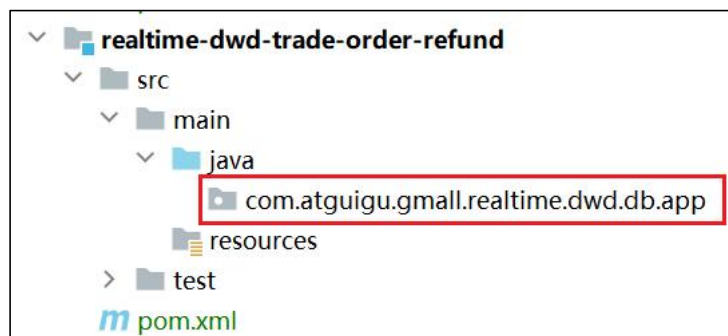
## 11.8 交易域退单事务事实表（练习）

### 11.8.1 创建 realtime-dwd-trade-order-refund 模块

- 1) 新建模块



## 2) 创建包结构



## 11.8.2 主要任务

从 Kafka 读取业务数据，筛选退单表数据，筛选满足条件的订单表数据，建立 MySQL-Lookup 字典表，关联三张表获得退单明细宽表。

## 11.8.3 思路分析

### 1) 设置 ttl

用户执行一次退单操作时，`order_refund_info` 会插入多条数据，同时 `order_info` 表的一条对应数据会发生修改，所以两张表不存在业务上的时间滞后问题，因此仅考虑可能的乱序即可，`ttl` 设置为 5s。

## 2) 筛选退单表数据

退单业务过程最细粒度的操作为一个订单中一个 SKU 的退单操作，退单表粒度与最细粒度相同，将其作为主表。

## 3) 筛选订单表数据

获取 `province_id` 维度。退单操作发生时，订单表的 `order_status` 字段值会由 1002（已支付）更新为 1005（退款中）。订单表中的数据要满足三个条件：

- (1) `order_status` 为 1005（退款中）；
- (2) 操作类型为 `update`；
- (3) 更新的字段为 `order_status`。

该字段发生变化时，变更数据中 `old` 字段下 `order_status` 的值不为 `null`（为 1002）。

## 4) 建立 MySQL-Lookup 字典表

获取退款类型名称和退款原因类型名称。

## 5) 关联这几张表获得退单明细宽表，写入 Kafka 退单明细主题

退单信息表 `order_refund_info` 的粒度为退单业务过程的最细粒度，将其作为主表。

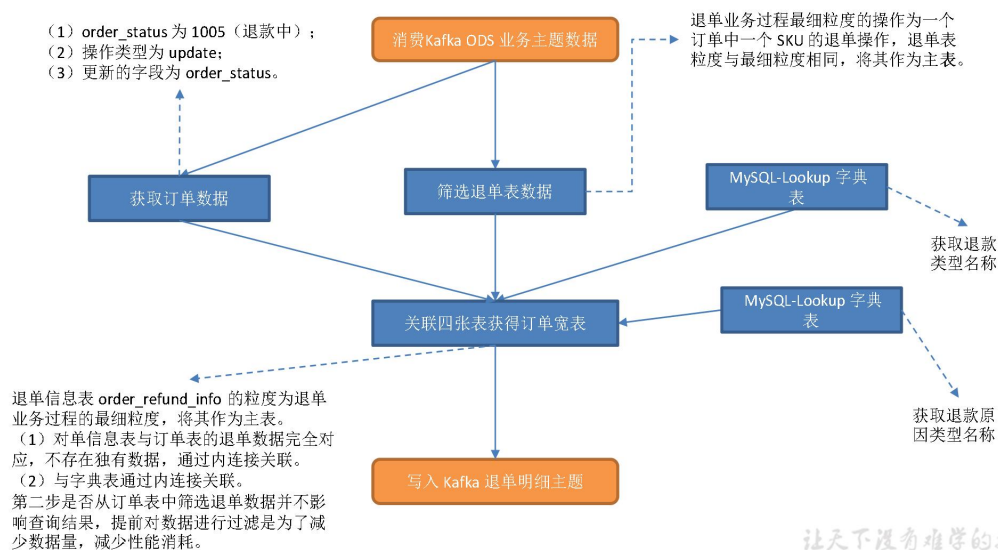
- (1) 退单信息表与订单表的退单数据完全对应，不存在独有数据，通过内连接关联
- (2) 与字典表通过内连接关联

第三步是否对订单表数据筛选并不影响查询结果，提前对数据进行过滤是为了减少数据量，减少资源消耗。下文同理，不再赘述。

## 11.8.4 图解



业务数据DWD层退单明细代码流程图



## 11.8.5 代码实现

在 realtime-dwd-trade-order-refund 模块的 com.atguigu.gmall.realtime.dwd.db.app 包下创建应用程序 DwdTradeOrderRefund, 继承 BaseSQLApp, 写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.db.app;

import com.atguigu.gmall.realtime.common.base.BaseSQLApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.SQLUtil;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.table.api.Table;
import org.apache.flink.table.api.bridge.java.StreamTableEnvironment;
import java.time.Duration;

public class DwdTradeOrderRefund extends BaseSQLApp {
    public static void main(String[] args) {
        new DwdTradeOrderRefund().start(
            10017,
            4,
            Constant.TOPIC_DWD_TRADE_ORDER_REFUND
        );
    }
}
```

```
@Override
public void handle(StreamExecutionEnvironment env,
                  StreamTableEnvironment tEnv) {

    tEnv.getConfig().setIdleStateRetention(Duration.ofSeconds(5));
    // 1.1 读取 topic_db
    readOdsDb(tEnv, Constant.TOPIC_DWD_TRADE_ORDER_REFUND);
    // 1.2 读取 字典表
    readBaseDic(tEnv);

    // 2. 过滤退单表数据 order_refund_info insert
    Table orderRefundInfo = tEnv.sqlQuery(
        "select " +
            "data['id'] id," +
            "data['user_id'] user_id," +
            "data['order_id'] order_id," +
            "data['sku_id'] sku_id," +
            "data['refund_type'] refund_type," +
            "data['refund_num'] refund_num," +
            "data['refund_amount'] refund_amount," +
            "data['refund_reason_type'] refund_reason_type," +
            "data['refund_reason_txt'] refund_reason_txt," +
            "data['create_time'] create_time," +
            "pt," +
            "ts " +
            "from topic_db " +
            "where `database`='gmall' " +
            "and `table`='order_refund_info' " +
            "and `type`='insert' ");
    tEnv.createTemporaryView("order_refund_info",
    orderRefundInfo);

    // 3. 过滤订单表中的退单数据: order_info update
    Table orderInfo = tEnv.sqlQuery(
        "select " +
            "data['id'] id," +
            "data['province_id'] province_id," +
            "`old` " +
            "from topic_db " +
            "where `database`='gmall' " +
            "and `table`='order_info' " +
            "and `type`='update' " +
            "and `old`['order_status'] is not null " +
```

```
        "and `data`['order_status']='1005' ");
tEnv.createTemporaryView("order_info", orderInfo);

// 4. join: 普通的和 lookup join
Table result = tEnv.sqlQuery(
    "select " +
        "ri.id," +
        "ri.user_id," +
        "ri.order_id," +
        "ri.sku_id," +
        "oi.province_id," +
        "date_format(ri.create_time,'yyyy-MM-dd') date_id," +
        "ri.create_time," +
        "ri.refund_type," +
        "dic1.info.dic_name," +
        "ri.refund_reason_type," +
        "dic2.info.dic_name," +
        "ri.refund_reason_txt," +
        "ri.refund_num," +
        "ri.refund_amount," +
        "ri.ts " +
        "from order_refund_info ri " +
        "join order_info oi " +
        "on ri.order_id=oi.id " +
        "join base_dic for system_time as of ri.pt as dic1 " +
        "on ri.refund_type=dic1.dic_code " +
        "join base_dic for system_time as of ri.pt as dic2 " +
        "on ri.refund_reason_type=dic2.dic_code ");

// 5. 写出到 kafka
tEnv.executeSql(
    "create table dwd_trade_order_refund(" +
        "id string," +
        "user_id string," +
        "order_id string," +
        "sku_id string," +
        "province_id string," +
        "date_id string," +
        "create_time string," +
        "refund_type_code string," +
        "refund_type_name string," +
        "refund_reason_type_code string," +
        "refund_reason_type_name string," +
        "refund_reason_txt string," +
```

```
        "refund_num string," +  
        "refund_amount string," +  
        "ts bigint " +  
        ") " +  
SQLUtil.getKafkaDDLSink(Constant.TOPIC_DWD_TRADE_ORDER_REFUND));  
  
result.executeInsert(Constant.TOPIC_DWD_TRADE_ORDER_REFUND);  
  
    }  
}
```

## 11.8.6 测试

- 1) 启动 zk、kafka、maxwell
- 2) 运行 DwdTradeOrderRefund
- 3) 启动命令行消费者监控 kafka 主题 dwd\_trade\_order\_refund
- 4) 运行数据生成器
- 5) 查看命令行 Kafka 消费者的输出

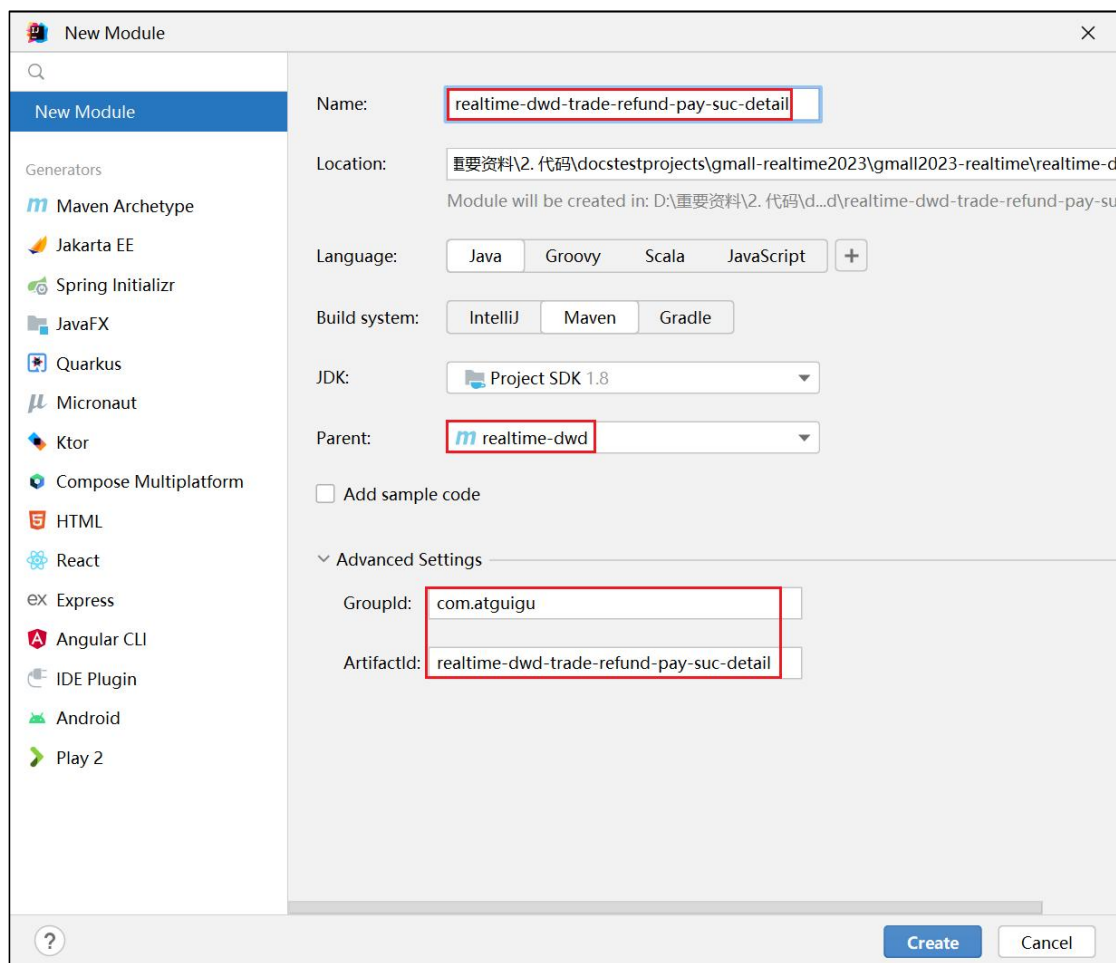
## 11.8.7 提交至 GitLab

略。

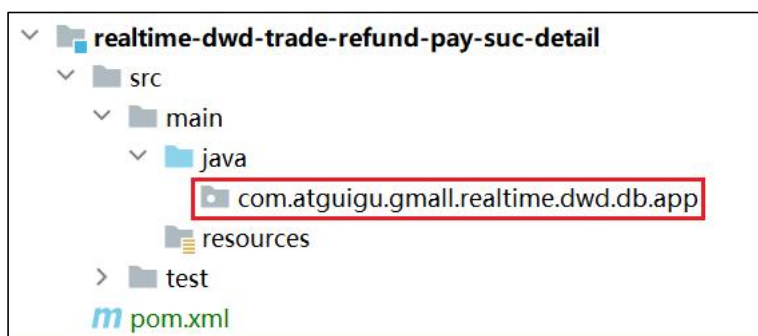
## 11.9 交易域退款成功事务事实表（练习）

### 11.9.1 创建 realtime-dwd-trade-refund-pay-suc-detail 模块

- 1) 新建模块



## 2) 创建包结构



## 11.9.2 主要任务

- (1) 从退款表中提取退款成功数据，并将字典表的 dic\_name 维度退化到表中
- (2) 从订单表中提取退款成功订单数据
- (3) 从退单表中提取退款成功的明细数据

## 11.9.3 思路分析

### 1) 设置 ttl

一次退款支付操作成功时，退款表 `refund_payment`、订单表 `order_info` 和退单表 `order_refund_info` 的对应数据会发生修改，几张表之间不存在业务时间上的滞后。与字典表的关联分析同上，不再赘述。因而，仅考虑可能的数据乱序即可。将 `ttl` 设置为 5s。

### 2) 建立 MySQL-Lookup 字典表

获取支付类型名称。

### 3) 读取退款表数据，筛选退款成功数据

退款表 `refund_payment` 的粒度为一个订单中一个 SKU 的退款记录，与退款业务过程的最细粒度相同，将其作为主表。

退款操作发生时，业务数据库的退款表会先插入一条数据，此时 `refund_status` 状态码为 1601（未支付），`callback_time` 为 null，片刻后退款完成时状态码由 1601 更新为 1602（已支付），同时将 `callback_time` 更新为退款支付成功的回调时间。

由上述分析可知，退款成功记录应满足三个条件：

- （1）数据操作类型为 `update`；
- （2）`refund_status` 为 1602；
- （3）修改的字段包含 `refund_status`。该条件的校验是通过判断 `old['refund_status']` 的值是否为 null 来实现的。

### 4) 读取订单表数据，过滤退款成功订单数据

用于获取 `user_id` 和 `province_id`。退款操作完后时，订单表的 `order_status` 字段会更新为 1006（退款完成），因此退单成功对应的订单数据应满足三个条件：

- （1）操作类型为 `update`；
- （2）`order_status` 为 1006；
- （3）修改了 `order_status` 字段。

`order_status` 值更改为 1006 之后对应的订单表数据就不会再发生变化，所以只要满足前两个条件，第三个条件必定满足。

### 5) 筛选退款成功的退单明细数据

用于获取退单件数 `refund_num`。退单成功时 `order_refund_info` 表中的 `refund_status` 字段会修改为 0705（退单完成状态码）。因此筛选条件有三：

- （1）操作类型为 `update`；

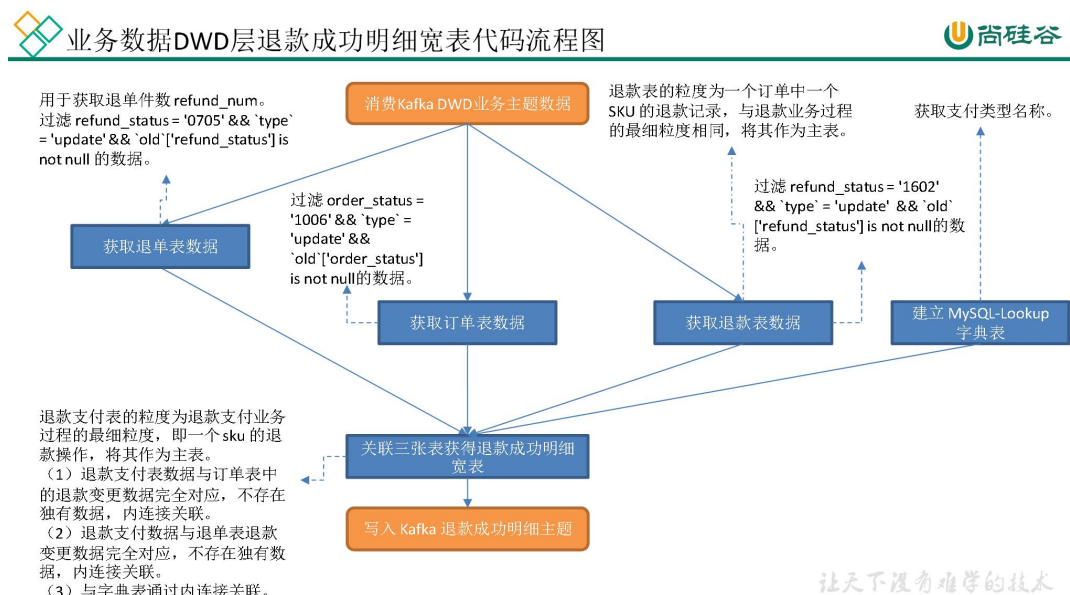
- (2) refund\_status 为 0705;
- (3) 修改了 refund\_status 字段。

#### 6) 关联四张表并写出到 Kafka 退款成功主题

退款支付表的粒度为退款支付业务过程的最细粒度，即一个 sku 的退款操作，将其作为主表。

- (1) 退款支付表数据与订单表中的退款变更数据完全对应，不存在独有数据，内连接关联。
- (2) 退款支付数据与退单表退款变更数据完全对应，不存在独有数据，内连接关联。
- (3) 与字典表通过内连接关联。

### 11.9.4 图解



### 11.9.5 代码实现

在 realtime-dwd-trade-refund-pay-suc-detail 模块的 com.atguigu.gmall.realtime.dwd.db.app 包下创建应用程序 DwdTradeRefundPaySucDetail，继承 BaseSQLApp，写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.db.app;

import com.atguigu.gmall.realtime.common.base.BaseSQLApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.SQLUtil;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
```

```
import org.apache.flink.table.api.Table;
import
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

import java.time.Duration;

public class DwdTradeRefundPaySucDetail extends BaseSQLApp {
    public static void main(String[] args) {
        new DwdTradeRefundPaySucDetail().start(
            10018,
            4,
            Constant.TOPIC_DWD_TRADE_REFUND_PAYMENT_SUCCESS
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        StreamTableEnvironment tEnv) {

        tEnv.getConfig().setIdleStateRetention(Duration.ofSeconds(5));

        // 1. 读取 topic_db
        readOdsDb(tEnv,
            Constant.TOPIC_DWD_TRADE_REFUND_PAYMENT_SUCCESS);
        // 2. 读取 字典表
        readBaseDic(tEnv);

        // 3. 过滤退款成功表数据
        Table refundPayment = tEnv.sqlQuery(
            "select " +
                "data['id'] id," +
                "data['order_id'] order_id," +
                "data['sku_id'] sku_id," +
                "data['payment_type'] payment_type," +
                "data['callback_time'] callback_time," +
                "data['total_amount'] total_amount," +
                "pt, " +
                "ts " +
                "from topic_db " +
                "where `database`='gmall' " +
                "and `table`='refund_payment' " +
                "and `type`='update' " +
                "and `old`['refund_status'] is not null " +
                "and `data`['refund_status']='1602'");
```



```
tEnv.createTemporaryView("refund_payment", refundPayment);

// 4. 过滤退单表中的退单成功的数据
Table orderRefundInfo = tEnv.sqlQuery(
    "select " +
        "data['order_id'] order_id," +
        "data['sku_id'] sku_id," +
        "data['refund_num'] refund_num " +
        "from topic_db " +
        "where `database`='gmall' " +
        "and `table`='order_refund_info' " +
        "and `type`='update' " +
        "and `old`['refund_status'] is not null " +
        "and `data`['refund_status']='0705'");
tEnv.createTemporaryView("order_refund_info",
orderRefundInfo);

// 5. 过滤订单表中的退款成功的数据
Table orderInfo = tEnv.sqlQuery(
    "select " +
        "data['id'] id," +
        "data['user_id'] user_id," +
        "data['province_id'] province_id " +
        "from topic_db " +
        "where `database`='gmall' " +
        "and `table`='order_info' " +
        "and `type`='update' " +
        "and `old`['order_status'] is not null " +
        "and `data`['order_status']='1006'");
tEnv.createTemporaryView("order_info", orderInfo);

// 6. 4 张表的 join
Table result = tEnv.sqlQuery(
    "select " +
        "rp.id," +
        "oi.user_id," +
        "rp.order_id," +
        "rp.sku_id," +
        "oi.province_id," +
        "rp.payment_type," +
        "dic.info.dic_name payment_type_name," +
        "date_format(rp.callback_time,'yyyy-MM-dd') date_id,"
+
        "rp.callback_time," +
```

```
        "ori.refund_num," +
        "rp.total_amount," +
        "rp.ts " +
        "from refund_payment rp " +
        "join order_refund_info ori " +
        "on rp.order_id=ori.order_id and rp.sku_id=ori.sku_id
" +

        "join order_info oi " +
        "on rp.order_id=oi.id " +
        "join base_dic for system_time as of rp.pt as dic " +
        "on rp.payment_type=dic.dic_code ");

// 7.写出到 kafka
tEnv.executeSql("create table
dwd_trade_refund_payment_success (" +
        "id string," +
        "user_id string," +
        "order_id string," +
        "sku_id string," +
        "province_id string," +
        "payment_type_code string," +
        "payment_type_name string," +
        "date_id string," +
        "callback_time string," +
        "refund_num string," +
        "refund_amount string," +
        "ts bigint " +
        ") " +

SQLUtil.getKafkaDDLSink(Constant.TOPIC_DWD_TRADE_REFUND_PAYMENT_S
UCCESS));

result.executeInsert(Constant.TOPIC_DWD_TRADE_REFUND_PAYMENT_SUCC
ESS);

    }
}
```

## 11.9.6 测试

- 1) 启动 zk、kafka、maxwell
- 2) 运行 DwdTradeRefundPaySucDetail
- 3) 启动命令行消费者监控 kafka 主题 dwd\_trade\_refund\_pay\_suc\_detail

4) 运行数据生成器

5) 查看命令行 Kafka 消费者的输出

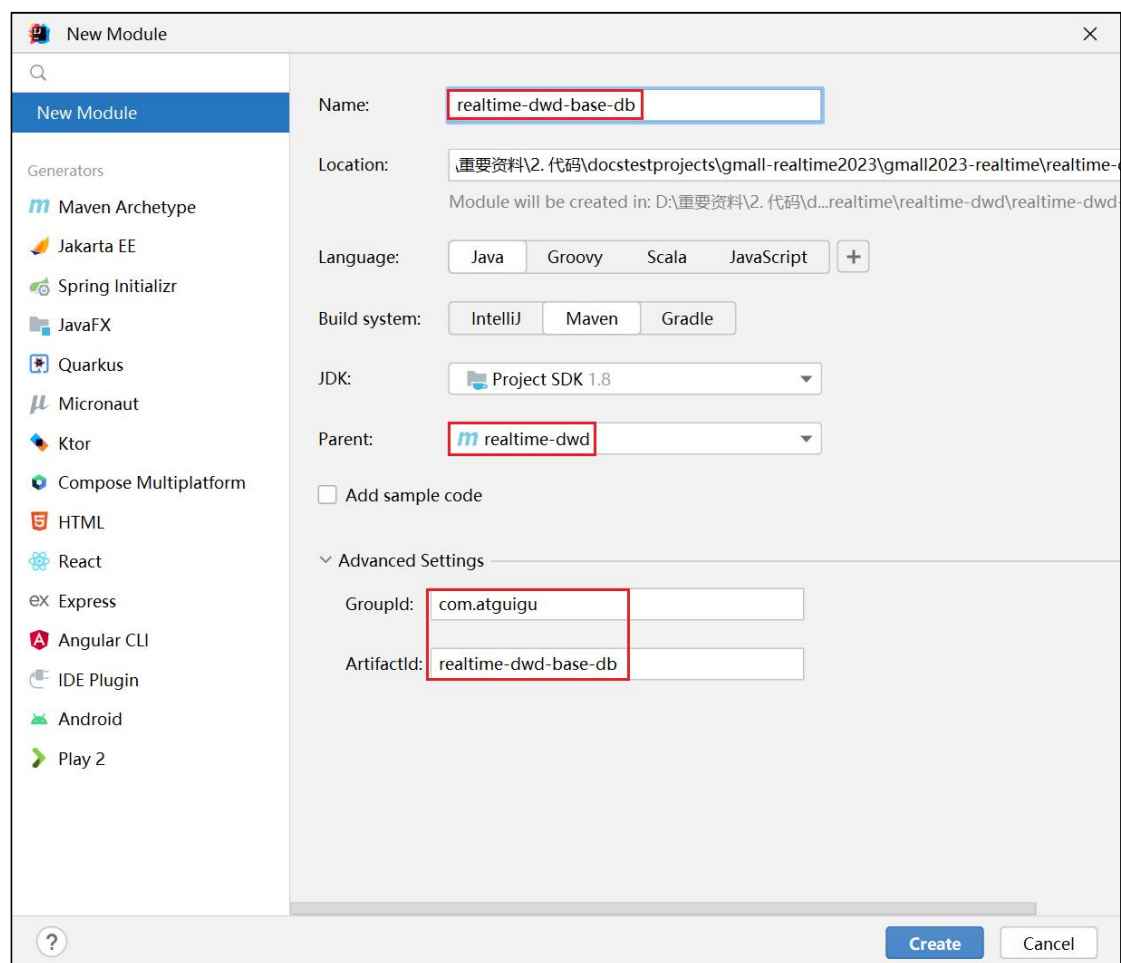
## 11.9.7 提交至 GitLab

略。

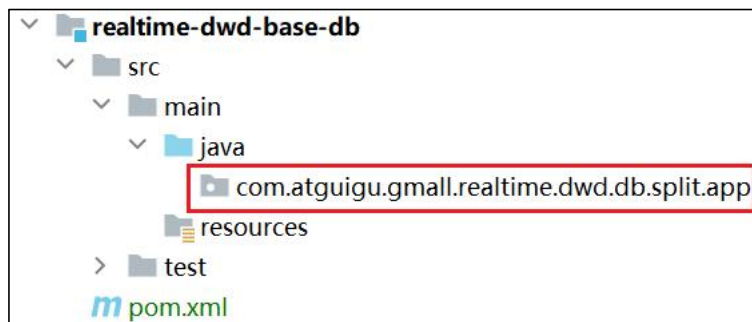
## 11.10 事实表动态分流

### 11.10.1 创建 realtime-dwd-base-db 模块

1) 新建模块



2) 创建包结构



## 11.10.2 主要任务

DWD 层余下的事实表都是从 topic\_db 中取业务数据库一张表的变更数据，按照某些条件过滤后写入 Kafka 的对应主题，它们处理逻辑相似且较为简单，可以结合配置表动态分流在同一个程序中处理。

注意：这里我们的优惠券使用和下单都是更新操作，我们放到一起，可以在后续 DWS 使用的时候，结合优惠券使用状态以及使用时间判断是优惠券下单还是支付操作。

## 11.10.3 思路分析

动态分流思路与 DIM 层同理。

### 11.10.3.1 配置表设计

1) 配置表字段取值略有差异。

- source\_table: 作为数据源的业务数据表名
- source\_type: 要筛选的原始数据操作类型
- sink\_table: 作为数据目的地的 Kafka 主题
- sink\_columns: 写出到 Kafka 的字段

2) 本节将生成四张事务事实表

工具域优惠券领取事务事实表

工具域优惠券使用（下单+支付）事务事实表

互动域收藏商品事务事实表

用户域用户注册事务事实表

3) 完整的配置表初始化语句如下



### 11.10.3.2 筛选条件

#### 1) 工具域优惠券领取事务事实表

用户领取优惠券后，业务数据库的优惠券领用表会新增一条数据，因此操作类型为 `insert` 的数据即优惠券领取数据。

#### 2) 工具域优惠券使用事务事实表

用户使用优惠券下单时，优惠券领用表的 `using_time` 字段会更新为下单时间；用户使用优惠券支付时，优惠券领用表的 `used_time` 字段会更新为支付时间，因此操作类型为 `update` 的数据即优惠券使用数据。

#### 3) 互动域收藏商品事务事实表

用户收藏商品时业务数据库的收藏表会插入一条数据，因此筛选操作类型为 `insert` 的数据即可。

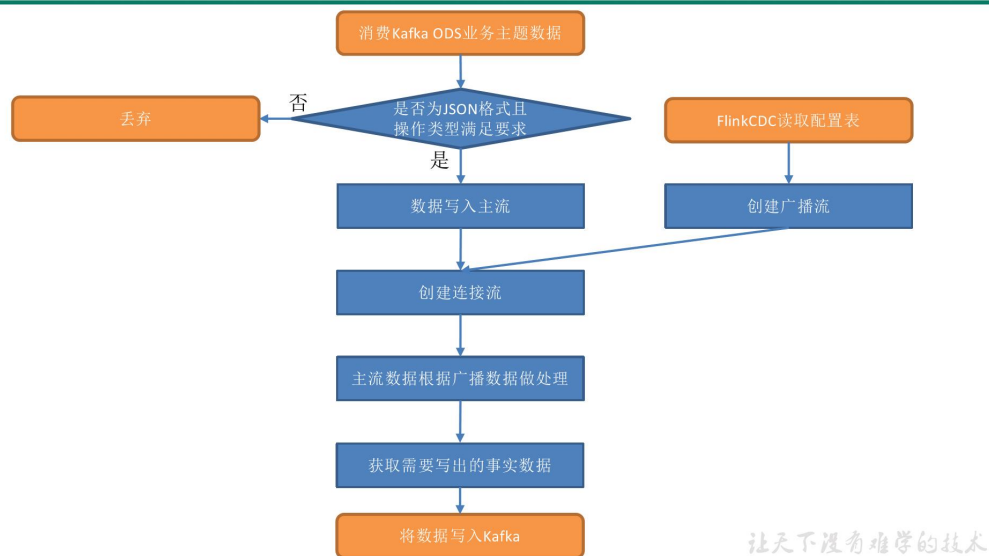
#### 4) 用户域用户注册事务事实表

用户注册时会在用户表中插入一条数据，筛选操作类型为 `insert` 的数据即可。

### 11.10.3.3 配置信息预加载

生产环境中，通常离线数仓的上线早于实时数仓，若采用 `Lambda` 架构，离线实时共用一套采集系统，实时数仓上线之前数据采集通道已在运行。启动分流应用，事实数据会立即进入主流，很可能满足条件的事实数据到来时配置表信息尚未加载，此时就会导致数据丢失。针对这个问题，此处同样需要对配置表信息进行预加载，加载方式与维度分流相同。

## 11.10.4 图解



让天下没有难学的技术

## 11.10.5 代码实现

### 1) 实体类 TableProcessDwd

在 common 模块的 bean 包下创建实体类 TableProcessDwd，如下。

```
package com.atguigu.gmall.realtime.common.bean;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
public class TableProcessDwd {
    // 来源表名
    String sourceTable;

    // 来源类型
    String sourceType;

    // 目标表名
    String sinkTable;

    // 输出字段
    String sinkColumns;

    // 配置表操作类型
    String op;
```

```
}
```

## 2) 修改工具类 FlinkSinkUtil

在 FlinkSinkUtil 中添加方法 getKafkaSink(), 如下。

```
public static Sink<Tuple2<JSONObject, TableProcessDwd>>
getKafkaSink() {
    return KafkaSink.<Tuple2<JSONObject, TableProcessDwd>>builder()
        .setBootstrapServers(Constant.KAFKA_BROKERS)
        .setRecordSerializer(new
KafkaRecordSerializationSchema<Tuple2<JSONObject,
TableProcessDwd>>() {
            @Nullable
            @Override
            public ProducerRecord<byte[], byte[]>
serialize(Tuple2<JSONObject, TableProcessDwd> dataWithConfig,
KafkaSinkContext context,
                                                    Long
timestamp) {
                String topic = dataWithConfig.f1.getSinkTable();
                JSONObject data = dataWithConfig.f0;
                data.remove("op_type");
                return new ProducerRecord<>(topic,
data.toJSONString().getBytes(StandardCharsets.UTF_8));
            }
        })
        .setDeliveryGuarantee(DeliveryGuarantee.EXACTLY_ONCE)
        .setTransactionalIdPrefix("atguigu-" + new
Random().nextLong())
        .setProperty("transaction.timeout.ms", 15 * 60 * 1000 + "")
        .build();
}
```

## 3) 主程序

在 realtime-dwd-base-db 模块的 com.atguigu.gmall.realtime.dwd.db.split.app 包下创建应用程序 DwdBaseDb, 继承 BaseApp, 写入以下内容。

```
package com.atguigu.gmall.realtime.dwd.db.split.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.bean.TableProcessDwd;
import com.atguigu.gmall.realtime.common.constant.Constant;
```

```
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import com.atguigu.gmall.realtime.common.util.JdbcUtil;
import com.ververica.cdc.connectors.mysql.source.MySqlSource;
import com.ververica.cdc.connectors.mysql.table.StartupOptions;
import
com.ververica.cdc.debezium.JsonDebeziumDeserializationSchema;
import lombok.extern.slf4j.Slf4j;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.FilterFunction;
import org.apache.flink.api.common.functions.MapFunction;
import org.apache.flink.api.common.state.BroadcastState;
import org.apache.flink.api.common.state.MapStateDescriptor;
import org.apache.flink.api.common.state.ReadOnlyBroadcastState;
import org.apache.flink.api.java.tuple.Tuple2;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.datastream.BroadcastStream;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera
tor;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import
org.apache.flink.streaming.api.functions.co.BroadcastProcessFunc
tion;
import org.apache.flink.util.Collector;

import java.util.*;

@Slf4j
public class DwdBaseDb extends BaseApp {
    public static void main(String[] args) {
        new DwdBaseDb().start(10019,
            4,
            "dwd_base_db",
            Constant.TOPIC_DB
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        DataStreamSource<String> stream) {
```



```
// 1. 对消费的数据, 做数据清洗
SingleOutputStreamOperator<JSONObject> etlStream =
etl(stream);
// 2. 通过 flink cdc 读取配置表的数据
SingleOutputStreamOperator<TableProcessDwd> configStream =
readTableProcess(env);
// 3. 数据流去 connect 配置流
SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDwd>> dataWithConfigStream = connect(etlStream,
configStream);
// 5. 删除不需要的字段
SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDwd>> resultStream =
deleteNotNeedColumns(dataWithConfigStream);
// 6. 写出到 Kafka 中
writeToKafka(resultStream);

}

private void
writeToKafka(SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDwd>> resultStream) {
    resultStream.sinkTo(FlinkSinkUtil.getKafkaSink());
}

private SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDwd>> deleteNotNeedColumns(
    SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDwd>> dataWithConfigStream) {
    return dataWithConfigStream
        .map(new MapFunction<Tuple2<JSONObject,
TableProcessDwd>, Tuple2<JSONObject, TableProcessDwd>>() {
            @Override
            public Tuple2<JSONObject, TableProcessDwd>
map(Tuple2<JSONObject, TableProcessDwd> dataWithConfig) throws
Exception {
                JSONObject data = dataWithConfig.f0;
                List<String> columns = new
ArrayList<>(Arrays.asList(dataWithConfig.f1.getSinkColumns()).split(", "));

                data.keySet().removeIf(key
```

```
-> !columns.contains(key));
        return dataWithConfig;
    }
    });
}

private SingleOutputStreamOperator<Tuple2<JSONObject,
TableProcessDwd>> connect(
    SingleOutputStreamOperator<JSONObject> dataStream,
    SingleOutputStreamOperator<TableProcessDwd> configStream)
{
    // 1. 把配置流做成广播流
    // key: 表名:type    user_info:ALL
    // value: TableProcess
    MapStateDescriptor<String, TableProcessDwd>
mapStateDescriptor = new MapStateDescriptor<String,
TableProcessDwd>("table_process_dwd", String.class,
TableProcessDwd.class);
    BroadcastStream<TableProcessDwd> broadcastStream =
configStream.broadcast(mapStateDescriptor);
    // 2. 数据流去 connect 广播流
    return dataStream
        .connect(broadcastStream)
        .process(new BroadcastProcessFunction<JSONObject,
TableProcessDwd, Tuple2<JSONObject, TableProcessDwd>>() {

            private HashMap<String, TableProcessDwd> map;

            @Override
            public void open(Configuration parameters) throws
Exception {
                // open 中没有办法访问状态!!!
                map = new HashMap<>();
                // 1. 去 mysql 中查询 table_process 表所有数据
                java.sql.Connection mysqlConn =
JdbcUtil.getMysqlConnection();
                List<TableProcessDwd> tableProcessDwdList =
JdbcUtil.queryList(mysqlConn,
                    "select * from
gmall2023_config.table_process_dwd",
                    TableProcessDwd.class,
                    true
                );
            }
        });
}
```

```
        for (TableProcessDwd tableProcessDwd :
tableProcessDwdList) {

            String key =
getKey(tableProcessDwd.getSourceTable(),
tableProcessDwd.getSourceType());
            map.put(key, tableProcessDwd);

        }
        JdbcUtil.closeConnection(mysqlConn);
    }

    // 4. 处理数据流中的数据：从广播状态中读取配置信息
    @Override
    public void processElement(JSONObject jsonObj,
        ReadOnlyContext context,

Collector<Tuple2<JSONObject, TableProcessDwd>> out) throws Exception
    {

        ReadOnlyBroadcastState<String,
TableProcessDwd> state =
context.getBroadcastState(mapStateDescriptor);
        String key = getKey(jsonObj.getString("table"),
jsonObj.getString("type"));
        TableProcessDwd tableProcessDwd =
state.get(key);

        if (tableProcessDwd == null) { // 如果状态中没有
查到，则去 map 中查找
            tableProcessDwd = map.get(key);
            if (tableProcessDwd != null) {
                log.info("在 map 中查找到 " + key);
            }
        } else {
            log.info("在 状态 中查找到 " + key);
        }
        if (tableProcessDwd != null) { // 这条数据找到了
对应的配置信息

            JSONObject data =
jsonObj.getJSONObject("data");
            out.collect(Tuple2.of(data,
tableProcessDwd));
        }
    }
}
```

```
        }
    }

    // 3. 处理广播流中的数据：把配置信息存入到广播状态中
    @Override
    public void
processBroadcastElement(TableProcessDwd tableProcessDwd,
                        Context context,
Collector<Tuple2<JSONObject, TableProcessDwd>> out) throws Exception
{
    BroadcastState<String, TableProcessDwd> state =
context.getBroadcastState(mapStateDescriptor);
    String key =
getKey(tableProcessDwd.getSourceTable(),
tableProcessDwd.getSourceType());

    if ("d".equals(tableProcessDwd.getOp())) {
        // 删除状态
        state.remove(key);
        // map 中的配置也要删除
        map.remove(key);
    } else {
        // 更新或者添加状态
        state.put(key, tableProcessDwd);
    }
}

private String getKey(String table, String type) {
    return table + ":" + type;
}

});

}

private SingleOutputStreamOperator<TableProcessDwd>
readTableProcess(StreamExecutionEnvironment env) {
    // useSSL=false
    Properties props = new Properties();
    props.setProperty("useSSL", "false");
    MySQLSource<String> mysqlSource =
MySQLSource.<String>builder()
        .hostname(Constant.MYSQL_HOST)
        .port(Constant.MYSQL_PORT)
```

```
        .databaseList("gmall2023_config") // set captured
database, If you need to synchronize the whole database, Please set
tableList to ".*".
        .tableList("gmall2023_config.table_process_dwd") //
set captured table
        .username(Constant.MYSQL_USER_NAME)
        .password(Constant.MYSQL_PASSWORD)
        .jdbcProperties(props)
        .deserializer(new JsonDebeziumDeserializationSchema())
// converts SourceRecord to JSON String
        .startupOptions(StartupOptions.initial()) // 默认值:
initial 第一次启动读取所有数据(快照), 然后通过 binlog 实时监控变化数据
        .build();

    return env
        .fromSource(mysqlSource,
WatermarkStrategy.noWatermarks(), "cdc-source")
        .setParallelism(1) // 并行度设置为 1
        .map(new MapFunction<String, TableProcessDwd>() {
            @Override
            public TableProcessDwd map(String value) throws
Exception {
                JSONObject obj = JSON.parseObject(value);
                String op = obj.getString("op");
                TableProcessDwd tp;
                if ("d".equals(op)) {
                    tp = obj.getJSONObject("before",
TableProcessDwd.class);
                } else {
                    tp = obj.getJSONObject("after",
TableProcessDwd.class);
                }
                tp.setOp(op);

                return tp;
            }
        })
        .setParallelism(1);
}

private SingleOutputStreamOperator<JSONObject>
etl(DataStreamSource<String> stream) {
    return stream
        .filter(new FilterFunction<String>() {
```

```
@Override
public boolean filter(String value) throws Exception
{
    try {
        JSONObject obj = JSON.parseObject(value);
        String db = obj.getString("database");
        String type = obj.getString("type");
        String data = obj.getString("data");

        return "gmall".equals(db)
            && ("insert".equals(type)
            || "update".equals(type))
            && data != null
            && data.length() > 2;

    } catch (Exception e) {
        log.warn("不是正确的 json 格式的数据: " +
value);

        return false;
    }

}

})
.map(JSON::parseObject);
}
```

### 11.10.6 测试

1) 启动 zk、kafka、maxwell

2) 运行 DwdBaseDb

3) 监控 kafka 目标主题

启动命令行消费者监控以下主题：

- dwd\_tool\_coupon\_get
- dwd\_tool\_coupon\_use
- dwd\_interaction\_favor\_add
- dwd\_user\_register

4) 运行数据生成器

5) 查看命令行 Kafka 消费者的输出

### 11.10.7 提交至 GitLab

略。

## 第 12 章 数仓开发之 DWS 层

DWS 层设计要点：

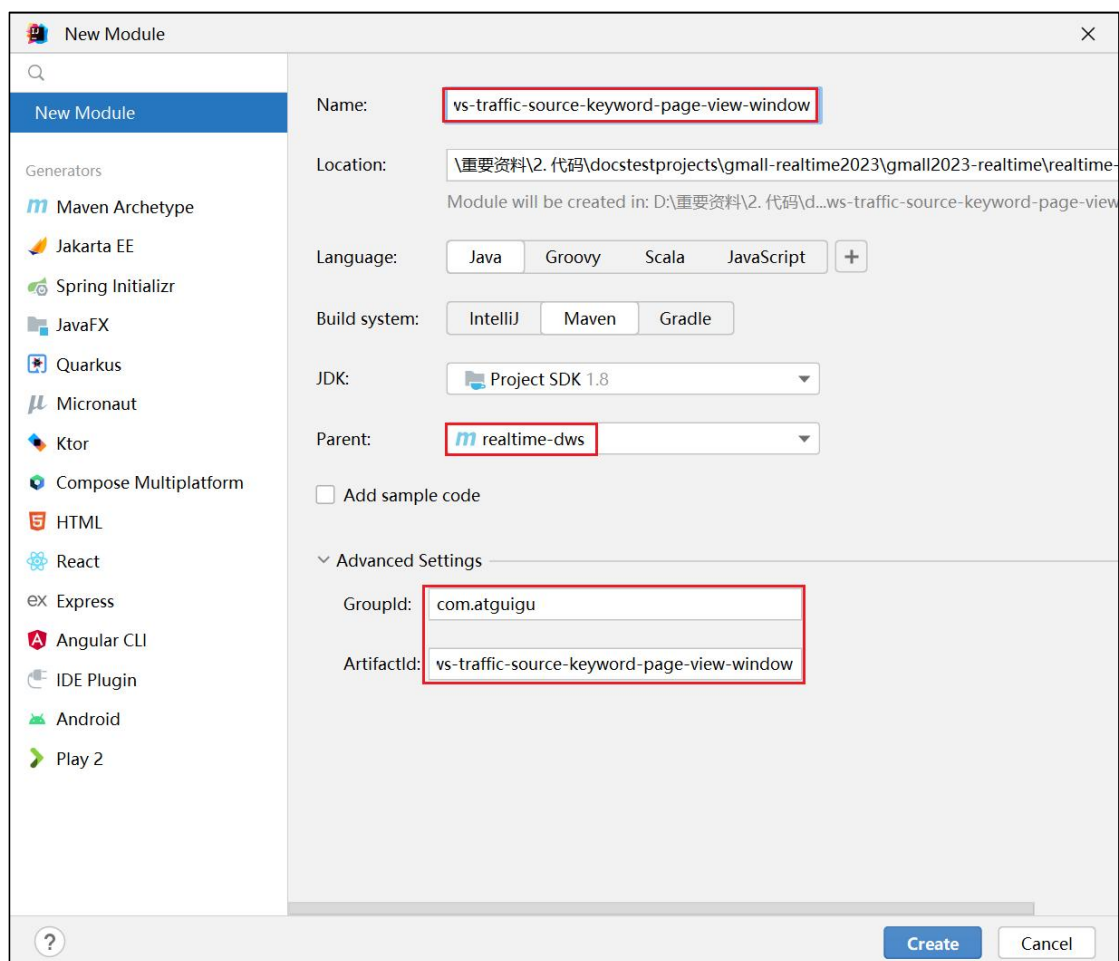
- (1) DWS 层的设计参考指标体系。
- (2) DWS 层表名的命名规范为 `dws_数据域_统计粒度_业务过程_统计周期 (window)`

注：window 表示窗口对应的时间范围

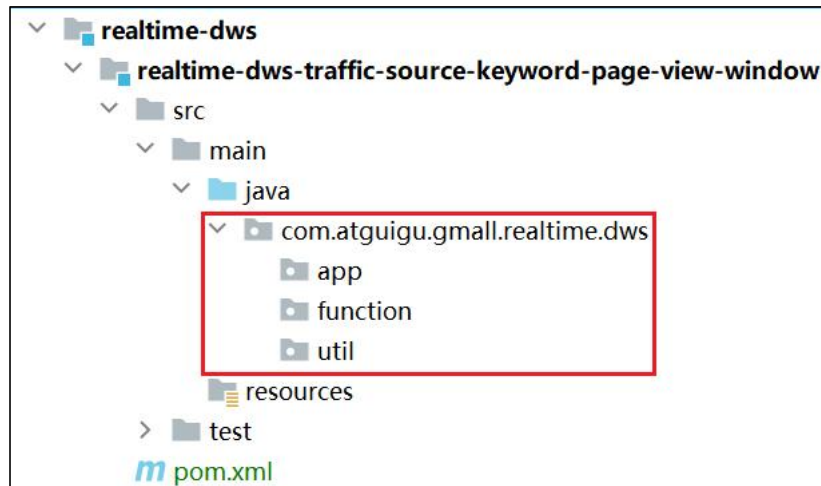
### 12.1 流量域搜索关键词粒度页面浏览各窗口汇总表

#### 12.1.1 创建 realtime-dws-traffic-source-keyword-page-view-window

##### 1) 新建模块



##### 2) 创建包结构



### 12.1.2 主要任务

从 Kafka 页面浏览明细主题读取数据，过滤搜索行为，使用自定义 UDTF（一进多出）函数对搜索内容分词。统计各窗口各关键词出现频次，写入 Doris。

### 12.1.3 思路分析

本程序将使用 FlinkSQL 实现。分词是个一进多出的过程，需要一个 UDTF 函数来实现，FlinkSQL 没有提供相关的内置函数，所以要自定义 UDTF 函数。

自定义函数的逻辑在代码中实现，要完成分词功能，需要导入相关依赖，此处将借助 IK 分词器完成分词。

最终要将数据写入 Doris，需要补充相关依赖。具体步骤如下：

#### 1) 分词处理

##### （1）创建分词工具类

定义分词方法，借助 IK 分词器提供的工具将输入的关键词拆分成多个词，返回一个 List 集合。

##### （2）创建自定义函数类

继承 Flink 的 TableFunction 类，调用分词工具类的分词方法，实现分词逻辑。

##### （3）注册函数

##### （4）从 Kafka 页面浏览明细主题读取数据并设置水位线

##### （5）过滤搜索行为

满足以下三个条件的即为搜索行为数据：

- page 字段下 item 字段不为 null；



- page 字段下 last\_page\_id 为 search;
- page 字段下 item\_type 为 keyword。

(6) 分词

(7) 分组、开窗、聚合计算

按照拆分后的关键词分组。统计每个词的出现频次，补充窗口起始时间、结束时间和统计日期字段，用于插入数据到 Doris 的时候进行分区。

(8) 将聚合数据写到 Doris

## 2) 将数据写入 Doris

(1) 建表

① Doris 建表首先要明确表引擎和数据模型，本项目没有特殊需求，表引擎使用默认的 olap 即可，接下来考虑数据模型。Flink 程序可以通过异步分界线快照算法实现精准一次语义，前提有二。

- 一是 Source 端是可以重置读取位置的数据源。本项目 Source 端使用了 Kafka，满足此要求。
- 二是 Sink 端需要实现事务，或者支持幂等写入。如果实现事务，程序故障重启时可以回滚清除未被状态后端记录的数据，从而保证精准一次。为了实现事务，我们需要继承抽象类 TwoPhaseCommitSinkFunction，重写抽象方法。如果 Sink 端是支持幂等写入的数据库，只要保证至少一次即可。Doris 官方提供的 Connector 底层实现了两阶段提交，可以保证精准一次。DWS 层某些应用的去重策略为了提升时效性，会导致一定程度的数据放大，人为引入重复数据（见下文去重策略分析）。因此，只是保证精准一次并不能满足需求，我们还要根据唯一键去重。Doris 提供了幂等写入，可以通过 Aggregate 模型的 replace 方式或 Unique 模型实现，二者的内部实现方式和数据存储方式完全一致。此处选用 Aggregate 模型的 replace 方式。

② 分区

分区可以在查询数据时帮助我们避免全表扫描，提升查询效率。本项目将会聚合 OLAP 数据库中的当日数据并将结果通过 SpringBoot 服务传递给前端页面，进行可视化展示。显然，按天分区最为合理。

Doris 有两种分区方式：Range Partition 和 List Partition，前者通过划分区间实现分区，后者通过枚举的方式实现分区，按天分区显然是在划分时间范围，应选用 Range Partition。

考虑下一个问题，Range Partition 的分区划分通过 less than 语法或以区间的形式完成，用户需要定期创建未来分区，删除历史分区，开发人员需要耗费大量精力维护。如果能把这些工作交给 Doris 来完成可以大大降低维护成本。Doris 的动态分区可以实现这样的需求。动态分区可以实现表级分区的生命周期管理，建表时设定动态分区规则，FE 会启动一个后台线程，根据建表时设定的规则创建或删除分区，降低维护成本。动态分区及表级部分配置项解读如下。

- `dynamic_partition.enable`: 是否开启动态分区特性，可指定 true 或 false,默认为 true
- `dynamic_partition.time_unit`: 动态分区调度的单位，可指定 HOUR、DAY、WEEK、MONTH。本项目按天分区，指定为 DAY，后缀格式为 yyyyMMdd。
- `dynamic_partition.start`: 动态分区的起始偏移，为负数。根据 `time_unit` 属性的不同，以当天（星期/月）为基准，分区范围在此偏移之前的分区将会被删除。如果不填写默认值为 `Integer.Min_VALUE` 即-2147483648，即不删除历史分区。通常实时项目不需要保留历史数据，理论上仅保留当日数据即可，但考虑这样一种情况，数仓上线一段时间后，对零点前后生成的数据做运算，可能系统时间已进入第二天，而第一天的分区数据计算仍未完成，此时就有必要保留前一日分区。因此将其设为-1，仅保留昨日及之后的数据。
- `dynamic_partition.end`: 动态分区的结束偏移，为正数。根据 `time_unit` 属性的不同，以当天（星期/月）为基准，提前创建对应范围的分区。本项目按天分区，提前一日创建分区即可，该参数设为 1，避免数据写入时分区不存在而报错。
- `dynamic_partition.prefix`: 动态创建的分区名前缀，此处设为 `par`。
- `dynamic_partition.buckets`: 动态创建的分区所对应分桶数量。分桶数越多，单次查询就可以定位到更加精确的数据集，查询效率更高，但同时数据会更加分散，可能会导致大量小文件的产生，分桶数应在二者之间寻找平衡，生产环境需要结合实际情况调整。最佳分桶数的确定属于项目调优内容，本文不过多介绍，此处设置为 10。
- `dynamic_partition.replication_num`: 动态创建的分区所对应的副本数量，如果不填写，则默认为该表创建时指定的副本数量。此处不设该参数，副本数由 `replication_num` 参数指定。
- `dynamic_partition.hot_partition_num`: 指定最新的多少个分区为热分区。对于热分区，系统会自动设置其 `storage_medium` 参数为 SSD，并且设置 `storage_cooldown_time`。上

文提到, 对零点前后生成数据的运算可能会有昨日分区数据的参与, 为了不影响效率, 昨日分区同样优先使用 SSD 介质存储, 该参数设置为 1。

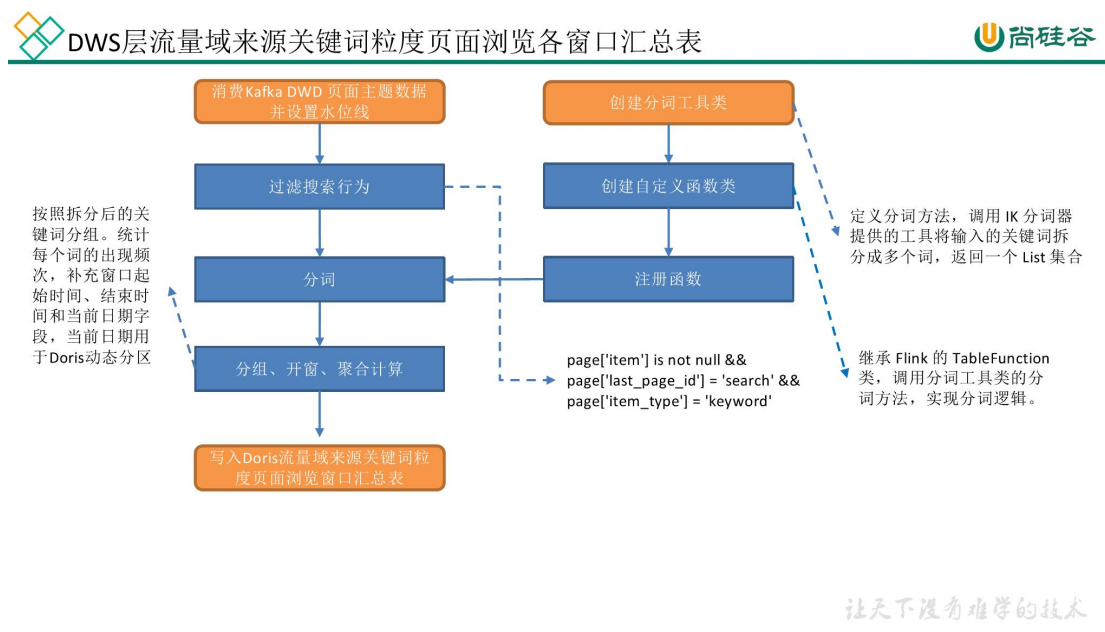
➤ `dynamic_partition.create_history_partition`: 默认为 `false`。当置为 `true` 时, Doris 会自动创建所有历史分区, 当期望创建的分区个数大于 `max_dynamic_partition_num` 值时, 操作将被禁止。当不指定 `start` 属性时, 该参数不生效。实时数仓不对历史数据做处理, 无须创建历史分区, 因此该参数保持默认值即可。

➤ `replication_num`: 指定表副本数, 副本数不可超过 IP 互不相同的 BE 节点数量。本项目 BE 分布在三台节点, 副本数最大为 3。该参数设置为 3。

## (2) 写出方式

Doris 官方提供了 Flink Doris Connector 可以实现 Flink SQL 及 Flink 流处理与 Doris 的数据交互。

### 12.1.4 图解



### 12.1.5 Doris 建表语句

#### 1) 建库

```
create database gmall2023_realtime;
```

#### 2) 建表

```
drop table if exists  
gmall2023_realtime.dws_traffic_source_keyword_page_view_window;
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
create table if not exists
gmall2023_realtime.dws_traffic_source_keyword_page_view_window
(
    `stt`          DATETIME COMMENT '窗口起始时间',
    `edt`          DATETIME COMMENT '窗口结束时间',
    `cur_date`     DATE COMMENT '当天日期',
    `keyword`      VARCHAR(128) COMMENT '搜索关键词',
    `keyword_count` BIGINT REPLACE COMMENT '搜索关键词出现次数'
) engine = olap aggregate key (`stt`, `edt`, `cur_date`, `keyword`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
    "replication_num" = "3",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "par",
    "dynamic_partition.buckets" = "10"
);
```

## 12.1.6 代码实现

### 1) 所需依赖

此处需要添加 IK 分词相关依赖及 Doris 连接器依赖，如下。

```
<dependency>
    <groupId>org.apache.doris</groupId>
    <artifactId>flink-doris-connector-1.17</artifactId>
</dependency>

<dependency>
    <groupId>com.janeluo</groupId>
    <artifactId>ikanalyzer</artifactId>
</dependency>
```

### 2) 创建分词工具类 IkUtil

此处的工具类仅用于本节应用，不属于通用工具类，因此放在 realtime-dws-traffic-source-keyword-page-view-window 模块下。下文的分词函数同理。

在 com.atguigu.gmall.realtime.dws.util 包下创建 IkUtil，写入以下内容。

```
package com.atguigu.gmall.realtime.dws.util;

import org.wltea.analyzer.core.IKSegmenter;
import org.wltea.analyzer.core.Lexeme;
```

```
import java.io.IOException;
import java.io.Reader;
import java.io.StringReader;
import java.util.HashSet;
import java.util.Set;

public class IkUtil {

    public static Set<String> split(String s) {
        Set<String> result = new HashSet<>();
        // String => Reader

        Reader reader = new StringReader(s);
        // 智能分词
        // max_word
        IKSegmenter ikSegmenter = new IKSegmenter(reader, true);

        try {
            Lexeme next = ikSegmenter.next();
            while (next != null) {
                String word = next.getLexemeText();
                result.add(word);
                next = ikSegmenter.next();
            }

        } catch (IOException e) {
            throw new RuntimeException(e);
        }

        return result;
    }
}
```

### 3) 创建分词函数 KwSplit

在 com.atguigu.gmall.realtime.dws.function 包下创建分词函数 KwSplit，写入以下内容。

```
package com.atguigu.gmall.realtime.dws.function;

import com.atguigu.gmall.realtime.dws.util.IkUtil;
import org.apache.flink.table.annotation.DataTypeHint;
import org.apache.flink.table.annotation.FunctionHint;
import org.apache.flink.table.functions.TableFunction;
import org.apache.flink.types.Row;
```

```
import java.util.Set;

@FunctionHint(output = @DataTypeHint("row<keyword string>"))
public class KwSplit extends TableFunction<Row> {
    public void eval(String kw) {
        if (kw == null) {
            return;
        }
        // "华为手机白色手机"
        Set<String> keywords = IkUtil.split(kw);
        for (String keyword : keywords) {
            collect(Row.of(keyword));
        }
    }
}
```

#### 4) 修改常量类 Constant

在 Constant 中添加以下常量

```
public static final String DORIS_FE_NODES =
    "hadoop102:7030,hadoop103:7030,hadoop104:7030";

public static final String DORIS_DATABASE = "gmall2023_realtime";
```

#### 5) 主程序

在 com.atguigu.gmall.realtime.dws.app 下创建 DwsTrafficSourceKeywordPageViewWindow, 写入以下内容。

```
package com.atguigu.gmall.realtime.dws.app;

import com.atguigu.gmall.realtime.common.base.BaseSQLApp;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.util.SQLUtil;
import com.atguigu.gmall.realtime.dws.function.KwSplit;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;

import org.apache.flink.table.api.Table;
import
org.apache.flink.table.api.bridge.java.StreamTableEnvironment;

public class DwsTrafficSourceKeywordPageViewWindow extends
BaseSQLApp {
    public static void main(String[] args) {
        new DwsTrafficSourceKeywordPageViewWindow().start(
```

```
        10021,
        4,
        "dws_traffic_source_keyword_page_view_window"
    );
}

@Override
public void handle(StreamExecutionEnvironment env,
                   StreamTableEnvironment tEnv) {
    // 1. 读取 页面日志
    tEnv.executeSql("create table page_log(" +
        " page map<string, string>, " +
        " ts bigint, " +
        " et as to_timestamp_ltz(ts, 3), " +
        " watermark for et as et - interval '5' second " +
        ") " +
    SQLUtil.getKafkaDDLSource("dws_traffic_source_keyword_page_view_w
indow", Constant.TOPIC_DWD_TRAFFIC_PAGE));

    // 2. 读取搜索关键词
    Table kwTable = tEnv.sqlQuery("select " +
        "page['item'] kw, " +
        "et " +
        "from page_log " +
        "where ( page['last_page_id'] ='search' " +
        "       or page['last_page_id'] ='home' " +
        "       )" +
        "and page['item_type']='keyword' " +
        "and page['item'] is not null ");
    tEnv.createTemporaryView("kw_table", kwTable);

    // 3. 自定义分词函数
    tEnv.createTemporaryFunction("kw_split", KwSplit.class);

    Table keywordTable = tEnv.sqlQuery("select " +
        " keyword, " +
        " et " +
        "from kw_table " +
        "join lateral table(kw_split(kw)) on true ");
    tEnv.createTemporaryView("keyword_table", keywordTable);

    // 3. 开窗聚和 tvf
    Table result = tEnv.sqlQuery("select " +
```

```
        " date_format(window_start, 'yyyy-MM-dd HH:mm:ss') stt,
" +
        " date_format(window_end, 'yyyy-MM-dd HH:mm:ss') edt,
" +
        " date_format(window_start, 'yyyyMMdd') cur_date, " +
        " keyword, " +
        " count(*) keyword_count " +
        "from table( tumble(table keyword_table,
descriptor(et), interval '5' second ) ) " +
        "group by window_start, window_end, keyword ");
// 5. 写出到 doris 中
tEnv.executeSql("create table
dws_traffic_source_keyword_page_view_window(" +
        " stt string, " + // 2023-07-11 14:14:14
        " edt string, " +
        " cur_date string, " +
        " keyword string, " +
        " keyword_count bigint " +
        ")with(" +
        " 'connector' = 'doris'," +
        " 'fenodes' = '" + Constant.DORIS_FE_NODES + "'," +
        " 'table.identifier' = '" + Constant.DORIS_DATABASE +
".dws_traffic_source_keyword_page_view_window'," +
        " 'username' = 'root'," +
        " 'password' = '000000'," +
        " 'sink.properties.format' = 'json'," +
        " 'sink.buffer-count' = '4'," +
        " 'sink.buffer-size' = '4086'," +
        " 'sink.enable-2pc' = 'false'," + // 测试阶段可以关闭
两阶段提交,方便测试
        " 'sink.properties.read_json_by_line' = 'true' " +
        ")");

result.executeInsert("dws_traffic_source_keyword_page_view_window
");
    }
}
```

### 12.1.7 测试

- 1) 启动 zk、kafka、HDFS（用于保存检查点）、flume（fl.sh start）、Doris
- 2) 运行 DwdBaseLog、DwsTrafficSourceKeywordPageViewWindow
- 3) 运行数据生成器



## 4) 查看 Doris 目标表数据

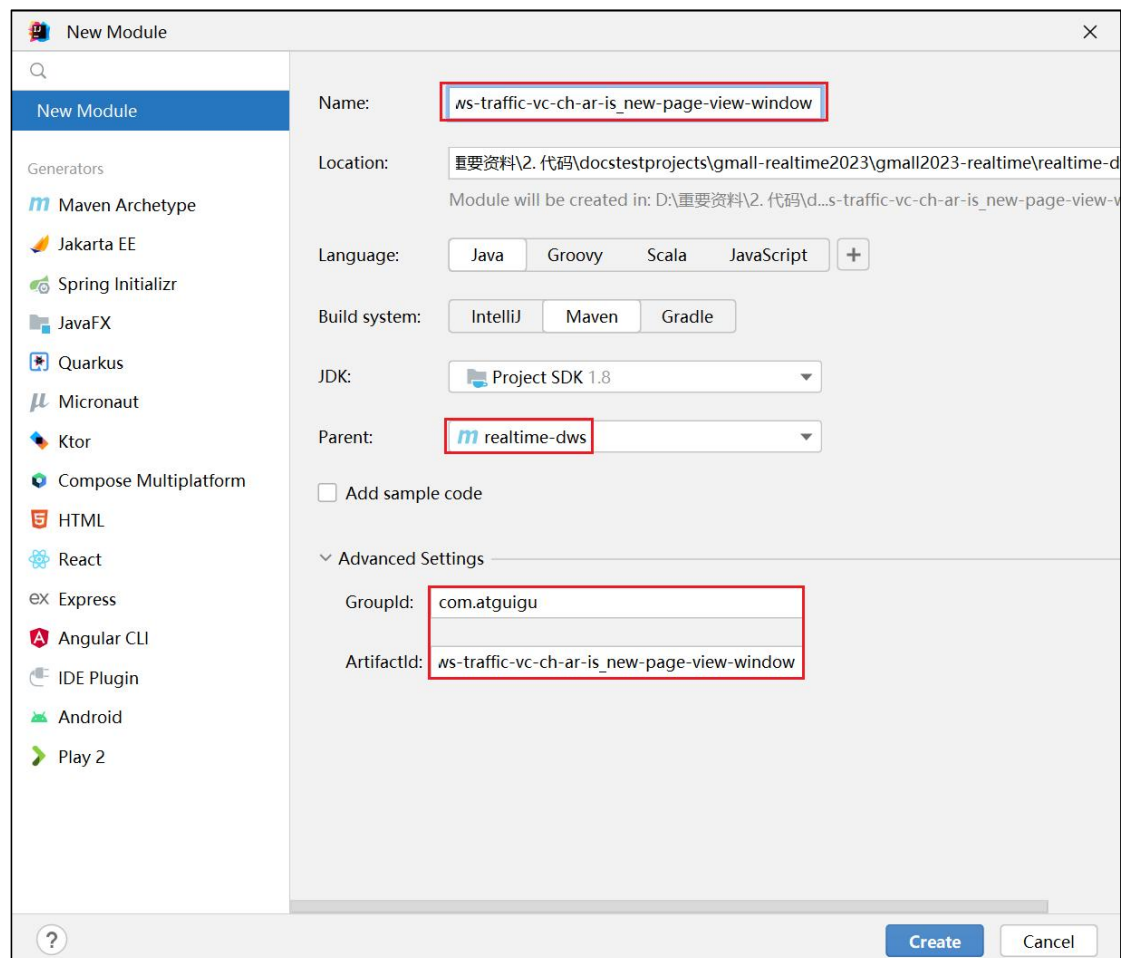
### 12.1.8 提交至 GitLab

略。

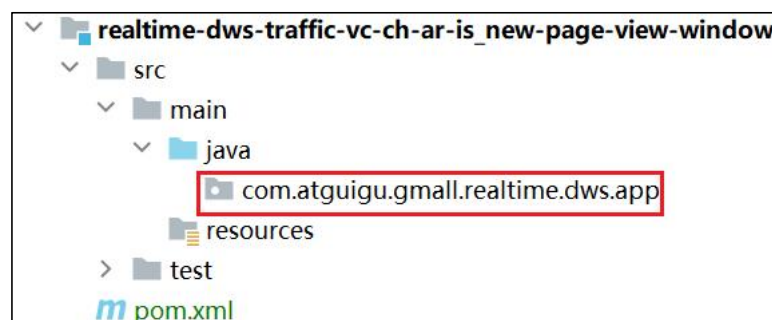
## 12.2 流量域版本-渠道-地区-访客类别粒度页面浏览各窗口汇总表

### 12.2.1 创建 realtime-dws-traffic-vc-ch-ar-is\_new-page-view-window

#### 1) 新建模块



#### 2) 创建包结构



### 12.2.2 主要任务

DWS 层是为 ADS 层服务的，通过对指标体系的分析，本节汇总表中需要有会话数、页面浏览数、浏览总时长、独立访客数四个度量字段。本节的任务是统计这四个指标，并将维度和度量数据写入 Doris 汇总表。

### 12.2.3 思路分析

#### 1) 知识储备

`@JSONField(serialize = false)`

`@JSONField` 是 FastJson 提供的注解，该注解提供了很多属性，用于定制序列化和反序列化过程。`serialize` 属性是控制字段是否被序列化的，默认为 `true`，设置为 `false` 意味着将 JSON 对象序列化为 JSON 字符串时该字段不会出现在 JSON 字符串中，但如果 JSON 字符串存在该字段，反序列化为 JSON 对象时仍会存在该字段的值，后者是有 `deserialize` 属性控制的，默认值同样为 `true`。

#### 2) 执行步骤

会话数、页面浏览数、浏览总时长、独立访客四个指标均与页面浏览有关，可以由 DWD 层页面日志事实表中获得。

- (1) 读取页面主题数据，封装为流
- (2) 转换数据结构，按照设备 id 分组
- (3) 统计页面浏览时长、页面浏览数、会话数和独立访客，转换数据结构

创建实体类，将页面浏览数置为 1（只要有一条页面浏览日志，则页面浏览数加一），获取日志中的页面浏览时长，赋值给实体类的同名字段。接下来判断 `last_page_id` 是否为 `null`，如果是，说明页面是首页，开启了一个新的会话，将会话数置为 1，否则置为 0。最后在状态中维护末次访问日期，如果状态中的日期为 `null` 或不为当日，则将独立访客数置为 1，否则置为 0。独立访客每日重置，所以状态最多维护一天即可，为了减少内存开销，设置状态 `ttl` 为 1 天。

补充维度字段，窗口起始和结束时间置为空字符串。下游要根据水位线开窗，所以要补充事件时间字段，此处将日志生成时间 `ts` 作为事件时间字段即可。最后将实体类对象发往下游。

- (4) 设置水位线；

(5) 按照维度字段分组;

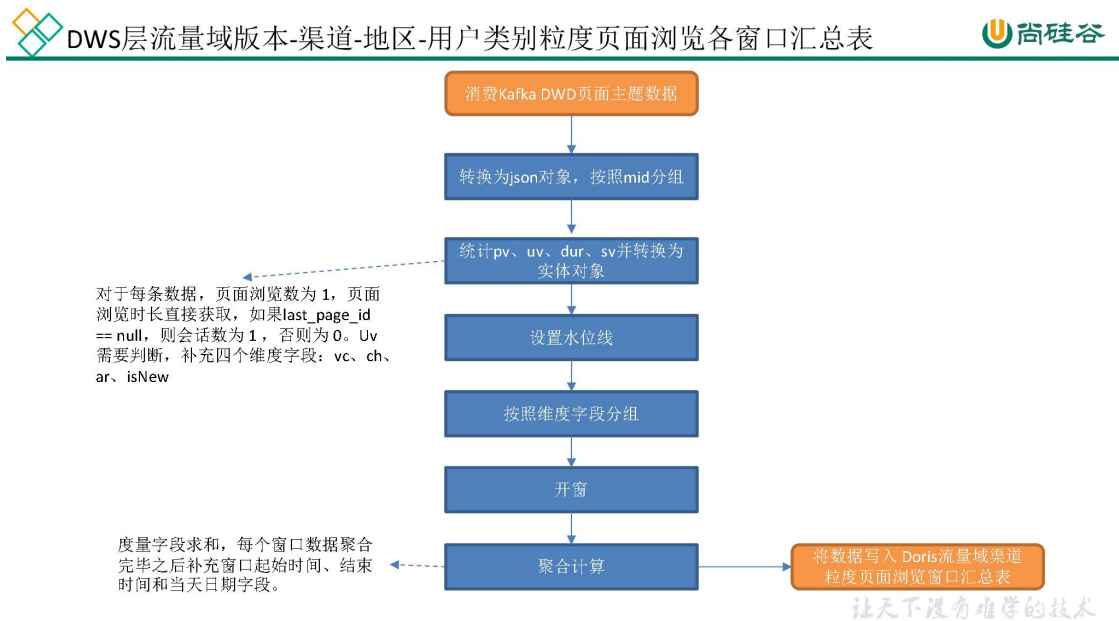
(6) 开窗

(7) 聚合计算

度量字段求和, 每个窗口数据聚合完毕之后补充窗口起始时间、结束时间字段以及统计日期字段。

(8) 将数据写入 Doris。

## 12.2.4 图解



## 12.2.5 Doris 建表语句

```

drop table if exists
gmall2023_realtime.dws_traffic_vc_ch_ar_is_new_page_view_window;
create table if not exists
gmall2023_realtime.dws_traffic_vc_ch_ar_is_new_page_view_window
(
  `stt`      DATETIME COMMENT '窗口起始时间',
  `edt`      DATETIME COMMENT '窗口结束时间',
  `cur_date` DATE COMMENT '当天日期',
  `vc`       VARCHAR(256) COMMENT '版本号',
  `ch`       VARCHAR(256) COMMENT '渠道',
  `ar`       TINYINT COMMENT '地区',
  `is_new`   TINYINT COMMENT '新老访客状态标记',
  `uv_ct`    BIGINT REPLACE COMMENT '独立访客数',

```

```
`sv_ct`    BIGINT REPLACE COMMENT '会话数',
`pv_ct`    BIGINT REPLACE COMMENT '页面浏览数',
`dur_sum`  BIGINT REPLACE COMMENT '累计访问时长'
) engine = olap aggregate key
(`stt`,`edt`,`cur_date`,`vc`,`ch`,`ar`,`is_new`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
"replication_num" = "3",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.end" = "3",
"dynamic_partition.prefix" = "par",
"dynamic_partition.buckets" = "10"
);
```

## 12.2.6 代码实现

### 1) 创建实体类 TrafficPageViewBean

在 common 模块的 bean 目录下创建 TrafficPageViewBean

```
package com.atguigu.gmall.realtime.common.bean;

import com.alibaba.fastjson.annotation.JSONField;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class TrafficPageViewBean {
    // 窗口起始时间
    private String stt;
    // 窗口结束时间
    private String edt;
    // 当天日期
    private String cur_date;
    // app 版本号
    private String vc;
    // 渠道
    private String ch;
    // 地区
    private String ar;
```

```
// 新老访客状态标记
private String isNew ;

// 独立访客数
private Long uvCt;
// 会话数
private Long svCt;
// 页面浏览数
private Long pvCt;
// 累计访问时长
private Long durSum;
// 时间戳
@JSONField(serialize = false) // 要不要序列化这个字段
private Long ts;
@JSONField(serialize = false) // 要不要序列化这个字段
private String sid;
}
```

## 2) 创建 DorisMapFunction 函数

在 common 模块的 function 包下创建 DorisMapFunction 函数。

将数据写入 Doris 时字段名称必须完全一致，否则目标表相应字段取值为 null。我们的实体类字段使用了小驼峰作为命名规范，但是 Doris 表字段使用的是下划线命名方式，此处定义工具类，用于完成小驼峰至下划线命名的转换。

```
package com.atguigu.gmall.realtime.common.function;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.PropertyNamingStrategy;
import com.alibaba.fastjson.serializer.SerializeConfig;
import org.apache.flink.api.common.functions.MapFunction;

public class DorisMapFunction<T> implements MapFunction<T, String>
{
    @Override
    public String map(T bean) throws Exception {
        SerializeConfig conf = new SerializeConfig();

        conf.setPropertyNamingStrategy(PropertyNamingStrategy.SnakeCase);
        return JSON.toJSONString(bean, conf);
    }
}
```

## 3) 修改 FlinkSinkUtil

修改 FlinkSinkUtil, 补充 getDorisSink 方法

```
public static DorisSink<String> getDorisSink(String table, String
labelPrefix) {
    Properties props = new Properties();
    props.setProperty("format", "json");
    props.setProperty("read_json_by_line", "true"); // 每行一条 json
数据
    return DorisSink.<String>builder()
        .setDorisReadOptions(DorisReadOptions.builder().build()
)
        .setDorisOptions(DorisOptions.builder() // 设置 doris 的连
接参数
            .setFenodes(Constant.DORIS_FE_NODES)
            .setTableIdentifier(table)
            .setUsername("root")
            .setPassword("000000")
            .build()
        )
        .setDorisExecutionOptions(DorisExecutionOptions.builder
() // 执行参数
            .setLabelPrefix(labelPrefix) // stream-load 导入数
据时 label 的前缀
            .disable2PC() // 开启两阶段提交后,labelPrefix 需要全局
唯一,为了测试方便禁用两阶段提交
            .setBufferCount(3) // 批次条数: 默认 3
            .setBufferSize(1024 * 1024) // 批次大小: 默认 1M
            .setCheckInterval(3000) // 批次输出间隔 上述三个批次
的限制条件是或的关系
            .setMaxRetries(3)
            .setStreamLoadProp(props) // 设置 stream load 的数据
格式 默认是 csv,需要改成 json
            .build()
            .setSerializer(new SimpleStringSerializer())
            .build();
        }
    }
```

#### 4) 主程序

在 com.atguigu.gmall.realtime.dws.app 下创建 DwsTrafficVcChArIsNewPageViewWindow, 写入以下内容。

```
package com.atguigu.gmall.realtime.dws.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载, 可百度访问: [尚硅谷官网](#)

```
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.bean.TrafficPageViewBean;
import com.atguigu.gmall.realtime.common.constant.Constant;
import
com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.ReduceFunction;
import org.apache.flink.api.common.state.StateTtlConfig;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.api.common.time.Time;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera
tor;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import
org.apache.flink.streaming.api.functions.windowing.ProcessWindowF
unction;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventT
imeWindows;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.time.Duration;

import static
org.apache.flink.api.common.state.StateTtlConfig.StateVisibility.
NeverReturnExpired;
import static
org.apache.flink.api.common.state.StateTtlConfig.UpdateType.OnCre
ateAndWrite;

public class DwsTrafficVcChArIsNewPageViewWindow extends BaseApp {
    public static void main(String[] args) {
```

```
new DwsTrafficVcChArIsNewPageViewWindow().start(  
    10022,  
    4,  
    "dws_traffic_vc_ch_ar_is_new_page_view_window",  
    Constant.TOPIC_DWD_TRAFFIC_PAGE  
);  
  
}  
  
@Override  
public void handle(StreamExecutionEnvironment env,  
DataStreamSource<String> stream) {  
    // 1. 解析数据, 封装 pojo 中  
    SingleOutputStreamOperator<TrafficPageViewBean> beanStream =  
parseToPojo(stream);  
    // 2. 开窗聚合  
    SingleOutputStreamOperator<TrafficPageViewBean> result =  
windowAndAgg(beanStream);  
  
    // 3. 写出到 doris 中  
    writeToDoris(result);  
}  
  
private void  
writeToDoris(SingleOutputStreamOperator<TrafficPageViewBean>  
result) {  
  
    result  
        .map(new DorisMapFunction<>())  
        .sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA  
TABASE + ".dws_traffic_vc_ch_ar_is_new_page_view_window",  
"dws_traffic_vc_ch_ar_is_new_page_view_window"));  
  
}  
  
private SingleOutputStreamOperator<TrafficPageViewBean>  
windowAndAgg(  
    SingleOutputStreamOperator<TrafficPageViewBean>  
beanStream) {  
    return beanStream  
        .assignTimestampsAndWatermarks(  
            WatermarkStrategy  
                .<TrafficPageViewBean>forBoundedOutOfOr  
derness(Duration.ofSeconds(5L))  
                .withTimestampAssigner((bean, ts) ->
```



```
bean.getTs())
                .withIdleness(Duration.ofSeconds(120L))
            )
            .keyBy(bean -> bean.getVc() + "_" + bean.getCh() + "_"
+ bean.getAr() + "_" + bean.getIsNew())
            .window(TumblingEventTimeWindows.of(org.apache.flink.
streaming.api.windowing.time.Time.seconds(5L)))
            .reduce(
                new ReduceFunction<TrafficPageViewBean>() {
                    @Override
                    public TrafficPageViewBean
reduce(TrafficPageViewBean value1,

TrafficPageViewBean value2) throws Exception {
                        value1.setPvCt(value1.getPvCt() +
value2.getPvCt());
                        value1.setUvCt(value1.getUvCt() +
value2.getUvCt());
                        value1.setSvCt(value1.getSvCt() +
value2.getSvCt());
                        value1.setDurSum(value1.getDurSum() +
value2.getDurSum());
                        return value1;
                    }
                },
                new ProcessWindowFunction<TrafficPageViewBean,
TrafficPageViewBean, String, TimeWindow>() {
                    @Override
                    public void process(String key,
Context ctx,

Iterable<TrafficPageViewBean> elements, // 有且仅有一个值：前面聚合的最
最终结果

Collector<TrafficPageViewBean> out) throws Exception {
                        TrafficPageViewBean bean =
elements.iterator().next();

bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()))
;

bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));
```

```
bean.setCur_date(DateFormatUtil.tsToDateForPartition(ctx.window().
getStart()));

        out.collect(bean);
    }
}

);

}

    private SingleOutputStreamOperator<TrafficPageViewBean>
parseToPojo(DataStreamSource<String> stream) {
    return stream
        .map(JSON::parseObject)
        .keyBy(obj ->
obj.getJSONObject("common").getString("mid")) // 顺便计算下 uv, 按照
mid 进行分组, 找到同一个 mid 的第一条记录, uv 置为1, 其他的 uv 为 0
        .process(new KeyedProcessFunction<String, JSONObject,
TrafficPageViewBean>() {

            private ValueState<String> lastVisitDateState;

            @Override
            public void open(Configuration parameters) throws
Exception {
                lastVisitDateState =
getRuntimeContext().getState(new
ValueStateDescriptor<String>("lastVisitDate", String.class));
            }

            @Override
            public void processElement(JSONObject obj,
Context ctx,

Collector<TrafficPageViewBean> out) throws Exception {
                JSONObject page = obj.getJSONObject("page");
                JSONObject common =
obj.getJSONObject("common");
                Long ts = obj.getLong("ts");
                String today = DateFormatUtil.tsToDate(ts);

                Long pv = 1L;
                Long durSum = page.getLong("during_time");
                // uv
```

```
String lastVisitDate =
lastVisitDateState.value();
Long uv = 0L;
if (!today.equals(lastVisitDate)) {
    // 是当天的第一次
    uv = 1L;
    // 更新状态
    lastVisitDateState.update(today);
}

TrafficPageViewBean bean = new
TrafficPageViewBean();
bean.setVc(common.getString("vc"));
bean.setCh(common.getString("ch"));
bean.setAr(common.getString("ar"));
bean.setIsNew(common.getString("is_new"));

bean.setPvCt(pv);
bean.setUvCt(uv);
bean.setDurSum(durSum);
bean.setTs(ts);

bean.setSid(common.getString("sid"));

out.collect(bean);

}
}))
.keyBy(TrafficPageViewBean::getSid) //按照 sid 分组,
计算 sv 使用
.process(new KeyedProcessFunction<String,
TrafficPageViewBean, TrafficPageViewBean>() {

    private ValueState<Boolean> isFirstState;

    @Override
    public void open(Configuration parameters) throws
Exception {
        ValueStateDescriptor<Boolean> desc = new
ValueStateDescriptor<>("isFirst", Boolean.class);
        StateTtlConfig conf = new
StateTtlConfig.Builder(Time.hours(1))
            .setStateVisibility(NeverReturnExpired)
            .setUpdateType(OnCreateAndWrite)
```

```
        .useProcessingTime()
        .build();
        desc.enableTimeToLive(conf);
        isFirstState =
getRuntimeContext().getState(desc);

    }

    @Override
    public void processElement(TrafficPageViewBean
bean,

                                Context ctx,

Collector<TrafficPageViewBean> out) throws Exception {

        if (isFirstState.value() == null) {
            bean.setSvCt(1L);
            isFirstState.update(true);
        } else {
            bean.setSvCt(0L);
        }
        out.collect(bean);
    }
    });
}
```

### 12.2.7 测试

- 1) 启动 zk、kafka、HDFS、flume (fl.sh start)、Doris
- 2) 运行 DwdBaseLog、DwsTrafficVcChArIsNewPageViewWindow
- 3) 运行数据生成器
- 4) 查看 Doris 目标表数据

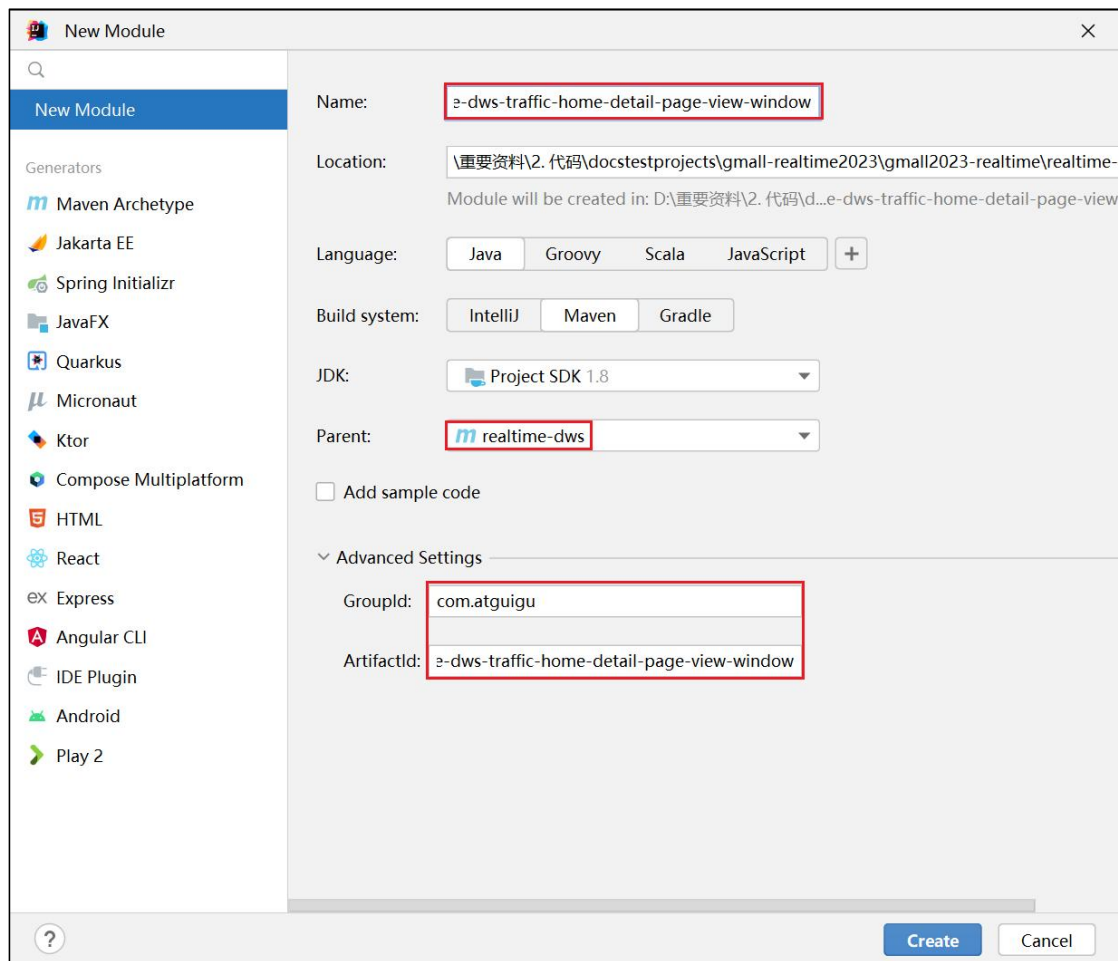
### 12.2.8 提交至 GitLab

略。

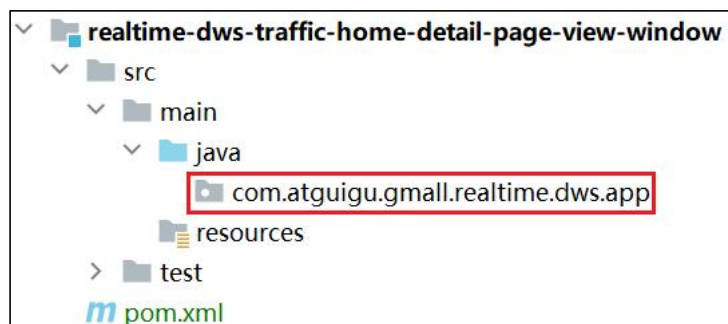
## 12.3 流量域首页、详情页页面浏览各窗口汇总表

### 12.3.1 创建 realtime-dws-traffic-home-detail-page-view-window

- 1) 新建模块



## 2) 创建包结构



## 12.3.2 主要任务

从 Kafka 页面日志主题读取数据，统计当日的首页和商品详情页独立访客数。

## 12.3.3 思路分析

### 1) 读取 Kafka 页面主题数据

### 2) 转换数据结构

将流中数据由 String 转换为 JSONObject。

### 3) 过滤数据

仅保留 `page_id` 为 `home` 或 `good_detail` 的数据, 因为本程序统计的度量仅与这两个页面有关, 其它数据无用。

### 4) 设置水位线

### 5) 按照 `mid` 分组

### 6) 统计首页和商品详情页独立访客数, 转换数据结构

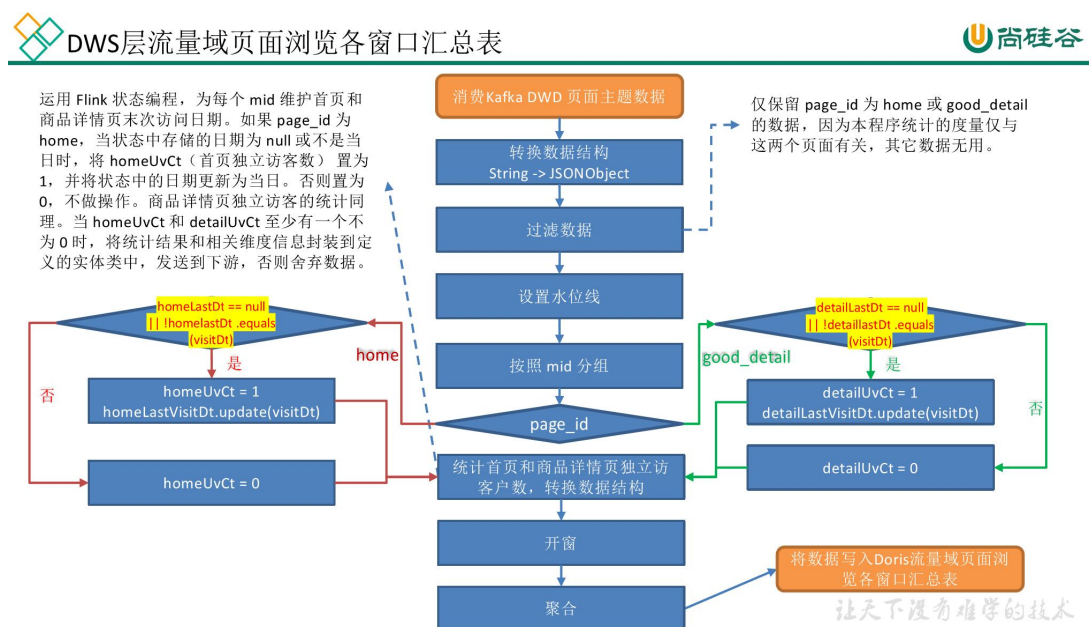
运用 Flink 状态编程, 为每个 `mid` 维护首页和商品详情页末次访问日期。如果 `page_id` 为 `home`, 当状态中存储的日期为 `null` 或不是当日时, 将 `homeUvCt` (首页独立访客数) 置为 1, 并将状态中的日期更新为当日。否则置为 0, 不做操作。商品详情页独立访客的统计同理。当 `homeUvCt` 和 `detailUvCt` 至少有一个不为 0 时, 将统计结果和相关维度信息封装到定义的实体类中, 发送到下游, 否则舍弃数据。

### 7) 开窗

### 8) 聚合

### 9) 将数据写出到 Doris

## 12.3.4 图解



## 12.3.5 Doris 建表语句

```
drop table if exists  
gmall2023_realtime.dws_traffic_home_detail_page_view_window;  
create table if not exists
```

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网

```
gmall2023_realtime.dws_traffic_home_detail_page_view_window
(
    `stt`          DATETIME COMMENT '窗口起始时间',
    `edt`          DATETIME COMMENT '窗口结束时间',
    `cur_date`     DATE COMMENT '当天日期',
    `home_uv_ct`   BIGINT REPLACE COMMENT '首页独立访客数',
    `good_detail_uv_ct` BIGINT REPLACE COMMENT '商品详情页独立访客数'
) engine = olap aggregate key (`stt`, `edt`, `cur_date`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
    "replication_num" = "3",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "par",
    "dynamic_partition.buckets" = "10"
);
```

## 12.3.6 代码实现

### 1) 实体类

在 common 模块的 bean 包下创建 TrafficHomeDetailViewBean

```
package com.atguigu.gmall.realtime.common.bean;

import com.alibaba.fastjson.annotation.JSONField;
import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class TrafficHomeDetailViewBean {
    // 窗口起始时间
    String stt;
    // 窗口结束时间
    String edt;
    // 当天日期
    String curDate;
    // 首页独立访客数
    Long homeUvCt;
    // 商品详情页独立访客数
    Long goodDetailUvCt;
    // 时间戳
    @JSONField(serialize = false)
```

```
Long ts;  
}
```

## 2) 主程序

在 com.atguigu.gmall.realtime.dws.app 包下创建 DwsTrafficHomeDetailViewWindow

```
package com.atguigu.gmall.realtime.dws.app;  
  
import com.alibaba.fastjson.JSON;  
import com.alibaba.fastjson.JSONObject;  
import com.atguigu.gmall.realtime.common.base.BaseApp;  
import  
com.atguigu.gmall.realtime.common.bean.TrafficHomeDetailViewB  
ean;  
import com.atguigu.gmall.realtime.common.constant.Constant;  
import  
com.atguigu.gmall.realtime.common.function.DorisMapFunction;  
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;  
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;  
import org.apache.flink.api.common.eventtime.WatermarkStrategy;  
import org.apache.flink.api.common.functions.ReduceFunction;  
import org.apache.flink.api.common.state.ValueState;  
import org.apache.flink.api.common.state.ValueStateDescriptor;  
import org.apache.flink.configuration.Configuration;  
import org.apache.flink.streaming.api.datastream.DataStreamSource;  
import  
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera  
tor;  
import  
org.apache.flink.streaming.api.environment.StreamExecutionEnviron  
ment;  
import  
org.apache.flink.streaming.api.functions.KeyedProcessFunction;  
import  
org.apache.flink.streaming.api.functions.windowing.ProcessAllWind  
owFunction;  
import  
org.apache.flink.streaming.api.windowing.assigners.TumblingEventT  
imeWindows;  
import org.apache.flink.streaming.api.windowing.time.Time;  
import  
org.apache.flink.streaming.api.windowing.windows.TimeWindow;  
import org.apache.flink.util.Collector;  
  
import java.time.Duration;
```



```
public class DwsTrafficHomeDetailPageViewWindow extends BaseApp {
    public static void main(String[] args) {
        new DwsTrafficHomeDetailPageViewWindow().start(
            10023,
            4,
            "dws_traffic_home_detail_page_view_window",
            Constant.TOPIC_DWD_TRAFFIC_PAGE
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        DataStreamSource<String> stream) {
        // 1. 过滤, 解析成 pojo 类型
        SingleOutputStreamOperator<TrafficHomeDetailPageViewBean>
beanStream = parseToPojo(stream);

        // 2. 开窗聚合
        SingleOutputStreamOperator<TrafficHomeDetailPageViewBean>
resultStream = windowAndAgg(beanStream);

        // 3. 写出到 doris
        writeToDoris(resultStream);
    }

    private void
writeToDoris(SingleOutputStreamOperator<TrafficHomeDetailPageView
Bean> resultStream) {
        resultStream
            .map(new DorisMapFunction<>())
            .sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA
TATABASE + ".dws_traffic_home_detail_page_view_window",
"dws_traffic_home_detail_page_view_window"));
    }

    private
SingleOutputStreamOperator<TrafficHomeDetailPageViewBean>
windowAndAgg(SingleOutputStreamOperator<TrafficHomeDetailPageView
Bean> beanStream) {
        return beanStream
            .assignTimestampsAndWatermarks(
                WatermarkStrategy
```

```
                .<TrafficHomeDetailViewBean>forBoundedOutOfOrderness(Duration.ofSeconds(5L))
                .withTimestampAssigner((bean, ts) ->
bean.getTs())
                .withIdleness(Duration.ofSeconds(120L))
            )
            .windowAll(TumblingEventTimeWindows.of(Time.seconds(
5L)))
            .reduce(
                new
ReduceFunction<TrafficHomeDetailViewBean>() {
                    @Override
                    public TrafficHomeDetailViewBean
reduce(TrafficHomeDetailViewBean value1,
TrafficHomeDetailViewBean value2) throws Exception {
                        value1.setHomeUvCt(value1.getHomeUvCt()
+ value2.getHomeUvCt());
value1.setGoodDetailUvCt(value1.getGoodDetailUvCt() +
value2.getGoodDetailUvCt());
                        return value1;
                    }
                },
                new
ProcessAllWindowFunction<TrafficHomeDetailViewBean,
TrafficHomeDetailViewBean, TimeWindow>() {
                    @Override
                    public void process(Context ctx,
Iterable<TrafficHomeDetailViewBean> elements,
Collector<TrafficHomeDetailViewBean> out) throws Exception {
                        TrafficHomeDetailViewBean bean =
elements.iterator().next();
bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()))
;
bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));
```

```
bean.setCurDate(DateFormatUtil.tsToDateForPartition(ctx.window().
getStart()));

        out.collect(bean);

    }

}

);

}

private
SingleOutputStreamOperator<TrafficHomeDetailViewBean>
parseToPojo(DataStreamSource<String> stream) {
    return stream
        .map(JSON::parseObject)
        .keyBy(obj ->
obj.getJSONObject("common").getString("mid"))
        .process(new KeyedProcessFunction<String, JSONObject,
TrafficHomeDetailViewBean>() {

            private ValueState<String> homeState;
            private ValueState<String> goodDetailState;

            @Override
            public void open(Configuration parameters) throws
Exception {
                homeState = getRuntimeContext().getState(new
ValueStateDescriptor<String>("home", String.class));
                goodDetailState =
getRuntimeContext().getState(new
ValueStateDescriptor<String>("goodDetail", String.class));
            }

            @Override
            public void processElement(JSONObject obj,
                Context ctx,

Collector<TrafficHomeDetailViewBean> out) throws Exception {
                String pageId =
obj.getJSONObject("page").getString("page_id");

                Long ts = obj.getLong("ts");
                String today = DateFormatUtil.tsToDate(ts);
```

```
String lastHomeDate = homeState.value();
String lastGoodDetailDate =
goodDetailState.value();
Long homeCt = 0L;
Long goodDetailCt = 0L;
// 是首页，并且今天和状态不相等，则是当天的第一个首页
if ("home".equals(pageId)
&& !today.equals(lastHomeDate)) {
    homeCt = 1L;
    homeState.update(today);
} else if ("good_detail".equals(pageId)
&& !today.equals(lastGoodDetailDate)) {
    goodDetailCt = 1L;
    goodDetailState.update(today);
}

if (homeCt + goodDetailCt == 1) {
    out.collect(new
TrafficHomeDetailViewBean(
        "", "",
        "",
        homeCt, goodDetailCt,
        ts
    ));
}
}
});
}
```

### 12.3.7 测试

- 1) 启动 zk、kafka、HDFS、flume (fl.sh start)、Doris
- 2) 运行 DwdBaseLog、DwsTrafficHomeDetailViewWindow
- 3) 运行数据生成器
- 4) 查看 Doris 目标表数据

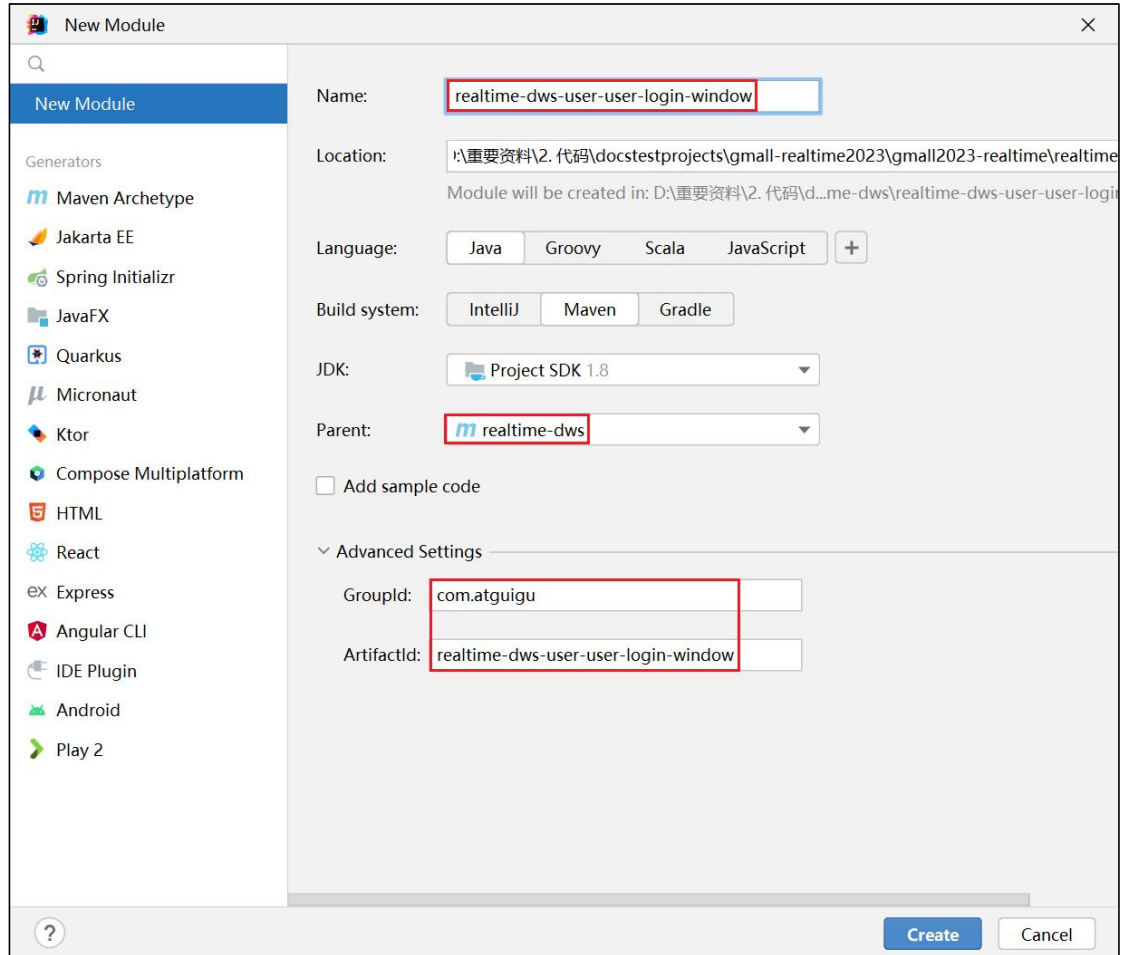
### 12.3.8 提交至 GitLab

略。

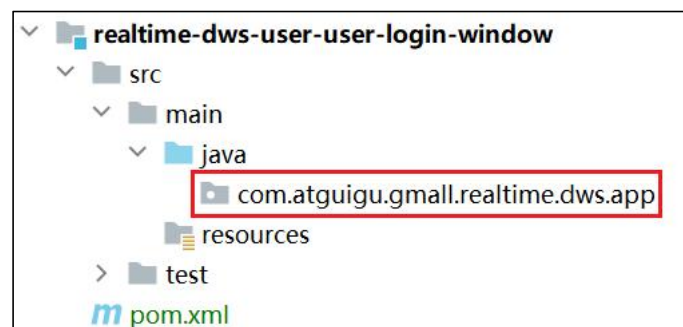
## 12.4 用户域用户登录各窗口汇总表

## 12.4.1 创建 realtime-dws-user-user-login-window

### 1) 新建模块



### 2) 创建包结构



## 12.4.2 主要任务

从 Kafka 页面日志主题读取数据，统计七日回流用户和当日独立用户数。

## 12.4.3 思路分析

之前的活跃用户，一段时间未活跃（流失），今日又活跃了，就称为回流用户。此处要求统计回流用户总数。规定当日登录，且自上次登录之后至少 7 日未登录的用户为回流用户。

#### 1) 读取 Kafka 页面主题数据

#### 2) 转换数据结构

流中数据由 String 转换为 JSONObject。

#### 3) 过滤数据

统计的指标与用户有关，uid 不为 null 的数据才是有用的。

此外，登录分为两种情况：

- 用户打开应用后自动登录
- 用户打开应用后没有登录，浏览部分页面后跳转到登录页面，中途登录

对于情况 1，登录操作发生在会话首页，所以保留首页即可；

对于情况 2，登录操作发生在 login 页面，login 页面之后必然会跳转到其它页面，保留 login 之后的页面即可记录情况 2 的登录操作。

综上，我们应保留 uid 不为 null 且 last\_page\_id 为 null 或 last\_page\_id 为 login 的浏览记录。

#### 4) 设置水位线

#### 5) 按照 uid 分组

不同用户的登录记录互不相干，各自处理。

#### 6) 统计回流用户数和独立用户数

运用 Flink 状态编程，记录用户末次登录日期。

- 若状态中的末次登录日期不为 null，进一步判断。
  - 如果末次登录日期不等于当天日期则独立用户数 uuCt 记为 1，并将状态中的末次登录日期更新为当日，进一步判断。
    - 如果当天日期与末次登录日期之差大于等于 8 天则回流用户数 backCt 置为 1。
    - 否则 backCt 置为 0。
  - 若末次登录日期为当天，则 uuCt 和 backCt 均为 0，此时本条数据不会影响统计结果，舍弃，不再发往下游。

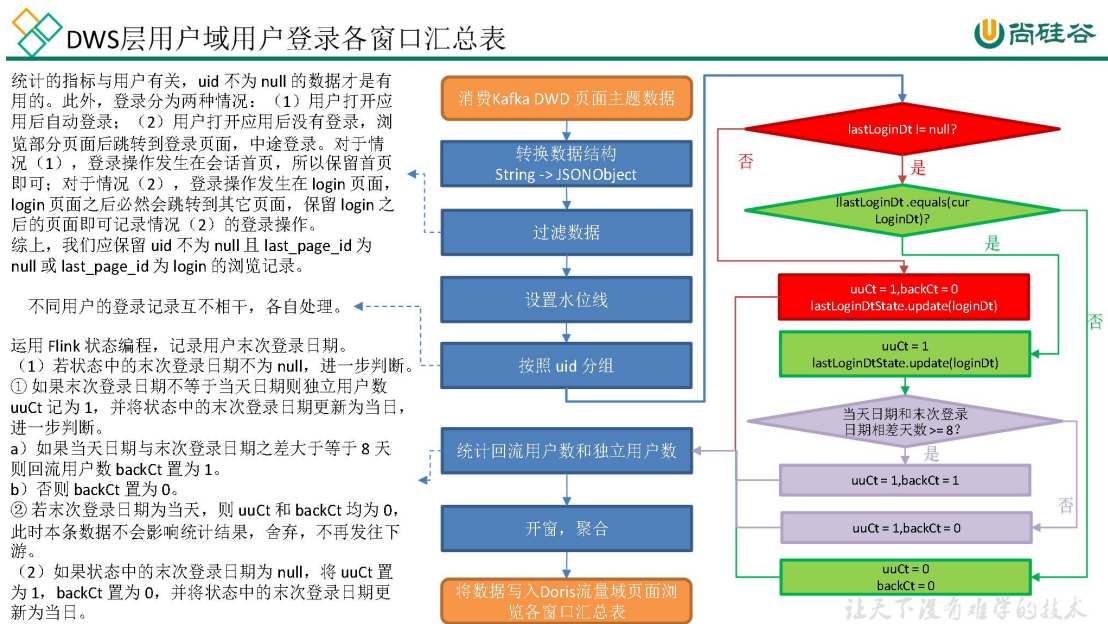
- 如果状态中的末次登录日期为 null，将 uuCt 置为 1，backCt 置为 0，并将状态中的末次登录日期更新为当日。

## 7) 开窗，聚合

度量字段求和，补充窗口起始和结束时间，统计日期字段，用于 Doris 分区。

## 8) 写入 Doris

## 12.4.4 图解



## 12.4.5 Doris 建表语句

```

drop table if exists gmall2023_realtime.dws_user_user_login_window;
create table if not exists
gmall2023_realtime.dws_user_user_login_window
(
  `stt`      DATETIME COMMENT '窗口起始时间',
  `edt`      DATETIME COMMENT '窗口结束时间',
  `cur_date` DATE COMMENT '当天日期',
  `back_ct`  BIGINT REPLACE COMMENT '回流用户数',
  `uu_ct`    BIGINT REPLACE COMMENT '独立用户数'
) engine = olap aggregate key (`stt`, `edt`, `cur_date`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
  "replication_num" = "3",
  "dynamic_partition.enable" = "true",

```

```
"dynamic_partition.time_unit" = "DAY",  
"dynamic_partition.end" = "3",  
"dynamic_partition.prefix" = "par",  
"dynamic_partition.buckets" = "10"  
);
```

## 12.4.6 代码实现

### 1) 实体类

在 common 模块的 bean 包下创建 UserLoginBean

```
package com.atguigu.gmall.realtime.common.bean;  
  
import com.alibaba.fastjson.annotation.JSONField;  
import lombok.AllArgsConstructor;  
import lombok.Data;  
import lombok.NoArgsConstructor;  
  
@Data  
@AllArgsConstructor  
@NoArgsConstructor  
public class UserLoginBean {  
    // 窗口起始时间  
    String stt;  
    // 窗口终止时间  
    String edt;  
    // 当天日期  
    String curDate;  
    // 回流用户数  
    Long backCt;  
    // 独立用户数  
    Long uuCt;  
    // 时间戳  
    @JSONField(serialize = false)  
    Long ts;  
}
```

### 2) 主程序

在本节新建模块 com.atguigu.gmall.realtime.dws.app 包下创建 DwsUserUserLoginWindow

```
package com.atguigu.gmall.realtime.dws.app;  
  
import com.alibaba.fastjson.JSON;  
import com.alibaba.fastjson.JSONObject;  
import com.atguigu.gmall.realtime.common.base.BaseApp;  
import com.atguigu.gmall.realtime.common.bean.UserLoginBean;
```



```
import com.atguigu.gmall.realtime.common.constant.Constant;
import
com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.FilterFunction;
import org.apache.flink.api.common.functions.ReduceFunction;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera
tor;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import
org.apache.flink.streaming.api.functions.windowing.AllWindowFunc
tion;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventT
imeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.time.Duration;

public class DwsUserUserLoginWindow extends BaseApp {
    public static void main(String[] args) {
        new DwsUserUserLoginWindow().start(
            10024,
            4,
            "dws_user_user_login_window",
            Constant.TOPIC_DWD_TRAFFIC_PAGE
        );
    }

    @Override
```

```
public void handle(StreamExecutionEnvironment env,
DataStreamSource<String> stream) {
    // 1. 先过滤出所有登录记录
    SingleOutputStreamOperator<JSONObject> loginLogStream =
filterLoginLog(stream);

    // 2. 再解析封装到 pojo 中: 如果是当然首次登录, 则置为 1 否则置为 0
    // 回流用户: 判断今天和最后一次登录日期的差值是否大于 7
    SingleOutputStreamOperator<UserLoginBean> beanStream =
parseToPojo(loginLogStream);

    // 3. 开窗聚合
    SingleOutputStreamOperator<UserLoginBean> resultStream =
windowAndAgg(beanStream);

    // 4. 写出
    writeToDoris(resultStream);
}

private void
writeToDoris(SingleOutputStreamOperator<UserLoginBean>
resultStream) {
    resultStream
        .map(new DorisMapFunction<>())
        .sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA
TABASE + ".dws_user_user_login_window",
"dws_user_user_login_window"));
}

private SingleOutputStreamOperator<UserLoginBean>
windowAndAgg(SingleOutputStreamOperator<UserLoginBean> beanStream)
{
    return beanStream
        .assignTimestampsAndWatermarks(
            WatermarkStrategy
                .<UserLoginBean>forBoundedOutOfOrdernes
s(Duration.ofSeconds(5L))
                .withTimestampAssigner((bean, ts) ->
bean.getTs())
                .withIdleness(Duration.ofSeconds(120L))
        )
        .windowAll(TumblingEventTimeWindows.of(Time.seconds(
5L)))
        .reduce(
```

```
        new ReduceFunction<UserLoginBean>() {
            @Override
            public UserLoginBean reduce(UserLoginBean
value1,
                                     UserLoginBean
value2) throws Exception {
                value1.setUuCt(value1.getUuCt() +
value2.getUuCt());
                value1.setBackCt(value1.getBackCt() +
value2.getBackCt());
                return value1;
            }
        },
        new AllWindowFunction<UserLoginBean,
UserLoginBean, TimeWindow>() {
            @Override
            public void apply(TimeWindow window,
                             Iterable<UserLoginBean>
values,
                             Collector<UserLoginBean> out)
throws Exception {
                UserLoginBean bean =
values.iterator().next();

                bean.setStt(DateFormatUtil.tsToDateTime(window.getStart()));

                bean.setEdt(DateFormatUtil.tsToDateTime(window.getEnd()));

                bean.setCurDate(DateFormatUtil.tsToDateForPartition(window.getSta
rt()));

                out.collect(bean);
            }
        }
    );
}

private SingleOutputStreamOperator<UserLoginBean>
parseToPojo(SingleOutputStreamOperator<JSONObject> stream) {
    return stream
        .keyBy(obj ->
obj.getJSONObject("common").getString("uid"))
        .process(new KeyedProcessFunction<String, JSONObject,
```

```
UserLoginBean>() {

    private ValueState<String> lastLoginDateState;

    @Override
    public void open(Configuration parameters) throws
Exception {

        lastLoginDateState =
getRuntimeContext().getState(new
ValueStateDescriptor<String>("lastLoginDate", String.class));
    }

    @Override
    public void processElement(JSONObject obj,
                                Context ctx,
                                Collector<UserLoginBean>
out) throws Exception {

        Long ts = obj.getLong("ts");
        String today = DateFormatUtil.tsToDate(ts);

        String lastLoginDate =
lastLoginDateState.value();

        Long uuCt = 0L;
        Long backCt = 0L;
        if (!today.equals(lastLoginDate)) {
            // 今天的第一次登录
            uuCt = 1L;
            lastLoginDateState.update(today);

            // 计算回流：曾经登录过
            if (lastLoginDate != null) { //
                long lastLoginTs =
DateFormatUtil.dateToTs(lastLoginDate);

                // 7 日回流
                if ((ts - lastLoginTs) / 1000 / 60 / 60 /
24 > 7) {

                    backCt = 1L;
                }
            }
        }

        if (uuCt == 1) {
```

```
        out.collect(new UserLoginBean("", "", "",
backCt, uuCt, ts));
    }

    }

    });

}

private SingleOutputStreamOperator<JSONObject>
filterLoginLog(DataStreamSource<String> stream) {
    return stream
        .map(JSON::parseObject)
        .filter(new FilterFunction<JSONObject>() {
            @Override
            public boolean filter(JSONObject value) throws
Exception {
                /*
                登录日志:
                自动登录
                uid != null && last_page_id == null

                手动登录
                A -> 登录页面 -> B(登录成功)

                udi != null && last_page_id = login

                合并优化: uid != null && (last_page_id == null ||
last_page_id = login)
                */
                String uid =
value.getJSONObject("common").getString("uid");
                String lastPageId =
value.getJSONObject("page").getString("last_page_id");

                return uid != null && (lastPageId == null ||
"login".equals(lastPageId));
            }
        });
}
```

## 12.4.7 测试

- 1) 启动 zk、kafka、HDFS、flume (f1.sh start)、Doris
- 2) 运行 DwdBaseLog、DwsUserUserLoginWindow
- 3) 运行数据生成器
- 4) 查看 Doris 目标表数据

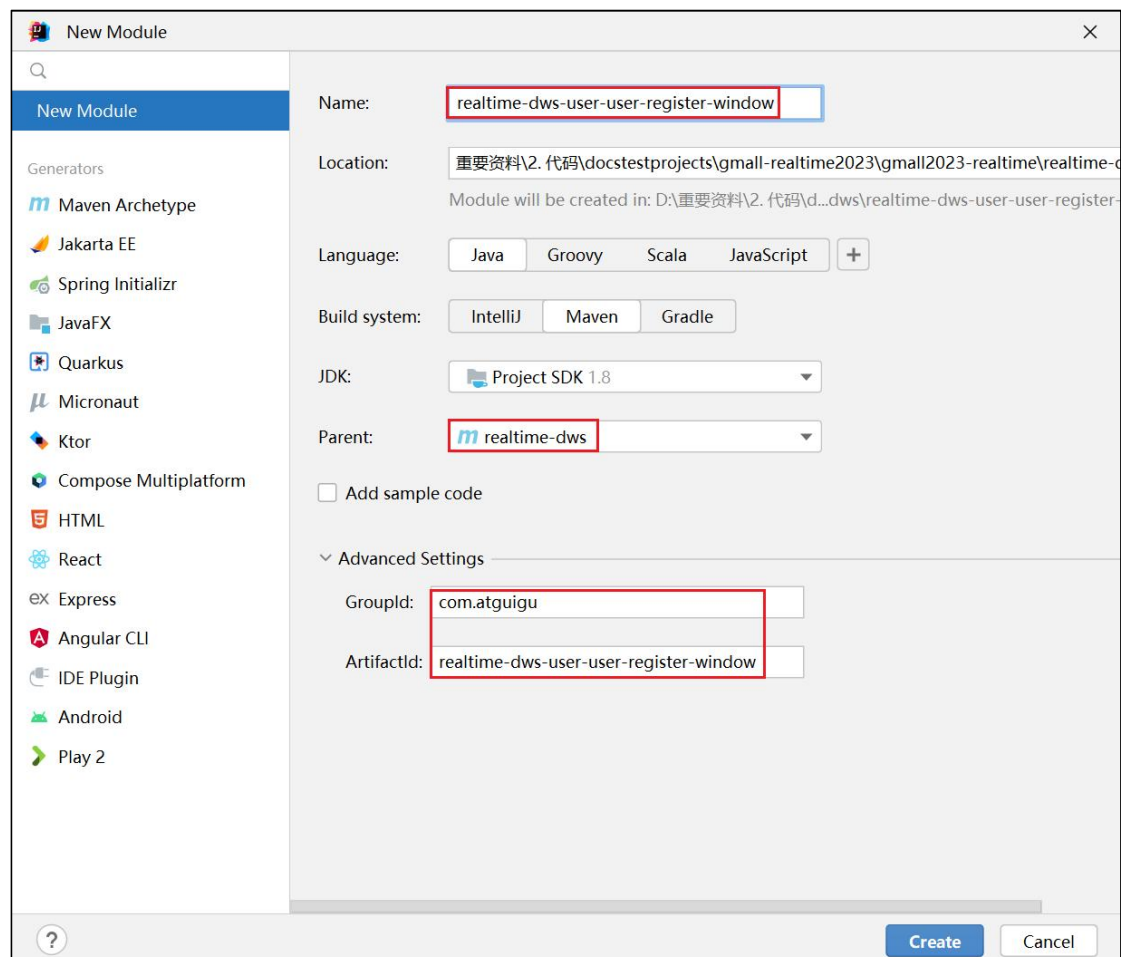
## 12.4.8 提交至 GitLab

略。

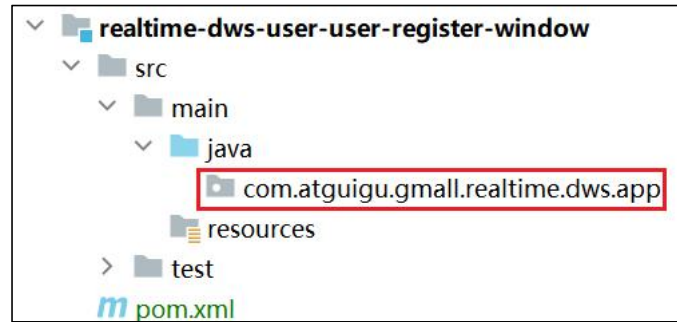
## 12.5 用户域用户注册各窗口汇总表 (练习)

### 12.5.1 创建 realtime-dws-user-user-register-window

- 1) 新建模块



- 2) 创建包结构



## 12.5.2 主要任务

从 DWD 层用户注册表中读取数据，统计各窗口注册用户数，写入 Doris。

## 12.5.3 思路分析

1) 读取 Kafka 用户注册主题数据

2) 转换数据结构

String 转换为 JSONObject。

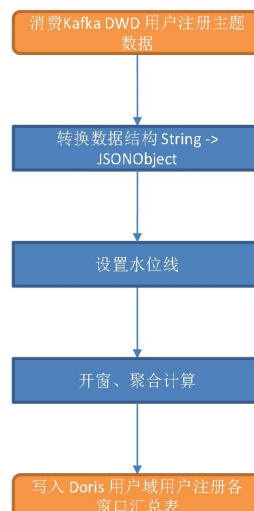
3) 设置水位线

4) 开窗、聚合

5) 写入 Doris

## 12.5.4 图解

 DWS层用户域用户注册各窗口汇总表



让天下没有难学的技术

## 12.5.5 Doris 建表语句

```
drop table if exists
gmall2023_realtime.dws_user_user_register_window;
```

```
create table if not exists
gmall2023_realtime.dws_user_user_register_window
(
    `stt`          DATETIME COMMENT '窗口起始时间',
    `edt`          DATETIME COMMENT '窗口结束时间',
    `cur_date`     DATE COMMENT '当天日期',
    `register_ct`  BIGINT REPLACE COMMENT '注册用户数'
) engine = olap aggregate key (`stt`, `edt`, `cur_date`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
    "replication_num" = "3",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "par",
    "dynamic_partition.buckets" = "10"
);
```

## 12.5.6 代码实现

### 1) 实体类

在 common 模块的 bean 包下创建实体类 UserRegisterBean

```
package com.atguigu.gmall.realtime.common.bean;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class UserRegisterBean {
    // 窗口起始时间
    String stt;
    // 窗口终止时间
    String edt;
    // 当天日期
    String curDate;
    // 注册用户数
    Long registerCt;
}
```



## 2) 主程序

在本节新建模块下创建主程序 `DwsUserUserRegisterWindow`

```
package com.atguigu.gmall.realtime.dws.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.bean.UserRegisterBean;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.AggregateFunction;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.streaming.api.functions.windowing.ProcessAllWindowFunction;
import org.apache.flink.streaming.api.windowing.assigners.TumblingEventTimeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.time.Duration;

public class DwsUserUserRegisterWindow extends BaseApp {
    public static void main(String[] args) {
        new DwsUserUserRegisterWindow().start(
            10025,
            4,
            "dws_user_user_register_window",
            Constant.TOPIC_DWD_USER_REGISTER
        );
    }

    @Override
```

```
public void handle(StreamExecutionEnvironment env,
DataStreamSource<String> stream) {
    stream
        .map(JSON::parseObject)
        .assignTimestampsAndWatermarks(
            WatermarkStrategy
                .<JSONObject>forBoundedOutOfOrderness(D
uration.ofSeconds(5L))
                .withTimestampAssigner((obj, ts) ->
obj.getLong("create_time")) // fastjson 会自动把 datetime 转成 long
                .withIdleness(Duration.ofSeconds(120L))
        )
        .windowAll(TumblingEventTimeWindows.of(Time.seconds(
5L)))
        .aggregate(
            new AggregateFunction<JSONObject, Long, Long>()
{
    @Override
    public Long createAccumulator() {
        return 0L;
    }

    @Override
    public Long add(JSONObject value, Long acc)
{
        return acc + 1;
    }

    @Override
    public Long getResult(Long acc) {
        return acc;
    }

    @Override
    public Long merge(Long acc1, Long acc2) {
        return acc1 + acc2;
    }
},
            new ProcessAllWindowFunction<Long,
UserRegisterBean, TimeWindow>() {
    @Override
    public void process(Context ctx,
                        Iterable<Long> elements,
```

```
Collector<UserRegisterBean> out) throws Exception {
    Long result =
elements.iterator().next();

    out.collect(new
UserRegisterBean(DateFormatUtil.tsToDateTime(ctx.window().getStar
t()),
DateFormatUtil.tsToDateTime(ctx.window().getEnd()),
DateFormatUtil.tsToDateForPartition(ctx.window().getEnd()),
result
));
}
}
)
.map(new DorisMapFunction<>())
.sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA
TABASE + ".dws_user_user_register_window",
"dws_user_user_register_window"));
}
}
```

### 12.5.7 测试

- 1) 启动 zk、kafka、HDFS、Maxwell、Doris
- 2) 运行 DwdBaseDb、DwsUserUserRegisterWindow
- 3) 运行数据生成器
- 4) 查看 Doris 目标表数据

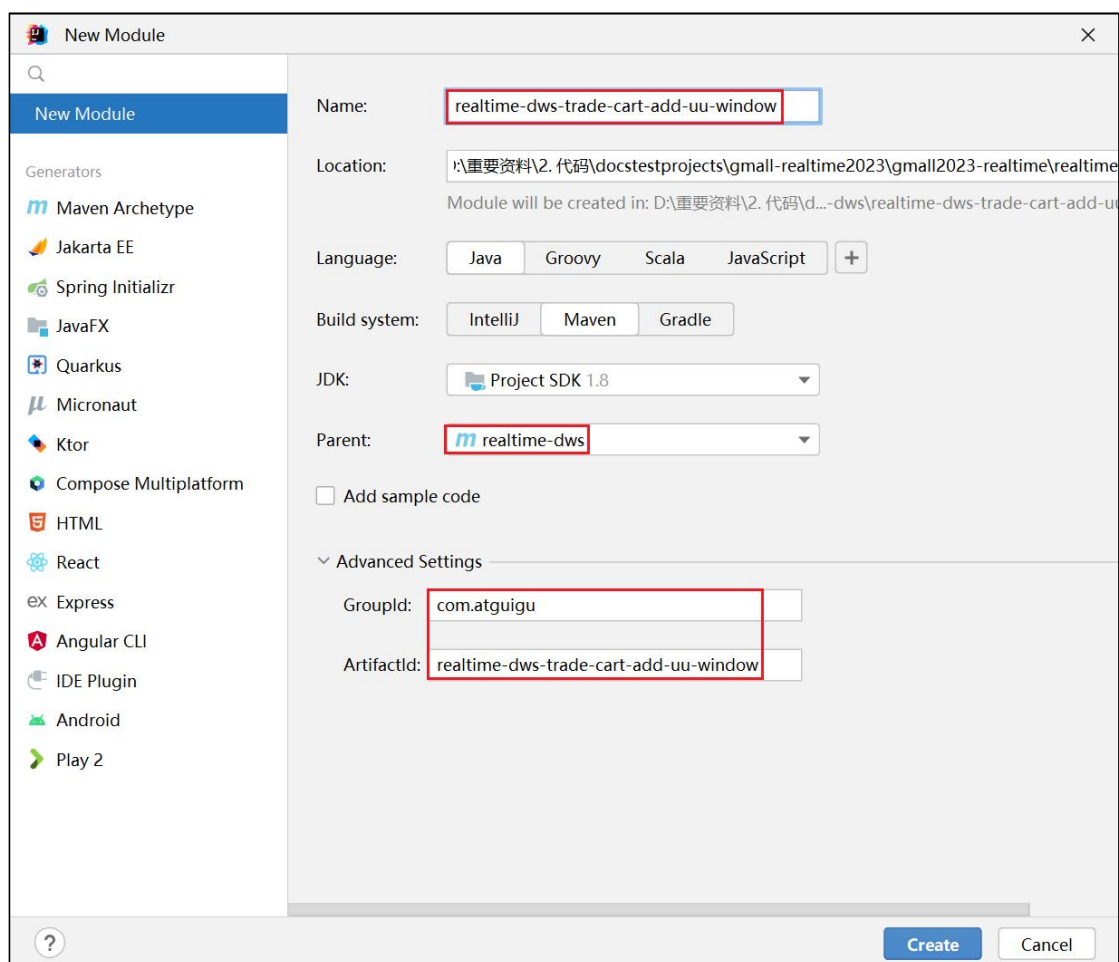
### 12.5.8 提交至 GitLab

略。

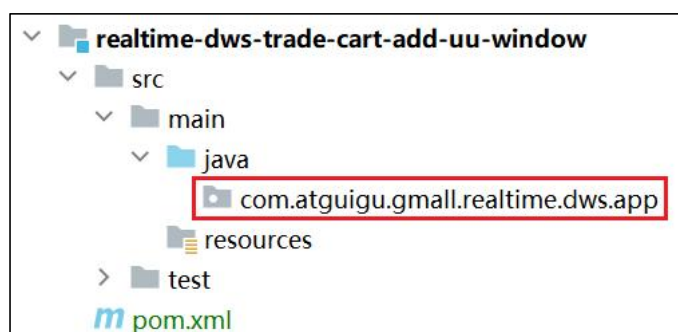
## 12.6 交易域加购各窗口汇总表

### 12.6.1 创建 realtime-dws-trade-cart-add-uu-window

- 1) 新建模块



## 2) 创建包结构



## 12.6.2 主要任务

从 Kafka 读取用户加购明细数据，统计各窗口加购独立用户数，写入 Doris。

## 12.6.3 思路分析

### 1) 从 Kafka 加购明细主题读取数据

### 2) 转换数据结构

将流中数据由 String 转换为 JSONObject。

## 3) 设置水位线

## 4) 按照用户 id 分组

## 5) 过滤独立用户加购记录

运用 Flink 状态编程，将用户末次加购日期维护到状态中。

如果末次加购日期为 null 或者不等于当天日期，则保留数据并更新状态，否则丢弃，不做操作。

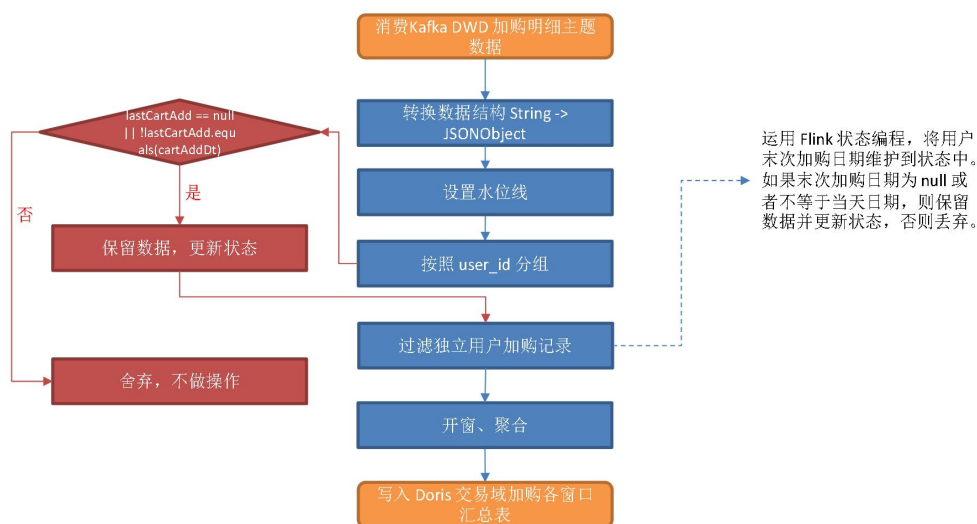
## 6) 开窗、聚合

统计窗口中数据条数即为加购独立用户数，补充窗口起始时间、关闭时间，统计日期，发送到下游。

## 7) 将数据写入 Doris。

## 12.6.4 图解

DWS层交易域加购各窗口汇总表



让天下没有难学的技术

## 12.6.5 Doris 建表语句

```

drop table if exists
gmall2023_realtime.dws_trade_cart_add_uu_window;
create table if not exists
gmall2023_realtime.dws_trade_cart_add_uu_window
(
    `stt` DATETIME COMMENT '窗口起始时间',
    `edt` DATETIME COMMENT '窗口结束时间',

```

```
`cur_date`          DATE COMMENT '当天日期',
`cart_add_uu_ct`    BIGINT REPLACE COMMENT '加购独立用户数'
) engine = olap aggregate key (`stt`, `edt`, `cur_date`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
"replication_num" = "3",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.end" = "3",
"dynamic_partition.prefix" = "par",
"dynamic_partition.buckets" = "10"
);
```

## 12.6.6 代码实现

### 1) 实体类

在 common 模块的 bean 包下创建实体类 CartAddUuBean

```
package com.atguigu.gmall.realtime.common.bean;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class CartAddUuBean {
    // 窗口起始时间
    String stt;
    // 窗口闭合时间
    String edt;
    // 当天日期
    String curDate;
    // 加购独立用户数
    Long cartAddUuCt;
}
```

### 2) 主程序

在本节新建模块下创建主程序 DwsTradeCartAddUuWindow

```
package com.atguigu.gmall.realtime.dws.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.bean.CartAddUuBean;
```

```
import com.atguigu.gmall.realtime.common.constant.Constant;
import
com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.ReduceFunction;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import
org.apache.flink.streaming.api.functions.windowing.ProcessAllWindowFunction;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventTimeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.time.Duration;

public class DwsTradeCartAddUuWindow extends BaseApp {
    public static void main(String[] args) {
        new DwsTradeCartAddUuWindow().start(
            10026,
            4,
            "dws_trade_cart_add_uu_window",
            Constant.TOPIC_DWD_TRADE_CART_ADD
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        DataStreamSource<String> stream) {
        stream
            .map(JSON::parseObject)
```

```
.assignTimestampsAndWatermarks (
    WatermarkStrategy
        .<JSONObject>forBoundedOutOfOrderness (Duration.of
Seconds (5L))
        .withTimestampAssigner ((obj, ts) ->
obj.getLong ("ts") * 1000)
        .withIdleness (Duration.ofSeconds (120L))
    )
    .keyBy (obj -> obj.getString ("user_id"))
    .process (new KeyedProcessFunction<String, JSONObject,
CartAddUuBean> () {

        private ValueState<String> lastCartAddDateState;

        @Override
        public void open (Configuration parameters) {
            lastCartAddDateState =
getRuntimeContext ().getState (new
ValueStateDescriptor<String> ("lastCartAddDate", String.class));
        }

        @Override
        public void processElement (JSONObject jsonObj,
Context context,
Collector<CartAddUuBean> out)
throws Exception {
            String lastCartAddDate =
lastCartAddDateState.value ();
            long ts = jsonObj.getLong ("ts") * 1000;
            String today = DateFormatUtil.tsToDate (ts);

            if (!today.equals (lastCartAddDate)) {
                lastCartAddDateState.update (today);

                out.collect (new CartAddUuBean ("", "", "", 1L));
            }

        }
    })
    .windowAll (TumblingEventTimeWindows.of (Time.seconds (5L))
)
    .reduce (
        new ReduceFunction<CartAddUuBean> () {
```



```
        @Override
        public CartAddUuBean reduce(CartAddUuBean value1,
                                     CartAddUuBean value2) {
            value1.setCartAddUuCt(value1.getCartAddUuCt()
+ value2.getCartAddUuCt());
            return value1;
        }
    },
    new ProcessAllWindowFunction<CartAddUuBean,
CartAddUuBean, TimeWindow>() {
        @Override
        public void process(Context ctx,
                             Iterable<CartAddUuBean> elements,
                             Collector<CartAddUuBean> out)
throws Exception {
            CartAddUuBean bean =
elements.iterator().next();

            bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()))
;

            bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));

            bean.setCurDate(DateFormatUtil.tsToDateForPartition(ctx.window().
getStart()));

            out.collect(bean);
        }
    }
)

.map(new DorisMapFunction<>())
.sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DATAB
ASE + ".dws_trade_cart_add_uu_window",
"dws_trade_cart_add_uu_window"));

}
}
```

## 12.6.7 测试

- 1) 启动 zk、kafka、HDFS、Maxwell、Doris
- 2) 运行 DwdTradeCartAdd、DwsTradeCartAddUuWindow

3) 运行数据生成器

4) 查看 Doris 目标表数据

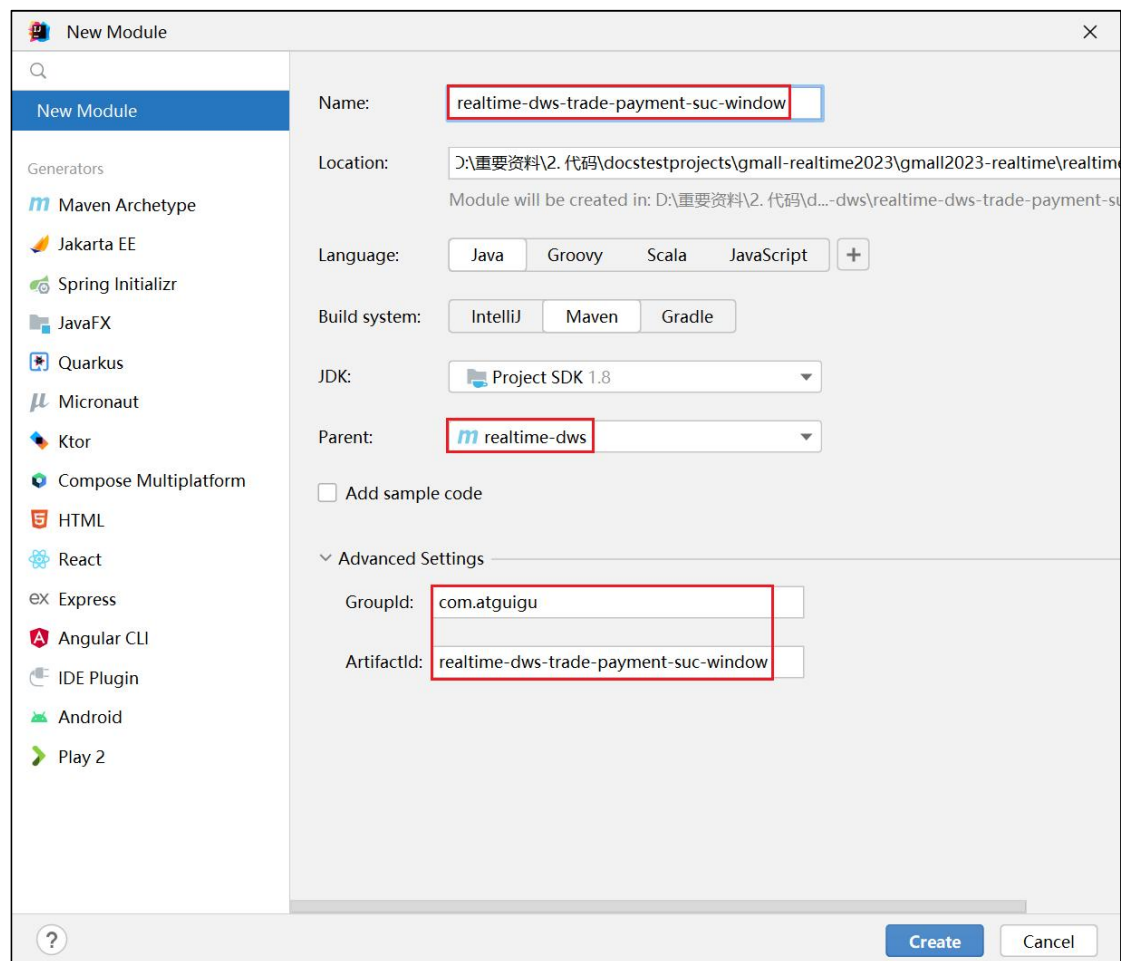
## 12.6.8 提交至 GitLab

略。

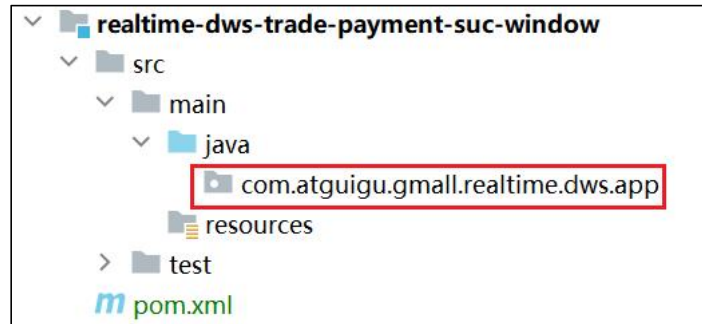
## 12.7 交易域支付成功各窗口汇总表 (练习)

### 12.7.1 创建 realtime-dws-trade-payment-suc-window

1) 新建模块



2) 创建包结构



### 12.7.2 主要任务

从 Kafka 读取交易域支付成功主题数据,统计支付成功独立用户数和首次支付成功用户数。

### 12.7.3 思路分析

1) 从 Kafka 支付成功明细主题读取数据

2) 转换数据结构

String 转换为 JSONObject。

3) 设置水位线

4) 按照 user\_id 分组

5) 统计独立支付人数和新增支付人数

运用 Flink 状态编程,在状态中维护用户末次支付日期。

若末次支付日期为 null,则将首次支付用户数和支付独立用户数均置为 1;否则首次支付用户数置为 0,判断末次支付日期是否为当日,如果不是当日则支付独立用户数置为 1,否则置为 0。最后将状态中的支付日期更新为当日。

6) 开窗、聚合

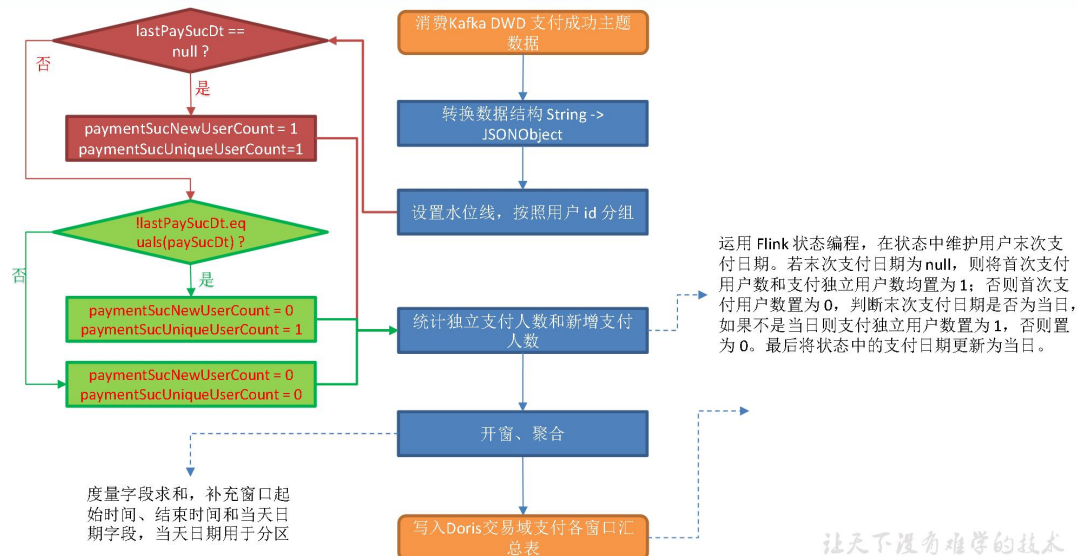
度量字段求和,补充窗口起始时间和结束时间字段,ts 字段置为当前系统时间戳。

7) 写出到 Doris

### 12.7.4 图解



## DWS层交易域支付成功各窗口汇总表



## 12.7.5 Doris 建表语句

```
drop table if exists
gmall2023_realtime.dws_trade_payment_suc_window;
create table if not exists
gmall2023_realtime.dws_trade_payment_suc_window
(
    `stt` DATETIME COMMENT '窗口起始时间',
    `edt` DATETIME COMMENT '窗口结束时间',
    `cur_date` DATE COMMENT '当天日期',
    `payment_suc_unique_user_count` BIGINT REPLACE COMMENT '支付成功独立用户数',
    `payment_suc_new_user_count` BIGINT REPLACE COMMENT '支付成功新用户数'
) engine = olap aggregate key (`stt`, `edt`, `cur_date`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
    "replication_num" = "3",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "par",
    "dynamic_partition.buckets" = "10"
);
```

## 12.7.6 代码实现

## 1) 实体类

在 common 模块的 bean 目录下创建实体类 TradePaymentBean

```
package com.atguigu.gmall.realtime.common.bean;

import com.alibaba.fastjson.annotation.JSONField;
import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class TradePaymentBean {
    // 窗口起始时间
    String stt;
    // 窗口终止时间
    String edt;
    // 当天日期
    String curDate;
    // 支付成功独立用户数
    Long paymentSucUniqueUserCount;
    // 支付成功新用户数
    Long paymentSucNewUserCount;
    // 时间戳
    @JSONField(serialize = false)
    Long ts;
}
```

## 2) 主程序

在本节新建模块下创建主程序 DwsTradePaymentSucWindow

```
package com.atguigu.gmall.realtime.dws.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.bean.TradePaymentBean;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.ReduceFunction;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
```

```
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import
org.apache.flink.streaming.api.functions.windowing.ProcessAllWindowFunction;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventTimeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.time.Duration;

public class DwsTradePaymentSucWindow extends BaseApp {
    public static void main(String[] args) {
        new DwsTradePaymentSucWindow().start(
            10027,
            4,
            "dws_trade_payment_suc_window",
            Constant.TOPIC_DWD_TRADE_ORDER_PAYMENT_SUCCESS
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        DataStreamSource<String> stream) {
        stream
            .map(JSON::parseObject)
            .keyBy(obj -> obj.getString("user_id"))
            .process(new KeyedProcessFunction<String, JSONObject,
TradePaymentBean>() {

                private ValueState<String> lastPayDateState;

                @Override
                public void open(Configuration parameters) throws
Exception {
```

```
        lastPayDateState =
getRuntimeContext().getState(new
ValueStateDescriptor<String>("lastPayDate", String.class));
    }

    @Override
    public void processElement(JSONObject obj,
                               Context ctx,
                               Collector<TradePaymentBean> out) throws Exception {
        String lastPayDate = lastPayDateState.value();
        long ts = obj.getLong("ts") * 1000;
        String today = DateFormatUtil.tsToDate(ts);

        long payUuCount = 0L;
        long payNewCount = 0L;
        if (!today.equals(lastPayDate)) { // 今天第一次
            lastPayDateState.update(today);
            payUuCount = 1L;

            if (lastPayDate == null) {
                // 表示这个用户曾经没有支付过，是一个新用户支
                payNewCount = 1L;
            }
        }

        if (payUuCount == 1) {
            out.collect(new TradePaymentBean("", "", "",
payUuCount, payNewCount, ts));
        }
    }
})
.assignTimestampsAndWatermarks(
    WatermarkStrategy
        .<TradePaymentBean>forBoundedOutOfOrder
ness(Duration.ofSeconds(5L))
        .withTimestampAssigner((bean, ts) ->
bean.getTs())
        .withIdleness(Duration.ofSeconds(120L))
)
.windowAll(TumblingEventTimeWindows.of(Time.seconds(
```

```
5L)))

        .reduce(
            new ReduceFunction<TradePaymentBean>() {
                @Override
                public TradePaymentBean
reduce(TradePaymentBean value1,
                                                TradePaymentBean
value2) throws Exception {

value1.setPaymentSucNewUserCount(value1.getPaymentSucNewUserCount
() + value2.getPaymentSucNewUserCount());

value1.setPaymentSucUniqueUserCount(value1.getPaymentSucUniqueUse
rCount() + value2.getPaymentSucUniqueUserCount());

                return value1;
            }
        ),
        new ProcessAllWindowFunction<TradePaymentBean,
TradePaymentBean, TimeWindow>() {
            @Override
            public void process(Context ctx,

Iterable<TradePaymentBean> elements,

Collector<TradePaymentBean> out) throws Exception {
                TradePaymentBean bean =
elements.iterator().next();

bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()))
;

bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));

bean.setCurDate(DateFormatUtil.tsToDateForPartition(ctx.window().
getStart()));

                out.collect(bean);
            }
        }
    )
    .map(new DorisMapFunction<>())
    .sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA
TABASE + ".dws_trade_payment_suc_window",
```



```
"dws_trade_payment_suc_window")));  
  
    }  
}
```

### 12.7.7 测试

1) 启动 zk、kafka、HDFS、Maxwell、Doris

2) 运行应用程序

(1) DwdTradeOrderDetail

(2) DwdTradeOrderPaySucDetail

(3) DwsTradePaymentSucWindow

3) 运行数据生成器

4) 查看 Doris 目标表数据

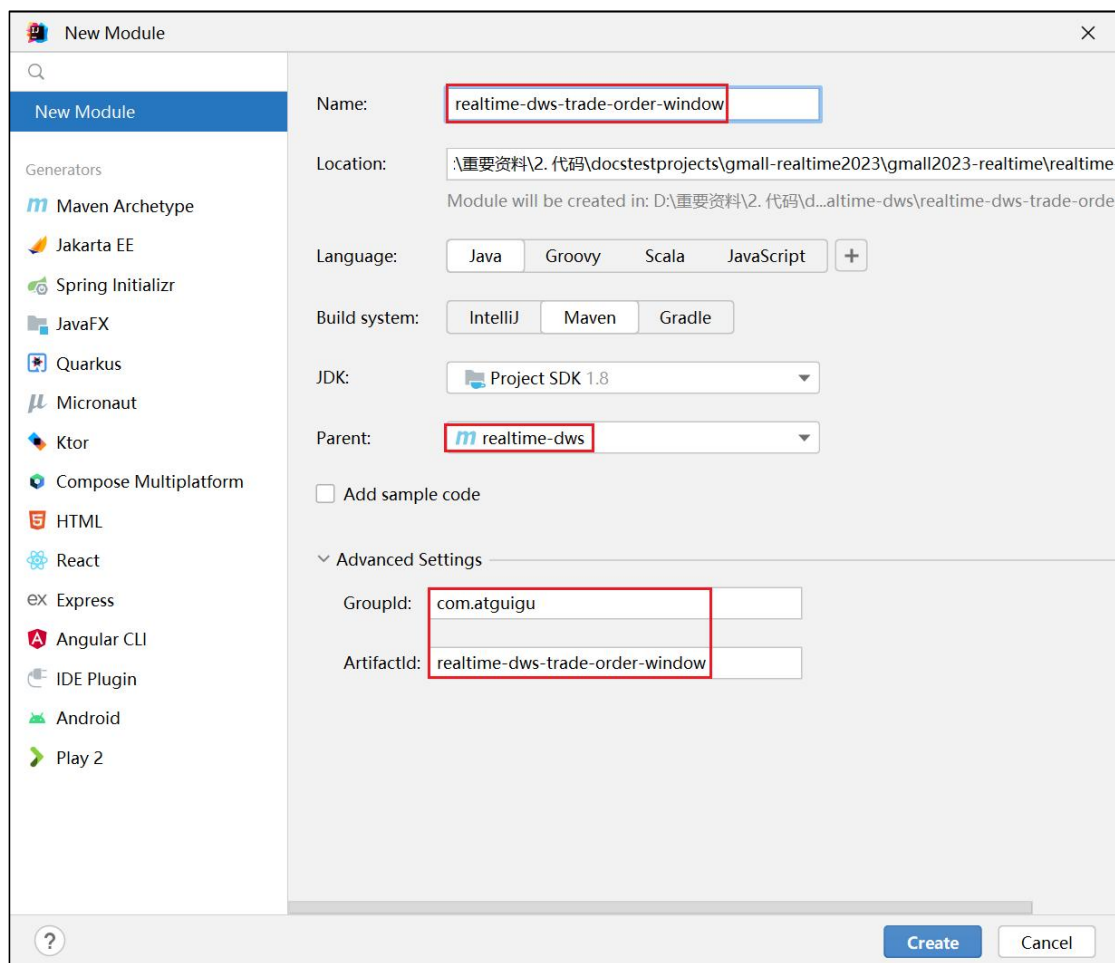
### 12.7.8 提交至 GitLab

略。

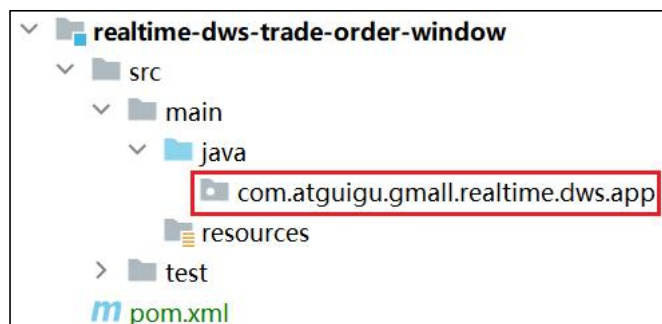
## 12.8 交易域下单各窗口汇总表 (练习)

### 12.8.1 创建 realtime-dws-trade-order-window

1) 新建模块



## 2) 创建包结构



## 12.8.2 主要任务

从 Kafka 订单明细主题读取数据，统计当日下单独立用户数和首次下单用户数，封装为实体类，写入 Doris。

## 12.8.3 思路分析

### 1) 从 Kafka 订单明细主题读取数据

### 2) 转换数据结构

Kafka 订单明细主题的数据是通过 Kafka-Connector 从订单预处理主题读取后进行过滤获取的, Kafka-Connector 会过滤掉主题中的 null 数据, 因此订单明细主题不存在为 null 的数据, 直接转换数据结构即可。

## 3) 设置水位线

## 4) 按照用户 id 分组

## 5) 计算度量字段的值

运用 Flink 状态编程, 在状态中维护用户末次下单日期。

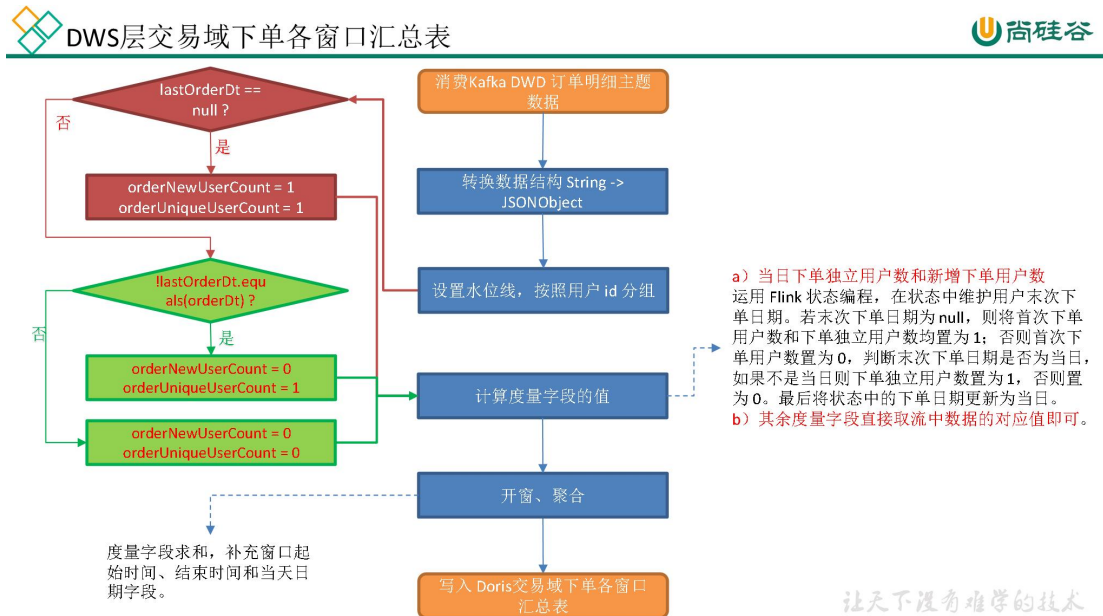
若末次下单日期为 null, 则将首次下单用户数和下单独立用户数均置为 1; 否则首次下单用户数置为 0, 判断末次下单日期是否为当日, 如果不是当日则下单独立用户数置为 1, 否则置为 0。最后将状态中的下单日期更新为当日。

## 6) 开窗、聚合

度量字段求和, 补充窗口起始时间和结束时间字段, ts 字段置为当前系统时间戳。

## 7) 写出到 Doris。

## 12.8.4 图解



## 12.8.5 Doris 建表语句

```
drop table if exists gmall2023_realtime.dws_trade_order_window;
create table if not exists gmall2023_realtime.dws_trade_order_window
(
    `stt` DATETIME COMMENT '窗口起始时间',
```

```
`edt`                                DATETIME COMMENT '窗口结束时间',
`cur_date`                            DATE COMMENT '当天日期',
`order_unique_user_count` BIGINT REPLACE COMMENT '下单独立用户数'
',
`order_new_user_count`    BIGINT REPLACE COMMENT '下单新用户数'
) engine = olap aggregate key (`stt`, `edt`, `cur_date`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
"replication_num" = "3",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.end" = "3",
"dynamic_partition.prefix" = "par",
"dynamic_partition.buckets" = "10"
);
```

## 12.8.6 代码实现

### 1) 实体类

在 common 模块的 bean 包下创建实体类 TradeOrderBean

```
package com.atguigu.gmall.realtime.common.bean;

import com.alibaba.fastjson.annotation.JSONField;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;

@Data
@AllArgsConstructor
@Builder
public class TradeOrderBean {
    // 窗口起始时间
    String stt;
    // 窗口关闭时间
    String edt;
    // 当天日期
    String curDate;
    // 下单独立用户数
    Long orderUniqueUserCount;
    // 下单新用户数
    Long orderNewUserCount;
    // 时间戳
    @JSONField(serialize = false)
```

```
Long ts;  
}
```

## 2) 主程序

在本节新建模块下创建主程序 `DwsTradeOrderWindow`

```
package com.atguigu.gmall.realtime.dws.app;  
  
import com.alibaba.fastjson.JSONObject;  
import com.atguigu.gmall.realtime.common.base.BaseApp;  
import com.atguigu.gmall.realtime.common.bean.TradeOrderBean;  
import com.atguigu.gmall.realtime.common.constant.Constant;  
import  
com.atguigu.gmall.realtime.common.function.DorisMapFunction;  
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;  
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;  
import org.apache.flink.api.common.eventtime.WatermarkStrategy;  
import org.apache.flink.api.common.functions.ReduceFunction;  
import org.apache.flink.api.common.state.ValueState;  
import org.apache.flink.api.common.state.ValueStateDescriptor;  
import org.apache.flink.configuration.Configuration;  
import org.apache.flink.streaming.api.datastream.DataStreamSource;  
import  
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;  
import  
org.apache.flink.streaming.api.functions.KeyedProcessFunction;  
import  
org.apache.flink.streaming.api.functions.windowing.ProcessAllWindowFunction;  
import  
org.apache.flink.streaming.api.windowing.assigners.TumblingEventTimeWindows;  
import org.apache.flink.streaming.api.windowing.time.Time;  
import  
org.apache.flink.streaming.api.windowing.windows.TimeWindow;  
import org.apache.flink.util.Collector;  
  
import java.time.Duration;  
  
public class DwsTradeOrderWindow extends BaseApp {  
    public static void main(String[] args) {  
        new DwsTradeOrderWindow().start(  
            10028,  
            4,  

```

```
        "dws_trade_order_window",
        Constant.TOPIC_DWD_TRADE_ORDER_DETAIL
    );
}

@Override
public void handle(StreamExecutionEnvironment env,
                   DataStreamSource<String> stream) {
    stream
        .map(JSONObject::parseObject)
        .assignTimestampsAndWatermarks(
            WatermarkStrategy
                .<JSONObject>forBoundedOutOfOrderness(Duration.of
Seconds(5L))
                .withTimestampAssigner((obj, ts) ->
obj.getLong("ts") * 1000)
                .withIdleness(Duration.ofSeconds(120L))
        )
        .keyBy(obj -> obj.getString("user_id"))
        .process(new KeyedProcessFunction<String, JSONObject,
TradeOrderBean>() {

            private ValueState<String> lastOrderDateState;

            @Override
            public void open(Configuration parameters) {
                lastOrderDateState =
getRuntimeContext().getState(new
ValueStateDescriptor<String>("lastOrderDate", String.class));
            }

            @Override
            public void processElement(JSONObject value,
                                       Context ctx,
                                       Collector<TradeOrderBean> out)
throws Exception {
                long ts = value.getLong("ts") * 1000;

                String today = DateFormatUtil.tsToDate(ts);
                String lastOrderDate = lastOrderDateState.value();

                long orderUu = 0L;
                long orderNew = 0L;
                if (!today.equals(lastOrderDate)) {
```

```
        orderUu = 1L;
        lastOrderDateState.update(today);

        if (lastOrderDate == null) {
            orderNew = 1L;
        }

    }
    if (orderUu == 1) {
        out.collect(new TradeOrderBean("", "", "",
orderUu, orderNew, ts));
    }
}
}))
.windowAll(TumblingEventTimeWindows.of(Time.seconds(5L))
)

.reduce(
    new ReduceFunction<TradeOrderBean>() {
        @Override
        public TradeOrderBean reduce(TradeOrderBean
value1,
                                     TradeOrderBean value2) {
            value1.setOrderUniqueUserCount(value1.getOrderUniqueUserCount() +
value2.getOrderUniqueUserCount());

            value1.setOrderNewUserCount(value1.getOrderNewUserCount() +
value2.getOrderNewUserCount());

            return value1;
        }
    },
    new ProcessAllWindowFunction<TradeOrderBean,
TradeOrderBean, TimeWindow>() {
        @Override
        public void process(Context ctx,
                             Iterable<TradeOrderBean>
elements,
                             Collector<TradeOrderBean> out)
throws Exception {
            TradeOrderBean bean =
elements.iterator().next();

            bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()))
;
        }
    }
);
```

```
bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));

bean.setCurDate(DateFormatUtil.tsToDateForPartition(ctx.window().
getStart()));

        out.collect(bean);
    }
}

)
.map(new DorisMapFunction<>())
.sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DATABASE + ".dws_trade_order_window", "dws_trade_order_window"));

}
}
```

### 12.8.7 测试

- 1) 启动 zk、kafka、HDFS、Maxwell、Doris
- 2) 运行应用程序 DwdTradeOrderDetail、DwsTradeOrderWindow
- 3) 运行数据生成器
- 4) 查看 Doris 目标表数据

### 12.8.8 提交至 GitLab

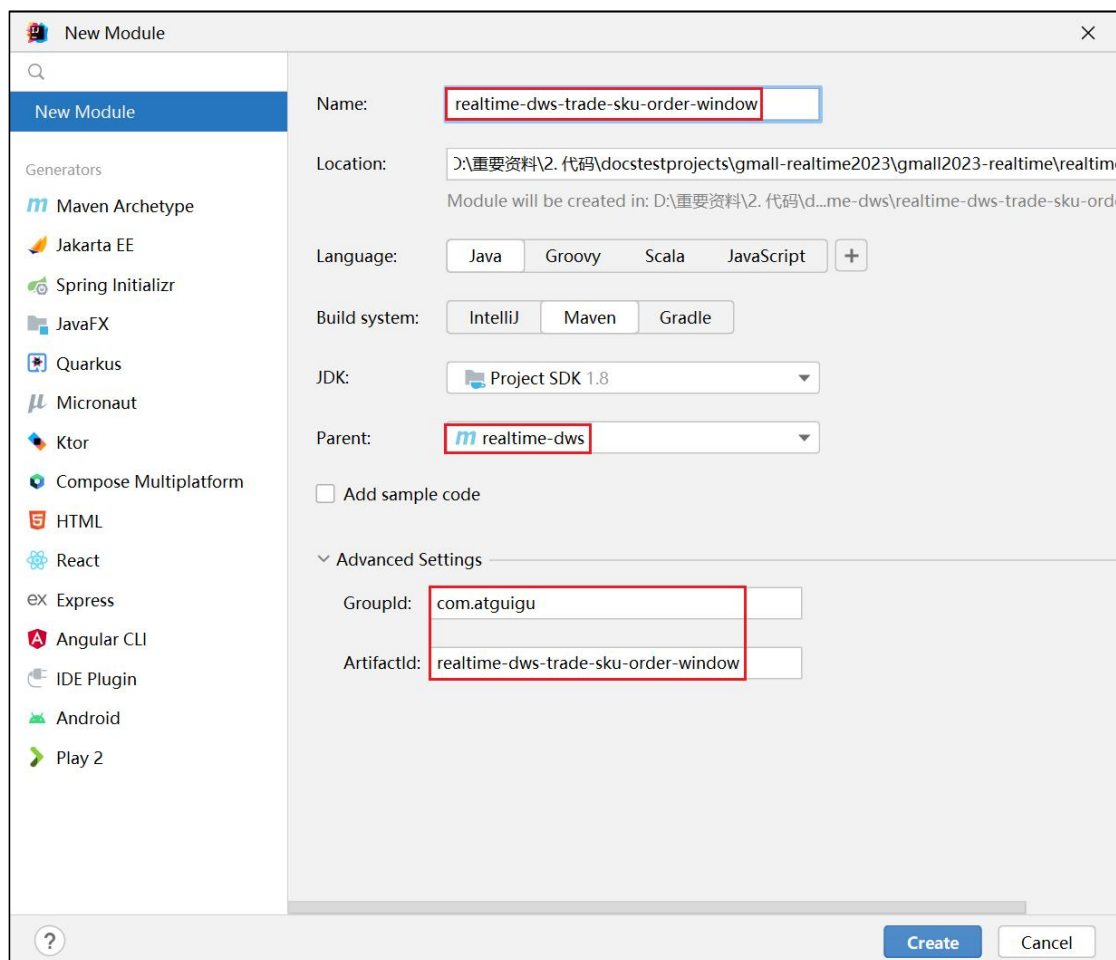
略。

## 12.9 交易域 SKU 粒度下单各窗口汇总表

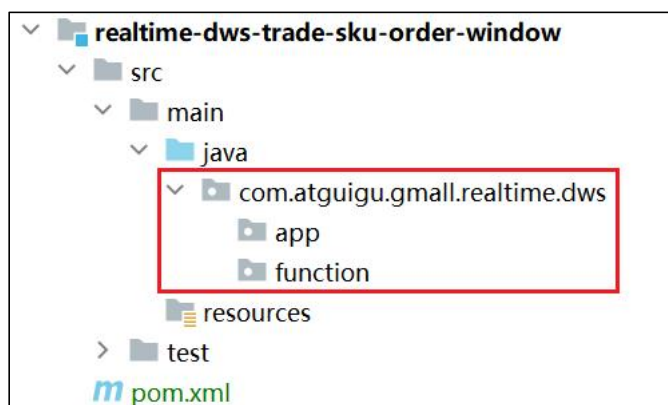
### 12.9.1 创建 realtime-dws-trade-sku-order-window

- 1) 新建模块





## 2) 创建包结构



## 12.9.2 主要任务

从 Kafka 订单明细主题读取数据，过滤 null 数据并按照唯一键对数据去重，按照 SKU 维度分组，统计原始金额、活动减免金额、优惠券减免金额和订单金额，并关联维度信息，将数据写入 Doris 交易域 SKU 粒度下单各窗口汇总表

## 12.9.3 思路分析

与上文提到的 DWS 层宽表相比，本程序新增了去重以及维度关联操作。维度表保存在 Hbase，首先要在 HBaseUtil 工具类中补充查询方法。

### 12.9.3.1 去重

我们在 DWD 层提到，订单明细表数据生成过程中会形成回撤流。left join 生成的数据集中，相同唯一键的数据可能会有多条。上文已有讲解，不再赘述。回撤数据在 Kafka 中以 null 值的形式存在，只需要简单判断即可过滤。我们需要考虑的是如何对其余数据去重。

对回撤流数据生成过程进行分析，可以发现，字段内容完整数据的生成一定晚于不完整数据的生成，要确保统计结果的正确性，我们应保留字段内容最全的数据，基于以上论述，内容最全的数据生成时间最晚。我们要做的是设法仅保留最后一条数据。重复数据对下游统计的影响主要在于金额等度量字段的重复计算。考虑这样的策略：在第一条数据到来时将其保存到状态中，然后直接发往下游。后面的数据将状态中数据的度量字段取反，发到下游，再将状态更新为当前数据，将当前数据发送到下游的窗口处理函数。如果窗口闭合时部分事件对应的修正记录迟到，将会导致数据迟到，针对这样的情况，我们通过 allowedLateness() 方法指定窗口延迟关闭时间，水位线升高到可以闭合窗口时，立即向下游发送汇总结果，之后在延迟时间内到来的每一条数据都会触发窗口的再次计算。这样只要窗口的延迟闭合时间设置合理，就可以保证最终结果的准确性，同时又不会因去重损失时效性。

### 12.9.3.2 旁路缓存优化

外部数据源的查询常常是流式计算的性能瓶颈。以本程序为例，每次查询都要连接 HBase，数据传输需要做序列化、反序列化，还有网络传输，严重影响时效性。可以通过旁路缓存对查询进行优化。

旁路缓存模式是一种非常常见的按需分配缓存模式。所有请求优先访问缓存，若缓存命中，直接获得数据返回给请求者。如果未命中则查询数据库，获取结果后，将其返回并写入缓存以备后续请求使用。

#### 1) 旁路缓存策略应注意两点

- (1) 缓存要设过期时间，不然冷数据会常驻缓存，浪费资源。
- (2) 要考虑维度数据是否会发生变化，如果发生变化要主动清除缓存。

#### 2) 缓存的选型

一般两种：堆缓存或者独立缓存服务 (memcache, redis)

堆缓存，性能更好，效率更高，因为数据访问路径更短。但是难于管理，其它进程无法维护缓存中的数据。

独立缓存服务（redis，memcache），会有创建连接、网络 IO 等消耗，较堆缓存略差，但性能尚可。独立缓存服务便于维护和扩展，对于数据会发生变化且数据量很大的场景更加适用，此处选择独立缓存服务，将 redis 作为缓存介质。

### 3) 实现步骤

#### (1) 查询时

从缓存中获取数据。

- 如果查询结果不为 null，则返回结果。
- 如果缓存中获取的结果为 null，则从 HBase 表中查询数据。
  - 如果结果非空则将数据写入缓存后返回结果。
  - 否则提示用户：没有对应的维度数据
  - 注意：缓存中的数据要设置超时时间，本程序设置为 1 天。此外，如果原表数据发生变化，要删除对应缓存。为了实现此功能，需要对维度分程序做如下修改：

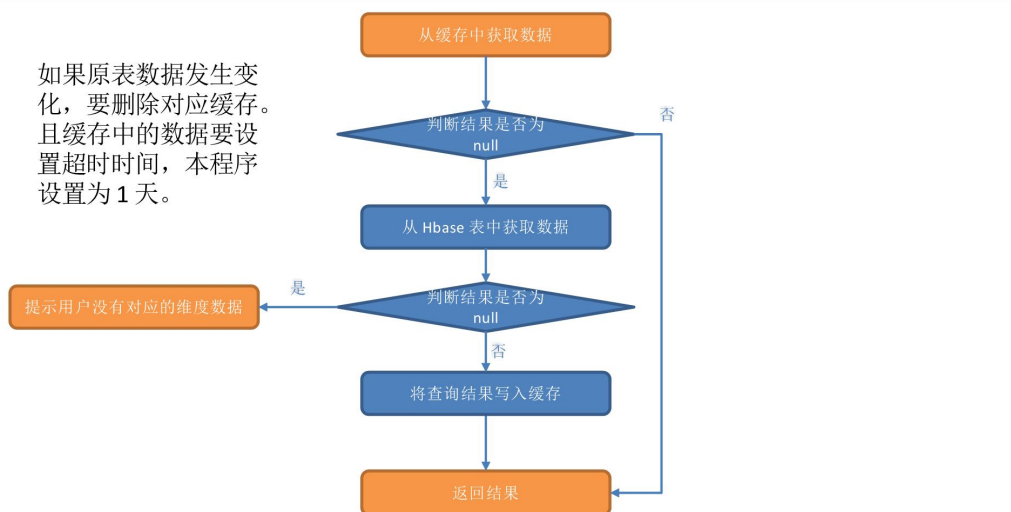
#### (2) 维度变更时

- 如果维度数据的变更类型为 insert，则对缓存无影响。
- 如果维度数据的变更类型为 update 或 delete，则清除缓存。

旁路缓存图解

旁路缓存思路

尚硅谷



让天下没有难学的技术

### 12.9.3.3 异步 IO

在 Flink 流处理过程中，经常需要和外部系统进行交互，如通过维度表补全事实表中的维度字段。

默认情况下，在 Flink 算子中，单个并行子任务只能以同步方式与外部系统交互：将请求发送到外部存储，IO 阻塞，等待请求返回，然后继续发送下一个请求。这种方式将大量时间耗费在了等待结果上。

为了提高处理效率，可以有两种思路。

- (1) 增加算子的并行度，但需要耗费更多的资源。
- (2) 异步 IO。

Flink 在 1.2 中引入了 Async I/O，将 IO 操作异步化。在异步模式下，单个并行子任务可以连续发送多个请求，按照返回的先后顺序对请求进行处理，发送请求后不需要阻塞式等待，省去了大量的等待时间，大幅提高了流处理效率。

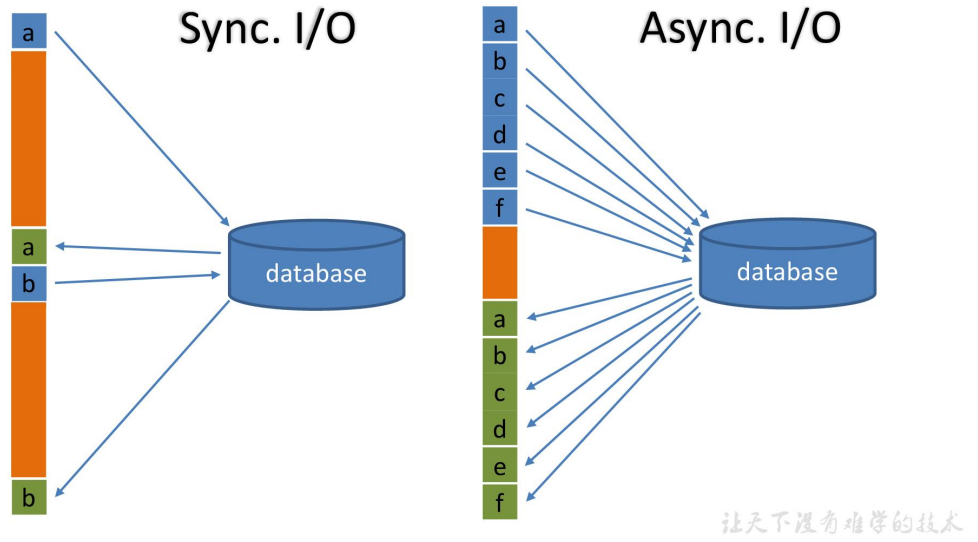
Async I/O 是阿里巴巴贡献给社区的特性，呼声很高，可用于解决与外部系统交互时网络延迟成为系统瓶颈的问题。

异步查询实际上是把维表的查询操作托管给单独的线程池完成，这样不会因为某一个查询造成阻塞，因此单个并行子任务可以连续发送多个请求，从而提高并发效率。对于涉及网络 IO 的操作，可以显著减少因为请求等待带来的性能损耗。

#### 12.9.3.4 异步 IO 图解

#### 12.9.3.5 HBaseUtil 查询方法思路

异步 IO 思路



根据指定参数：HBase 连接对象，命名空间，表名，rowKey 查找指定行的所有列，把整行数据封装到一个 JSONObject 对象中返回。模板方法设计模式

#### 12.9.3.6 执行步骤

- 1) 从 Kafka 订单明细主题读取数据
- 2) 转换数据结构
- 3) 按照唯一键去重
- 4) 转换数据结构

JSONObject 转换为实体类 TradeSkuOrderBean。

- 5) 按照 user\_id 分组，统计下单独立用户数。

运用 Flink 状态编程，记录每个用户的末次下单日期，末次下单日期为 null 或末次下单日期不等于当日则为当日下单独立用户

- 6) 设置水位线

- 7) 分组、开窗、聚合

按照维度信息分组，度量字段求和，并在窗口闭合后补充窗口起始时间和结束时间。将时间戳置为当前系统时间。

- 8) 维度关联，补充与分组无关的维度字段

(1) 关联 sku\_info 表

获取 sku\_name, tm\_id, category3\_id, spu\_id

(2) 关联 spu\_info 表

获取 spu\_name。

(3) 关联 base\_trademark 表

获取 tm\_name。

(4) 关联 base\_category3 表

获取 name (三级品类名称), 获取 category2\_id。

(5) 关联 base\_category2 表

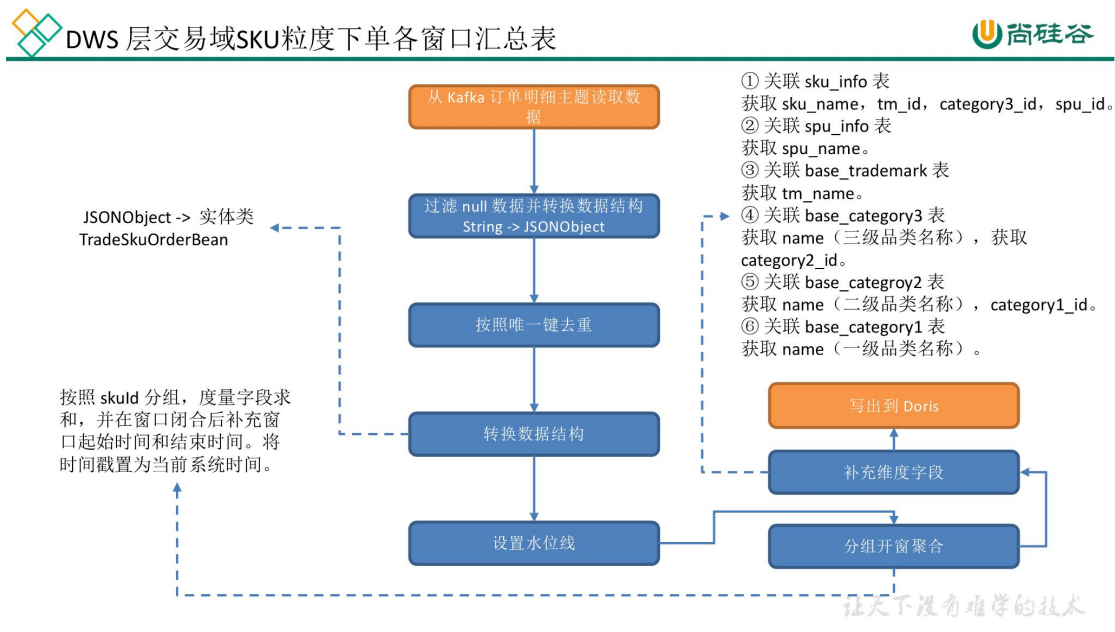
获取 name (二级品类名称), category1\_id。

(6) 关联 base\_category1 表

获取 name (一级品类名称)。

9) 写出到 Doris。

## 12.9.4 图解



## 12.9.5 Doris 建表语句

```
drop table if exists gmall2023_realtime.dws_trade_sku_order_window;
create table if not exists
gmall2023_realtime.dws_trade_sku_order_window
```

更多 [Java](#) - [大数据](#) - [前端](#) - [python](#) 人工智能资料下载，可百度访问：[尚硅谷官网](#)

```
(
    `stt`          DATETIME COMMENT '窗口起始时间',
    `edt`          DATETIME COMMENT '窗口结束时间',
    `cur_date`     DATE COMMENT '当天日期',
    `trademark_id` SMALLINT COMMENT '品牌 ID',
    `trademark_name` CHAR(255) COMMENT '品牌名称',
    `category1_id` SMALLINT COMMENT '一级品类 ID',
    `category1_name` CHAR(128) COMMENT '一级品类名称',
    `category2_id` SMALLINT COMMENT '二级品类 ID',
    `category2_name` CHAR(128) COMMENT '二级品类名称',
    `category3_id` SMALLINT COMMENT '三级品类 ID',
    `category3_name` CHAR(128) COMMENT '三级品类名称',
    `sku_id`       INT COMMENT 'SKU_ID',
    `sku_name`     CHAR(255) COMMENT 'SKU 名称',
    `spu_id`       INT COMMENT 'SPU_ID',
    `spu_name`     CHAR(255) COMMENT 'SPU 名称',
    `original_amount` DECIMAL(16, 2) REPLACE COMMENT '原始金额',
    `activity_reduce_amount` DECIMAL(16, 2) REPLACE COMMENT '活动减免金额',
    `coupon_reduce_amount` DECIMAL(16, 2) REPLACE COMMENT '优惠券减免金额',
    `order_amount` DECIMAL(16, 2) REPLACE COMMENT '下单金额'
) ENGINE = olap AGGREGATE KEY
(`stt`, `edt`, `cur_date`, `trademark_id`, `trademark_name`, `category1_id`, `category1_name`, `category2_id`, `category2_name`, `category3_id`, `category3_name`, `sku_id`, `sku_name`, `spu_id`, `spu_name`)
PARTITION BY RANGE(`cur_date`) ()
DISTRIBUTED BY HASH(`stt`) BUCKETS 10
PROPERTIES (
    "replication_num" = "3",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "par",
    "dynamic_partition.buckets" = "10"
);
```

## 12.9.6 代码实现

### 12.9.6.1 同步查询+旁路缓存模式

#### 1) 创建模板方法设计模式模板接口 DimFunction

在 common 模块的 function 包下创建接口 DimFunction，写入以下内容。



```
package com.atguigu.gmall.realtime.common.function;

import com.alibaba.fastjson.JSONObject;

public interface DimFunction<T> {
    String getRowKey(T bean);
    String getTableName();
    void addDims(T bean, JSONObject dim);
}
```

## 2) 实体类 TradeSkuOrderBean

在 common 模块的 bean 目录下创建 TradeSkuOrderBean

```
package com.atguigu.gmall.realtime.common.bean;

import com.alibaba.fastjson.annotation.JSONField;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.math.BigDecimal;

@Data
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class TradeSkuOrderBean {
    @JSONField(serialize = false)
    String orderDetailId;
    // 窗口起始时间
    String stt;
    // 窗口结束时间
    String edt;
    // 当天日期
    String curDate;
    // 品牌 ID
    String trademarkId;
    // 品牌名称
    String trademarkName;
    // 一级品类 ID
    String category1Id;
    // 一级品类名称
    String category1Name;
```



```
// 二级品类 ID
String category2Id;
// 二级品类名称
String category2Name;
// 三级品类 ID
String category3Id;
// 三级品类名称
String category3Name;
// sku_id
String skuId;
// sku 名称
String skuName;
// spu_id
String spuId;
// spu 名称
String spuName;
// 原始金额
BigDecimal originalAmount;
// 活动减免金额
BigDecimal activityReduceAmount;
// 优惠券减免金额
BigDecimal couponReduceAmount;
// 下单金额
BigDecimal orderAmount;
// 时间戳
@JSONField(serialize = false)
Long ts;
}
```

### 3) 修改 HBaseUtil, 补充 getRow 方法

```
/**
 * 根据参数从 hbase 指定的表中查询一行数据
 *
 * @param hbaseConn hbase 链接
 * @param nameSpace 命名空间
 * @param table 表名
 * @param rowKey rowKey
 * @return 把一行查询到的所有列封装到一个 JSONObject 对象中
 */
public static <T> T getRow(Connection hbaseConn,
                           String nameSpace,
                           String table,
                           String rowKey,
                           Class<T> tClass,
```

```
        boolean... isUnderlineToCamel) {
    boolean defaultIsUToC = false; // 默认不执行下划线转驼峰

    if (isUnderlineToCamel.length > 0) {
        defaultIsUToC = isUnderlineToCamel[0];
    }

    try (Table table =
hbaseConn.getTable(tableName.valueOf(nameSpace, table))) { //
jdk1.7 : 可以自动释放资源
        Get get = new Get(Bytes.toBytes(rowKey));
        Result result = table.get(get);
        // 4. 把查询到的一行数据,封装到一个对象中: JSONObject
        // 4.1 一行中所有的列全部解析出来
        List<Cell> cells = result.listCells(); // 一个 Cell 表示这行中
的一列
        T t = tClass.newInstance();
        for (Cell cell : cells) {
            // 取出每列的列名(json 对象的中的 key)和列值(json 对象中的
value)
            String key =
Bytes.toString(CellUtil.cloneQualifier(cell));
            if (defaultIsUToC) { // 需要下划线转驼峰: a_a => aA a_aaaa_aa
=> aAaaaaAa
                key =
CaseFormat.LOWER_UNDERSCORE.to(CaseFormat.LOWER_CAMEL, key);
            }
            String value = Bytes.toString(CellUtil.cloneValue(cell));

            BeanUtils.setProperty(t, key, value);
        }
        return t;
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

#### 4) 创建 RedisUtil

##### (1) 所需依赖

此处需要引入 Jedis 相关依赖, 如下。

```
<dependency>
<groupId>redis.clients</groupId>
<artifactId>jedis</artifactId>
```

```
</dependency>
```

已在 common 模块引入。

## (2) 创建 RedisUtil

在 common 模块的 util 包下创建 RedisUtil。

此处要注意：Redis 默认支持 16 个数据库，编号为 0-15，客户端与 Redis 建立连接后默认选择 0 号数据库，生产环境下 0 号库可能压力较大，使用人数较多，为了避免 key 发生冲突，此处选择任意非 0 号数据库。生产环境下数据库的选择视具体情况而定。

写入以下内容

```
package com.atguigu.gmall.realtime.common.util;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.constant.Constant;
import org.apache.commons.pool2.impl.GenericObjectPoolConfig;
import redis.clients.jedis.Jedis;
import redis.clients.jedis.JedisPool;
import redis.clients.jedis.JedisPoolConfig;

public class RedisUtil {

    private final static JedisPool pool;

    static {
        GenericObjectPoolConfig config = new JedisPoolConfig();
        config.setMaxTotal(300);
        config.setMaxIdle(10);
        config.setMinIdle(2);

        config.setTestOnCreate(true);
        config.setTestOnBorrow(true);
        config.setTestOnReturn(true);

        config.setMaxWaitMillis(10 * 1000);

        pool = new JedisPool(config, "hadoop102", 6379);
    }

    public static Jedis getJedis() {
        // Jedis jedis = new Jedis("hadoop102", 6379);
    }
}
```

```
Jedis jedis = pool.getResource();
jedis.select(4); // 直接选择 4 号库

return jedis;
}

/**
 * 从 redis 读取维度数据
 *
 * @param jedis    jedis 对象
 * @param tableName 表名
 * @param id       维度的 id 值
 * @return 这条维度组成的 JSONObject 对象
 */
public static JSONObject readDim(Jedis jedis, String tableName,
String id) {
    String key = getKey(tableName, id);
    String jsonStr = jedis.get(key);
    if (jsonStr != null) {
        return JSON.parseObject(jsonStr);
    }
    return null;
}

public static void writeDim(Jedis jedis, String tableName, String
id, JSONObject dim) {
    /*jedis.set(getKey(tableName, id), dim.toJSONString()); // 写
入字符串
    jedis.expire(getKey(tableName, id), 2 * 24 * 60 * 60); // 设
置过期时间*/

    jedis.setex(getKey(tableName, id), Constant.TWO_DAY_SECONDS,
dim.toJSONString());
}

public static String getKey(String tableName, String id) {
    return tableName + ":" + id;
}

public static void closeJedis(Jedis jedis) {
    if (jedis != null) {
        jedis.close(); // 如果 jedis 客户端是 new Jedis() 得到的, 则是
```

关闭客户端.如果是通过连接池得到的,则归还

```
    }  
    }  
}
```

### 5) 修改 HBaseSinkFunction

修改 realtime-dim 模块下 function 包下的 HBaseSinkFunction, 补充维度变更时缓存的清除操作。

(1) 定义成员变量 jedis

```
private Jedis jedis;
```

(2) 在 open 方法中补充 Jedis 连接的获取操作。

```
@Override  
public void open(Configuration parameters) throws Exception {  
    conn = HBaseUtil.getHBaseConnection();  
    jedis = RedisUtil.getJedis();  
}
```

(3) 在 close 方法中补充 Jedis 连接的释放操作

```
@Override  
public void close() throws Exception {  
    HBaseUtil.closeHBaseConn(conn);  
    RedisUtil.closeJedis(jedis);  
}
```

(4) 在 invoke 方法中补充缓存清除操作

```
TableProcessDim tableProcessDim = dataWithConfig.f1;  
// 如果是维度是 update 或 delete 则删除缓存中的维度数据  
if ("delete".equals(opType) || "update".equals(opType)) {  
    String key = RedisUtil.getKey(  
        tableProcessDim.getSinkTable(),  
        data.getString(tableProcessDim.getSinkRowKey()));  
    jedis.del(key);  
}
```

### 6) 定义同步关联函数 MapDimFunction

在 realtime-dws-trade-sku-order 模块 function 包下创建 MapDimFunction

```
package com.atguigu.gmall.realtime.dws.function;  
  
import com.alibaba.fastjson.JSONObject;  
import com.atguigu.gmall.realtime.common.function.DimFunction;  
import com.atguigu.gmall.realtime.common.util.HBaseUtil;  
import com.atguigu.gmall.realtime.common.util.RedisUtil;  
import lombok.extern.slf4j.Slf4j;
```

```
import org.apache.flink.api.common.functions.RichMapFunction;
import org.apache.flink.configuration.Configuration;
import org.apache.hadoop.hbase.client.Connection;
import redis.clients.jedis.Jedis;

@Slf4j
public abstract class MapDimFunction<T> extends RichMapFunction<T,
T> implements DimFunction<T> {

    private Connection hbaseConn;
    private Jedis jedis;

    @Override
    public void open(Configuration parameters) throws Exception {
        hbaseConn = HBaseUtil.getHBaseConnection();

        jedis = RedisUtil.getJedis();
    }

    @Override
    public void close() throws Exception {
        HBaseUtil.closeHBaseConn(hbaseConn);
    }

    @Override
    public T map(T bean) throws Exception {
        // 用 bean 对象中获取一个 rowKey, 由于 bean 是泛型, 所以在父类中无法直接获取
        // 等到子类 new 对象的时候, 会确定 T 的类型, 需要让子类去提供这个 rowKey

        // 1. 先去 redis 读取维度
        JSONObject dim = RedisUtil.readDim(jedis, getTableName(),
        getRowKey(bean));

        if (dim == null) {
            // 3. 如果没有读到, 则去 hbase 读取,
            dim = HBaseUtil.getRow(hbaseConn,
                "gmall",
                getTableName(),
                getRowKey(bean),
                JSONObject.class);

            // 4. 并把读到的维度缓存到 redis 中
            RedisUtil.writeDim(jedis, getTableName(), getRowKey(bean),
```

```
dim);  
        log.info("走 hbase: " + getTableName() + " " +  
getRowKey(bean));  
  
    }else{  
        // 2. 如果读到, 执行下面的操作  
        log.info("走 redis: " + getTableName() + " " +  
getRowKey(bean));  
    }  
  
    // 补充维度  
    addDims(bean, dim);  
    return bean;  
}  
}
```

## 7) 同步关联测试程序

在 realtime-dws-trade-sku-order 模块 app 包下创建 DwsTradeSkuOrderWindowSyncCache

```
package com.atguigu.gmall.realtime.dws.app;  
  
import com.alibaba.fastjson.JSON;  
import com.alibaba.fastjson.JSONObject;  
import com.atguigu.gmall.realtime.common.base.BaseApp;  
import com.atguigu.gmall.realtime.common.bean.TradeSkuOrderBean;  
import com.atguigu.gmall.realtime.common.constant.Constant;  
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;  
import com.atguigu.gmall.realtime.dws.function.MapDimFunction;  
import org.apache.flink.api.common.eventtime.WatermarkStrategy;  
import org.apache.flink.api.common.functions.MapFunction;  
import org.apache.flink.api.common.functions.ReduceFunction;  
import org.apache.flink.api.common.state.StateTtlConfig;  
import org.apache.flink.api.common.state.ValueState;  
import org.apache.flink.api.common.state.ValueStateDescriptor;  
import org.apache.flink.api.common.time.Time;  
import org.apache.flink.configuration.Configuration;  
import org.apache.flink.streaming.api.datastream.DataStreamSource;  
import  
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera  
tor;  
import  
org.apache.flink.streaming.api.environment.StreamExecutionEnviron  
ment;  
import  
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
```

```
import
org.apache.flink.streaming.api.functions.windowing.ProcessWindowF
unction;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventT
imeWindows;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.math.BigDecimal;
import java.time.Duration;

public class DwsTradeSkuOrderWindowSyncCache extends BaseApp {
    public static void main(String[] args) {
        new DwsTradeSkuOrderWindowSyncCache().start(
            10029,
            4,
            "dws_trade_sku_order_window",
            Constant.TOPIC_DWD_TRADE_ORDER_DETAIL
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
DataStreamSource<String> stream) {
        // 1. 解析成 pojo 类型
        SingleOutputStreamOperator<TradeSkuOrderBean> beanStream =
parseToPojo(stream);

        // 2. 按照 order_detail_id 去重
        beanStream = distinctByOrderId(beanStream);

        // 3. 按照 sku_id 分组, 开窗, 聚合
        SingleOutputStreamOperator<TradeSkuOrderBean>
beanStreamWithoutDims = windowAndAgg(beanStream);

        // 4. join 维度
        joinDim(beanStreamWithoutDims);
    }

    private void
```



```
joinDim(SingleOutputStreamOperator<TradeSkuOrderBean> stream) {
    /*
        每来一条数据，都需要去维度层(hbase)中查找对应的维度，补充到 bean
        中

        6 张维度表：
        dim_sku_info sku_id
            sku_name spu_id tm_id c3_id
        dim_spu_info spu_id
            spu_name
        dim_base_trademark tm_id
            tm_name
        dim_base_category3 c3_id
            c3_name c2_id
        dim_base_category2 c2_id
            c2_name c1_id
        dim_base_category3 c1_id
            c1_name

    */
    // 补充 sku 信息
    SingleOutputStreamOperator<TradeSkuOrderBean> skuStream =
stream
        .map(new MapDimFunction<TradeSkuOrderBean>() {
            @Override
            public String getTableName() {
                return "dim_sku_info";
            }

            @Override
            public String getRowKey(TradeSkuOrderBean bean) {
                return bean.getSkuId();
            }

            @Override
            public void addDims(TradeSkuOrderBean bean,
                                JSONObject dim) {
                bean.setSkuName(dim.getString("sku_name"));
                bean.setSpuId(dim.getString("spu_id"));
                bean.setTrademarkId(dim.getString("tm_id"));
                bean.setCategory3Id(dim.getString("category3_id"));
            }
        })
```

```
        });  
        SingleOutputStreamOperator<TradeSkuOrderBean> spuStream =  
skuStream.map(new MapDimFunction<TradeSkuOrderBean>() {  
            @Override  
            public String getRowKey(TradeSkuOrderBean bean) {  
                return bean.getSpuId();  
            }  
  
            @Override  
            public String getTableName() {  
                return "dim_spu_info";  
            }  
  
            @Override  
            public void addDims(TradeSkuOrderBean bean, JSONObject dim)  
{  
                bean.setSpuName(dim.getString("spu_name"));  
            }  
        });  
  
        SingleOutputStreamOperator<TradeSkuOrderBean> tmStream =  
spuStream  
        .map(new MapDimFunction<TradeSkuOrderBean>() {  
            @Override  
            public String getRowKey(TradeSkuOrderBean bean) {  
                return bean.getTrademarkId();  
            }  
  
            @Override  
            public String getTableName() {  
                return "dim_base_trademark";  
            }  
  
            @Override  
            public void addDims(TradeSkuOrderBean bean,  
                                JSONObject dim) {  
                bean.setTrademarkName(dim.getString("tm_name"));  
            }  
        });  
  
        SingleOutputStreamOperator<TradeSkuOrderBean> c3Stream =  
tmStream.map(new MapDimFunction<TradeSkuOrderBean>() {
```

```
@Override
public String getRowKey(TradeSkuOrderBean bean) {
    return bean.getCategory3Id();
}

@Override
public String getTableName() {
    return "dim_base_category3";
}

@Override
public void addDims(TradeSkuOrderBean bean, JSONObject dim)
{
    bean.setCategory3Name(dim.getString("name"));
    bean.setCategory2Id(dim.getString("category2_id"));
}
});

SingleOutputStreamOperator<TradeSkuOrderBean> c2Stream =
c3Stream.map(new MapDimFunction<TradeSkuOrderBean>() {
    @Override
    public String getRowKey(TradeSkuOrderBean bean) {
        return bean.getCategory2Id();
    }

    @Override
    public String getTableName() {
        return "dim_base_category2";
    }

    @Override
    public void addDims(TradeSkuOrderBean bean, JSONObject dim)
    {
        bean.setCategory2Name(dim.getString("name"));
        bean.setCategory1Id(dim.getString("category1_id"));
    }
});

SingleOutputStreamOperator<TradeSkuOrderBean> c1Stream =
c2Stream.map(new MapDimFunction<TradeSkuOrderBean>() {
    @Override
    public String getRowKey(TradeSkuOrderBean bean) {
        return bean.getCategory1Id();
    }
});
```

```
    }

    @Override
    public String getTableName() {
        return "dim_base_category1";
    }

    @Override
    public void addDims(TradeSkuOrderBean bean, JSONObject dim)
{
        bean.setCategory1Name(dim.getString("name"));
    }
    });

    c1Stream.print();

}

private SingleOutputStreamOperator<TradeSkuOrderBean>
windowAndAgg(
    SingleOutputStreamOperator<TradeSkuOrderBean> beanStream) {
    return beanStream
        .assignTimestampsAndWatermarks(
            WatermarkStrategy
                .<TradeSkuOrderBean>forBoundedOutOfOrderness(Duration.ofSeconds(3))
                .withTimestampAssigner((bean, ts) -> bean.getTs())
                .withIdleness(Duration.ofSeconds(120))
        )
        .keyBy(TradeSkuOrderBean::getSkuId)
        .window(TumblingEventTimeWindows.of(org.apache.flink.streaming.api.windowing.time.Time.seconds(5)))
        .reduce(
            new ReduceFunction<TradeSkuOrderBean>() {
                @Override
                public TradeSkuOrderBean reduce(TradeSkuOrderBean
value1,
                                                    TradeSkuOrderBean
value2) throws Exception {

                    value1.setOriginalAmount(value1.getOriginalAmount().add(value2.getOriginalAmount()));

                    value1.setOrderAmount(value1.getOrderAmount().add(value2.getOrder
```

```
Amount())));

value1.setActivityReduceAmount(value1.getActivityReduceAmount().add(value2.getActivityReduceAmount()));

value1.setCouponReduceAmount(value1.getCouponReduceAmount().add(value2.getCouponReduceAmount()));

        return value1;
    }

    },
    new ProcessWindowFunction<TradeSkuOrderBean,
TradeSkuOrderBean, String, TimeWindow>() {
        @Override
        public void process(String skuId,
                                Context ctx,
                                Iterable<TradeSkuOrderBean>
elements,
                                Collector<TradeSkuOrderBean> out)
throws Exception {
            TradeSkuOrderBean bean =
elements.iterator().next();

            bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()));
            ;

            bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));

            bean.setCurDate(DateFormatUtil.tsToDateForPartition(ctx.window().getEnd()));

            out.collect(bean);
        }
    }
);

}

private SingleOutputStreamOperator<TradeSkuOrderBean>
distinctById(
    SingleOutputStreamOperator<TradeSkuOrderBean> beanStream) {
    /*
```

去重逻辑：按照 order\_detail\_id 分组

思路：1

使用 session 窗口，当窗口关闭的时候，同一个详情的多条数据，一定都来齐了，找到最完整的那个。

需要 dwd 层的数据添加一个字段，表示这条数据的生成时间，时间大的那个就是最完整的

| 详情 id | sku_id | 金额  | 活动   | 优惠券  | 系统时间 |
|-------|--------|-----|------|------|------|
| 1     | 100    | 100 | null | null | 1    |
| null  |        |     |      |      |      |
| 1     | 100    | 100 | 有值   | null | 2    |
| 1     | 100    | 100 | 有值   | 有值   | 3    |

优点：简单

缺点：实效性低

窗口内的最后一条数据到了之后，经过一个 gap (5s) 才能计算出结果

思路：2

定时器

当第一条数据到的时候，注册一个 5s 后触发的定时器，把这条存入到状态中

每来一条数据，就和状态中的比较时间，时间大的保存到状态中。当定时器触发的时候，则状态中存储的一定是时间最大的那个

优点：简单

确定：实效性低

窗口内的第一条数据到了之后，5s 才能计算出结果

思路：3 抵消

|      | 详情 id | sku_id | 左表金额 | 右表 1 活动 | 右表 2 优惠券 |
|------|-------|--------|------|---------|----------|
| 第一条  | 1     | 100    | 100  | null    | null     |
| 直接输出 |       |        |      |         |          |
|      | 1     | 100    | -100 | null    | null     |
| 直接输出 |       |        |      |         |          |
| 第二条  | 1     | 100    | 100  | 200     | null     |
| 直接输出 |       |        |      |         |          |
|      | 1     | 100    | -100 | -200    | null     |
| 直接输出 |       |        |      |         |          |
| 第三条  | 1     | 100    | 100  | 200     | 300      |
| 直接输出 |       |        |      |         |          |

优点：实效性高

缺点：写放大

|      |     |       |        |            |            |      |
|------|-----|-------|--------|------------|------------|------|
|      | 优化: | 详情 id | sku_id | 左表金额       | 右表 1 活动    | 右表 2 |
| 优惠券  | 第一条 | 1     | 100    | 100        | null       | null |
| 直接输出 | 第二条 | 1     | 100    | 100+(-100) | 200        | null |
| 直接输出 | 第三条 | 1     | 100    | 100+(-100) | 200+(-200) | 300  |
| 直接输出 |     |       |        |            |            |      |

优点: 实效性高

思路 4:

如果需要聚合的值都在左表, 不需要等最后最完整的数据.

只需要输出第一条就行了.

优点: 实效性最高

缺点: 特殊情况. 聚和的值都在左表

```

*/
return beanStream
    .keyBy(TradeSkuOrderBean::getOrderDetailId)
    .process(new KeyedProcessFunction<String,
TradeSkuOrderBean, TradeSkuOrderBean>() {

        private ValueState<TradeSkuOrderBean> lastBeanState;

        @Override
        public void open(Configuration parameters) throws
Exception {
            ValueStateDescriptor<TradeSkuOrderBean> des = new
ValueStateDescriptor<>("lastBean", TradeSkuOrderBean.class);
            StateTtlConfig conf = new
StateTtlConfig.Builder(Time.seconds(60))
                .useProcessingTime()
                .updateTtlOnCreateAndWrite()
                .neverReturnExpired()
                .build();
            des.enableTimeToLive(conf);
            lastBeanState = getRuntimeContext().getState(des);
        }

        @Override

```

```
        public void processElement(TradeSkuOrderBean
currentBean,

                                Context ctx,
                                Collector<TradeSkuOrderBean>
out) throws Exception {
            TradeSkuOrderBean lastBean =
lastBeanState.value();
            if (lastBean == null) { // 第1条数据来了, 则直接输出
                out.collect(currentBean);
            } else {
                // 把上条数据的值取反, 然后和当前的 bean 中的值相加,
输出计算后的值

lastBean.setOrderAmount(currentBean.getOrderAmount().subtract(las
tBean.getOrderAmount()));

lastBean.setOriginalAmount(currentBean.getOriginalAmount().subtra
ct(lastBean.getOriginalAmount()));

lastBean.setActivityReduceAmount(currentBean.getActivityReduceAmo
unt().subtract(lastBean.getActivityReduceAmount()));

lastBean.setCouponReduceAmount(currentBean.getCouponReduceAmount()
.subtract(lastBean.getCouponReduceAmount()));
                out.collect(lastBean);
            }

            // 把当前的 bean 更新到状态中
            lastBeanState.update(currentBean);
        }
    });
}

private SingleOutputStreamOperator<TradeSkuOrderBean>
parseToPojo(DataStreamSource<String> stream) {
    return stream
        .map(new MapFunction<String, TradeSkuOrderBean>() {
            @Override
            public TradeSkuOrderBean map(String value) throws
Exception {
                JSONObject obj = JSON.parseObject(value);
```



```
        return TradeSkuOrderBean.builder()
            .skuId(obj.getString("sku_id"))
            .orderDetailId(obj.getString("id"))
            .originalAmount(obj.getBigDecimal("split_original_amount"))
            .orderAmount(obj.getBigDecimal("split_total_amount"))
            .activityReduceAmount(obj.getBigDecimal("split_activity_amount") == null ? new BigDecimal("0.0") :
obj.getBigDecimal("split_activity_amount"))
            .couponReduceAmount(obj.getBigDecimal("split_coupon_amount") == null ? new BigDecimal("0.0") :
obj.getBigDecimal("split_coupon_amount"))
            .ts(obj.getLong("ts") * 1000)
            .build();
    }

    });
}
```

### 12.9.6.2 异步 IO+旁路缓存

#### 1) 修改 RedisUtil

##### (1) 所需依赖

Jedis 不支持异步读取 Redis，所以我们使用支持异步查询的 Redis 客户端：lettuce。此处需要引入 lettuce 相关依赖，如下。

```
<dependency>
    <groupId>io.lettuce</groupId>
    <artifactId>lettuce-core</artifactId>
</dependency>
```

已在 common 模块引入。

##### (2) 在 RedisUtil 中补充异步查询相关方法

```
/**
 * 获取到 redis 的异步连接
 *
 * @return 异步链接对象
 */
public static StatefulRedisConnection<String, String>
getRedisAsyncConnection() {
    RedisClient redisClient =
RedisClient.create("redis://hadoop102:6379/2");
```

```
        return redisClient.connect();
    }

    /**
     * 关闭 redis 的异步连接
     *
     * @param redisAsyncConn
     */
    public static void
    closeRedisAsyncConnection(StatefulRedisConnection<String, String>
    redisAsyncConn) {
        if (redisAsyncConn != null) {
            redisAsyncConn.close();
        }
    }

    /**
     * 异步的方式从 redis 读取维度数据
     * @param redisAsyncConn 异步连接
     * @param tableName 表名
     * @param id id 的值
     * @return 读取到维度数据,封装的 json 对象中
     */
    public static JSONObject
    readDimAsync(StatefulRedisConnection<String, String>
    redisAsyncConn,

                String tableName,
                String id) {
        RedisAsyncCommands<String, String> asyncCommand =
    redisAsyncConn.async();
        String key = getKey(tableName, id);
        try {
            String json = asyncCommand.get(key).get();
            if (json != null) {
                return JSON.parseObject(json);
            }
        } catch (Exception e) {
            throw new RuntimeException(e);
        }

        return null;
    }
}
```

```
/**
 * 把维度异步的写入到 redis 中
 * @param redisAsyncConn 到 redis 的异步连接
 * @param tableName 表名
 * @param id id 的值
 * @param dim 要写入的维度数据
 */
public static void writeDimAsync(StatefulRedisConnection<String,
String> redisAsyncConn,
                                String tableName,
                                String id,
                                JSONObject dim) {
    // 1. 得到异步命令
    RedisAsyncCommands<String, String> asyncCommand =
redisAsyncConn.async();

    String key = getKey(tableName, id);
    // 2. 写入到 string 中: 顺便还设置的 ttl
    asyncCommand.setex(key, Constant.TWO_DAY_SECONDS,
dim.toJSONString());
}
```

## 2) 修改 HBaseUtil

在 HBaseUtil 中补充异步查询相关方法

```
/**
 * 获取到 Hbase 的异步连接
 *
 * @return 得到异步连接对象
 */
public static AsyncConnection getHBaseAsyncConnection() {
    Configuration conf = new Configuration();
    conf.set("hbase.zookeeper.quorum", "hadoop102");
    conf.set("hbase.zookeeper.property.clientPort", "2181");
    try {
        return ConnectionFactory.createAsyncConnection(conf).get();
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}

/**
 * 关闭 hbase 异步连接
 */
```

```
* @param asyncConn 异步连接
*/
public static void closeAsyncHbaseConnection(AsyncConnection
asyncConn) {
    if (asyncConn != null) {
        try {
            asyncConn.close();
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}

/**
 * 异步的从 hbase 读取维度数据
 *
 * @param hBaseAsyncConn hbase 的异步连接
 * @param nameSpace      命名空间
 * @param tableName      表名
 * @param rowKey          rowKey
 * @return 读取到的维度数据，封装到 json 对象中。
 */
public static JSONObject readDimAsync(AsyncConnection
hBaseAsyncConn,
                                     String nameSpace,
                                     String tableName,
                                     String rowKey) {
    AsyncTable<AdvancedScanResultConsumer> asyncTable =
hBaseAsyncConn
        .getTable(TableName.valueOf(nameSpace, tableName));

    Get get = new Get(Bytes.toBytes(rowKey));
    try {
        // 获取 result
        Result result = asyncTable.get(get).get();
        List<Cell> cells = result.listCells(); // 一个 Cell 表示这行中
的一列
        JSONObject dim = new JSONObject();
        for (Cell cell : cells) {
            // 取出每列的列名(json 对象的中的 key)和列值(json 对象中的
value)
            String key =
Bytes.toString(CellUtil.cloneQualifier(cell));
            String value = Bytes.toString(CellUtil.cloneValue(cell));
```

```
        dim.put(key, value);
    }

    return dim;

} catch (Exception e) {
    e.printStackTrace();
    throw new RuntimeException(e);
}
}
```

### 3) 创建 AsyncDimFunction 函数

在 common 模块的 function 包下创建函数 AsyncDimFunction，写入以下内容。

```
package com.atguigu.gmall.realtime.common.function;

import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.util.HBaseUtil;
import com.atguigu.gmall.realtime.common.util.RedisUtil;
import io.lettuce.core.api.StatefulRedisConnection;
import lombok.extern.slf4j.Slf4j;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.functions.async.ResultFuture;
import org.apache.flink.streaming.api.functions.async.RichAsyncFunction;
import org.apache.hadoop.hbase.client.AsyncConnection;

import java.util.Collections;
import java.util.concurrent.CompletableFuture;
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Supplier;

@Slf4j
public abstract class AsyncDimFunction<T> extends
    RichAsyncFunction<T, T> implements DimFunction<T>{

    private StatefulRedisConnection<String, String> redisAsyncConn;
    private AsyncConnection hBaseAsyncConn;

    @Override
    public void open(Configuration parameters) throws Exception {
```

```
        redisAsyncConn = RedisUtil.getRedisAsyncConnection();
        hBaseAsyncConn = HBaseUtil.getHBaseAsyncConnection();

    }

    @Override
    public void close() throws Exception {
        RedisUtil.closeRedisAsyncConnection(redisAsyncConn);
        HBaseUtil.closeAsyncHbaseConnection(hBaseAsyncConn);
    }

    // 异步调用, 流中每来一个元素, 这个方法执行一次
    // 参数 1: 需要异步处理的元素
    // 参数 2: 元素被异步处理完之后, 放入到 ResultFuture 中, 则会输出到后面的流中
    @Override
    public void asyncInvoke(T bean,
                           ResultFuture<T> resultFuture) throws
Exception {

        // jdbc8 提供的专门用执行异步代码的框架
        // 1. 从 redis 去维度 第一个异步操作
        // 2. 从 hbase 去读维度 第二个异步操作
        // 3. 把维度补充到 bean 中

        CompletableFuture
            .supplyAsync(new Supplier<JSONObject>() {
                @Override
                public JSONObject get() {
                    // 1. 从 redis 去维度 . 把读取到的维度返回
                    return RedisUtil.readDimAsync(redisAsyncConn,
getTableName(), getRowKey(bean));
                }
            })
            .thenApplyAsync(new Function<JSONObject, JSONObject>() {
                @Override
                public JSONObject apply(JSONObject dimFromRedis) {
                    //2. 当 redis 没有读到维度的时候, 从 hbase 读取维度数据
                    JSONObject dim = dimFromRedis;
                    if (dim == null) { // redis 中没有读到
                        dim = HBaseUtil.readDimAsync(hBaseAsyncConn,
"gmall", getTableName(), getRowKey(bean));
                    }
                    // 把维度写入到 Redis 中
                    RedisUtil.writeDimAsync(redisAsyncConn,
```

```
getTableName(), getRowKey(bean), dim);
        log.info("走的是 hbase " + getTableName() + " "
+ getRowKey(bean));

        }else{
            log.info("走的是 redis " + getTableName() + " "
+ getRowKey(bean));
        }

        return dim;
    }
})
.thenAccept(new Consumer<JSONObject>() {
    @Override
    public void accept(JSONObject dim) {
        // 3. 补充维度到 bean 中.
        addDims(bean, dim);
        // 4. 把结果输出

resultFuture.complete(Collections.singleton(bean));
    }
});
}
```

#### 4) 主程序

在 realtime-dws-trade-sku-order 模块 app 包下创建 DwsTradeSkuOrderWindow

```
package com.atguigu.gmall.realtime.dws.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.bean.TradeSkuOrderBean;
import com.atguigu.gmall.realtime.common.constant.Constant;
import
com.atguigu.gmall.realtime.common.function.AsyncDimFunction;
import
com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.MapFunction;
import org.apache.flink.api.common.functions.ReduceFunction;
import org.apache.flink.api.common.state.StateTtlConfig;
```

```
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.api.common.time.Time;
import org.apache.flink.configuration.Configuration;
import org.apache.flink.streaming.api.datastream.AsyncDataStream;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera
tor;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import
org.apache.flink.streaming.api.functions.windowing.ProcessWindowF
unction;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventT
imeWindows;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.math.BigDecimal;
import java.time.Duration;
import java.util.concurrent.TimeUnit;

public class DwsTradeSkuOrderWindow extends BaseApp {
    public static void main(String[] args) {
        new DwsTradeSkuOrderWindow().start(
            10029,
            4,
            "dws_trade_sku_order_window",
            Constant.TOPIC_DWD_TRADE_ORDER_DETAIL
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
DataStreamSource<String> stream) {
```



```
// 1. 解析成 pojo 类型
SingleOutputStreamOperator<TradeSkuOrderBean> beanStream =
parseToPojo(stream);

// 2. 按照 order_detail_id 去重
beanStream = distinctByOrderId(beanStream);

// 3. 按照 sku_id 分组, 开窗, 聚合
SingleOutputStreamOperator<TradeSkuOrderBean>
beanStreamWithoutDims = windowAndAgg(beanStream);

// 4. join 维度
SingleOutputStreamOperator<TradeSkuOrderBean>
beanStreamWithDims = joinDim(beanStreamWithoutDims);

// 5. 结果写入到 doris
writeToDoris(beanStreamWithDims);

}

private void
writeToDoris(SingleOutputStreamOperator<TradeSkuOrderBean> stream)
{

    stream.map(new DorisMapFunction<>())
//        .print();
        .sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA
TABASE + ".dws_trade_sku_order_window",
"dws_trade_sku_order_window"));

}

private SingleOutputStreamOperator<TradeSkuOrderBean>
joinDim(SingleOutputStreamOperator<TradeSkuOrderBean> stream) {
    SingleOutputStreamOperator<TradeSkuOrderBean> skuStream =
AsyncDataStream.unorderedWait(
        stream,
        new AsyncDimFunction<TradeSkuOrderBean>() {
            @Override
            public String getRowKey(TradeSkuOrderBean bean) {
                return bean.getSkuId();
            }
        }

        @Override
```

```
        public String getTableName() {
            return "dim_sku_info";
        }

        @Override
        public void addDims (TradeSkuOrderBean bean, JSONObject
dim) {
            bean.setSkuName(dim.getString("sku_name"));
            bean.setSpuId(dim.getString("spu_id"));
            bean.setTrademarkId(dim.getString("tm_id"));

            bean.setCategory3Id(dim.getString("category3_id"));
        },
        120,
        TimeUnit.SECONDS
    );

    SingleOutputStreamOperator<TradeSkuOrderBean> spuStream =
AsyncDataStream.unorderedWait(
    skuStream,
    new AsyncDimFunction<TradeSkuOrderBean>() {
        @Override
        public String getRowKey(TradeSkuOrderBean bean) {
            return bean.getSpuId();
        }

        @Override
        public String getTableName() {
            return "dim_spu_info";
        }

        @Override
        public void addDims (TradeSkuOrderBean bean, JSONObject
dim) {
            bean.setSpuName(dim.getString("spu_name"));
        },
        120,
        TimeUnit.SECONDS
    );

    SingleOutputStreamOperator<TradeSkuOrderBean> tmStream =
AsyncDataStream.unorderedWait(
```

```
        spuStream,
        new AsyncDimFunction<TradeSkuOrderBean>() {
            @Override
            public String getRowKey(TradeSkuOrderBean bean) {
                return bean.getTrademarkId();
            }

            @Override
            public String getTableName() {
                return "dim_base_trademark";
            }

            @Override
            public void addDims(TradeSkuOrderBean bean,
                                JSONObject dim) {
                bean.setTrademarkName(dim.getString("tm_name"));
            }
        },
        120,
        TimeUnit.SECONDS
    );

    SingleOutputStreamOperator<TradeSkuOrderBean> c3Stream =
    AsyncDataStream.unorderedWait(
        tmStream,
        new AsyncDimFunction<TradeSkuOrderBean>() {
            @Override
            public String getRowKey(TradeSkuOrderBean bean) {
                return bean.getCategory3Id();
            }

            @Override
            public String getTableName() {
                return "dim_base_category3";
            }

            @Override
            public void addDims(TradeSkuOrderBean bean, JSONObject
dim) {
                bean.setCategory3Name(dim.getString("name"));
                bean.setCategory2Id(dim.getString("category2_id"));
            }
        },
```

```
        120,
        TimeUnit.SECONDS
    );
    SingleOutputStreamOperator<TradeSkuOrderBean> c2Stream =
AsyncDataStream.unorderedWait(
    c3Stream,
    new AsyncDimFunction<TradeSkuOrderBean>() {
        @Override
        public String getRowKey(TradeSkuOrderBean bean) {
            return bean.getCategory2Id();
        }

        @Override
        public String getTableName() {
            return "dim_base_category2";
        }

        @Override
        public void addDims(TradeSkuOrderBean bean, JSONObject
dim) {
            bean.setCategory2Name(dim.getString("name"));
            bean.setCategory1Id(dim.getString("category1_id"));
        }
    },
    120,
    TimeUnit.SECONDS
);

return AsyncDataStream.unorderedWait(
    c2Stream,
    new AsyncDimFunction<TradeSkuOrderBean>() {
        @Override
        public String getRowKey(TradeSkuOrderBean bean) {
            return bean.getCategory1Id();
        }

        @Override
        public String getTableName() {
            return "dim_base_category1";
        }

        @Override
```

```
        public void addDims (TradeSkuOrderBean bean, JSONObject
dim) {
            bean.setCategory1Name (dim.getString ("name"));
        }
    },
    120,
    TimeUnit.SECONDS
);

}

private SingleOutputStreamOperator<TradeSkuOrderBean>
windowAndAgg (
    SingleOutputStreamOperator<TradeSkuOrderBean> beanStream) {
    return beanStream
        .assignTimestampsAndWatermarks (
            WatermarkStrategy
                .<TradeSkuOrderBean>forBoundedOutOfOrderness (Dura
tion.ofSeconds (5L))
                .withTimestampAssigner ((bean, ts) -> bean.getTs ())
                .withIdleness (Duration.ofSeconds (120L))
        )
        .keyBy (TradeSkuOrderBean::getSkuId)
        .window (TumblingEventTimeWindows.of (org.apache.flink.st
reaming.api.windowing.time.Time.seconds (5L)))
        .reduce (
            new ReduceFunction<TradeSkuOrderBean> () {
                @Override
                public TradeSkuOrderBean reduce (TradeSkuOrderBean
value1,
                                                    TradeSkuOrderBean
value2) throws Exception {
                    value1.setOriginalAmount (value1.getOriginalAmount ().add (value2.ge
tOriginalAmount ()));
                    value1.setOrderAmount (value1.getOrderAmount ().add (value2.getOrder
Amount ()));
                    value1.setActivityReduceAmount (value1.getActivityReduceAmount ().a
dd (value2.getActivityReduceAmount ()));
                    value1.setCouponReduceAmount (value1.getCouponReduceAmount ().add (v
alue2.getCouponReduceAmount ()));
                }
            }
        )
    }
}
```

```
        return value1;
    }

    },
    new ProcessWindowFunction<TradeSkuOrderBean,
TradeSkuOrderBean, String, TimeWindow>() {
        @Override
        public void process(String skuId,
                             Context ctx,
                             Iterable<TradeSkuOrderBean>
elements,
                             Collector<TradeSkuOrderBean> out)
throws Exception {
            TradeSkuOrderBean bean =
elements.iterator().next();

            bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()))
;

            bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));

            bean.setCurDate(DateFormatUtil.tsToDateForPartition(ctx.window().
getEnd()));

            out.collect(bean);
        }
    }
);

}
```

```
private SingleOutputStreamOperator<TradeSkuOrderBean>
distinctByOrderId(
    SingleOutputStreamOperator<TradeSkuOrderBean> beanStream) {
    /*
    去重逻辑：按照 order_detail_id 分组
    思路：1
        使用 session 窗口，当窗口关闭的时候，同一个详情的多条数据，一
        定都来齐了.找到最完整的那个。
        需要 dwd 层的数据添加一个字段，表示这条数据的生成时间，时间
        大的那个就是最完整的
```

| 详情 id | sku_id | 金额  | 活动   | 优惠券  | 系统时间 |
|-------|--------|-----|------|------|------|
| 1     | 100    | 100 | null | null | 1    |



第三条      1      100      100+(-100) 200+(-200)      300

直接输出

优点：实效性高

思路 4：

如果需要聚合的值都在左表，不需要等最后最完整的数据。

只需要输出第一条就行了。

优点：实效性最高

缺点：特殊情况。聚和的值都在左表

```
*/
return beanStream
    .keyBy(TradeSkuOrderBean::getOrderDetailId)
    .process(new KeyedProcessFunction<String,
TradeSkuOrderBean, TradeSkuOrderBean>() {

        private ValueState<TradeSkuOrderBean> lastBeanState;

        @Override
        public void open(Configuration parameters) throws
Exception {
            ValueStateDescriptor<TradeSkuOrderBean> des = new
ValueStateDescriptor<>("lastBean", TradeSkuOrderBean.class);
            StateTtlConfig conf = new
StateTtlConfig.Builder(Time.seconds(60L))
                .useProcessingTime()
                .updateTtlOnCreateAndWrite()
                .neverReturnExpired()
                .build();
            des.enableTimeToLive(conf);
            lastBeanState = getRuntimeContext().getState(des);
        }

        @Override
        public void processElement(TradeSkuOrderBean
currentBean,

                                Context ctx,
                                Collector<TradeSkuOrderBean>
out) throws Exception {
            TradeSkuOrderBean lastBean =
lastBeanState.value();
```



```
        if (lastBean == null) { // 第1条数据来了, 则直接输出
            out.collect(currentBean);
        } else {
            // 把上条数据的值取反, 然后和当前的 bean 中的值相加,
            输出计算后的值

            lastBean.setOrderAmount(currentBean.getOrderAmount().subtract(las
            tBean.getOrderAmount()));

            lastBean.setOriginalAmount(currentBean.getOriginalAmount().subtra
            ct(lastBean.getOriginalAmount()));

            lastBean.setActivityReduceAmount(currentBean.getActivityReduceAmo
            unt().subtract(lastBean.getActivityReduceAmount()));

            lastBean.setCouponReduceAmount(currentBean.getCouponReduceAmount()
            .subtract(lastBean.getCouponReduceAmount()));
            out.collect(lastBean);
        }

        // 把当前的 bean 更新到状态中
        lastBeanState.update(currentBean);

    }
});

}

private SingleOutputStreamOperator<TradeSkuOrderBean>
parseToPojo(DataStreamSource<String> stream) {
    return stream
        .map(new MapFunction<String, TradeSkuOrderBean>() {
            @Override
            public TradeSkuOrderBean map(String value) throws
            Exception {

                JSONObject obj = JSON.parseObject(value);

                return TradeSkuOrderBean.builder()
                    .skuId(obj.getString("sku_id"))
                    .orderDetailId(obj.getString("id"))
                    .originalAmount(obj.getBigDecimal("split_orig
                    inal_amount"))
                    .orderAmount(obj.getBigDecimal("split_total_a
                    mount"))
```

```
                .activityReduceAmount(obj.getBigDecimal("split_activity_amount") == null ? new BigDecimal("0.0") :  
obj.getBigDecimal("split_activity_amount"))  
                .couponReduceAmount(obj.getBigDecimal("split_coupon_amount") == null ? new BigDecimal("0.0") :  
obj.getBigDecimal("split_coupon_amount"))  
                .ts(obj.getLong("ts") * 1000)  
                .build();  
        }  
    }  
};
```

## 12.9.7 测试

### 1) 同步关联测试

- (1) 启动 zk、kafka、Maxwell、HDFS、HBase、Redis、Doris
- (2) 运行应用程序
  - DimApp
  - DwdTradeOrderDetail
  - DwsTradeSkuOrderWindowSyncCache
- (3) 运行数据生成器
- (4) 查看控制台输出

### 2) 异步关联测试

- (1) 启动 zk、kafka、Maxwell、HDFS、HBase、Redis、Doris
- (2) 运行应用程序 DimApp、DwdTradeOrderDetail、DwsTradeSkuOrderWindow
- (3) 运行数据生成器
- (4) 查看 Doris 目标表数据

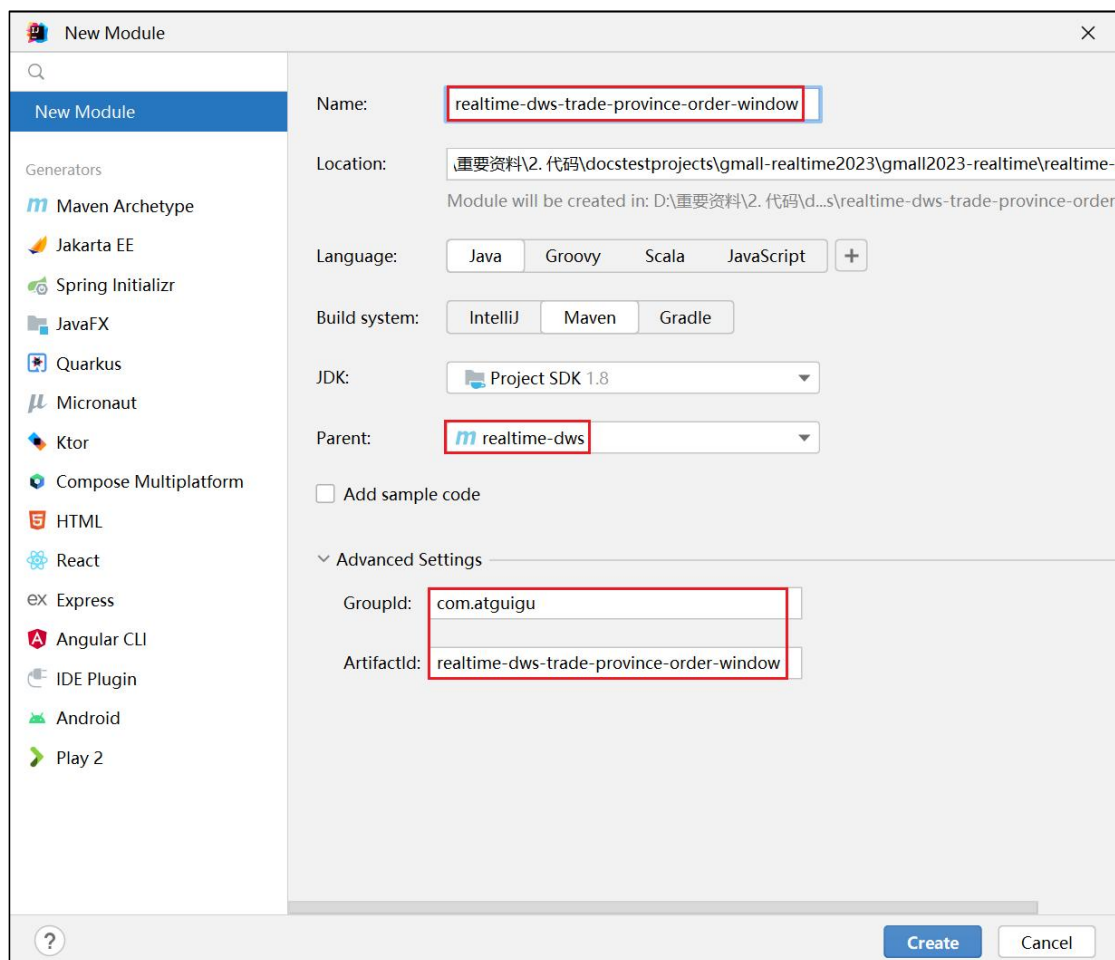
## 12.9.8 提交至 GitLab

略。

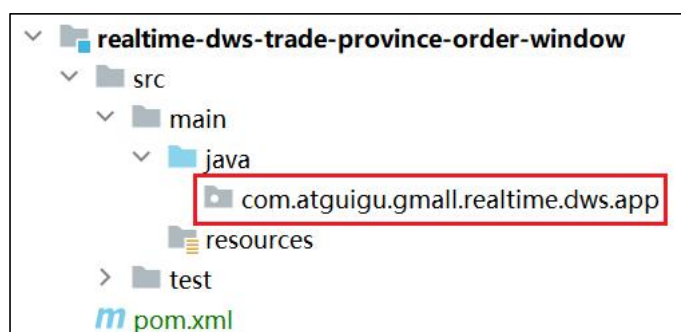
## 12.10 交易域省份粒度下单各窗口汇总表

### 12.10.1 创建 realtime-dws-trade-province-order-window

#### 1) 新建模块



## 2) 创建包结构



## 12.10.2 主要任务

从 Kafka 读取订单明细数据，过滤 null 数据并按照唯一键对数据去重，统计各省份各窗口订单数和订单金额，将数据写入 Doris 交易域省份粒度下单各窗口汇总表。

## 12.10.3 思路分析

1) 从 Kafka 下单明细主题读取数据

2) 过滤 null 数据并转换数据结构

3) 按照唯一键去重

4) 转换数据结构

JSONObject 转换为实体类 TradeProvinceOrderWindow。

5) 设置水位线

6) 按照省份 ID 分组

provinceId 可以唯一标识数据。

7) 开窗

8) 聚合计算

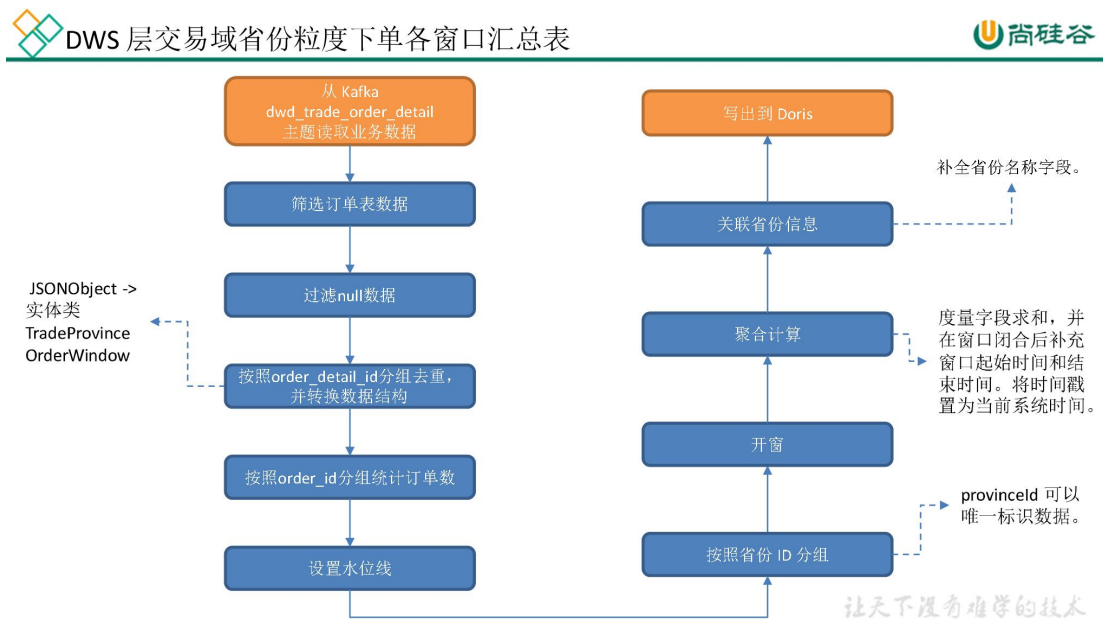
度量字段求和，并在窗口闭合后补充窗口起始时间、结束时间以及当前统计日期。

9) 关联省份信息

补全省份名称字段。

10) 写出到 Doris。

## 12.10.4 图解



## 12.10.5 Doris 建表语句

```
drop table if exists
gmall2023_realtime.dws_trade_province_order_window;
create table if not exists
gmall2023_realtime.dws_trade_province_order_window
```

```
(
    `stt`          DATETIME COMMENT '窗口起始时间',
    `edt`          DATETIME COMMENT '窗口结束时间',
    `cur_date`     DATE COMMENT '当天日期',
    `province_id`  TINYINT COMMENT '省份 ID',
    `province_name` CHAR(128) COMMENT '省份名称',
    `order_count`  BIGINT REPLACE COMMENT '累计下单次数',
    `order_amount` DECIMAL(16, 2) REPLACE COMMENT '累计下单金额'
) engine = olap aggregate key (`stt`,
`edt`, `cur_date`, `province_id`, `province_name`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
    "replication_num" = "3",
    "dynamic_partition.enable" = "true",
    "dynamic_partition.time_unit" = "DAY",
    "dynamic_partition.end" = "3",
    "dynamic_partition.prefix" = "par",
    "dynamic_partition.buckets" = "10"
);
```

## 12.10.6 代码实现

### 1) 实体类

在 common 模块的 bean 包下创建实体类 TradeProvinceOrderBean

```
package com.atguigu.gmall.realtime.common.bean;

import com.alibaba.fastjson.annotation.JSONField;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;

import java.math.BigDecimal;
import java.util.Set;

@Data
@AllArgsConstructor
@Builder
public class TradeProvinceOrderBean {
    // 窗口起始时间
    String stt;
    // 窗口结束时间
    String edt;
```

```
// 当天日期
String curDate;
// 省份 ID
String provinceId;
// 省份名称
@Builder.Default
String provinceName = "";

// 累计下单次数
Long orderCount;
// 累计下单金额
BigDecimal orderAmount;

// 时间戳
@JSONField(serialize = false)
Long ts;
@JSONField(serialize = false)
String orderDetailId;
@JSONField(serialize = false)
Set<String> orderIdSet;
}
```

## 2) 主程序

在本节新建模块下创建主程序 DwsTradeProvinceOrderWindow

```
package com.atguigu.gmall.realtime.dws.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import com.atguigu.gmall.realtime.common.bean.TradeProvinceOrderBean;
import com.atguigu.gmall.realtime.common.constant.Constant;
import com.atguigu.gmall.realtime.common.function.AsyncDimFunction;
import com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.MapFunction;
import org.apache.flink.api.common.functions.ReduceFunction;
import org.apache.flink.api.common.state.ValueState;
import org.apache.flink.api.common.state.ValueStateDescriptor;
import org.apache.flink.configuration.Configuration;
```

```
import org.apache.flink.streaming.api.datastream.AsyncDataStream;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.datastream.SingleOutputStreamOperator;
import
org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import
org.apache.flink.streaming.api.functions.KeyedProcessFunction;
import
org.apache.flink.streaming.api.functions.windowing.ProcessWindowFunction;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventTimeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.time.Duration;
import java.util.HashSet;
import java.util.concurrent.TimeUnit;

public class DwsTradeProvinceOrderWindow extends BaseApp {
    public static void main(String[] args) {
        new DwsTradeProvinceOrderWindow().start(
            10020,
            4,
            "dws_trade_province_order_window",
            Constant.TOPIC_DWD_TRADE_ORDER_DETAIL
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
        DataStreamSource<String> stream) {
        SingleOutputStreamOperator<TradeProvinceOrderBean>
reducedStream = stream
            .map(new MapFunction<String, TradeProvinceOrderBean>()
{
            @Override
            public TradeProvinceOrderBean map(String value)
```

```
throws Exception {
    JSONObject obj = JSON.parseObject(value);

    HashSet<String> set = new HashSet<>();
    set.add(obj.getString("order_id"));

    return TradeProvinceOrderBean.builder()
        .orderDetailId(obj.getString("id"))
        .orderAmount(obj.getBigDecimal("split_total_amount"))
        .provinceId(obj.getString("province_id"))
        .ts(obj.getLong("ts") * 1000)
        .orderIdSet(set)
        .build();
}

})
.keyBy(TradeProvinceOrderBean::getOrderDetailId) // 按照详情 id 去重
.process(new KeyedProcessFunction<String, TradeProvinceOrderBean, TradeProvinceOrderBean>() {

    private ValueState<Boolean> isFirstState;

    @Override
    public void open(Configuration parameters) throws Exception {
        isFirstState = getRuntimeContext().getState(new ValueStateDescriptor<Boolean>("isFirst", Boolean.class));
        // 一定给状态添加 ttl 省略
    }

    @Override
    public void processElement(TradeProvinceOrderBean value,

                                Context ctx,

                                Collector<TradeProvinceOrderBean> out) throws Exception {
        // 因为后期需要聚合的数据都在左表, 所以, 可以只去当前详情 id 的第一条数据即可
        if (isFirstState.value() == null) {
            isFirstState.update(true);
            out.collect(value);
        }
    }
})
```



```
        }
    }
    })
    .assignTimestampsAndWatermarks(
        WatermarkStrategy
            .<TradeProvinceOrderBean>forBoundedOutOfOrderness(Duration.ofSeconds(5L))
            .withTimestampAssigner((bean, ts) ->
                bean.getTs())
            .withIdleness(Duration.ofSeconds(120L))
    )
    .keyBy(TradeProvinceOrderBean::getProvinceId) // 分组
开窗聚合
    .window(TumblingEventTimeWindows.of(Time.seconds(5L))
    )
    .reduce(
        new ReduceFunction<TradeProvinceOrderBean>() {
            @Override
            public TradeProvinceOrderBean
reduce(TradeProvinceOrderBean value1,
TradeProvinceOrderBean value2) throws Exception {
value1.setOrderAmount(value1.getOrderAmount().add(value2.getOrder
Amount()));
value1.getOrderIdSet().addAll(value2.getOrderIdSet());
            return value1;
        }
    ),
    new
ProcessWindowFunction<TradeProvinceOrderBean,
TradeProvinceOrderBean, String, TimeWindow>() {
        @Override
        public void process(String provinceId,
            Context ctx,
Iterable<TradeProvinceOrderBean> elements,
Collector<TradeProvinceOrderBean> out) throws Exception {
            TradeProvinceOrderBean bean =
elements.iterator().next();
```

```
bean.setStt(DateFormatUtil.tsToDateTime(ctx.window().getStart()))
;

bean.setEdt(DateFormatUtil.tsToDateTime(ctx.window().getEnd()));

bean.setCurDate(DateFormatUtil.tsToDateForPartition(ctx.window().
getStart()));

                                bean.setOrderCount((long)
bean.getOrderIdSet().size());
                                out.collect(bean);
                                }
                                }
);

AsyncDataStream
    .unorderedWait( // 异步的方式补充维度
        reducedStream,
        new AsyncDimFunction<TradeProvinceOrderBean>()
    {
        @Override
        public String
getRowKey(TradeProvinceOrderBean bean) {
            return bean.getProvinceId();
        }

        @Override
        public String getTableName() {
            return "dim_base_province";
        }

        @Override
        public void addDims(TradeProvinceOrderBean
bean,
                                JSONObject dim) {

bean.setProvinceName(dim.getString("name"));
        }
    },
    120,
    TimeUnit.SECONDS
)
    .map(new DorisMapFunction<>())
    .sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA
```

```
TABASE + ".dws_trade_province_order_window",  
"dws_trade_province_order_window"));  
  
}  
}
```

### 12.10.7 测试

- 1) 启动 zk、kafka、Maxwell、HDFS、HBase、Redis、Doris
- 2) 运行应用程序 DimApp、DwdTradeOrderDetail、DwsTradeProvinceOrderWindow
- 3) 运行数据生成器
- 4) 查看 Doris 目标表数据

### 12.10.8 提交至 GitLab

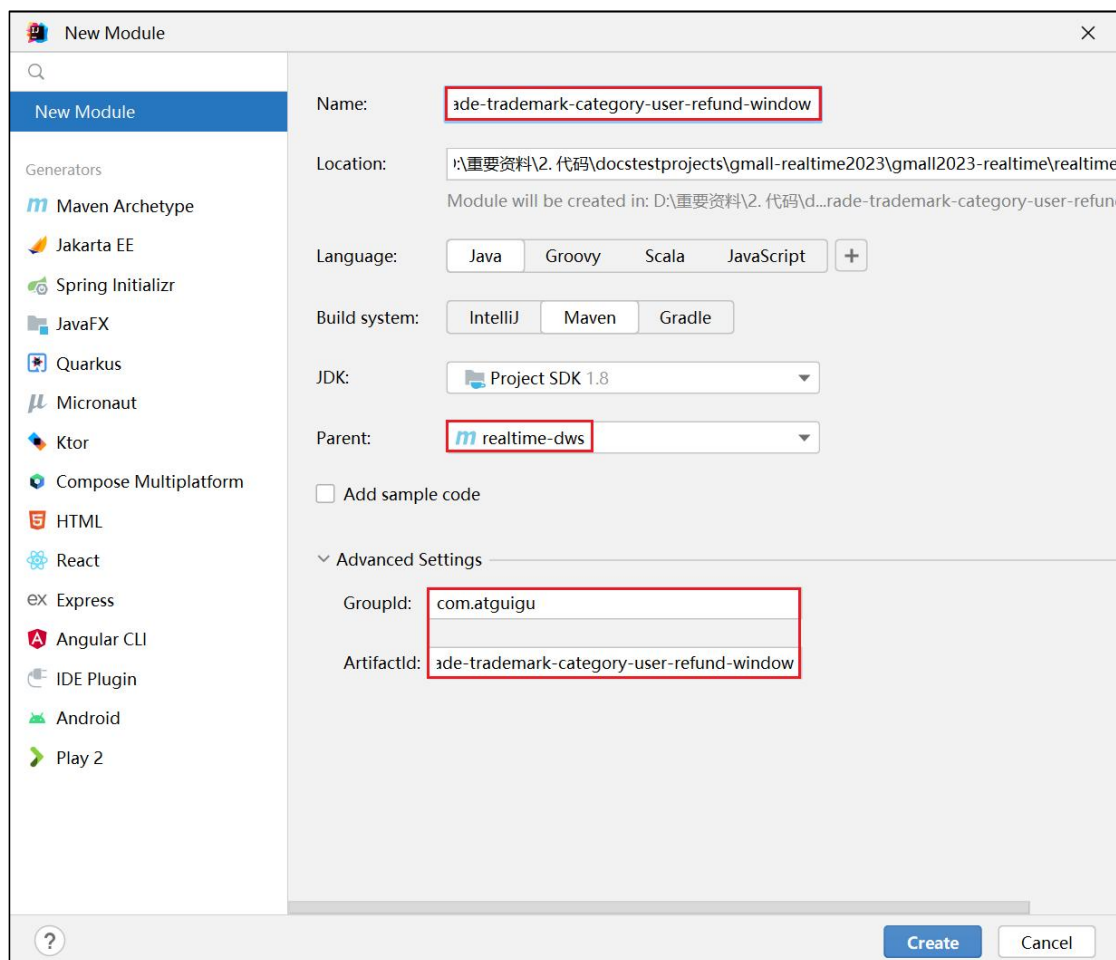
略。

## 12.11 交易域品牌-品类-用户粒度退单各窗口汇总表（练习）

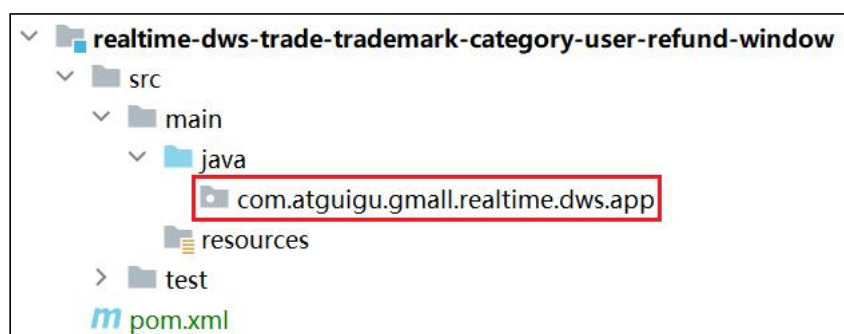
### 12.11.1 创建模块

- 1) 新建模块

创建 realtime-dws-trade-trademark-category-user-refund-window



## 2) 创建包结构



## 12.11.2 主要任务

从 Kafka 读取退单明细数据，关联与分组相关的维度信息后分组，统计各分组各窗口的订单数和订单金额，补充与分组无关的维度信息，将数据写入 Doris 交易域品牌-品类-用户粒度退单各窗口汇总表

## 12.11.3 思路分析

### 1) 从 Kafka 退单明细主题读取数据

## 2) 转换数据结构

JSONObject 转换为实体类 TradeTrademarkCategoryUserRefundBean。

## 3) 补充与分组相关的维度信息

(1) 关联 sku\_info 表

(2) 获取 tm\_id, category3\_id。

## 4) 设置水位线

## 5) 分组、开窗、聚合

按照维度信息分组，度量字段求和，并在窗口闭合后补充窗口起始时间、结束时间以及当前统计时间。

## 6) 补充与分组无关的维度信息

(1) 关联 base\_trademark 表

获取 tm\_name。

(2) 关联 base\_category3 表

获取 name (三级品类名称)，获取 category2\_id。

(3) 关联 base\_categroy2 表

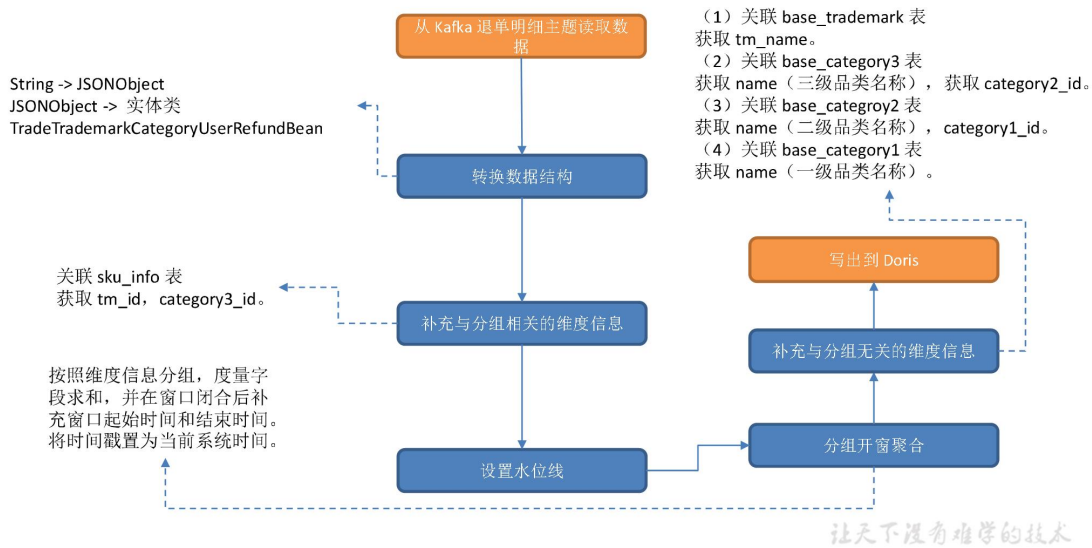
获取 name (二级品类名称)，category1\_id。

(4) 关联 base\_category1 表

获取 name (一级品类名称)。

## 7) 写出到 Doris

## 12.11.4 图解



### 12.11.5 Doris 建表语句

```
drop table if exists
gmall2023_realtime.dws_trade_trademark_category_user_refund_window;

create table if not exists
gmall2023_realtime.dws_trade_trademark_category_user_refund_window
(
    `stt`          DATETIME COMMENT '窗口起始时间',
    `edt`          DATETIME COMMENT '窗口结束时间',
    `cur_date`     DATE COMMENT '当天日期',
    `trademark_id` SMALLINT COMMENT '品牌 ID',
    `trademark_name` CHAR(256) COMMENT '品牌名称',
    `category1_id` SMALLINT COMMENT '一级品类 ID',
    `category1_name` CHAR(128) COMMENT '一级品类名称',
    `category2_id` SMALLINT COMMENT '二级品类 ID',
    `category2_name` CHAR(128) COMMENT '二级品类名称',
    `category3_id` SMALLINT COMMENT '三级品类 ID',
    `category3_name` CHAR(128) COMMENT '三级品类名称',
    `user_id`      INT COMMENT '用户 ID',
    `refund_count` INT REPLACE COMMENT '退单次数'
) engine = olap aggregate key
(`stt`, `edt`, `cur_date`, `trademark_id`, `trademark_name`, `category1_id`, `category1_name`, `category2_id`, `category2_name`, `category3_id`, `category3_name`, `user_id`)
partition by range(`cur_date`) ()
distributed by hash(`stt`) buckets 10
properties (
```

```
"replication_num" = "3",
"dynamic_partition.enable" = "true",
"dynamic_partition.time_unit" = "DAY",
"dynamic_partition.end" = "3",
"dynamic_partition.prefix" = "par",
"dynamic_partition.buckets" = "10"
);
```

## 12.11.6 代码实现

### 1) 实体类

在 common 模块的 bean 包下创建实体类 TradeTrademarkCategoryUserRefundBean

```
package com.atguigu.gmall.realtime.common.bean;

import com.alibaba.fastjson.annotation.JSONField;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;

import java.util.Set;

@Data
@AllArgsConstructor
@Builder
public class TradeTrademarkCategoryUserRefundBean {
    // 窗口起始时间
    String stt;
    // 窗口结束时间
    String edt;
    // 当天日期
    String curDate;
    // 品牌 ID
    String trademarkId;
    // 品牌名称
    String trademarkName;
    // 一级品类 ID
    String category1Id;
    // 一级品类名称
    String category1Name;
    // 二级品类 ID
    String category2Id;
    // 二级品类名称
    String category2Name;
```

```
// 三级品类 ID
String category3Id;
// 三级品类名称
String category3Name;

// 用户 ID
String userId;
// 退单次数
Long refundCount;

// 时间戳
@JSONField(serialize = false)
Long ts;
@JSONField(serialize = false)
String skuId;
@JSONField(serialize = false)
Set<String> orderIdSet;
}
```

## 2) 主程序

在本节新建模块下创建主程序 `DwsTradeTrademarkCategoryUserRefundWindow`

```
package com.atguigu.gmall.realtime.dws.app;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import com.atguigu.gmall.realtime.common.base.BaseApp;
import
com.atguigu.gmall.realtime.common.bean.TradeTrademarkCategoryUser
RefundBean;
import com.atguigu.gmall.realtime.common.constant.Constant;
import
com.atguigu.gmall.realtime.common.function.AsyncDimFunction;
import
com.atguigu.gmall.realtime.common.function.DorisMapFunction;
import com.atguigu.gmall.realtime.common.util.DateFormatUtil;
import com.atguigu.gmall.realtime.common.util.FlinkSinkUtil;
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.api.common.functions.MapFunction;
import org.apache.flink.api.common.functions.ReduceFunction;
import org.apache.flink.streaming.api.datastream.AsyncDataStream;
import org.apache.flink.streaming.api.datastream.DataStreamSource;
import
org.apache.flink.streaming.api.datastream.SingleOutputStreamOpera
tor;
```



```
import
org.apache.flink.streaming.api.environment.StreamExecutionEnviron
ment;
import
org.apache.flink.streaming.api.functions.windowing.ProcessWindowF
unction;
import
org.apache.flink.streaming.api.windowing.assigners.TumblingEventT
imeWindows;
import org.apache.flink.streaming.api.windowing.time.Time;
import
org.apache.flink.streaming.api.windowing.windows.TimeWindow;
import org.apache.flink.util.Collector;

import java.time.Duration;
import java.util.Collections;
import java.util.HashSet;
import java.util.concurrent.TimeUnit;

public class DwsTradeTrademarkCategoryUserRefundWindow extends
BaseApp {
    public static void main(String[] args) {
        new DwsTradeTrademarkCategoryUserRefundWindow().start(
            10031,
            4,
            "dws_trade_trademark_category_user_refund_window",
            Constant.TOPIC_DWD_TRADE_ORDER_REFUND
        );
    }

    @Override
    public void handle(StreamExecutionEnvironment env,
DataStreamSource<String> stream) {

SingleOutputStreamOperator<TradeTrademarkCategoryUserRefundBean>
beanStream = stream
        .map(new MapFunction<String,
TradeTrademarkCategoryUserRefundBean>() {
            @Override
            public TradeTrademarkCategoryUserRefundBean
map(String value) {
                JSONObject obj = JSON.parseObject(value);
                return
TradeTrademarkCategoryUserRefundBean.builder()
```

```
                .orderIdSet(new
HashSet<>(Collections.singleton(obj.getString("order_id"))))
                .skuId(obj.getString("sku_id"))
                .userId(obj.getString("user_id"))
                .ts(obj.getLong("ts") * 1000)
                .build();
        }
    });
    // 补充 keyBy 字段维度

SingleOutputStreamOperator<TradeTrademarkCategoryUserRefundBean>
reducedStream = AsyncDataStream
    .unorderedWait(
        beanStream,
        new
AsyncDimFunction<TradeTrademarkCategoryUserRefundBean>() {
            @Override
            public String
getRowKey(TradeTrademarkCategoryUserRefundBean bean) {
                return bean.getSkuId();
            }

            @Override
            public String getTableName() {
                return "dim_sku_info";
            }

            @Override
            public void
addDims(TradeTrademarkCategoryUserRefundBean bean, JSONObject dim)
{
                bean.setTrademarkId(dim.getString("tm_id"));

                bean.setCategory3Id(dim.getString("category3_id"));

            }
        },
        120,
        TimeUnit.SECONDS
    )
    .assignTimestampsAndWatermarks(
        WatermarkStrategy
            .<TradeTrademarkCategoryUserRefundBean>
```

```
forBoundedOutOfOrderness (Duration.ofSeconds (5L))
    .withTimestampAssigner((bean, ts) ->
bean.getTs())
    .withIdleness (Duration.ofSeconds (120L))

)
    .keyBy(bean -> bean.getUserId() + "_" +
bean.getCategory3Id() + "_" + bean.getTrademarkId())
    .window(TumblingEventTimeWindows.of (Time.seconds (5L))
)

    .reduce(
        new
ReduceFunction<TradeTrademarkCategoryUserRefundBean>() {
            @Override
            public TradeTrademarkCategoryUserRefundBean
reduce (TradeTrademarkCategoryUserRefundBean value1,

TradeTrademarkCategoryUserRefundBean value2) throws Exception {

value1.getOrderIdSet().addAll (value2.getOrderIdSet());
            return value1;
        }
    ),
    new
ProcessWindowFunction<TradeTrademarkCategoryUserRefundBean,
TradeTrademarkCategoryUserRefundBean, String, TimeWindow>() {
        @Override
        public void process (String s,
            Context ctx,

Iterable<TradeTrademarkCategoryUserRefundBean> elements,

Collector<TradeTrademarkCategoryUserRefundBean> out) throws
Exception {
            TradeTrademarkCategoryUserRefundBean
bean = elements.iterator().next();

bean.setStt (DateFormatUtil.tsToDateTime (ctx.window().getStart()))
;

bean.setEdt (DateFormatUtil.tsToDateTime (ctx.window().getEnd()));
```

```
bean.setCurDate(DateFormatUtil.tsToDate(ctx.window().getStart()));
// doris 的分区字段: 年月日带连字符也可以

        bean.setRefundCount((long)
bean.getOrderIdSet().size());

        out.collect(bean);
    }
}

);

SingleOutputStreamOperator<TradeTrademarkCategoryUserRefundBean>
tmStream = AsyncDataStream.unorderedWait(
    reducedStream,
    new
AsyncDimFunction<TradeTrademarkCategoryUserRefundBean>() {
    @Override
    public String
getRowKey(TradeTrademarkCategoryUserRefundBean bean) {
        return bean.getTrademarkId();
    }

    @Override
    public String getTableName() {
        return "dim_base_trademark";
    }

    @Override
    public void
addDims(TradeTrademarkCategoryUserRefundBean bean,
        JSONObject dim) {

bean.setTrademarkName(dim.getString("tm_name"));
    }
},
120,
TimeUnit.SECONDS
);

SingleOutputStreamOperator<TradeTrademarkCategoryUserRefundBean>
c3Stream = AsyncDataStream.unorderedWait(
    tmStream,
```

```
        new
AsyncDimFunction<TradeTrademarkCategoryUserRefundBean>() {
    @Override
    public String
getRowKey(TradeTrademarkCategoryUserRefundBean bean) {
        return bean.getCategory3Id();
    }

    @Override
    public String getTableName() {
        return "dim_base_category3";
    }

    @Override
    public void
addDims(TradeTrademarkCategoryUserRefundBean bean, JSONObject dim)
{
        bean.setCategory3Name(dim.getString("name"));

bean.setCategory2Id(dim.getString("category2_id"));
    }
    },
    120,
    TimeUnit.SECONDS
);

SingleOutputStreamOperator<TradeTrademarkCategoryUserRefundBean>
c2Stream = AsyncDataStream.unorderedWait(
    c3Stream,
    new
AsyncDimFunction<TradeTrademarkCategoryUserRefundBean>() {
        @Override
        public String
getRowKey(TradeTrademarkCategoryUserRefundBean bean) {
            return bean.getCategory2Id();
        }

        @Override
        public String getTableName() {
            return "dim_base_category2";
        }

        @Override
        public void
```

```
addDims(TradeTrademarkCategoryUserRefundBean bean, JSONObject dim)
{
    bean.setCategory2Name(dim.getString("name"));

    bean.setCategory1Id(dim.getString("category1_id"));
    },
    120,
    TimeUnit.SECONDS
);

SingleOutputStreamOperator<TradeTrademarkCategoryUserRefundBean>
resultStream = AsyncDataStream.unorderedWait(
    c2Stream,
    new
AsyncDimFunction<TradeTrademarkCategoryUserRefundBean>() {
    @Override
    public String
getRowKey(TradeTrademarkCategoryUserRefundBean bean) {
        return bean.getCategory1Id();
    }

    @Override
    public String getTableName() {
        return "dim_base_category1";
    }

    @Override
    public void
addDims(TradeTrademarkCategoryUserRefundBean bean, JSONObject dim)
{
    bean.setCategory1Name(dim.getString("name"));
    },
    120,
    TimeUnit.SECONDS
);

resultStream
    .map(new DorisMapFunction<>())
    .sinkTo(FlinkSinkUtil.getDorisSink(Constant.DORIS_DA
TABASE + ".dws_trade_trademark_category_user_refund_window",
```

```
"dws_trade_trademark_category_user_refund_window")));  
  
}  
}
```

### 12.11.7 测试

1) 启动 zk、kafka、Maxwell、HDFS、HBase、Redis、Doris

2) 运行应用程序

(1) DimApp

(2) DwdTradeOrderRefund

(3) DwsTradeTrademarkCategoryUserRefundWindow

3) 运行数据生成器

4) 查看 Doris 目标表数据

### 12.11.8 提交至 GitLab

略。